

TRUST Keywords Reference

Generated by Doxygen 1.9.8

1 Keywords Reference Manual	1
1.0.1 champ_generique_base	1
1.0.2 chimie	2
1.0.3 class_generic	2
1.0.4 comment	2
1.0.5 condlim_base	2
1.0.6 correlation_base	4
1.0.7 discretisation_base	4
1.0.8 domaine_base	4
1.0.9 field_base	4
1.0.10 front_field_base	5
1.0.11 interface_base	6
1.0.12 interpolation_ibm_base	6
1.0.13 interprete	6
1.0.14 loi_etat_base	9
1.0.15 loi_fermeture_base	10
1.0.16 loi_horaire	10
1.0.17 milieu_base	10
1.0.18 modele_turbulence_scal_base	10
1.0.19 mor_eqn	10
1.0.20 moyenne_imposee_deriv	11
1.0.21 nom	11
1.0.22 objet_lecture	12
1.0.23 partitionneur_deriv	15
1.0.24 pb_champ_evaluateur	16
1.0.25 pb_gen_base	16
1.0.26 porosites	17
1.0.27 precond_base	17
1.0.28 preconditionneur_petsc_deriv	17
1.0.29 prepro_ibm_base	17
1.0.30 reorder_mesh	18
1.0.31 schema_temps_base	18
1.0.32 solveur_implicite_base	18
1.0.33 solveur_petsc_deriv	19
1.0.34 source_base	19
1.0.35 sous_zone	20
1.0.36 transparent_medium_radiation_model	20
1.0.37 turbulence_parois_base	20
1.0.38 turbulence_parois_scalaire_base	20
1.1 Keywords derived from champ_generique_base	20
1.1.1 Keywords derived from champ_post_de_champs_post	21
1.2 Keywords derived from chimie	27

1.3 Keywords derived from class_generic	27
1.3.1 Keywords derived from dt_start	27
1.3.2 Keywords derived from solveur_sys_base	28
1.4 Keywords derived from comment	31
1.5 Keywords derived from condlim_base	31
1.5.1 Keywords derived from dirichlet	37
1.5.2 Keywords derived from neumann	39
1.5.3 Keywords derived from neumann_homogene	41
1.5.4 Keywords derived from neumann_paro	41
1.5.5 Keywords derived from paroi_echange_externe_impose	42
1.5.6 Keywords derived from paroi_echange_global_impose	42
1.5.7 Keywords derived from paroi_fixe	43
1.5.8 Keywords derived from symetrie	43
1.6 Keywords derived from correlation_base	43
1.6.1 Keywords derived from flux_parietal_base	43
1.7 Keywords derived from discretisation_base	44
1.8 Keywords derived from domaine_base	46
1.8.1 Keywords derived from domaine	46
1.9 Keywords derived from field_base	46
1.9.1 Keywords derived from champ_don_base	47
1.9.2 Keywords derived from champ_fonc_med	53
1.9.3 Keywords derived from champ_input_base	55
1.10 Keywords derived from front_field_base	55
1.10.1 Keywords derived from ch_front_input	60
1.10.2 Keywords derived from champ_front_composite	60
1.10.3 Keywords derived from champ_front_fonc_txyz	61
1.10.4 Keywords derived from champ_front_tabule	61
1.11 Keywords derived from interface_base	61
1.11.1 Keywords derived from saturation_base	62
1.12 Keywords derived from interpolation_ibm_base	63
1.12.1 Keywords derived from interpolation_ibm_elem_fluid	63
1.12.2 Keywords derived from interpolation_ibm_mean_gradient	66
1.13 Keywords derived from interprete	67
1.13.1 Keywords derived from analyse_angle	91
1.13.2 Keywords derived from extract_2d_from_3d	91
1.13.3 Keywords derived from extruder	92
1.13.4 Keywords derived from imprimer_flux	92
1.13.5 Keywords derived from interprete_geometrique_base	93
1.13.6 Keywords derived from mailler	93
1.13.7 Keywords derived from raffiner_isotrope	93
1.13.8 Keywords derived from read_file	94
1.13.9 Keywords derived from read_med	94

1.13.10 Keywords derived from regroupebord	95
1.13.11 Keywords derived from scatter	96
1.13.12 Keywords derived from tetraedriser	96
1.13.13 Keywords derived from transformer	99
1.13.14 Keywords derived from triangulate	100
1.13.15 Keywords derived from write_file	101
1.14 Keywords derived from loi_etat_base	101
1.14.1 Keywords derived from loi_etat_gaz_parfait_base	102
1.14.2 Keywords derived from loi_etat_gaz_reel_base	104
1.14.3 Keywords derived from loi_etat_tppi_base	104
1.15 Keywords derived from loi_fermeture_base	105
1.16 Keywords derived from loi_horaire	105
1.17 Keywords derived from milieu_base	106
1.17.1 Keywords derived from fluide_base	107
1.18 Keywords derived from modele_turbulence_scal_base	112
1.19 Keywords derived from mor_eqn	114
1.19.1 Keywords derived from eqn_base	114
1.20 Keywords derived from moyenne_imposee_deriv	139
1.21 Keywords derived from nom	141
1.22 Keywords derived from objet_lecture	141
1.22.1 Keywords derived from bloc_lecture	156
1.22.2 Keywords derived from bord_base	156
1.22.3 Keywords derived from convection_deriv	157
1.22.4 Keywords derived from defbord	160
1.22.5 Keywords derived from diffusion_deriv	161
1.22.6 Keywords derived from format_file_base	163
1.22.7 Keywords derived from lecture_bloc_moment_base	164
1.22.8 Keywords derived from mailler_base	164
1.22.9 Keywords derived from modele_turbulence_hyd_deriv	165
1.22.10 Keywords derived from op_non_conservativtifs_deriv	170
1.22.11 Keywords derived from parametre_equation_base	170
1.22.12 Keywords derived from postraitement_base	171
1.22.13 Keywords derived from sonde_base	174
1.22.14 Keywords derived from spec_pdc_r_base	177
1.22.15 Keywords derived from stat_post_deriv	177
1.22.16 Keywords derived from traitement_particulier_base	178
1.22.17 Keywords derived from type_diffusion_turbulente_multiphase_deriv	180
1.22.18 Keywords derived from type_perte_charge_deriv	181
1.23 Keywords derived from partitionneur_deriv	182
1.24 Keywords derived from pb_champ_evaluateur	184
1.25 Keywords derived from pb_gen_base	185
1.25.1 Keywords derived from pb_base	185

1.26 Keywords derived from porosites	222
1.27 Keywords derived from precondition_base	222
1.28 Keywords derived from preconditionneur_petsc_deriv	223
1.29 Keywords derived from prepro_ibm_base	225
1.30 Keywords derived from reorder_mesh	226
1.31 Keywords derived from schema_temps_base	226
1.31.1 Keywords derived from sch_cn_iteratif	246
1.31.2 Keywords derived from schema_implicite_base	250
1.32 Keywords derived from solveur_implicite_base	260
1.32.1 Keywords derived from simplifier	261
1.33 Keywords derived from solveur_petsc_deriv	266
1.34 Keywords derived from source_base	273
1.35 Keywords derived from sous_zone	281
1.36 Keywords derived from transparent_medium_radiation_model	282
1.37 Keywords derived from turbulence_paro_base	282
1.38 Keywords derived from turbulence_paro_scalaire_base	282
Index	283

1 Keywords Reference Manual

This manual is the complete reference for all keywords accepted by TRUST .data input files. Keywords are extracted automatically from `// XD` annotations embedded in the C++ source code and assembled by the trustify pipeline — the content here always reflects the current state of the source.

Keywords are grouped by their base class. Each section below covers one family: the base class description is shown first, followed by all concrete keywords that inherit from it, organised recursively by intermediate parent when the hierarchy has more than one level.

1.0.1 champ_generique_base

- `champ_generique_base`
 - `champ_post_de_champs_post`
 - * `champ_post_operateur_base`
 - `divergence`
 - `gradient`
 - * `champ_post_operateur_eqn`
 - * `champ_post_statistiques_base`
 - `correlation`
 - `ecart_type`
 - `moyenne`
 - * `extraction`
 - * `interpolation`
 - * `morceau_equation`
 - * `reduction_0d`
 - * `tparo_vef`
 - * `transformation`
 - `predefini`
 - `refchamp`

1.0.2 chimie

- [chimie](#)

1.0.3 class_generic

- [class_generic](#)
 - [dt_start](#)
 - * [dt_calc_dt_calc](#)
 - * [dt_calc_dt_fixe](#)
 - * [dt_calc_dt_min](#)
 - [solveur_sys_base](#)
 - * [amg](#)
 - * [cholesky](#)
 - * [optimal](#)
 - * [petsc](#)
 - [amgx](#)
 - [cudss](#)
 - [petsc_gpu](#)
 - * [solv_gcp](#)
 - [gcp_ns](#)
 - * [solv_gmres](#)

1.0.4 comment

- [comment](#)

1.0.5 condlim_base

- [condlim_base](#)
 - [dirichlet](#)
 - * [frontiere_ouverte_alpha_impose](#)
 - [entree_supersonique](#)
 - * [frontiere_ouverte_concentration_imposee](#)
 - * [frontiere_ouverte_temperature_imposee](#)
 - [entree_temperature_imposee_h](#)
 - [frontiere_ouverte_temperature_imposee_rayo_transp](#)
 - * [frontiere_ouverte_vitesse_imposee](#)
 - [frontiere_ouverte_vitesse_imposee_sortie](#)
 - [frontiere_ouverte_rho_u_impose](#)
 - * [paroi_defilante](#)
 - * [paroi_deplacement_impose](#)
 - * [paroi_knudsen_non_negligeable](#)
 - * [paroi_temperature_imposee](#)
 - [temperature_imposee_paro](#)
 - * [scalaire_impose_paro](#)

- `echange_externe_radiatif`
- `echange_interne_global_impose`
- `echange_interne_global_parfait`
- `echange_interne_impose`
- `echange_interne_parfait`
- `frontiere_ouverte_fraction_massique_imposee`
- `neumann`
 - * `frontiere_ouverte`
 - `frontiere_ouverte_rayo_transp`
 - * `frontiere_ouverte_gradient_pression_impose`
 - `frontiere_ouverte_gradient_pression_impose_vefprep1b`
 - * `frontiere_ouverte_gradient_pression_libre_vef`
 - * `frontiere_ouverte_gradient_pression_libre_vefprep1b`
 - * `frontiere_ouverte_pression_imposee`
 - * `frontiere_ouverte_pression_imposee_orlansky`
 - * `frontiere_ouverte_pression_moyenne_imposee`
 - * `sortie_libre_temperature_imposee_h`
- `neumann_homogene`
 - * `neumann_paro_i_adiabatique`
- `neumann_paro_i`
 - * `paroi_flux_impose_rayo_transp`
- `paroi`
- `paroi_adiabatique`
- `paroi_contact`
- `paroi_contact_fictif`
- `paroi_contact_rayo`
- `paroi_decalee_robin`
- `paroi_echange_contact_correlation_vdf`
- `paroi_echange_contact_correlation_vef`
- `paroi_echange_contact_vdf`
- `paroi_echange_externe_impose`
 - * `paroi_echange_externe_impose_h`
 - * `paroi_echange_externe_impose_rayo_transp`
- `paroi_echange_global_impose`
 - * `echange_couplage_thermique`
- `paroi_fixe`
 - * `paroi_fixe_iso_genepi2_sans_contribution_aux_vitesses_sommets`
- `paroi_flux_impose`
- `paroi_pression_imposee`
- `periodic`
- `robin_vef`
- `symetrie`
 - * `sortie_supersonique`

1.0.6 correlation_base

- [correlation_base](#)
 - [flux_parietal_base](#)
 - * [flux_parietal_nusselt](#)

1.0.7 discretisation_base

- [discretisation_base](#)
 - [coloc](#)
 - [dg](#)
 - [ef](#)
 - [ef_axi](#)
 - [ijk](#)
 - [polymac_cdo](#)
 - [polymac_hfv](#)
 - [polymac_mpfa](#)
 - [vdf](#)
 - [vef](#)

1.0.8 domaine_base

- [domaine_base](#)
 - [domaine](#)
 - * [domaineaxi1d](#)
 - [domaine_64](#)
 - [domaine_ijk](#)

1.0.9 field_base

- [field_base](#)
 - [champ_don_base](#)
 - * [champ_composite](#)
 - [champ_musig](#)
 - * [champ_don_lu](#)
 - * [champ_fonc_fonction_txyz_morceaux](#)
 - * [champ_fonc_interp](#)
 - * [champ_fonc_reprise](#)
 - * [champ_fonc_t](#)
 - * [champ_fonc_tabule](#)
 - [champ_fonc_fonction](#)
 - [champ_fonc_fonction_txyz](#)
 - * [champ_fonc_tabule_morceaux](#)
 - [champ_fonc_tabule_morceaux_interp](#)

- * [champ_init_canal_sinal](#)
- * [champ_morceaux](#)
- * [champ_parametrique](#)
- * [champ_som_lu_vdf](#)
- * [champ_som_lu_vef](#)
- * [champ_tabule_temps](#)
 - [champ_tabule_lu](#)
- * [champ_uniforme_morceaux](#)
 - [champ_uniforme_morceaux_tabule_temps](#)
 - [valeur_totale_sur_volume](#)
- * [field_func_txyz](#)
- * [field_func_xyz](#)
- * [init_par_partie](#)
- * [tayl_green](#)
- * [uniform_field](#)
- [champ_fonc_med](#)
 - * [champ_fonc_med_table_temps](#)
 - * [champ_fonc_med_tabule](#)
- [champ_input_base](#)
 - * [champ_input_p0](#)
 - * [champ_input_p0_composite](#)
- [champ_ostwald](#)

1.0.10 [front_field_base](#)

- [front_field_base](#)
 - [boundary_field_inward](#)
 - [ch_front_input](#)
 - * [ch_front_input_uniforme](#)
 - [champ_front_bruite](#)
 - [champ_front_calc](#)
 - [champ_front_composite](#)
 - * [champ_front_musig](#)
 - [champ_front_debit](#)
 - [champ_front_debit_massique](#)
 - [champ_front_debit_qc_vdf](#)
 - [champ_front_debit_qc_vdf_fonc_t](#)
 - [champ_front_fonc_pois_ipsn](#)
 - [champ_front_fonc_pois_tube](#)
 - [champ_front_fonc_t](#)
 - [champ_front_fonc_txyz](#)
 - * [champ_front_xyz_tabule](#)
 - [champ_front_fonc_xyz](#)
 - [champ_front_fonction](#)
 - [champ_front_lu](#)
 - [champ_front_med](#)

- champ_front_normal_vef
- champ_front_parametrique
- champ_front_pression_from_u
- champ_front_recyclage
- champ_front_tabule
 - * champ_front_tabule_lu
- champ_front_tangentiel_vef
- champ_front_uniforme
- champ_front_xyz_debit

1.0.11 interface_base

- interface_base
 - interface_baer_nunziato
 - interface_sigma_constant
 - saturation_base
 - * saturation_constant
 - * saturation_sodium

1.0.12 interpolation_ibm_base

- interpolation_ibm_base
 - interpolation_ibm_aucune
 - interpolation_ibm_elem_fluid
 - * interpolation_ibm_hybride
 - * interpolation_ibm_power_law_tbl
 - * interpolation_ibm_thermal_wall_law
 - interpolation_ibm_mean_gradient
 - * interpolation_ibm_power_law_tbl_u_star

1.0.13 interprete

- interprete
 - analyse_angle
 - * analyse_angle_64
 - associate
 - axi
 - bidim_axi
 - calculer_moments
 - cgns_to_lata
 - criteres_convergence
 - debog
 - declarer_bord_perio

- [decoupebord](#)
- [decouper_bord_coincident](#)
- [dilate](#)
- [dimension](#)
- [disable_tu](#)
- [discretiser_domaine](#)
- [discretize](#)
- [distance_paro](#)
- [ecrire_champ_med](#)
- [ecrire_fichier_xyz_valeur](#)
- [ecrire_med](#)
- [ecriturelecturespecial](#)
- [execute_parallel](#)
- [extract_2d_from_3d](#)
 - * [extract_2daxi_from_3d](#)
- [extraire_domaine](#)
- [extraire_plan](#)
- [extraire_surface](#)
- [extrudebord](#)
- [extrudeparoi](#)
- [extruder](#)
 - * [extruder_en3](#)
- [extruder_en20](#)
- [facsec_expert](#)
- [fin](#)
- [imprimer_flux](#)
 - * [imprimer_flux_sum](#)
- [integrer_champ_med](#)
- [interprete_geometrique_base](#)
 - * [create_domain_from_sub_domain](#)
- [lata_to_cgns](#)
- [lata_to_med](#)
- [lata_to_other](#)
- [lire_ideas](#)
- [lml_to_lata](#)
- [mailler](#)
 - * [mailler_64](#)
- [maillerparallel](#)
- [mass_source](#)
- [merge_med](#)
- [mkdir](#)
- [modif_bord_to_raccord](#)
- [modifydomaineaxi1d](#)
- [moyenne_volumique](#)
- [multigrid_solver](#)

- multiplefiles
- my_comm_group
- nettoiepasnoeuds
- op_conv_ef_stab_dg_elem
- op_conv_ef_stab_polymac_cdo_face
- op_conv_ef_stab_polymac_hfv_elem
- op_conv_ef_stab_polymac_hfv_face
- op_conv_ef_stab_polymac_mpfa_face
- option_cgns
- option_dg
- option_ijk
- option_interpolation
- option_polymac_family
- option_vdf
- orientefacesbord
- parallel_io_parameters
- partition
- partition_multi
- pilote_icoco
- polyedriser
- postraiter_domaine
- precisiongeom
- raffiner_anisotrope
- raffiner_isotrope
 - * raffiner_simplexes_64
- raffiner_isotrope_parallele
- read
- read_file
 - * read_file_bin
 - * read_unsupported_ascii_file_from_icem
- read_med
 - * lire_med_64
- read_tgrid
- rectify_mesh
- redresser_hexaedres_vdf
- refine_mesh
- regroupebord
 - * regroupebord_64
- remove_elem
- remove_invalid_internal_boundaries
- reorienter_tetraedres
- reorienter_triangles
- resequencing
- residuals
- rotation

- scatter
 - * scattermed
- solve
- stat_per_proc_perf_log
- supprime_bord
- system
- test_solveur
- test_sse_kernels
- testeur
- testeur_medcoupling
- tetraedriser
 - * tetraedriser_homogene
 - * tetraedriser_homogene_compact
 - * tetraedriser_homogene_fin
 - * tetraedriser_par_prisme
- transformer
 - * transformer_64
- triangulate
 - * trianguler_fin
 - * trianguler_h
- verifier_qualite_raffinements
- verifier_simplexes
- verifiercoin
- write
- write_file
 - * ecrire_fichier_formatte

1.0.14 loi_etat_base

- loi_etat_base
 - loi_etat_gaz_parfait_base
 - * binaire_gaz_parfait_qc
 - * binaire_gaz_parfait_wc
 - * multi_gaz_parfait_qc
 - * multi_gaz_parfait_wc
 - * perfect_gaz_qc
 - * perfect_gaz_wc
 - * rhot_gaz_parfait_qc
 - loi_etat_gaz_reel_base
 - * rhot_gaz_reel_qc
 - loi_etat_tpqi_base
 - * coolprop_qc
 - * coolprop_wc
 - * eos_qc
 - * eos_wc

1.0.15 loi_fermeture_base

- [loi_fermeture_base](#)
 - [loi_fermeture_test](#)

1.0.16 loi_horaire

- [loi_horaire](#)

1.0.17 milieu_base

- [milieu_base](#)
 - [constituant](#)
 - [espece](#)
 - [fluide_base](#)
 - * [fluide_incompressible](#)
 - [fluide_ostwald](#)
 - * [fluide_quasi_compressible](#)
 - * [fluide_sodium_gaz](#)
 - * [fluide_sodium_liquide](#)
 - * [fluide_stiffened_gas](#)
 - * [fluide_weakly_compressible](#)
 - [milieu_elasticite](#)
 - [solide](#)

1.0.18 modele_turbulence_scal_base

- [modele_turbulence_scal_base](#)
 - [modele_turbulence_scal_null](#)
 - [prandtl](#)
 - [schmidt](#)

1.0.19 mor_eqn

- [mor_eqn](#)
 - [eqn_base](#)
 - * [aire_interfaciale](#)
 - * [conduction](#)
 - [conduction_ibm](#)
 - * [cons_euler](#)
 - [energy_euler](#)
 - [fraction_euler](#)
 - [masse_euler](#)
 - * [convection_diffusion_chaleur_qc](#)

- [convection_diffusion_chaleur_turbulent_qc](#)
- * [convection_diffusion_chaleur_wc](#)
- * [convection_diffusion_concentration](#)
 - [convection_diffusion_concentration_turbulent](#)
- * [convection_diffusion_espece_binaire_qc](#)
 - [convection_diffusion_espece_binaire_turbulent_qc](#)
- * [convection_diffusion_espece_binaire_wc](#)
- * [convection_diffusion_espece_multi_qc](#)
- * [convection_diffusion_espece_multi_turbulent_qc](#)
- * [convection_diffusion_espece_multi_wc](#)
- * [convection_diffusion_temperature](#)
 - [convection_diffusion_temperature_ibm](#)
- * [convection_diffusion_temperature_ibm_turbulent](#)
- * [convection_diffusion_temperature_turbulent](#)
- * [echelle_temporelle_turbulente](#)
- * [energie_cinetique_turbulente](#)
- * [energie_cinetique_turbulente_wit](#)
- * [energie_multiphase](#)
- * [energie_multiphase_enthalpie](#)
- * [equation_navier_cauchy](#)
- * [masse_multiphase](#)
- * [momentum_euler](#)
- * [navier_stokes_standard](#)
 - [navier_stokes_ibm](#)
 - [navier_stokes_ibm_turbulent](#)
 - [navier_stokes_qc](#)
 - [navier_stokes_turbulent](#)
 - [navier_stokes_turbulent_qc](#)
 - [navier_stokes_wc](#)
- * [qdm_multiphase](#)
- * [taux_dissipation_turbulent](#)

1.0.20 moyenne_imposee_deriv

- [moyenne_imposee_deriv](#)
 - [moyenne_imposee_connexion_approchee](#)
 - [moyenne_imposee_connexion_exacte](#)
 - [moyenne_imposee_interpolation](#)
 - [moyenne_imposee_logarithmique](#)
 - [moyenne_imposee_profil](#)

1.0.21 nom

- [nom](#)
 - [nom_anonyme](#)

1.0.22 objet_lecture

- [objet_lecture](#)
 - [bloc_convection](#)
 - [bloc_couronne](#)
 - [bloc_decouper](#)
 - [bloc_diffusion](#)
 - [bloc_diffusion_standard](#)
 - [bloc_ef](#)
 - [bloc_fichier](#)
 - [bloc_lec_champ_init_canal_sinal](#)
 - [bloc_lecture](#)
 - * [bloc_criteres_convergence](#)
 - * [solveur_petsc_option_cli](#)
 - [bloc_lecture_poro](#)
 - [bloc_op_non_conservativtifs](#)
 - [bloc_origine_cotes](#)
 - [bloc_pave](#)
 - [bloc_pdf_model](#)
 - [bloc_sutherland](#)
 - [bloc_tube](#)
 - [bord_base](#)
 - * [bord](#)
 - * [groupe_faces](#)
 - * [internes](#)
 - * [raccord](#)
 - [champ_a_post](#)
 - [champs_posts](#)
 - [champs_posts_fichier](#)
 - [coarsen_operator_uniform](#)
 - [condinit](#)
 - [condlimlu](#)
 - [convection_deriv](#)
 - * [convection_amont](#)
 - * [convection_amont_old](#)
 - * [convection_btd](#)
 - * [convection_centre](#)
 - * [convection_centre4](#)
 - * [convection_centre_old](#)
 - * [convection_di_l2](#)
 - * [convection_ef](#)
 - * [convection_ef_stab](#)
 - * [convection_generic](#)
 - * [convection_hll](#)
 - * [convection_kquick](#)
 - * [convection_muscl](#)
 - * [convection_muscl3](#)

- * convection_muscl_new
- * convection_muscl_old
- * convection_negligeable
- * convection_quick
- * convection_rusanov
- * convection_supg
- defbord
 - * defbord_2
 - * defbord_3
- definition_champ
- definition_champs_fichier
- deuxentiers
- deuxmots
- diffusion_deriv
 - * diffusion_negligeable
 - * diffusion_option
 - * diffusion_p1ncp1b
 - * diffusion_stab
 - * diffusion_standard
 - * diffusion_turbulente_multiphase
 - * diffusion_p1b
- dt_impr_nusselt_mean_only
- dt_impr_ustar_mean_only
- entierfloat
- floatfloat
- fonction_champ_reprise
- form_a_nb_points
- format_file_base
 - * binaire
 - * formatte
 - * pdi
 - * pdi_expert
 - * single_hdf
 - * xyz
- format_lata_to_med
- format_trustlata
- info_med
- interface_posts
- lecture_bloc_moment_base
 - * calcul
 - * centre_de_gravite
- mailler_base
 - * domain
 - * epsilon
 - * pave
 - pave_64
- modele_turbulence_hyd_deriv

- * mod_turb_hyd_ss_maille
 - longueur_melange
 - sous_maille_smag
 - sous_maille_wale
 - * modele_turbulence_hyd_0_eq_base
 - * modele_turbulence_hyd_null
- nom_postraitement
- op_implicit
- op_non_conservatifs_deriv
 - * termes_non_conservatifs_hll
 - * termes_non_conservatifs_negligeable
- parametre_equation_base
 - * parametre_diffusion_implicit
 - * parametre_implicit
- postraitement_base
 - * postraitement
 - corps_postraitement
- quatremots
- reaction
- remove_elem_bloc
- sonde
- sonde_base
 - * circle
 - * circle_3
 - * numero_elem_sur_maitre
 - * plan
 - * points
 - point
 - segmentpoints
 - * position_like
 - * radius
 - * segment
 - * segmentfacesx
 - * segmentfacesy
 - * segmentfacesz
 - * volume
- sondes_fichier
- sous_zone_valeur
- spec_pdc_base
 - * longitudinale
 - * transversale
- stat_post_deriv
 - * stat_post_correlation
 - * stat_post_ecart_type
 - * stat_post_moyenne
 - * stat_post_t_deb
 - * stat_post_t_fin
- stats_posts

- stats_posts_fichier
- stats_serie_posts
- stats_serie_posts_fichier
- traitement_particulier
- traitement_particulier_base
 - * canal
 - * chmoy_faceperio
 - * ec
 - * profils_thermo
 - * temperature
 - * thi
- troisf
- troismots
- type_diffusion_turbulente_multiphase_deriv
 - * type_diffusion_turbulente_multiphase_aire_interfaciale
 - * type_diffusion_turbulente_multiphase_l_melange
 - * type_diffusion_turbulente_multiphase_prandtl
 - * type_diffusion_turbulente_multiphase_sgdh
 - * type_diffusion_turbulente_multiphase_smago
 - * type_diffusion_turbulente_multiphase_wale
- type_perte_charge_deriv
 - * type_perte_charge_dp
 - * type_perte_charge_dp_regul
- type_postraitement_ft_lata
- type_un_post
- un_pb
- un_point
- un_postraitement
- un_postraitement_spec
- verifiercoin_bloc

1.0.23 partitionneur_deriv

- partitionneur_deriv
 - partitionneur_fichier_decoupage
 - partitionneur_fichier_med
 - partitionneur_metis
 - partitionneur_metis_64
 - partitionneur_parmetis
 - partitionneur_partition
 - partitionneur_sous_dom
 - partitionneur_sous_domaines
 - partitionneur_tranche
 - partitionneur_union

1.0.24 pb_champ_evaluateur

- [pb_champ_evaluateur](#)

1.0.25 pb_gen_base

- [pb_gen_base](#)
 - [coupled_problem](#)
 - [pb_base](#)
 - * [pb_conduction](#)
 - * [pb_conduction_ibm](#)
 - * [pb_euler](#)
 - * [pb_hydraulique](#)
 - * [pb_hydraulique_cloned_concentration](#)
 - * [pb_hydraulique_cloned_concentration_turbulent](#)
 - * [pb_hydraulique_concentration](#)
 - * [pb_hydraulique_concentration_scalaires_passifs](#)
 - * [pb_hydraulique_concentration_turbulent](#)
 - * [pb_hydraulique_concentration_turbulent_scalaires_passifs](#)
 - * [pb_hydraulique_ibm](#)
 - * [pb_hydraulique_ibm_turbulent](#)
 - * [pb_hydraulique_list_concentration](#)
 - * [pb_hydraulique_list_concentration_turbulent](#)
 - * [pb_hydraulique_melange_binaire_qc](#)
 - * [pb_hydraulique_melange_binaire_turbulent_qc](#)
 - * [pb_hydraulique_melange_binaire_wc](#)
 - * [pb_hydraulique_turbulent](#)
 - * [pb_multiphase](#)
 - [pb_multiphase_enthalpie](#)
 - [pb_multiphase_hem](#)
 - * [pb_post](#)
 - * [pb_thermohydraulique](#)
 - * [pb_thermohydraulique_cloned_concentration](#)
 - * [pb_thermohydraulique_cloned_concentration_turbulent](#)
 - * [pb_thermohydraulique_concentration](#)
 - * [pb_thermohydraulique_concentration_scalaires_passifs](#)
 - * [pb_thermohydraulique_concentration_turbulent](#)
 - * [pb_thermohydraulique_concentration_turbulent_scalaires_passifs](#)
 - * [pb_thermohydraulique_especes_qc](#)
 - * [pb_thermohydraulique_especes_turbulent_qc](#)
 - * [pb_thermohydraulique_especes_wc](#)
 - * [pb_thermohydraulique_ibm](#)
 - * [pb_thermohydraulique_ibm_turbulent](#)
 - * [pb_thermohydraulique_list_concentration](#)
 - * [pb_thermohydraulique_list_concentration_turbulent](#)
 - * [pb_thermohydraulique_qc](#)
 - * [pb_thermohydraulique_scalaires_passifs](#)
 - * [pb_thermohydraulique_turbulent](#)
 - * [pb_thermohydraulique_turbulent_qc](#)

- * [pb_thermohydraulique_turbulent_scalaires_passifs](#)
- * [pb_thermohydraulique_wc](#)
- * [problem_read_generic](#)
- * [probleme_elasticite_lineaire](#)
- [pbc_med](#)

1.0.26 porosites

- [porosites](#)

1.0.27 precondition_base

- [precond_base](#)
 - [precondsolv](#)
 - [ssor](#)
 - [ssor_bloc](#)

1.0.28 preconditionneur_petsc_deriv

- [preconditionneur_petsc_deriv](#)
 - [preconditionneur_petsc_block_jacobi_icc](#)
 - [preconditionneur_petsc_block_jacobi_ilu](#)
 - [preconditionneur_petsc_boomeramg](#)
 - [preconditionneur_petsc_c_amg](#)
 - [preconditionneur_petsc_diag](#)
 - [preconditionneur_petsc_eisentat](#)
 - [preconditionneur_petsc_ilu_mumps](#)
 - [preconditionneur_petsc_jacobi](#)
 - [preconditionneur_petsc_lu](#)
 - [preconditionneur_petsc_null](#)
 - [preconditionneur_petsc_pilut](#)
 - [preconditionneur_petsc_sa_amg](#)
 - [preconditionneur_petsc_spai](#)
 - [preconditionneur_petsc_ssor](#)

1.0.29 prepro_ibm_base

- [prepro_ibm_base](#)
 - [prepro_ibm_ponderation](#)
 - [prepro_ibm_uzawa](#)

1.0.30 reorder_mesh

- [reorder_mesh](#)

1.0.31 schema_temps_base

- [schema_temps_base](#)
 - [euler_scheme](#)
 - [leap_frog](#)
 - [runge_kutta_ordre_2](#)
 - [runge_kutta_ordre_2_classique](#)
 - [runge_kutta_ordre_3](#)
 - [runge_kutta_ordre_3_classique](#)
 - [runge_kutta_ordre_4](#)
 - [runge_kutta_ordre_4_classique](#)
 - [runge_kutta_ordre_4_classique_3_8](#)
 - [runge_kutta_rationnel_ordre_2](#)
 - [sch_cn_iteratif](#)
 - * [sch_cn_ex_iteratif](#)
 - [schema_adams_bashforth_order_2](#)
 - [schema_adams_bashforth_order_3](#)
 - [schema_implicite_base](#)
 - * [schema_adams_moulton_order_2](#)
 - * [schema_adams_moulton_order_3](#)
 - * [schema_backward_differentiation_order_2](#)
 - * [schema_backward_differentiation_order_3](#)
 - * [schema_euler_implicite](#)
 - [schema_predictor_corrector](#)

1.0.32 solveur_implicite_base

- [solveur_implicite_base](#)
 - [newmark](#)
 - [simpler](#)
 - * [piso](#)
 - [implicite](#)
 - [simple](#)
 - [solveur_u_p](#)
 - * [sets](#)
 - [ice](#)
 - [solveur_lineaire_std](#)

1.0.33 solveur_petsc_deriv

- [solveur_petsc_deriv](#)
 - [solveur_petsc_bicgstab](#)
 - [solveur_petsc_cholesky](#)
 - [solveur_petsc_cholesky_lapack](#)
 - [solveur_petsc_cholesky_mumps_blr](#)
 - [solveur_petsc_cholesky_out_of_core](#)
 - [solveur_petsc_cholesky_pastix](#)
 - [solveur_petsc_cholesky_superlu](#)
 - [solveur_petsc_cholesky_umfpack](#)
 - [solveur_petsc_cli](#)
 - [solveur_petsc_cli_quiet](#)
 - [solveur_petsc_gcp](#)
 - [solveur_petsc_gmres](#)
 - [solveur_petsc_ibicgstab](#)
 - [solveur_petsc_lu](#)
 - [solveur_petsc_pipecg](#)

1.0.34 source_base

- [source_base](#)
 - [acceleration](#)
 - [boussinesq_concentration](#)
 - [boussinesq_temperature](#)
 - [canal_perio](#)
 - [coriolis](#)
 - [correction_antal](#)
 - [correction_tomiyama](#)
 - [darcy](#)
 - [derivee_forme](#)
 - [dirac](#)
 - [dispersion_bulles](#)
 - [dp_impose](#)
 - [echange_thermique_volumique](#)
 - [flux_interfacial](#)
 - [forchheimer](#)
 - [frottement_interfacial](#)
 - [perte_charge_anisotrope](#)
 - [perte_charge_circulaire](#)
 - [perte_charge_directionnelle](#)
 - [perte_charge_isotrope](#)
 - [perte_charge_reguliere](#)
 - [perte_charge_singuliere](#)

- [portance_interfaciale](#)
- [puissance_thermique](#)
- [radioactive_decay](#)
- [source_constituant](#)
- [source_generique](#)
- [source_meca_grad_pression_thermique](#)
- [source_pdf](#)
- [source_qdm](#)
- [source_qdm_lambdaup](#)
- [source_th_tdivu](#)
- [terme_puissance_thermique_echange_impose](#)
- [travail_pression](#)

1.0.35 sous_zone

- [sous_zone](#)

1.0.36 transparent_medium_radiation_model

- [transparent_medium_radiation_model](#)

1.0.37 turbulence_paro_base

- [turbulence_paro_base](#)
 - [negligeable](#)

1.0.38 turbulence_paro_scalaire_base

- [turbulence_paro_scalaire_base](#)
 - [negligeable_scalaire](#)

1.1 Keywords derived from champ_generique_base

champ_generique_base

not set

- [Keywords derived from champ_post_de_champs_post](#)

predefini**Inherits from:** [champ_generique_base](#)

This keyword is used to post process predefined postprocessing fields.

Parameters:

- **pb_champ** (*type:* [deuxmots](#)) { Pb_champ nom_pb nom_champ } : nom_pb is the problem name and nom_champ is the selected field name. The available keywords for the field name are: energie_cinetique_totale, energie_cinetique_elem, viscosite_turbulente, viscous_force_x, viscous_force_y, viscous_force_z, pressure_force_x, pressure_force_y, pressure_force_z, total_force_x, total_force_y, total_force_z, viscous_force, pressure_force, total_force

refchamp**Synonyms:** champ_post_refchamp**Inherits from:** [champ_generique_base](#)

Field of problem

Parameters:

- **[nom_source]** (*type:* string) The alias name for the field
- **pb_champ** (*type:* [deuxmots](#)) { Pb_champ nom_pb nom_champ } : nom_pb is the problem name and nom_champ is the selected field name.

1.1.1 Keywords derived from champ_post_de_champs_post**champ_post_de_champs_post****Inherits from:** [champ_generique_base](#)

not_set

Parameters:

- **[source]** (*type:* [champ_generique_base](#)) the source field.
- **[sources]** (*type:* list of [Champ_generique_base](#)) XXX
- **[nom_source]** (*type:* string) To name a source field with the nom_source keyword
- **[source_reference]** (*type:* string) not_set
- **[sources_reference]** (*type:* list of [Nom_anonyme](#)) List of name.

- [Keywords derived from champ_post_operateur_base](#)
- [Keywords derived from champ_post_statistiques_base](#)

champ_post_operateur_eqn**Synonyms:** operateur_eqn**Inherits from:** [champ_post_de_champs_post](#)

Post-process equation operators/sources

Parameters:

- **[numero_source]** (*type:* int) the source to be post-processed (its number). If you have only one source term, numero_source will correspond to 0 if you want to post-process that unique source
- **[numero_op]** (*type:* int) numero_op will be 0 (diffusive operator) or 1 (convective operator) or 2 (gradient operator) or 3 (divergence operator).
- **[numero_masse]** (*type:* int) numero_masse will be 0 for the mass equation operator in Pb_multiphase.
- **[sans_solveur_masse]** (*type:* flag) not_set

- **[compo]** (*type*: int) If you want to post-process only one component of a vector field, you can specify the number of the component after compo keyword. By default, it is set to -1 which means that all the components will be post-processed. This feature is not available in VDF discretization.
 - **[source]** (*type*: [champ_generique_base](#)) the source field.
 - **[sources]** (*type*: list of Champ_generique_base) XXX
 - **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
 - **[source_reference]** (*type*: string) not_set
 - **[sources_reference]** (*type*: list of Nom_anonyme) List of name.
-

extraction

Synonyms: champ_post_extraction

Inherits from: [champ_post_de_champs_post](#)

To create a surface field (values at the boundary) of a volume field

Parameters:

- **domaine** (*type*: string) name of the volume field
 - **nom_frontiere** (*type*: string) boundary name where the values of the volume field will be picked
 - **[methode]** (*type*: string into ['trace', 'champ_frontiere']) name of the extraction method (trace by_default or champ_frontiere)
 - **[source]** (*type*: [champ_generique_base](#)) the source field.
 - **[sources]** (*type*: list of Champ_generique_base) XXX
 - **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
 - **[source_reference]** (*type*: string) not_set
 - **[sources_reference]** (*type*: list of Nom_anonyme) List of name.
-

interpolation

Synonyms: champ_post_interpolation

Inherits from: [champ_post_de_champs_post](#)

To create a field which is an interpolation of the field given by the keyword source.

Parameters:

- **localisation** (*type*: string) type_loc indicate where is done the interpolation (elem for element or som for node).
 - **[methode]** (*type*: string) The optional keyword methode is limited to calculer_champ_post for the moment.
 - **[domaine]** (*type*: string) the domain name where the interpolation is done (by default, the calculation domain)
 - **[optimisation_sous_maillage]** (*type*: string into ['default', 'yes', 'no']) not_set
 - **[source]** (*type*: [champ_generique_base](#)) the source field.
 - **[sources]** (*type*: list of Champ_generique_base) XXX
 - **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
 - **[source_reference]** (*type*: string) not_set
 - **[sources_reference]** (*type*: list of Nom_anonyme) List of name.
-

morceau_equation**Synonyms:** champ_post_morceau_equation**Inherits from:** [champ_post_de_champs_post](#)

To calculate a field related to a piece of equation. For the moment, the field which can be calculated is the stability time step of an operator equation. The problem name and the unknown of the equation should be given by Source refChamp { Pb_Champ problem_name unknown_field_of_equation }

Parameters:

- **type** (*type*: string) can only be operateur for equation operators.
- **[numero]** (*type*: int) numero will be 0 (diffusive operator) or 1 (convective operator) or 2 (gradient operator) or 3 (divergence operator).
- **[unite]** (*type*: string) will specify the field unit
- **option** (*type*: string into ['stabilite', 'flux_bords', 'flux_surfacique_bords']) option is stability for time steps or flux_bords for boundary fluxes or flux_surfacique_bords for boundary surfacic fluxes
- **[compo]** (*type*: int) compo will specify the number component of the boundary flux (for boundary fluxes, in this case compo permits to specify the number component of the boundary flux choosen).
- **[source]** (*type*: [champ_generique_base](#)) the source field.
- **[sources]** (*type*: list of Champ_generique_base) XXX
- **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
- **[source_reference]** (*type*: string) not_set
- **[sources_reference]** (*type*: list of Nom_anonyme) List of name.

reduction_0d**Synonyms:** champ_post_reduction_0d**Inherits from:** [champ_post_de_champs_post](#)

To calculate the min, max, sum, average, weighted sum, weighted average, weighted sum by porosity, weighted average by porosity, euclidian norm, normalized euclidian norm, L1 norm, L2 norm of a field.

Parameters:

- **methode** (*type*: string into ['min', 'max', 'moyenne', 'average', 'moyenne_ponderee', 'weighted_average', 'somme', 'sum', 'somme_ponderee', 'weighted_sum', 'somme_ponderee_porosite', 'weighted_sum_porosity', 'euclidian_norm', 'normalized_euclidian_norm', 'l1_norm', 'l2_norm', 'valeur_a_gauche', 'left_value']) name of the reduction method: - min for the minimum value, - max for the maximum value, - average (or moyenne) for a mean, - weighted_average (or moyenne_ponderee) for a mean ponderated by integration volumes, e.g: cell volumes for temperature and pressure in VDF, volumes around faces for velocity and temperature in VEF, - sum (or somme) for the sum of all the values of the field, - weighted_sum (or somme_ponderee) for a weighted sum (integral), - weighted_average_porosity (or moyenne_ponderee_porosite) and weighted_sum_porosity (or somme_ponderee_porosite) for the mean and sum weighted by the volumes of the elements, only for ELEM localisation, - euclidian_norm for the euclidian norm, - normalized_euclidian_norm for the euclidian norm normalized, - L1_norm for norm L1, - L2_norm for norm L2
- **[source]** (*type*: [champ_generique_base](#)) the source field.
- **[sources]** (*type*: list of Champ_generique_base) XXX
- **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
- **[source_reference]** (*type*: string) not_set
- **[sources_reference]** (*type*: list of Nom_anonyme) List of name.

tparoi_vef**Synonyms:** champ_post_tparoi_vef**Inherits from:** [champ_post_de_champs_post](#)

This keyword is used to post process (only for VEF discretization) the temperature field with a slight difference on boundaries with Neumann condition where law of the wall is applied on the temperature field. `nom_pb` is the problem name and `field_name` is the selected field name. A keyword (`temperature_physique`) is available to post process this field without using `Definition_champs`.

Parameters:

- **[source]** (*type*: [champ_generique_base](#)) the source field.
- **[sources]** (*type*: list of [Champ_generique_base](#)) XXX
- **[nom_source]** (*type*: string) To name a source field with the `nom_source` keyword
- **[source_reference]** (*type*: string) not_set
- **[sources_reference]** (*type*: list of `Nom_anonyme`) List of name.

transformation**Synonyms:** champ_post_transformation**Inherits from:** [champ_post_de_champs_post](#)

To create a field with a transformation using source fields and `x`, `y`, `z`, `t`. If you use in your datafile source `refChamp { Pb_champ pb pression }`, the field `pression` may be used in the expression with the name `pression_natif_dom`; this latter is the same as `pression`. If you specify `nom_source` in `refChamp` bloc, you should use the alias given to pressure field. This is avail for all equations unknowns in transformation.

Parameters:

- **methode** (*type*: string into ['produit_scalaire', 'norme', 'vecteur', 'formule', 'composante']) methode 0 methode norme : will calculate the norm of a vector given by a source field methode `produit_scalaire` : will calculate the dot product of two vectors given by two sources fields methode `composante` numero integer : will create a field by extracting the integer component of a field given by a source field methode `formule` expression 1 : will create a scalar field located to elements using expressions with `x,y,z,t` parameters and field names given by a source field or several sources fields. methode `vecteur` expression `N f1(x,y,z,t) fN(x,y,z,t)` : will create a vector field located to elements by defining its `N` components with `N` expressions with `x,y,z,t` parameters and field names given by a source field or several sources fields.
- **[unite]** (*type*: string) will specify the field unit
- **[expression]** (*type*: list of str) expression 1 see methodes `formule` and `vecteur`
- **[numero]** (*type*: int) numero 1 see methode `composante`
- **[localisation]** (*type*: string) localisation 1 `type_loc` indicate where is done the interpolation (`elem` for element or `som` for node). The optional keyword `methode` is limited to `calculer_champ_post` for the moment
- **[source]** (*type*: [champ_generique_base](#)) the source field.
- **[sources]** (*type*: list of [Champ_generique_base](#)) XXX
- **[nom_source]** (*type*: string) To name a source field with the `nom_source` keyword
- **[source_reference]** (*type*: string) not_set
- **[sources_reference]** (*type*: list of `Nom_anonyme`) List of name.

1.1.1.1 Keywords derived from champ_post_operateur_base

champ_post_operateur_base

Inherits from: [champ_post_de_champs_post](#)

not_set

Parameters:

- **[source]** (*type*: [champ_generique_base](#)) the source field.
 - **[sources]** (*type*: list of Champ_generique_base) XXX
 - **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
 - **[source_reference]** (*type*: string) not_set
 - **[sources_reference]** (*type*: list of Nom_anonyme) List of name.
-

divergence

Synonyms: champ_post_operateur_divergence

Inherits from: [champ_post_operateur_base](#)

To calculate divergency of a given field.

Parameters:

- **[source]** (*type*: [champ_generique_base](#)) the source field.
 - **[sources]** (*type*: list of Champ_generique_base) XXX
 - **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
 - **[source_reference]** (*type*: string) not_set
 - **[sources_reference]** (*type*: list of Nom_anonyme) List of name.
-

gradient

Synonyms: champ_post_operateur_gradient

Inherits from: [champ_post_operateur_base](#)

To calculate gradient of a given field.

Parameters:

- **[source]** (*type*: [champ_generique_base](#)) the source field.
- **[sources]** (*type*: list of Champ_generique_base) XXX
- **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
- **[source_reference]** (*type*: string) not_set
- **[sources_reference]** (*type*: list of Nom_anonyme) List of name.

1.1.1.2 Keywords derived from champ_post_statistiques_base

champ_post_statistiques_base

Inherits from: [champ_post_de_champs_post](#)

not_set

Parameters:

- **t_deb** (*type*: float) Start of integration time
- **t_fin** (*type*: float) End of integration time
- **[source]** (*type*: [champ_generique_base](#)) the source field.

- **[sources]** (*type*: list of Champ_generique_base) XXX
 - **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
 - **[source_reference]** (*type*: string) not_set
 - **[sources_reference]** (*type*: list of Nom_anonyme) List of name.
-

correlation

Synonyms: champ_post_statistiques_correlation

Inherits from: [champ_post_statistiques_base](#)

to calculate the correlation between the two fields.

Parameters:

- **t_deb** (*type*: float) Start of integration time
 - **t_fin** (*type*: float) End of integration time
 - **[source]** (*type*: [champ_generique_base](#)) the source field.
 - **[sources]** (*type*: list of Champ_generique_base) XXX
 - **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
 - **[source_reference]** (*type*: string) not_set
 - **[sources_reference]** (*type*: list of Nom_anonyme) List of name.
-

ecart_type

Synonyms: champ_post_statistiques_ecart_type

Inherits from: [champ_post_statistiques_base](#)

to calculate the standard deviation (statistic rms) of the field nom_champ.

Parameters:

- **t_deb** (*type*: float) Start of integration time
 - **t_fin** (*type*: float) End of integration time
 - **[source]** (*type*: [champ_generique_base](#)) the source field.
 - **[sources]** (*type*: list of Champ_generique_base) XXX
 - **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
 - **[source_reference]** (*type*: string) not_set
 - **[sources_reference]** (*type*: list of Nom_anonyme) List of name.
-

moyenne

Synonyms: champ_post_statistiques_moyenne

Inherits from: [champ_post_statistiques_base](#)

to calculate the average of the field over time

Parameters:

- **[moyenne_convergee]** (*type*: [field_base](#)) This option allows to read a converged time averaged field in a .xyz file in order to calculate, when resuming the calculation, the statistics fields (rms, correlation) which depend on this average. In that case, the time averaged field is not updated during the resume of calculation. In this case, the time averaged field must be fully converged to avoid errors when calculating high order statistics.
 - **t_deb** (*type*: float) Start of integration time
-

- **t_fin** (*type*: float) End of integration time
- **[source]** (*type*: [champ_generique_base](#)) the source field.
- **[sources]** (*type*: list of Champ_generique_base) XXX
- **[nom_source]** (*type*: string) To name a source field with the nom_source keyword
- **[source_reference]** (*type*: string) not_set
- **[sources_reference]** (*type*: list of Nom_anonyme) List of name.

1.2 Keywords derived from chimie

chimie

Keyword to describe the chemical reactions

Parameters:

- **reactions** (*type*: list of Reaction) list of reactions
- **[modele_micro_melange]** (*type*: int) modele_micro_melange (0 by default)
- **[constante_modele_micro_melange]** (*type*: float) constante of modele (1 by default)
- **[espece_en_competition_micro_melange]** (*type*: string) espece in competition in reactions

1.3 Keywords derived from class_generic

class_generic

not_set

- [Keywords derived from dt_start](#)
- [Keywords derived from solveur_sys_base](#)

1.3.1 Keywords derived from dt_start

dt_start

Inherits from: [class_generic](#)

not_set

dt_calc_dt_calc

Synonyms: dt_calc

Inherits from: [dt_start](#)

The time step at first iteration is calculated in agreement with CFL condition.

dt_calc_dt_fixe

Synonyms: dt_fixe

Inherits from: [dt_start](#)

The first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).

Parameters:

- **value** (*type*: float) first time step.
-

dt_calc_dt_min

Synonyms: dt_min

Inherits from: [dt_start](#)

The first iteration is based on dt_min.

1.3.2 Keywords derived from solveur_sys_base

solveur_sys_base

Inherits from: [class_generic](#)

Basic class to solve the linear system.

- [Keywords derived from petsc](#)
- [Keywords derived from solv_gcp](#)

amg

Inherits from: [solveur_sys_base](#)

Wrapper for AMG preconditioner-based solver which switch for the best one on CPU/GPU Nvidia/GPU AMD

Parameters:

- **solveur** (*type*: string) not_set
 - **option_solveur** (*type*: [bloc_lecture](#)) not_set
-

cholesky

Inherits from: [solveur_sys_base](#)

Cholesky direct method.

Parameters:

- **[impr]** (*type*: flag) Keyword which may be used to print the resolution time.
 - **[quiet]** (*type*: flag) To disable printing of information
-

optimal

Inherits from: [solveur_sys_base](#)

Optimal is a solver which tests several solvers of the previous list to choose the fastest one for the considered linear system.

Parameters:

- **seuil** (*type*: float) Convergence threshold
 - **[impr]** (*type*: flag) To print the convergency of the fastest solver
 - **[quiet]** (*type*: flag) To disable printing of information
 - **[save_matrice | save_matrix]** (*type*: flag) To save the linear system (A, x, B) into a file
 - **[frequence_recalc]** (*type*: int) To set a time step period (by default, 100) for re-checking the fastest solver
 - **[nom_fichier_solveur]** (*type*: string) To specify the file containing the list of the tested solvers
 - **[fichier_solveur_non_recree]** (*type*: flag) To avoid the creation of the file containing the list
-

solv_gmres

Synonyms: gmres

Inherits from: [solveur_sys_base](#)

Preconditioned GMRES.

Parameters:

- **[diag]** (*type*: flag) Keyword to precondition with the diagonal
 - **[seuil]** (*type*: float) Value of the final residue. The solver ceases iterations when the Euclidean residue standard $||Ax-B||$ is less than this value. default value 1e-12.
-

- **[impr]** (*type*: flag) Keyword which is used to request display of the Euclidean residue standard each time this iterates
- **[save_matrice | save_matrix]** (*type*: int) To save the matrix in a file.
- **[quiet]** (*type*: flag) To not displaying any outputs of the solver.
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the solver.
- **[controle_residu]** (*type*: int) Keyword of Boolean type (by default 0). If set to 1, check the convergence after solve

1.3.2.1 Keywords derived from petsc

petsc

Inherits from: [solveur_sys_base](#)

Solver via Petsc API

Parameters:

- **solveur** (*type*: [solveur_petsc_deriv](#)) solver type and options

amgx

Inherits from: [petsc](#)

Solver via AmgX API

Parameters:

- **solveur** (*type*: string) not_set
- **option_solveur** (*type*: [bloc_lecture](#)) not_set

cuDSS

Inherits from: [petsc](#)

Solver via cuDSS API

Parameters:

- **solveur** (*type*: string) not_set
- **option_solveur** (*type*: [bloc_lecture](#)) not_set

petsc_gpu

Inherits from: [petsc](#)

GPU solver via Petsc API

Parameters:

- **solveur** (*type*: string) not_set
- **option_solveur** (*type*: [bloc_lecture](#)) not_set
- **[atol]** (*type*: float) Absolute threshold for convergence (same as seuil option)
- **[rtol]** (*type*: float) Relative threshold for convergence

1.3.2.2 Keywords derived from solv_gcp

solv_gcp

Synonyms: gcp

Inherits from: [solveur_sys_base](#)

Preconditioned conjugated gradient.

Parameters:

- **seuil** (*type*: float) Value of the final residue. The gradient ceases iteration when the Euclidean residue standard $||Ax-B||$ is less than this value.
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gcp.
- **[impr]** (*type*: flag) Keyword which is used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[quiet]** (*type*: flag) To not displaying any outputs of the solver.
- **[save_matrice | save_matrix]** (*type*: int) to save the matrix in a file.
- **[precond]** (*type*: [precond_base](#)) Keyword to define system preconditioning in order to accelerate resolution by the conjugated gradient. Many parallel preconditioning methods are not equivalent to their sequential counterpart, and you should therefore expect differences, especially when you select a high value of the final residue (seuil). The result depends on the number of processors and on the mesh splitting. It is sometimes useful to run the solver with no preconditioning at all. In particular: - when the solver does not converge during initial projection, - when comparing sequential and parallel computations. With no preconditioning, except in some particular cases (no open boundary), the sequential and the parallel computations should provide exactly the same results within fpu accuracy. If not, there might be a coding error or the system of equations is singular.
- **[precond_nul]** (*type*: flag) Keyword to not use a preconditioning method.
- **[precond_diagonal]** (*type*: flag) Keyword to use diagonal preconditioning.
- **[optimized]** (*type*: flag) This keyword triggers a memory and network optimized algorithms useful for strong scaling (when computing less than 100 000 elements per processor). The matrix and the vectors are duplicated, common items removed and only virtual items really used in the matrix are exchanged. Warning: this is experimental and known to fail in some VEF computations (L2 projection step will not converge). Works well in VDF.

gcp_ns

Inherits from: [solv_gcp](#)

not_set

Parameters:

- **solveur0** (*type*: [solveur_sys_base](#)) Solver type.
- **solveur1** (*type*: [solveur_sys_base](#)) Solver type.
- **seuil** (*type*: float) Value of the final residue. The gradient ceases iteration when the Euclidean residue standard $||Ax-B||$ is less than this value.
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gcp.
- **[impr]** (*type*: flag) Keyword which is used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[quiet]** (*type*: flag) To not displaying any outputs of the solver.
- **[save_matrice | save_matrix]** (*type*: int) to save the matrix in a file.

- **[precond]** (*type*: [precond_base](#)) Keyword to define system preconditioning in order to accelerate resolution by the conjugated gradient. Many parallel preconditioning methods are not equivalent to their sequential counterpart, and you should therefore expect differences, especially when you select a high value of the final residue (*seuil*). The result depends on the number of processors and on the mesh splitting. It is sometimes useful to run the solver with no preconditioning at all. In particular: - when the solver does not converge during initial projection, - when comparing sequential and parallel computations. With no preconditioning, except in some particular cases (no open boundary), the sequential and the parallel computations should provide exactly the same results within fpu accuracy. If not, there might be a coding error or the system of equations is singular.
- **[precond_nul]** (*type*: flag) Keyword to not use a preconditioning method.
- **[precond_diagonal]** (*type*: flag) Keyword to use diagonal preconditioning.
- **[optimized]** (*type*: flag) This keyword triggers a memory and network optimized algorithms useful for strong scaling (when computing less than 100 000 elements per processor). The matrix and the vectors are duplicated, common items removed and only virtual items really used in the matrix are exchanged. Warning: this is experimental and known to fail in some VEF computations (L2 projection step will not converge). Works well in VDF.

1.4 Keywords derived from comment

comment

Synonyms:

Comments in a data file.

Parameters:

- **comm** (*type*: string) Text to be commented.

1.5 Keywords derived from condlim_base

condlim_base

Basic class of boundary conditions.

- [Keywords derived from dirichlet](#)
- [Keywords derived from neumann](#)
- [Keywords derived from neumann_homogene](#)
- [Keywords derived from neumann_paro](#)
- [Keywords derived from paroi_echange_externe_impose](#)
- [Keywords derived from paroi_echange_global_impose](#)
- [Keywords derived from paroi_fixe](#)
- [Keywords derived from symetrie](#)

echange_externe_radiatif

Synonyms: paroi_echange_externe_radiatif

Inherits from: [condlim_base](#)

Combines radiative $(\sigma * \epsilon * (T^4 - T_{ext}^4))$ and convective $(h * (T - T_{ext}))$ heat transfer boundary conditions, where σ is the Stefan-Boltzmann constant, ϵ is the emi

Parameters:

- **h_imp** (*type*: string into ['h_imp', 't_ext', 'emissivite']) Heat exchange coefficient value (expressed in $W.m^{-2}.K^{-1}$).
- **himpc** (*type*: [front_field_base](#)) Boundary field type.

- **emissivite** (*type*: string into ['h_imp', 't_ext', 'emissivite']) Emissivity coefficient value.
 - **emissivitebc** (*type*: [front_field_base](#)) Boundary field type.
 - **t_ext** (*type*: string into ['h_imp', 't_ext', 'emissivite']) External temperature value (expressed in oC or K).
 - **ch** (*type*: [front_field_base](#)) Boundary field type.
 - **temp_unit** (*type*: string into ['temperature_unit']) Temperature unit
 - **temp_unit_val** (*type*: string into ['kelvin', 'celsius']) Temperature unit
-

echange_interne_global_impose

Synonyms: paroi_echange_interne_global_impose

Inherits from: [condlim_base](#)

Internal heat exchange boundary condition with global exchange coefficient.

Parameters:

- **h_imp** (*type*: string) Global exchange coefficient value. The global exchange coefficient value is expressed in W.m-2.K-1.
 - **ch** (*type*: [front_field_base](#)) Boundary field type.
-

echange_interne_global_parfait

Synonyms: paroi_echange_interne_global_parfait

Inherits from: [condlim_base](#)

Internal heat exchange boundary condition with perfect (infinite) exchange coefficient.

echange_interne_impose

Synonyms: paroi_echange_interne_impose

Inherits from: [condlim_base](#)

Internal heat exchange boundary condition with exchange coefficient.

Parameters:

- **h_imp** (*type*: string) Exchange coefficient value expressed in W.m-2.K-1.
 - **ch** (*type*: [front_field_base](#)) Boundary field type.
-

echange_interne_parfait

Synonyms: paroi_echange_interne_parfait

Inherits from: [condlim_base](#)

Internal heat exchange boundary condition with perfect (infinite) exchange coefficient.

frontiere_ouverte_fraction_massique_imposee

Inherits from: [condlim_base](#)

not_set

Parameters:

- **ch** (*type*: [front_field_base](#)) Boundary field type.
-

paroi

Inherits from: [condlim_base](#)

Impermeability condition at a wall called bord (edge) (standard flux zero). This condition must be associated with a wall type hydraulic condition.

paroi_adiabatique**Inherits from:** [condlim_base](#)

Normal zero flux condition at the wall called bord (edge).

paroi_contact**Inherits from:** [condlim_base](#)

Thermal condition between two domains. Important: the name of the boundaries in the two domains should be the same. (Warning: there is also an old limitation not yet fixed on the sequential algorithm in VDF to detect the matching faces on the two boundaries: faces should be ordered in the same way). The kind of condition depends on the discretization. In VDF, it is a heat exchange condition, and in VEF, a temperature condition.

Such a coupling requires coincident meshes for the moment. In case of non-coincident meshes, run is stopped and two external files are automatically generated in VEF (`connectivity_failed_boundary_name` and `connectivity_failed_pb_name.med`). In 2D, the keyword `Decouper_bord_coincident` associated to the `connectivity_failed_boundary_name` file allows to generate a new coincident mesh.

In 3D, for a first preliminary cut domain with HOMARD (fluid for instance), the second problem associated to `pb_name` (solide in a fluid/solid coupling problem) has to be submitted to HOMARD cutting procedure with `connectivity_failed_pb_name.med`.

Such a procedure works as while the primary refined mesh (fluid in our example) impacts the fluid/solid interface with a compact shape as described below (values 2 or 4 indicates the number of division from primary faces obtained in fluid domain at the interface after HOMARD cutting):

```
2-2-2-2-2-2
2-4-4-4-4-4-2 \; 2-2-2
2-4-4-4-4-2 \; 2-4-2
2-2-2-2-2 \; 2-2
OK
2-2 \; \; 2-2-2
2-4-2 \; 2-2
2-2 \; 2-2
NOT OK
```

Parameters:

- **autrepb** (*type*: string) Name of other problem.
- **nameb** (*type*: string) boundary name of the remote problem which should be the same than the local name

paroi_contact_fictif**Inherits from:** [condlim_base](#)

This keyword is derived from `paroi_contact` and is especially dedicated to compute coupled fluid/solid/fluid problem in case of thin material. Thanks to this option, solid is considered as a fictitious media (no mesh, no domain associated), and coupling is performed by considering instantaneous thermal equilibrium in it (for the moment).

Parameters:

- **autrepb** (*type*: string) Name of other problem.
- **nameb** (*type*: string) Name of bord.
- **conduct_fictif** (*type*: float) thermal conductivity
- **ep_fictive** (*type*: float) thickness of the fictitious media

paroi_contact_rayo**Inherits from:** [condlim_base](#)

Radiation Thermal condition between two domains. Important: the name of the boundaries in the two domains should be the same. (Warning: there is also an old limitation not yet fixed on the sequential algorithm in VDF to detect the matching faces on the two boundaries: faces should be ordered in the same way). The kind of condition depends on the discretization. In VDF, it is a heat exchange condition, and in VEF, a temperature condition.

Such a coupling requires coincident meshes for the moment. In case of non-coincident meshes, run is stopped and two external files are automatically generated in VEF (connectivity_failed_boundary_name and connectivity_failed_pb_name.med). In 2D, the keyword Decouper_bord_coincident associated to the connectivity_failed_boundary_name file allows to generate a new coincident mesh.

In 3D, for a first preliminary cut domain with HOMARD (fluid for instance), the second problem associated to pb_name (solide in a fluid/solid coupling problem) has to be submitted to HOMARD cutting procedure with connectivity_failed_pb_name.med.

Such a procedure works as while the primary refined mesh (fluid in our example) impacts the fluid/solid interface with a compact shape as described below (values 2 or 4 indicates the number of division from primary faces obtained in fluid domain at the interface after HOMARD cutting):

```
2-2-2-2-2-2
2-4-4-4-4-2 \; 2-2-2
2-4-4-4-4-2 \; 2-4-2
2-2-2-2-2 \; 2-2
OK
2-2 \; \; 2-2-2
2-4-2 \; 2-2
2-2 \; 2-2
NOT OK
```

Parameters:

- **autrepb** (*type*: string) Name of other problem.
- **nameb** (*type*: string) boundary name of the remote problem which should be the same than the local name
- **type_rayo** (*type*: string) Radiation type : transparent or semi-transparent

paroi_decalee_robin

Inherits from: [conclim_base](#)

This keyword is used to designate a Robin boundary condition ($a.u+b.du/dn=c$) associated with the Pironneau methodology for the wall laws. The value of given by the delta option is the distance between the mesh (where symmetry boundary condition is applied) and the fictious wall. This boundary condition needs the definition of the dedicated source terms (Source_Robin or Source_Robin_Scalaire) according the equations used.

Parameters:

- **delta** (*type*: float) not_set

paroi_echange_contact_correlation_vdf

Inherits from: [conclim_base](#)

Class to define a thermohydraulic 1D model which will apply to a boundary of 2D or 3D domain.

Warning : For parallel calculation, the only possible partition will be according the axis of the model with the keyword Tranche.

Parameters:

- **[dir]** (*type*: int) Direction (0 : axis X, 1 : axis Y, 2 : axis Z) of the 1D model.
- **[tinf]** (*type*: float) Inlet fluid temperature of the 1D model (oC or K).
- **[tsup]** (*type*: float) Outlet fluid temperature of the 1D model (oC or K).
- **[lambda_ | lambda]** (*type*: string) Thermal conductivity of the fluid (W.m-1.K-1).
- **[rho]** (*type*: string) Mass density of the fluid (kg.m-3) which may be a function of the temperature T.
- **[dt_impr]** (*type*: float) Printing period in name_of_data_file_time.dat files of the 1D model results.
- **[cp]** (*type*: float) Calorific capacity value at a constant pressure of the fluid (J.kg-1.K-1).
- **[mu]** (*type*: string) Dynamic viscosity of the fluid (kg.m-1.s-1) which may be a function of the temperature T.

- **[debit]** (*type*: float) Surface flow rate ($\text{kg.s}^{-1}.\text{m}^{-2}$) of the fluid into the channel.
- **[dh]** (*type*: float) Hydraulic diameter may be a function $f(x)$ with x position along the 1D axis ($x_{\text{inf}} \leq x \leq x_{\text{sup}}$)
- **[volume]** (*type*: string) Exact volume of the 1D domain (m^3) which may be a function of the hydraulic diameter (Dh) and the lateral surface (S) of the meshed boundary.
- **[nu]** (*type*: string) Nusselt number which may be a function of the Reynolds number (Re) and the Prandtl number (Pr).
- **[reprise_correlation]** (*type*: flag) Keyword in the case of a resuming calculation with this correlation.

`paroi_echange_contact_correlation_vef`

Inherits from: `condlim_base`

Class to define a thermohydraulic 1D model which will apply to a boundary of 2D or 3D domain.

Warning : For parallel calculation, the only possible partition will be according the axis of the model with the keyword `Tranche_geom`.

Parameters:

- **[dir]** (*type*: int) Direction (0 : axis X, 1 : axis Y, 2 : axis Z) of the 1D model.
- **[tinf]** (*type*: float) Inlet fluid temperature of the 1D model (oC or K).
- **[tsup]** (*type*: float) Outlet fluid temperature of the 1D model (oC or K).
- **[lambda_ | lambda]** (*type*: string) Thermal conductivity of the fluid ($\text{W.m}^{-1}.\text{K}^{-1}$).
- **[rho]** (*type*: string) Mass density of the fluid (kg.m^{-3}) which may be a function of the temperature T.
- **[dt_impr]** (*type*: float) Printing period in `name_of_data_file_time.dat` files of the 1D model results.
- **[cp]** (*type*: float) Calorific capacity value at a constant pressure of the fluid ($\text{J.kg}^{-1}.\text{K}^{-1}$).
- **[mu]** (*type*: string) Dynamic viscosity of the fluid ($\text{kg.m}^{-1}.\text{s}^{-1}$) which may be a function of the temperature T.
- **[debit]** (*type*: float) Surface flow rate ($\text{kg.s}^{-1}.\text{m}^{-2}$) of the fluid into the channel.
- **[n]** (*type*: int) Number of 1D cells of the 1D mesh.
- **[dh]** (*type*: string) Hydraulic diameter may be a function $f(x)$ with x position along the 1D axis ($x_{\text{inf}} \leq x \leq x_{\text{sup}}$)
- **[surface]** (*type*: string) Section surface of the channel which may be function $f(\text{Dh}, x)$ of the hydraulic diameter (Dh) and x position along the 1D axis ($x_{\text{inf}} \leq x \leq x_{\text{sup}}$)
- **[xinf]** (*type*: float) Position of the inlet of the 1D mesh on the axis direction.
- **[xsup]** (*type*: float) Position of the outlet of the 1D mesh on the axis direction.
- **[nu]** (*type*: string) Nusselt number which may be a function of the Reynolds number (Re) and the Prandtl number (Pr).
- **[emissivite_pour_rayonnement_entre_deux_plaques_quasi_infinies]** (*type*: float) Coefficient of emissivity for radiation between two quasi infinite plates.
- **[reprise_correlation]** (*type*: flag) Keyword in the case of a resuming calculation with this correlation.

paroi_echange_contact_vdf**Inherits from:** [condlim_base](#)

Boundary condition type to model the heat flux between two problems. Important: the name of the boundaries in the two problems should be the same.

Parameters:

- **autrepb** (*type*: string) Name of other problem.
- **nameb** (*type*: string) Name of bord.
- **temp** (*type*: string) Name of field.
- **h** (*type*: float) Value assigned to a coefficient (expressed in W.K-1m-2) that characterises the contact between the two mediums. In order to model perfect contact, h must be taken to be infinite. This value must obviously be the same in both the two problems blocks. The surface thermal flux exchanged between the two mediums is represented by : $q_i = h (T_1 - T_2)$ where $1/h = d_1/\lambda_1 + 1/val_h_contact + d_2/\lambda_2$ where d_i : distance between the node where T_i and the wall is found.

paroi_flux_impose**Inherits from:** [condlim_base](#)

Normal flux condition at the wall called bord (edge). The surface area of the flux (W.m-1 in 2D or W.m-2 in 3D) is imposed at the boundary according to the following convention: a positive flux is a flux that enters into the domain according to convention.

Parameters:

- **ch** (*type*: [front_field_base](#)) Boundary field type.

paroi_pression_imposee**Inherits from:** [condlim_base](#)

CL_Contrainte_Imposee/paroi_pression_imposee

Parameters:

- **ch** (*type*: [front_field_base](#)) Boundary field type.

periodic**Synonyms:** periodique**Inherits from:** [condlim_base](#)

1). For Navier-Stokes equations, this keyword is used to indicate that the horizontal inlet velocity values are the same as the outlet velocity values, at every moment. As regards meshing, the inlet and outlet edges bear the same name.; 2). For scalar transport equation, this keyword is used to set a periodic condition on scalar. The two edges dealing with this periodic condition bear the same name.

robin_vef**Inherits from:** [condlim_base](#)

Robin condition at the boundary (edge)

Parameters:

- **alpha** (*type*: float) Robin coefficient for the normal field
- **beta** (*type*: float) Robin coefficient for the tangent field
- **champ_front_normal_et_tangentiel_robin** (*type*: [front_field_base](#)) The boundary field

1.5.1 Keywords derived from dirichlet

dirichlet

Inherits from: [condlim_base](#)

Dirichlet condition at the boundary called bord (edge) : 1). For Navier-Stokes equations, velocity imposed at the boundary; 2). For scalar transport equation, scalar imposed at the boundary.

- [Keywords derived from frontiere_ouverte_alpha_imposee](#)
- [Keywords derived from frontiere_ouverte_temperature_imposee](#)
- [Keywords derived from frontiere_ouverte_vitesse_imposee](#)
- [Keywords derived from paroi_temperature_imposee](#)

frontiere_ouverte_concentration_imposee

Inherits from: [dirichlet](#)

Imposed concentration condition at an open boundary called bord (edge) (situation corresponding to a fluid inlet). This condition must be associated with an imposed inlet velocity condition.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.
-

paroi_defilante

Inherits from: [dirichlet](#)

Keyword to designate a condition where tangential velocity is imposed on the wall called bord (edge). If the velocity components set by the user is not tangential, projection is used.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.
-

paroi_deplacement_impose

Inherits from: [dirichlet](#)

CL_Deplacement_Impose/paroi_deplacement_impose

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.
-

paroi_knudsen_non_negligeable

Inherits from: [dirichlet](#)

Boundary condition for number of Knudsen (Kn) above 0.001 where slip-flow condition appears: the velocity near the wall depends on the shear stress : $Kn = l/L$ with l is the mean-free-path of the molecules and L a characteristic length scale.

$$U(y=0) - U_{wall} = k(dU/dY)$$

Where k is a coefficient given by several laws:

$$\text{Maxwell} : k = (2-s) \cdot l / s$$

$$\text{Bestok\&Karniadakis} : k = (2-s) / s \cdot L \cdot Kn / (1 + Kn)$$

$$\text{Xue\&Fan} : k = (2-s) / s \cdot L \cdot \tanh(Kn)$$

s is a value between 0 and 2 named accommodation coefficient. $s=1$ seems a good value.

Warning : The keyword is available for VDF calculation only for the moment.

Parameters:

- **name_champ_1** (type: string into ['vitesse_paro', 'k']) Field name.
 - **champ_1** (type: [front_field_base](#)) Boundary field type.
 - **name_champ_2** (type: string into ['vitesse_paro', 'k']) Field name.
 - **champ_2** (type: [front_field_base](#)) Boundary field type.
-

scalaire_impose_paro**Inherits from:** [dirichlet](#)

Imposed temperature condition at the wall called bord (edge).

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.

1.5.1.1 Keywords derived from frontiere_ouverte_alpha_impose**frontiere_ouverte_alpha_impose****Inherits from:** [dirichlet](#)

Imposed alpha condition at the open boundary.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.
-

entree_supersonique**Inherits from:** [frontiere_ouverte_alpha_impose](#)

Supersonic inlet boundary condition.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.

1.5.1.2 Keywords derived from frontiere_ouverte_temperature_imposee**frontiere_ouverte_temperature_imposee****Synonyms:** [frontiere_ouverte_enthalpie_imposee](#)**Inherits from:** [dirichlet](#)

Imposed temperature condition at the open boundary called bord (edge) (in the case of fluid inlet). This condition must be associated with an imposed inlet velocity condition. The imposed temperature value is expressed in oC or K.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.
-

entree_temperature_imposee_h**Inherits from:** [frontiere_ouverte_temperature_imposee](#)Particular case of class [frontiere_ouverte_temperature_imposee](#) for enthalpy equation.**Parameters:**

- **ch** (type: [front_field_base](#)) Boundary field type.
-

frontiere_ouverte_temperature_imposee_rayo_transp**Inherits from:** [frontiere_ouverte_temperature_imposee](#)

Radiation imposed temperature condition at the open boundary called bord (edge) (in the case of fluid inlet). This condition must be associated with an imposed inlet velocity condition. The imposed temperature value is expressed in C or K.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.

1.5.1.3 Keywords derived from `frontiere_ouverte_vitesse_imposee`

`frontiere_ouverte_vitesse_imposee`

Inherits from: [dirichlet](#)

Class for velocity-inlet boundary condition. The imposed velocity field at the inlet is vectorial and the imposed velocity values are expressed in m.s-1.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.

- [Keywords derived from `frontiere\_ouverte\_vitesse\_imposee\_sortie`](#)

1.5.1.3.1 Keywords derived from `frontiere_ouverte_vitesse_imposee_sortie`

`frontiere_ouverte_vitesse_imposee_sortie`

Inherits from: [frontiere_ouverte_vitesse_imposee](#)

Sub-class for velocity boundary condition. The imposed velocity field at the open boundary is vectorial and the imposed velocity values are expressed in m.s-1.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.

`frontiere_ouverte_rho_u_impose`

Inherits from: [frontiere_ouverte_vitesse_imposee_sortie](#)

This keyword is used to designate a condition of imposed mass rate at an open boundary called bord (edge). The imposed mass rate field at the inlet is vectorial and the imposed velocity values are expressed in kg.s-1. This boundary condition can be used only with the Quasi compressible model.

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.

1.5.1.4 Keywords derived from `paroi_temperature_imposee`

`paroi_temperature_imposee`

Inherits from: [dirichlet](#)

Imposed temperature condition at the wall called bord (edge).

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.

`temperature_imposee_parois`

Synonyms: `enthalpie_imposee_parois`

Inherits from: [paroi_temperature_imposee](#)

Imposed temperature condition at the wall called bord (edge).

Parameters:

- **ch** (type: [front_field_base](#)) Boundary field type.

1.5.2 Keywords derived from `neumann`

`neumann`

Inherits from: [condlim_base](#)

Neumann condition at the boundary called bord (edge) : 1). For Navier-Stokes equations, constraint imposed at the boundary; 2). For scalar transport equation, flux imposed at the boundary.

- [Keywords derived from `frontiere\_ouverte`](#)
- [Keywords derived from `frontiere\_ouverte\_gradient\_pression\_impose`](#)

frontiere_ouverte_gradient_pression_libre_vef**Inherits from:** [neumann](#)

Class for outlet boundary condition in VEF like Orlansky. There is no reference for pressure for these boundary conditions so it is better to add pressure condition (with `Frontiere_ouverte_pression_imposee`) on one or two cells (for symmetry in a channel) of the boundary where Orlansky conditions are imposed.

frontiere_ouverte_gradient_pression_libre_vefprep1b**Inherits from:** [neumann](#)

Class for outlet boundary condition in VEF P1B/P1NC like Orlansky.

frontiere_ouverte_pression_imposee**Inherits from:** [neumann](#)

Imposed pressure condition at the open boundary called bord (edge). The imposed pressure field is expressed in Pa.

Parameters:

- **ch** (*type:* [front_field_base](#)) Boundary field type.

frontiere_ouverte_pression_imposee_orlansky**Inherits from:** [neumann](#)

This boundary condition may only be used with VDF discretization. There is no reference for pressure for this boundary condition so it is better to add pressure condition (with `Frontiere_ouverte_pression_imposee`) on one or two cells (for symmetry in a channel) of the boundary where Orlansky conditions are imposed.

frontiere_ouverte_pression_moyenne_imposee**Inherits from:** [neumann](#)

Class for open boundary with pressure mean level imposed.

Parameters:

- **pext** (*type:* float) Mean pressure.

sortie_libre_temperature_imposee_h**Inherits from:** [neumann](#)

Open boundary for heat equation with enthalpy as unknown.

Parameters:

- **ch** (*type:* [front_field_base](#)) Boundary field type.

1.5.2.1 Keywords derived from frontiere_ouverte**frontiere_ouverte****Inherits from:** [neumann](#)

Boundary outlet condition on the boundary called bord (edge) (diffusion flux zero). This condition must be associated with a boundary outlet hydraulic condition.

Parameters:

- **var_name** (*type:* string into ['t_ext', 'c_ext', 'y_ext', 'k_eps_ext', 'k_omega_ext', 'fluctu_temperature_ext', 'flux_chaleur_turb_ext', 'v2_ext', 'a_ext', 'tau_ext', 'k_ext', 'omega_ext', 'h_ext', 'a_i_ext']) Field name.
- **ch** (*type:* [front_field_base](#)) Boundary field type.

frontiere_ouverte_rayo_transp**Inherits from:** [frontiere_ouverte](#)

Radiation boundary outlet condition on the boundary called bord (edge) (diffusion flux zero). This condition must be associated with a boundary outlet hydraulic condition.

Parameters:

- **var_name** (*type:* string into ['t_ext', 'c_ext', 'y_ext', 'k_eps_ext', 'k_omega_ext', 'fluctu_temperature_ext', 'flux_chaleur_turb_ext', 'v2_ext', 'a_ext', 'tau_ext', 'k_ext', 'omega_ext', 'h_ext', 'a_i_ext']) Field name.
- **ch** (*type:* [front_field_base](#)) Boundary field type.

1.5.2.2 Keywords derived from `frontiere_ouverte_gradient_pression_impose`**frontiere_ouverte_gradient_pression_impose****Inherits from:** [neumann](#)

Normal imposed pressure gradient condition on the open boundary called bord (edge). This boundary condition may be only used in VDF discretization. The imposed $\frac{\partial P}{\partial n}$ value is expressed in Pa.m-1.

Parameters:

- **ch** (*type:* [front_field_base](#)) Boundary field type.

frontiere_ouverte_gradient_pression_impose_vefprep1b**Inherits from:** [frontiere_ouverte_gradient_pression_impose](#)

Keyword for an outlet boundary condition in VEF P1B/P1NC on the gradient of the pressure.

Parameters:

- **ch** (*type:* [front_field_base](#)) Boundary field type.

1.5.3 Keywords derived from `neumann_homogene`**neumann_homogene****Inherits from:** [condlim_base](#)

Homogeneous neumann boundary condition

neumann_paro_adiabatique**Inherits from:** [neumann_homogene](#)

Adiabatic wall neumann boundary condition

1.5.4 Keywords derived from `neumann_paro`**neumann_paro****Inherits from:** [condlim_base](#)

Neumann boundary condition for mass equation (multiphase problem)

Parameters:

- **ch** (*type:* [front_field_base](#)) Boundary field type.

paroi_flux_impose_rayo_transp**Inherits from:** [neumann_paro](#)

Radiation neumann boundary condition (imposed flux)

Parameters:

- **ch** (*type:* [front_field_base](#)) Boundary field type.

1.5.5 Keywords derived from `paroi_echange_externe_impose`

`paroi_echange_externe_impose`

Inherits from: [condlim_base](#)

External type exchange condition with a heat exchange coefficient and an imposed external temperature.

Parameters:

- **h_or_t | h_imp** (*type*: string into ['h_imp', 't_ext']) Heat exchange coefficient value (expressed in W.m-2.K-1).
- **himpc** (*type*: [front_field_base](#)) Boundary field type.
- **t_or_h | text** (*type*: string into ['h_imp', 't_ext']) External temperature value (expressed in oC or K).
- **ch** (*type*: [front_field_base](#)) Boundary field type.

`paroi_echange_externe_impose_h`

Inherits from: [paroi_echange_externe_impose](#)

Particular case of class `paroi_echange_externe_impose` for enthalpy equation.

Parameters:

- **h_or_t | h_imp** (*type*: string into ['h_imp', 't_ext']) Heat exchange coefficient value (expressed in W.m-2.K-1).
- **himpc** (*type*: [front_field_base](#)) Boundary field type.
- **t_or_h | text** (*type*: string into ['h_imp', 't_ext']) External temperature value (expressed in oC or K).
- **ch** (*type*: [front_field_base](#)) Boundary field type.

`paroi_echange_externe_impose_rayo_transp`

Inherits from: [paroi_echange_externe_impose](#)

Radiation External type exchange condition with a heat exchange coefficient and an imposed external temperature.

Parameters:

- **h_or_t | h_imp** (*type*: string into ['h_imp', 't_ext']) Heat exchange coefficient value (expressed in W.m-2.K-1).
- **himpc** (*type*: [front_field_base](#)) Boundary field type.
- **t_or_h | text** (*type*: string into ['h_imp', 't_ext']) External temperature value (expressed in oC or K).
- **ch** (*type*: [front_field_base](#)) Boundary field type.

1.5.6 Keywords derived from `paroi_echange_global_impose`

`paroi_echange_global_impose`

Inherits from: [condlim_base](#)

Global type exchange condition (internal) that is to say that diffusion on the first fluid mesh is not taken into consideration.

Parameters:

- **h_imp** (*type*: string) Global exchange coefficient value. The global exchange coefficient value is expressed in W.m-2.K-1.
- **himpc** (*type*: [front_field_base](#)) Boundary field type.
- **text** (*type*: string) External temperature value. The external temperature value is expressed in oC or K.
- **ch** (*type*: [front_field_base](#)) Boundary field type.

echange_couplage_thermique**Inherits from:** [paroi_echange_global_impose](#)

Thermal coupling boundary condition

Parameters:

- **[temperature_paroil]** (*type:* [field_base](#)) Temperature
- **[flux_paroil]** (*type:* [field_base](#)) Wall heat flux

1.5.7 Keywords derived from paroi_fixe**paroi_fixe****Inherits from:** [condlim_base](#)

Keyword to designate a situation of adherence to the wall called bord (edge) (normal and tangential velocity at the edge is zero).

paroi_fixe_iso_geneppi2_sans_contribution_aux_vitesses_sommets**Inherits from:** [paroi_fixe](#)

Boundary condition to obtain iso Geneppi2, without interest

1.5.8 Keywords derived from symetrie**symetrie****Inherits from:** [condlim_base](#)

1). For Navier-Stokes equations, this keyword is used to designate a symmetry condition concerning the velocity at the boundary called bord (edge) (normal velocity at the edge equal to zero and tangential velocity gradient at the edge equal to zero); 2). For scalar transport equation, this keyword is used to set a symmetry condition on scalar on the boundary named bord (edge).

sortie_supersonique**Inherits from:** [symetrie](#)

Outlet supersonic boundary condition

1.6 Keywords derived from correlation_base**correlation_base**[not_set](#)

- [Keywords derived from flux_parietal_base](#)

1.6.1 Keywords derived from flux_parietal_base**flux_parietal_base****Inherits from:** [correlation_base](#)[not_set](#)**flux_parietal_nusselt****Synonyms:** nusselt**Inherits from:** [flux_parietal_base](#)[not_set](#)**Parameters:**

- **formula** (*type:* string) Nusselt formula as a function of Re and Pr

1.7 Keywords derived from discretisation_base

discretisation_base

Basic class for space discretization of thermohydraulic turbulent problems.

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.
-

coloc

Inherits from: [discretisation_base](#)

Co-localised cell-center discretization

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.
-

dg

Inherits from: [discretisation_base](#)

DG discretization

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.
-

ef

Inherits from: [discretisation_base](#)

Element Finite discretization.

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.
-

ef_axi

Inherits from: [discretisation_base](#)

Element Finite discretization.

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.
-

ijk

Inherits from: [discretisation_base](#)

IJK discretization.

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.
-

polymac_cdo

Synonyms: polymac

Inherits from: [discretisation_base](#)

PolyMAC_CDO discretization (PolyMAC_CDO discretization that is not compatible with pb_multi).

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.
-

polymac_hfv**Synonyms:** polymac_p0p1nc**Inherits from:** [discretisation_base](#)

PolyMAC_HFV discretization (previously PolyMAC_CDO discretization compatible with pb_multi).

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.

polymac_mpfa**Synonyms:** polymac_p0**Inherits from:** [discretisation_base](#)

PolyMAC_MPFA discretization (previously covimac discretization compatible with pb_multi).

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.

vdf**Inherits from:** [discretisation_base](#)

Finite difference volume discretization.

Parameters:

- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.

vef**Synonyms:** vefprep1b**Inherits from:** [discretisation_base](#)

Finite element volume discretization (P1NC/P1-bubble element). Since the 1.5.5 version, several new discretizations are available thanks to the optional keyword Read. By default, the VEFPreP1B keyword is equivalent to the former VEFPreP1B formulation (v1.5.4 and sooner). P0P1 (if used with the strong formulation for imposed pressure boundary) is equivalent to VEFPreP1B but the convergence is slower. VEFPreP1B dis is equivalent to VEFPreP1B dis Read dis { P0 P1 Changement_de_base_P1Bulle 1 Cl_pression_sommet_faible 0 }

Parameters:

- **[changement_de_base_p1bulle]** (*type:* int into [0, 1]) changement_de_base_p1bulle 1 This option may be used to have the P1NC/P0P1 formulation (value set to 0) or the P1NC/P1Bulle formulation (value set to 1, the default).
- **[p0]** (*type:* flag) Pressure nodes are added on element centres
- **[p1]** (*type:* flag) Pressure nodes are added on vertices
- **[pa]** (*type:* flag) Only available in 3D, pressure nodes are added on bones
- **[rt]** (*type:* flag) For P1NCP1B (in TrioCFD)
- **[modif_div_face_dirichlet]** (*type:* int into [0, 1]) This option (by default 0) is used to extend control volumes for the momentum equation.
- **[cl_pression_sommet_faible]** (*type:* int into [0, 1]) This option is used to specify a strong formulation (value set to 0, the default) or a weak formulation (value set to 1) for an imposed pressure boundary condition. The first formulation converges quicker and is stable in general cases. The second formulation should be used if there are several outlet boundaries with Neumann condition (see Ecoulement_Neumann test case for example).
- **[reorder]** (*type:* [reorder_mesh](#)) Reordering directive.

1.8 Keywords derived from `domaine_base`

`domaine_base`

base for most domains

- [Keywords derived from `domaine`](#)

`domaine_64`

Inherits from: [domaine_base](#)

Keyword to create a big (64b) domain.

`domaine_ijk`

Inherits from: [domaine_base](#)

domain for IJK simulation (used in TrioCFD)

Parameters:

- **nbelem** (*type*: list of int) Number of elements in each direction (integers, 2 or 3 values depending on dimension)
- **size_dom** (*type*: list of float) Domain size in each direction (floats, 2 or 3 values depending on dimension)
- **perio** (*type*: list of int) Is the direction periodic ? (0 or 1, 2 or 3 values depending on dimension)
- **nproc** (*type*: list of int) Number of procs in each direction (integers, 2 or 3 values depending on dimension)
- **[origin]** (*type*: list of float) Domain origin in each direction (floats, 2 or 3 values depending on dimension)
- **[ijk_splitting_ft_extension]** (*type*: int) not_set
- **[file_coords]** (*type*: [troismots](#)) not_set

1.8.1 Keywords derived from `domaine`

`domaine`

Inherits from: [domaine_base](#)

Keyword to create a domain.

`domaineaxis1d`

Inherits from: [domaine](#)

1D domain

1.9 Keywords derived from `field_base`

`field_base`

Synonyms: `champ_base`

Basic class of fields.

- [Keywords derived from `champ\_don\_base`](#)
- [Keywords derived from `champ\_fonc\_med`](#)
- [Keywords derived from `champ\_input\_base`](#)

`champ_ostwald`

Inherits from: [field_base](#)

This keyword is used to define the viscosity variation law:

$\mu(T) = K(T) \cdot (D:D/2)^{n-1/2}$

1.9.1 Keywords derived from champ_don_base

champ_don_base

Inherits from: [field_base](#)

Basic class for data fields (not calculated), p.e. physics properties.

- [Keywords derived from champ_composite](#)
- [Keywords derived from champ_fonc_tabule](#)
- [Keywords derived from champ_fonc_tabule_morceaux](#)
- [Keywords derived from champ_tabule_temps](#)
- [Keywords derived from champ_uniforme_morceaux](#)

champ_don_lu

Inherits from: [champ_don_base](#)

Field to read a data field (values located at the center of the cells) in a file.

Parameters:

- **dom** (*type*: string) Name of the domain.
 - **nb_comp** (*type*: int) Number of field components.
 - **file** (*type*: string) Name of the file. This file has the following format: nb_val_lues -> Number of values readen in th file Xi Yi Zi -> Coordinates readen in the file Ui Vi Wi -> Value of the field
-

champ_fonc_fonction_txyz_morceaux

Inherits from: [champ_don_base](#)

Field defined by analytical functions in each sub-domaine. On each zone, the value is defined as a function of x,y,z,t and of scalar value taken from a parameter field. This values is associated to the variable 'val' in the expression.

Parameters:

- **problem_name** (*type*: string) Name of the problem.
 - **inco** (*type*: string) Name of the field (for example: temperature).
 - **nb_comp** (*type*: int) Number of field components.
 - **data** (*type*: [bloc_lecture](#)) { Defaut val_def sous_domaine_1 val_1 ... sous_domaine_i val_i } By default, the value val_def is assigned to the field. It takes the sous_domaine_i identifier Sous_Domaine (sub_area) type object function, val_i. Sous_Domaine (sub_area) type objects must have been previously defined if the operator wishes to use a champ_fonc_fonction_txyz_morceaux type object.
-

champ_fonc_interp

Inherits from: [champ_don_base](#)

Field that is interpolated from a distant domain via MEDCoupling (remapper).

Parameters:

- **nom_champ** (*type*: string) Name of the field (for example: temperature).
 - **pb_loc** (*type*: string) Name of the local problem.
 - **pb_dist** (*type*: string) Name of the distant problem.
 - **[dom_loc]** (*type*: string) Name of the local domain.
 - **[dom_dist]** (*type*: string) Name of the distant domain.
 - **[default_value]** (*type*: string) Name of the distant domain.
-

- **nature** (*type*: string) Nature of the field (knowledge from MEDCoupling is required; IntensiveMaximum, IntensiveConservation, ...).
- **[use_overlapdec]** (*type*: string) Nature of the field (knowledge from MEDCoupling is required; IntensiveMaximum, IntensiveConservation, ...).

champ_fonc_reprise

Inherits from: [champ_don_base](#)

This field is used to read a data field in a save file (.xyz or .sauv) at a specified time. It is very useful, for example, to run a thermohydraulic calculation with velocity initial condition read into a save file from a previous hydraulic calculation.

Parameters:

- **[format]** (*type*: string into ['binaire', 'formatte', 'xyz', 'single_hdf', 'pdi']) Type of file (the file format). If xyz format is activated, the .xyz file from the previous calculation will be given for filename, and if formatte or binaire is choosen, the .sauv file of the previous calculation will be specified for filename. In the case of a parallel calculation, if the mesh partition does not changed between the previous calculation and the next one, the binaire format should be preferred, because is faster than the xyz format. If pdi is used, the same constraints/advantages as binaire apply, but it produces one (HDF5) file per node on the filesystem instead of having one file per processor. The single_hdf format is still supported but is obsolete, the PDI format is recommended.
- **filename** (*type*: string) Name of the save file.
- **pb_name** (*type*: string) Name of the problem.
- **champ** (*type*: string) Name of the problem unknown. It may also be the temporal average of a problem unknown (like moyenne_vitesse, moyenne_temperature,...)
- **[fonction]** (*type*: [fonction_champ_reprise](#)) Optional keyword to apply a function on the field being read in the save file (e.g. to read a temperature field in Celsius units and convert it for the calculation on Kelvin units, you will use: fonction 1 273.+val)
- **temps | time** (*type*: string) Time of the saved field in the save file or last_time. If you give the keyword last_time instead, the last time saved in the save file will be used.

champ_fonc_t

Inherits from: [champ_don_base](#)

Field that is constant in space and is a function of time.

Parameters:

- **val** (*type*: list of str) Values of field components (time dependant functions).

champ_init_canal_sinal

Inherits from: [champ_don_base](#)

For a parabolic profile on U velocity with an unpredictable disturbance on V and W and a sinusoidal disturbance on V velocity.

Parameters:

- **dim** (*type*: int) Number of field components.
 - **bloc** (*type*: [bloc_lec_champ_init_canal_sinal](#)) Parameters for the class champ_init_canal_sinal.
-

champ_morceaux**Inherits from:** [champ_don_base](#)

Field defined by subfields on a list of subdomains. Each field must be defined on the whole domain.

Parameters:

- **problem_name** (*type*: string) Name of the problem.
- **data** (*type*: [bloc_lecture](#)) { Default field_def sous_domaine_1 field_1 ... sous_domaine_i field_i } By default, the field field_def is assigned to the field. It takes the sous_domaine_i identifier Sous_Domaine (sub_area) type object field, field_i. Sous_Domaine (sub_area) type objects must have been previously defined if the operator wishes to use a champ_morceaux type object.

champ_parametrique**Inherits from:** [champ_don_base](#)

Parametric field

Parameters:

- **fichier** (*type*: string) Filename where fields are read

champ_som_lu_vdf**Inherits from:** [champ_don_base](#)

Keyword to read in a file values located at the nodes of a mesh in VDF discretization.

Parameters:

- **domain_name** (*type*: string) Name of the domain.
- **dim** (*type*: int) Value of the dimension of the field.
- **tolerance** (*type*: float) Value of the tolerance to check the coordinates of the nodes.
- **file** (*type*: string) name of the file This file has the following format: Xi Yi Zi -> Coordinates of the node Ui Vi Wi -> Value of the field on this node Xi+1 Yi+1 Zi+1 -> Next point Ui+1 Vi+1 Zi+1 -> Next value ...

champ_som_lu_vef**Inherits from:** [champ_don_base](#)

Keyword to read in a file values located at the nodes of a mesh in VEF discretization.

Parameters:

- **domain_name** (*type*: string) Name of the domain.
- **dim** (*type*: int) Value of the dimension of the field.
- **tolerance** (*type*: float) Value of the tolerance to check the coordinates of the nodes.
- **file** (*type*: string) Name of the file. This file has the following format: Xi Yi Zi -> Coordinates of the node Ui Vi Wi -> Value of the field on this node Xi+1 Yi+1 Zi+1 -> Next point Ui+1 Vi+1 Zi+1 -> Next value ...

field_func_txyz**Synonyms:** champ_fonc_txyz**Inherits from:** [champ_don_base](#)

Field defined by analytical functions. It makes it possible the definition of a field that depends on the time and the space.

Parameters:

- **dom** (*type*: string) Name of domain of calculation
- **val** (*type*: list of str) List of functions on (t,x,y,z).

field_func_xyz**Synonyms:** champ_fonc_xyz**Inherits from:** [champ_don_base](#)

Field defined by analytical functions. It makes it possible the definition of a field that depends on (x,y,z).

Parameters:

- **dom** (*type*: string) Name of domain of calculation.
 - **val** (*type*: list of str) List of functions on (x,y,z).
-

init_par_partie**Inherits from:** [champ_don_base](#)

ne marche que pour n_comp=1

Parameters:

- **n_comp** (*type*: int into [1]) not_set
 - **val1** (*type*: float) not_set
 - **val2** (*type*: float) not_set
 - **val3** (*type*: float) not_set
-

tayl_green**Inherits from:** [champ_don_base](#)

Class Tayl_green.

Parameters:

- **dim** (*type*: int) Dimension.
-

uniform_field**Synonyms:** champ_uniforme**Inherits from:** [champ_don_base](#)

Field that is constant in space and stationary.

Parameters:

- **val** (*type*: list of float) Values of field components.
-

1.9.1.1 Keywords derived from champ_composite**champ_composite****Inherits from:** [champ_don_base](#)

Composite field. Used in multiphase problems to associate data to each phase.

Parameters:

- **dim** (*type*: int) Number of field components.
 - **bloc** (*type*: [bloc_lecture](#)) Values Various pieces of the field, defined per phase. Part 1 goes to phase 1, etc...
-

champ_musig**Inherits from:** [champ_composite](#)

MUSIG field. Used in multiphase problems to associate data to each phase.

Parameters:

- **bloc** (*type*: [bloc_lecture](#)) Not set
-

1.9.1.2 Keywords derived from champ_fonc_tabule

champ_fonc_tabule**Inherits from:** [champ_don_base](#)

Field that is tabulated as a function of another field.

Parameters:

- **pb_field** (*type*: [bloc_lecture](#)) block similar to { pb1 field1 } or { pb1 field1 ... pbN fieldN }
- **dim** (*type*: int) Number of field components.
- **bloc** (*type*: [bloc_lecture](#)) Values (the table (the value of the field at any time is calculated by linear interpolation from this table) or the analytical expression (with keyword expression to use an analytical expression)).

-
- [Keywords derived from champ_fonc_fonction](#)

1.9.1.2.1 Keywords derived from champ_fonc_fonction

champ_fonc_fonction**Inherits from:** [champ_fonc_tabule](#)

Field that is a function of another field.

Parameters:

- **problem_name** (*type*: string) Name of problem.
- **inco** (*type*: string) Name of the field (for example: temperature).
- **expression** (*type*: list of str) Number of field components followed by the analytical expression for each field component.

champ_fonc_fonction_txyz**Inherits from:** [champ_fonc_fonction](#)

this refers to a field that is a function of another field and time and/or space coordinates

Parameters:

- **problem_name** (*type*: string) Name of problem.
- **inco** (*type*: string) Name of the field (for example: temperature).
- **expression** (*type*: list of str) Number of field components followed by the analytical expression for each field component.

1.9.1.3 Keywords derived from champ_fonc_tabule_morceaux

champ_fonc_tabule_morceaux**Synonyms:** champ_tabule_morceaux**Inherits from:** [champ_don_base](#)

Field defined by tabulated data in each sub-domaine. It makes possible the definition of a field which is a function of other fields.

Parameters:

- **domain_name** (*type*: string) Name of the domain.
- **nb_comp** (*type*: int) Number of field components.
- **data** (*type*: [bloc_lecture](#)) { Defaut val_def sous_domaine_1 val_1 ... sous_domaine_i val_i } By default, the value val_def is assigned to the field. It takes the sous_domaine_i identifier Sous_Domaine (sub_area) type object function, val_i. Sous_Domaine (sub_area) type objects must have been previously defined if the operator wishes to use a champ_fonc_tabule_morceaux type object.

champ_fonc_tabule_morceaux_interp

Inherits from: [champ_fonc_tabule_morceaux](#)

Field defined by tabulated data in each sub-domaine. It makes possible the definition of a field which is a function of other fields. Here we use MEDCoupling to interpolate fields between the two domains.

Parameters:

- **problem_name** (*type*: string) Name of the problem.
- **nb_comp** (*type*: int) Number of field components.
- **data** (*type*: [bloc_lecture](#)) { Defaut val_def sous_domaine_1 val_1 ... sous_domaine_i val_i } By default, the value val_def is assigned to the field. It takes the sous_domaine_i identifier Sous_Domaine (sub_area) type object function, val_i. Sous_Domaine (sub_area) type objects must have been previously defined if the operator wishes to use a champ_fonc_tabule_morceaux type object.

1.9.1.4 Keywords derived from champ_tabule_temps

champ_tabule_temps

Inherits from: [champ_don_base](#)

Field that is constant in space and tabulated as a function of time.

Parameters:

- **dim** (*type*: int) Number of field components.
- **bloc** (*type*: [bloc_lecture](#)) Values as a table. The value of the field at any time is calculated by linear interpolation from this table.

champ_tabule_lu

Inherits from: [champ_tabule_temps](#)

Uniform field, tabulated from a specified column file. Lines starting with # are ignored.

Parameters:

- **nb_comp** (*type*: int) Number of field components.
- **column_file** (*type*: string) Name of the column file.
- **dim** (*type*: int) Number of field components.

1.9.1.5 Keywords derived from champ_uniforme_morceaux

champ_uniforme_morceaux

Inherits from: [champ_don_base](#)

Field which is partly constant in space and stationary.

Parameters:

- **nom_dom** (*type*: string) Name of the domain to which the sub-areas belong.
 - **nb_comp** (*type*: int) Number of field components.
 - **data** (*type*: [bloc_lecture](#)) { Defaut val_def sous_zone_1 val_1 ... sous_zone_i val_i } By default, the value val_def is assigned to the field. It takes the sous_zone_i identifier Sous_Zone (sub_area) type object value, val_i. Sous_Zone (sub_area) type objects must have been previously defined if the operator wishes to use a Champ_Uniforme_Morceaux(partly_uniform_field) type object.
-

champ_uniforme_morceaux_tabule_temps**Inherits from:** [champ_uniforme_morceaux](#)

this type of field is constant in space on one or several sub_zones and tabulated as a function of time.

Parameters:

- **nom_dom** (*type*: string) Name of the domain to which the sub-areas belong.
- **nb_comp** (*type*: int) Number of field components.
- **data** (*type*: [bloc_lecture](#)) { Default val_def sous_zone_1 val_1 ... sous_zone_i val_i } By default, the value val_def is assigned to the field. It takes the sous_zone_i identifier Sous_Zone (sub_area) type object value, val_i. Sous_Zone (sub_area) type objects must have been previously defined if the operator wishes to use a Champ_Uniforme_Morceaux(partly_uniform_field) type object.

valeur_totale_sur_volume**Inherits from:** [champ_uniforme_morceaux](#)

Similar as Champ_Uniforme_Morceaux with the same syntax. Used for source terms when we want to specify a source term with a value given for the volume (eg: heat in Watts) and not a value per volume unit (eg: heat in Watts/m3).

Parameters:

- **nom_dom** (*type*: string) Name of the domain to which the sub-areas belong.
- **nb_comp** (*type*: int) Number of field components.
- **data** (*type*: [bloc_lecture](#)) { Default val_def sous_zone_1 val_1 ... sous_zone_i val_i } By default, the value val_def is assigned to the field. It takes the sous_zone_i identifier Sous_Zone (sub_area) type object value, val_i. Sous_Zone (sub_area) type objects must have been previously defined if the operator wishes to use a Champ_Uniforme_Morceaux(partly_uniform_field) type object.

1.9.2 Keywords derived from champ_fonc_med**champ_fonc_med****Inherits from:** [field_base](#)

Field to read a data field in a MED-format file .med at a specified time. It is very useful, for example, to resume a calculation with a new or refined geometry. The field post-processed on the new geometry at med format is used as initial condition for the resume.

Parameters:

- **[use_existing_domain]** (*type*: flag) whether to optimize the field loading by indicating that the field is supported by the same mesh that was initially loaded as the domain
- **[last_time]** (*type*: flag) to use the last time of the MED file instead of the specified time. Mutually exclusive with 'time' parameter.
- **[decoup]** (*type*: string) specify a partition file.
- **[mesh]** (*type*: string) Name of the mesh supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use_↔ existing_domain'.
- **domain** (*type*: string) Name of the domain supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use_existing_domain'.
- **file** (*type*: string) Name of the .med file.
- **field** (*type*: string) Name of field to load.
- **[loc]** (*type*: string into ['som', 'elem']) To indicate where the field is localised. Default to 'elem'.
- **[time]** (*type*: float) Timestep to load from the MED file. Mutually exclusive with 'last_time' flag.

champ_fonc_med_table_temps**Inherits from:** [champ_fonc_med](#)

Field defined as a fixed spatial shape scaled by a temporal coefficient

Parameters:

- **[table_temps]** (*type*: [bloc_lecture](#)) Table containing the temporal coefficient used to scale the field
- **[table_temps_lue]** (*type*: string) Name of the file containing the values of the temporal coefficient used to scale the field
- **[use_existing_domain]** (*type*: flag) whether to optimize the field loading by indicating that the field is supported by the same mesh that was initially loaded as the domain
- **[last_time]** (*type*: flag) to use the last time of the MED file instead of the specified time. Mutually exclusive with 'time' parameter.
- **[decoup]** (*type*: string) specify a partition file.
- **[mesh]** (*type*: string) Name of the mesh supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use_↔existing_domain'.
- **domain** (*type*: string) Name of the domain supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use_existing_domain'.
- **file** (*type*: string) Name of the .med file.
- **field** (*type*: string) Name of field to load.
- **[loc]** (*type*: string into ['som', 'elem']) To indicate where the field is localised. Default to 'elem'.
- **[time]** (*type*: float) Timestep to load from the MED file. Mutually exclusive with 'last_time' flag.

champ_fonc_med_tabule**Inherits from:** [champ_fonc_med](#)

not_set

Parameters:

- **[use_existing_domain]** (*type*: flag) whether to optimize the field loading by indicating that the field is supported by the same mesh that was initially loaded as the domain
- **[last_time]** (*type*: flag) to use the last time of the MED file instead of the specified time. Mutually exclusive with 'time' parameter.
- **[decoup]** (*type*: string) specify a partition file.
- **[mesh]** (*type*: string) Name of the mesh supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use_↔existing_domain'.
- **domain** (*type*: string) Name of the domain supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use_existing_domain'.
- **file** (*type*: string) Name of the .med file.
- **field** (*type*: string) Name of field to load.
- **[loc]** (*type*: string into ['som', 'elem']) To indicate where the field is localised. Default to 'elem'.
- **[time]** (*type*: float) Timestep to load from the MED file. Mutually exclusive with 'last_time' flag.

1.9.3 Keywords derived from champ_input_base

champ_input_base

Inherits from: [field_base](#)

not_set

Parameters:

- **nb_comp** (*type*: int) not_set
 - **nom** (*type*: string) not_set
 - **[initial_value]** (*type*: list of float) not_set
 - **probleme** (*type*: string) not_set
 - **[sous_zone]** (*type*: string) not_set
-

champ_input_p0

Inherits from: [champ_input_base](#)

not_set

Parameters:

- **nb_comp** (*type*: int) not_set
 - **nom** (*type*: string) not_set
 - **[initial_value]** (*type*: list of float) not_set
 - **probleme** (*type*: string) not_set
 - **[sous_zone]** (*type*: string) not_set
-

champ_input_p0_composite

Inherits from: [champ_input_base](#)

Field used to define a classical champ input p0 field (for ICoCo), but with a predefined field for the initial state.

Parameters:

- **[initial_field]** (*type*: [field_base](#)) The field used for initialization
- **[input_field]** (*type*: [champ_input_p0](#)) The input field for ICoCo
- **nb_comp** (*type*: int) not_set
- **nom** (*type*: string) not_set
- **[initial_value]** (*type*: list of float) not_set
- **probleme** (*type*: string) not_set
- **[sous_zone]** (*type*: string) not_set

1.10 Keywords derived from front_field_base

front_field_base

Synonyms: [champ_front_base](#)

Basic class for fields at domain boundaries.

- [Keywords derived from ch_front_input](#)
- [Keywords derived from champ_front_composite](#)
- [Keywords derived from champ_front_fonc_txyz](#)
- [Keywords derived from champ_front_tabule](#)

boundary_field_inward**Inherits from:** [front_field_base](#)

this field is used to define the normal vector field standard at the boundary in VDF or VEF discretization.

Parameters:

- **normal_value** (*type*: string) normal vector value (positive value for a vector oriented outside to inside) which can depend of the time.

champ_front_bruit**Inherits from:** [front_field_base](#)

Field which is variable in time and space in a random manner.

Parameters:

- **nb_comp** (*type*: int) Number of field components.
- **bloc** (*type*: [bloc_lecture](#)) { [N val L val] Moyenne m_1.....[m_i] Amplitude A_1.....[A_i]}: Random noise: If N and L are not defined, the ith component of the field varies randomly around an average value m_i with a maximum amplitude A_i. White noise: If N and L are defined, these two additional parameters correspond to L, the domain length and N, the number of nodes in the domain. Noise frequency will be between $2\pi/L$ and $2\pi N/(4L)$. For example, formula for velocity: $u=U_0(t)$ $v=U_1(t)U_j(t)=M_j+2A_j*\text{bruit_blanc}$ where [bruit](#)↔[_blanc](#) (white noise) is the formula given in the [mettre_a_jour](#) (update) method of the [Champ_front_bruit](#) (noise_boundary_field) (Refer to the [Champ_front_bruit.cpp](#) file).

champ_front_calc**Inherits from:** [front_field_base](#)

This keyword is used on a boundary to get a field from another boundary. The local and remote boundaries should have the same mesh. If not, the [Champ_front_recyclage](#) keyword could be used instead. It is used in the condition block at the limits of equation which itself refers to a problem called pb1. We are working under the supposition that pb1 is coupled to another problem.

Parameters:

- **problem_name** (*type*: string) Name of the other problem to which pb1 is coupled.
- **bord** (*type*: string) Name of the side which is the boundary between the 2 domains in the domain object description associated with the problem_name object.
- **field_name** (*type*: string) Name of the field containing the value that the user wishes to use at the boundary. The field_name object must be recognized by the problem_name object.

champ_front_debit**Inherits from:** [front_field_base](#)

This field is used to define a flow rate field instead of a velocity field for a Dirichlet boundary condition on Navier-↔Stokes equations.

Parameters:

- **ch** (*type*: [front_field_base](#)) uniform field in space to define the flow rate. It could be, for example, [champ](#)↔[front_uniforme](#), [ch_front_input_uniform](#) or [champ_front_fonc_txyz](#) that depends only on time.

champ_front_debit_massique**Inherits from:** [front_field_base](#)

This field is used to define a flow rate field using the density

Parameters:

- **ch** (*type*: [front_field_base](#)) uniform field in space to define the flow rate. It could be, for example, [champ](#)↔[front_uniforme](#), [ch_front_input_uniform](#) or [champ_front_fonc_txyz](#) that depends only on time.

champ_front_debit_qc_vdf**Inherits from:** [front_field_base](#)

This keyword is used to define a flow rate field for quasi-compressible fluids in VDF discretization. The flow rate is kept constant during a transient.

Parameters:

- **dimension** | **dim** (*type*: int) Problem dimension
 - **liste** (*type*: [bloc_lecture](#)) List of the mass flow rate values [kg/s/m2] with the following syntaxe: { val1 ... valdim }
 - **[moyen]** (*type*: string) Option to use rho mean value
 - **pb_name** (*type*: string) Problem name
-

champ_front_debit_qc_vdf_fonc_t**Inherits from:** [front_field_base](#)

This keyword is used to define a flow rate field for quasi-compressible fluids in VDF discretization. The flow rate could be constant or time-dependent.

Parameters:

- **dimension** | **dim** (*type*: int) Problem dimension
 - **liste** (*type*: [bloc_lecture](#)) List of the mass flow rate values [kg/s/m2] with the following syntaxe: { val1 ... valdim } where val1 ... valdim are constant or function of time.
 - **[moyen]** (*type*: string) Option to use rho mean value
 - **pb_name** (*type*: string) Problem name
-

champ_front_fonc_pois_ipsn**Inherits from:** [front_field_base](#)

Boundary field champ_front_fonc_pois_ipsn.

Parameters:

- **r_tube** (*type*: float) not_set
 - **umoy** (*type*: list of float) not_set
 - **r_loc** (*type*: list of float) not_set
-

champ_front_fonc_pois_tube**Inherits from:** [front_field_base](#)

Boundary field champ_front_fonc_pois_tube.

Parameters:

- **r_tube** (*type*: float) not_set
 - **umoy** (*type*: list of float) not_set
 - **r_loc** (*type*: list of float) not_set
 - **r_loc_mult** (*type*: list of int) not_set
-

champ_front_fonc_t**Inherits from:** [front_field_base](#)

Boundary field that depends only on time.

Parameters:

- **val** (*type*: list of str) Values of field components (mathematical expressions).
-

champ_front_fonc_xyz**Inherits from:** [front_field_base](#)

Boundary field which is not constant in space.

Parameters:

- **val** (*type*: list of str) Values of field components (mathematical expressions).
-

champ_front_fonction**Inherits from:** [front_field_base](#)

boundary field that is function of another field

Parameters:

- **dim** (*type*: int) Number of field components.
 - **inco** (*type*: string) Name of the field (for example: temperature).
 - **expression** (*type*: string) keyword to use a analytical expression like 10.*EXP(-0.1*val) where val be the keyword for the field.
-

champ_front_lu**Inherits from:** [front_field_base](#)boundary field which is given from data issued from a read file. The format of this file has to be the same that the one generated by `Ecrire_fichier_xyz_valeur`

Example for K and epsilon quantities to be defined for inlet condition in a boundary named 'entree':

entree frontiere_ouverte_K_Eps_impose Champ_Front_lu dom 2pb_K_EPS_PERIO_1006.306198.dat

Parameters:

- **domaine** | **domain** (*type*: string) Name of domain
 - **dim** (*type*: int) number of components
 - **file** (*type*: string) path for the read file
-

champ_front_med**Inherits from:** [front_field_base](#)Field allowing the loading of a boundary condition from a MED file using `Champ_fonc_med`**Parameters:**

- **champ_fonc_med** (*type*: [field_base](#)) a `champ_fonc_med` loading the values of the unknown on a domain boundary
-

champ_front_normal_vef**Inherits from:** [front_field_base](#)

Field to define the normal vector field standard at the boundary in VEF discretization.

Parameters:

- **mot** (*type*: string into ['valeur_normale']) Name of vector field.
 - **vit_tan** (*type*: float) normal vector value (positive value for a vector oriented outside to inside).
-

champ_front_parametrique**Inherits from:** [front_field_base](#)

Parametric boundary field

Parameters:

- **fichier** (*type*: string) Filename where boundary fields are read

champ_front_pression_from_u**Inherits from:** [front_field_base](#)

this field is used to define a pressure field depending of a velocity field.

Parameters:

- **expression** (*type*: string) value depending of a velocity (like $2 * u_{moy}^2$).

champ_front_recyclage**Inherits from:** [front_field_base](#)

This keyword is used on a boundary to get a field from another boundary.

It is to use, in a general way, on a boundary of a local_pb problem, a field calculated from a linear combination of an imposed field $g(x,y,z,t)$ with an instantaneous $f(x,y,z,t)$ and a spatial mean field $\langle f \rangle(t)$ or a temporal mean field $\langle f \rangle(x,y,z)$ extracted from a plane of a problem named pb (pb may be local_pb itself):

For each component i, the field F applied on the boundary will be:

$$F_i(x,y,z,t) = \alpha_i * g_i(x,y,z,t) + \chi_i * [f_i(x,y,z,t) - \beta_i * \langle f_i \rangle]$$

Parameters:

- **pb_champ_evaluateur** (*type*: [pb_champ_evaluateur](#)) not_set
- **[distance_plan]** (*type*: list of float) Vector which gives the distance between the boundary and the plane from where the field F will be extracted. By default, the vector is zero, that should imply the two domains have coincident boundaries.
- **[ampli_moyenne_imposee]** (*type*: list of float) 2|3 $\alpha(0)$ $\alpha(1)$ [$\alpha(2)$]: α_i coefficients (by default =1)
- **[ampli_moyenne_recyclee]** (*type*: list of float) 2|3 $\beta(0)$ $\beta(1)$ [$\beta(2)$]: β_i coefficients (by default =1)
- **[ampli_fluctuation]** (*type*: list of float) 2|3 $\gamma(0)$ $\gamma(1)$ [$\gamma(2)$]: γ_i coefficients (by default =1)
- **[direction_anisotrope]** (*type*: int into [1, 2, 3]) If an integer is given for direction (X:1, Y:2, Z:3, by default, direction is negative), the imposed field g will be 0 for the 2 other directions.
- **[moyenne_imposee]** (*type*: [moyenne_imposee_deriv](#)) Value of the imposed g field.
- **[moyenne_recyclee]** (*type*: string) Method used to perform a spatial or a temporal averaging of f field to specify $\langle f \rangle$. $\langle f \rangle$ can be the surface mean of f on the plane (surface option, see below) or it can be read from several files (for example generated by the `chmoy_faceperio` option of the `Traitement_particulier` keyword to obtain a temporal mean field). The option `methode_recyc` can be: `surfacique`, Surface mean for $\langle f \rangle$ from f values on the plane ; Or one of the following `methode_moy` options applied to read a temporal mean field $\langle f \rangle(x,y,z)$: `interpolation`, `connexion_approchee` or `connexion_exacte`
- **[fichier]** (*type*: string) not_set

champ_front_tangentiel_vef**Inherits from:** [front_field_base](#)

Field to define the tangential velocity vector field standard at the boundary in VEF discretization.

Parameters:

- **mot** (*type*: string into ['vitesse_tangentielle']) Name of vector field.
- **vit_tan** (*type*: float) Vector field standard [m/s].

champ_front_uniforme**Inherits from:** [front_field_base](#)

Boundary field which is constant in space and stationary.

Parameters:

- **val** (*type*: list of float) Values of field components.

champ_front_xyz_debit**Inherits from:** [front_field_base](#)

This field is used to define a flow rate field with a velocity profil which will be normalized to match the flow rate chosen.

Parameters:

- **[velocity_profil]** (*type*: [front_field_base](#)) velocity_profil 0 velocity field to define the profil of velocity.
- **flow_rate** (*type*: [front_field_base](#)) flow_rate 1 uniform field in space to define the flow rate. It could be, for example, champ_front_uniforme, ch_front_input_uniform or champ_front_fonc_t

1.10.1 Keywords derived from ch_front_input**ch_front_input****Inherits from:** [front_field_base](#)

not_set

Parameters:

- **nb_comp** (*type*: int) not_set
- **nom** (*type*: string) not_set
- **[initial_value]** (*type*: list of float) not_set
- **probleme** (*type*: string) not_set
- **[sous_zone]** (*type*: string) not_set

ch_front_input_uniforme**Inherits from:** [ch_front_input](#)

for coupling, you can use ch_front_input_uniforme which is a champ_front_uniforme, which use an external value. It must be used with Problem.setInputField.

Parameters:

- **nb_comp** (*type*: int) not_set
- **nom** (*type*: string) not_set
- **[initial_value]** (*type*: list of float) not_set
- **probleme** (*type*: string) not_set
- **[sous_zone]** (*type*: string) not_set

1.10.2 Keywords derived from champ_front_composite**champ_front_composite****Inherits from:** [front_field_base](#)

Composite front field. Used in multiphase problems to associate data to each phase.

Parameters:

- **dim** (*type*: int) Number of field components.
- **bloc** (*type*: [bloc_lecture](#)) Values Various pieces of the field, defined per phase. Part 1 goes to phase 1, etc...

champ_front_musig**Inherits from:** [champ_front_composite](#)

MUSIG front field. Used in multiphase problems to associate data to each phase.

Parameters:

- **bloc** (*type*: [bloc_lecture](#)) Not set

1.10.3 Keywords derived from champ_front_fonc_txyz**champ_front_fonc_txyz****Inherits from:** [front_field_base](#)

Boundary field which is not constant in space and in time.

Parameters:

- **val** (*type*: list of str) Values of field components (mathematical expressions).

champ_front_xyz_tabule**Inherits from:** [champ_front_fonc_txyz](#)

Space dependent field on the boundary, tabulated as a function of time.

Parameters:

- **val** (*type*: list of str) Values of field components (mathematical expressions).
- **bloc** (*type*: [bloc_lecture](#)) {nt1 t2 t3tn u1 [v1 w1 ...] u2 [v2 w2 ...] u3 [v3 w3 ...] ... un [vn wn ...] } Values are entered into a table based on n couples (ti, ui) if nb_comp value is 1. The value of a field at a given time is calculated by linear interpolation from this table.

1.10.4 Keywords derived from champ_front_tabule**champ_front_tabule****Inherits from:** [front_field_base](#)

Constant field on the boundary, tabulated as a function of time.

Parameters:

- **nb_comp** (*type*: int) Number of field components.
- **bloc** (*type*: [bloc_lecture](#)) {nt1 t2 t3tn u1 [v1 w1 ...] u2 [v2 w2 ...] u3 [v3 w3 ...] ... un [vn wn ...] } Values are entered into a table based on n couples (ti, ui) if nb_comp value is 1. The value of a field at a given time is calculated by linear interpolation from this table.

champ_front_tabule_lu**Inherits from:** [champ_front_tabule](#)

Constant field on the boundary, tabulated from a specified column file. Lines starting with # are ignored.

Parameters:

- **nb_comp** (*type*: int) Number of field components.
- **column_file** (*type*: string) Name of the column file.

1.11 Keywords derived from interface_base**interface_base**

Basic class for a liquid-gas interface (used in pb_multiphase)

Parameters:

- **[tension_superficielle | surface_tension]** (*type*: float) surface tension
- [Keywords derived from saturation_base](#)

interface_baer_nunziato

Inherits from: [interface_base](#)

Interface Baer Nunziato class

Parameters:

- **velocity_from_phase** | **phase_vitesse** (*type*: string) Name of phase used to take the velocity at the interface. The pressure is automatically taken from the other phase
- **[tension_superficielle** | **surface_tension]** (*type*: float) surface tension

interface_sigma_constant

Inherits from: [interface_base](#)

Liquid-gas interface with a constant surface tension sigma

Parameters:

- **[tension_superficielle** | **surface_tension]** (*type*: float) surface tension

1.11.1 Keywords derived from saturation_base

saturation_base

Inherits from: [interface_base](#)

fluid-gas interface with phase change (used in pb_multiphase)

Parameters:

- **[p_ref]** (*type*: float) not_set
- **[t_ref]** (*type*: float) not_set
- **[tension_superficielle** | **surface_tension]** (*type*: float) surface tension

saturation_constant

Inherits from: [saturation_base](#)

Class for saturation constant

Parameters:

- **[p_sat]** (*type*: float) Define the saturation pressure value (this is a required parameter)
- **[t_sat]** (*type*: float) Define the saturation temperature value (this is a required parameter)
- **[lvap]** (*type*: float) Latent heat of vaporization
- **[hlsat]** (*type*: float) Liquid saturation enthalpy
- **[hvsat]** (*type*: float) Vapor saturation enthalpy
- **[p_ref]** (*type*: float) not_set
- **[t_ref]** (*type*: float) not_set
- **[tension_superficielle** | **surface_tension]** (*type*: float) surface tension

saturation_sodium

Inherits from: [saturation_base](#)

Class for saturation sodium

Parameters:

- **[p_ref]** (*type*: float) Use to fix the pressure value in the closure law. If not specified, the value of the pressure unknown will be used
- **[t_ref]** (*type*: float) Use to fix the temperature value in the closure law. If not specified, the value of the temperature unknown will be used
- **[tension_superficielle** | **surface_tension]** (*type*: float) surface tension

1.12 Keywords derived from interpolation_ibm_base

interpolation_ibm_base

Base class for all the interpolation methods available in the Immersed Boundary Method (IBM).

Parameters:

- **[impr]** (*type*: flag) To print IBM-related data
 - **[nb_histo_boxes_impr]** (*type*: int) number of histogram boxes for printed data
 - **[est_dirichlet]** (*type*: [field_base](#)) Node field of booleans indicating whether the node belong to an element where the interface is
 - **[get_solid_points_from_prepro]** (*type*: flag) get IBM solid points from prepro.
 - **[get_is_dirichlet_from_prepro]** (*type*: flag) get IBM is_dirichlet from prepro.
 - **[get_corresp_elems_from_prepro]** (*type*: flag) get IBM corresp_elems from prepro.
-
- [Keywords derived from interpolation_ibm_elem_fluid](#)
 - [Keywords derived from interpolation_ibm_mean_gradient](#)

interpolation_ibm_aucune

Synonyms: ibm_aucune

Inherits from: [interpolation_ibm_base](#)

Immersed Boundary Method (IBM): no interpolation.

Parameters:

- **[impr]** (*type*: flag) To print IBM-related data
- **[nb_histo_boxes_impr]** (*type*: int) number of histogram boxes for printed data
- **[est_dirichlet]** (*type*: [field_base](#)) Node field of booleans indicating whether the node belong to an element where the interface is
- **[get_solid_points_from_prepro]** (*type*: flag) get IBM solid points from prepro.
- **[get_is_dirichlet_from_prepro]** (*type*: flag) get IBM is_dirichlet from prepro.
- **[get_corresp_elems_from_prepro]** (*type*: flag) get IBM corresp_elems from prepro.

1.12.1 Keywords derived from interpolation_ibm_elem_fluid

interpolation_ibm_elem_fluid

Synonyms: ibm_element_fluide, interpolation_ibm_element_fluide

Inherits from: [interpolation_ibm_base](#)

Immersed Boundary Method (IBM): fluid element interpolation.

Parameters:

- **[points_fluides]** (*type*: [field_base](#)) Node field giving the projection of the point below (points_solides) falling into the pure cell fluid
- **[points_solides]** (*type*: [field_base](#)) Node field giving the projection of the node on the immersed boundary
- **[elements_fluides]** (*type*: [field_base](#)) Node field giving the number of the element (cell) containing the pure fluid point
- **[correspondance_elements]** (*type*: [field_base](#)) Cell field giving the SALOME cell number
- **[get_fluid_points_from_prepro]** (*type*: flag) get IBM fluid points from prepro.
- **[get_fluid_elems_from_prepro]** (*type*: flag) get IBM fluid elems from prepro.

- **[impr]** (*type*: flag) To print IBM-related data
- **[nb_histo_boxes_impr]** (*type*: int) number of histogram boxes for printed data
- **[est_dirichlet]** (*type*: [field_base](#)) Node field of booleans indicating whether the node belong to an element where the interface is
- **[get_solid_points_from_prepro]** (*type*: flag) get IBM solid points from prepro.
- **[get_is_dirichlet_from_prepro]** (*type*: flag) get IBM is_dirichlet from prepro.
- **[get_corresp_elems_from_prepro]** (*type*: flag) get IBM corresp_elems from prepro.

interpolation_ibm_hybride

Synonyms: ibm_hybride

Inherits from: [interpolation_ibm_elem_fluid](#)

Immersed Boundary Method (IBM): hybrid (fluid/mean gradient) interpolation.

Parameters:

- **elements_solides** (*type*: [field_base](#)) Node field giving the element number containing the solid point
- **[points_fluides]** (*type*: [field_base](#)) Node field giving the projection of the point below (points_solides) falling into the pure cell fluid
- **[points_solides]** (*type*: [field_base](#)) Node field giving the projection of the node on the immersed boundary
- **[elements_fluides]** (*type*: [field_base](#)) Node field giving the number of the element (cell) containing the pure fluid point
- **[correspondance_elements]** (*type*: [field_base](#)) Cell field giving the SALOME cell number
- **[get_fluid_points_from_prepro]** (*type*: flag) get IBM fluid points from prepro.
- **[get_fluid_elems_from_prepro]** (*type*: flag) get IBM fluid elems from prepro.
- **[impr]** (*type*: flag) To print IBM-related data
- **[nb_histo_boxes_impr]** (*type*: int) number of histogram boxes for printed data
- **[est_dirichlet]** (*type*: [field_base](#)) Node field of booleans indicating whether the node belong to an element where the interface is
- **[get_solid_points_from_prepro]** (*type*: flag) get IBM solid points from prepro.
- **[get_is_dirichlet_from_prepro]** (*type*: flag) get IBM is_dirichlet from prepro.
- **[get_corresp_elems_from_prepro]** (*type*: flag) get IBM corresp_elems from prepro.

interpolation_ibm_power_law_tbl

Synonyms: ibm_power_law_tbl

Inherits from: [interpolation_ibm_elem_fluid](#)

Immersed Boundary Method (IBM): power law interpolation.

Parameters:

- **[formulation_wjsp]** (*type*: int) Choix formulation WJSP ou non
- **[formulation_linear_pwl]** (*type*: int) Choix formulation lineaire ou non
- **[points_fluides]** (*type*: [field_base](#)) Node field giving the projection of the point below (points_solides) falling into the pure cell fluid
- **[points_solides]** (*type*: [field_base](#)) Node field giving the projection of the node on the immersed boundary

- **[elements_fluides]** (*type: field_base*) Node field giving the number of the element (cell) containing the pure fluid point
- **[correspondance_elements]** (*type: field_base*) Cell field giving the SALOME cell number
- **[get_fluid_points_from_prepro]** (*type: flag*) get IBM fluid points from prepro.
- **[get_fluid_elems_from_prepro]** (*type: flag*) get IBM fluid elems from prepro.
- **[impr]** (*type: flag*) To print IBM-related data
- **[nb_histo_boxes_impr]** (*type: int*) number of histogram boxes for printed data
- **[est_dirichlet]** (*type: field_base*) Node field of booleans indicating whether the node belong to an element where the interface is
- **[get_solid_points_from_prepro]** (*type: flag*) get IBM solid points from prepro.
- **[get_is_dirichlet_from_prepro]** (*type: flag*) get IBM is_dirichlet from prepro.
- **[get_corresp_elems_from_prepro]** (*type: flag*) get IBM corresp_elems from prepro.

interpolation_ibm_thermal_wall_law

Synonyms: ibm_thermal_wall_law

Inherits from: [interpolation_ibm_elem_fluid](#)

Immersed Boundary Method (IBM): Interpolation thermal wall law.

Parameters:

- **[formulation_tplus]** (*type: int*) Choix formulation calcul T+ (Kader si rien)
- **boundary_type** (*type: int*) Choix type de cond limite
- **t_inlet** (*type: float*) Demande la température d'écoulement d'entrée moyenne
- **prandlt_mol** (*type: float*) Demande le Prandlt du fluide
- **[points_fluides]** (*type: field_base*) Node field giving the projection of the point below (points_solides) falling into the pure cell fluid
- **[points_solides]** (*type: field_base*) Node field giving the projection of the node on the immersed boundary
- **[elements_fluides]** (*type: field_base*) Node field giving the number of the element (cell) containing the pure fluid point
- **[correspondance_elements]** (*type: field_base*) Cell field giving the SALOME cell number
- **[get_fluid_points_from_prepro]** (*type: flag*) get IBM fluid points from prepro.
- **[get_fluid_elems_from_prepro]** (*type: flag*) get IBM fluid elems from prepro.
- **[impr]** (*type: flag*) To print IBM-related data
- **[nb_histo_boxes_impr]** (*type: int*) number of histogram boxes for printed data
- **[est_dirichlet]** (*type: field_base*) Node field of booleans indicating whether the node belong to an element where the interface is
- **[get_solid_points_from_prepro]** (*type: flag*) get IBM solid points from prepro.
- **[get_is_dirichlet_from_prepro]** (*type: flag*) get IBM is_dirichlet from prepro.
- **[get_corresp_elems_from_prepro]** (*type: flag*) get IBM corresp_elems from prepro.

1.12.2 Keywords derived from interpolation_ibm_mean_gradient

interpolation_ibm_mean_gradient

Synonyms: ibm_gradient_moyen, interpolation_ibm_gradient_moyen

Inherits from: [interpolation_ibm_base](#)

Immersed Boundary Method (IBM): mean gradient interpolation.

Parameters:

- **[points_solides]** (*type: field_base*) Node field giving the projection of the node on the immersed boundary
- **[est_dirichlet]** (*type: field_base*) Node field of booleans indicating whether the node belong to an element where the interface is
- **[correspondance_elements]** (*type: field_base*) Cell field giving the SALOME cell number
- **[elements_solides]** (*type: field_base*) Node field giving the element number containing the solid point
- **[get_solid_points_from_prepro]** (*type: flag*) get IBM solid points from prepro.
- **[get_solid_elems_from_prepro]** (*type: flag*) get IBM solid elems from prepro.
- **[get_is_dirichlet_from_prepro]** (*type: flag*) get IBM is_dirichlet from prepro.
- **[get_corresp_elems_from_prepro]** (*type: flag*) get IBM corresp_elems from prepro.
- **[impr]** (*type: flag*) To print IBM-related data
- **[nb_histo_boxes_impr]** (*type: int*) number of histogram boxes for printed data

interpolation_ibm_power_law_tbl_u_star

Synonyms: ibm_power_law_tbl_u_star

Inherits from: [interpolation_ibm_mean_gradient](#)

Immersed Boundary Method (IBM): law u star.

Parameters:

- **points_solides** (*type: field_base*) Node field giving the projection of the node on the immersed boundary
- **est_dirichlet** (*type: field_base*) Node field of booleans indicating whether the node belong to an element where the interface is
- **correspondance_elements** (*type: field_base*) Cell field giving the SALOME cell number
- **elements_solides** (*type: field_base*) Node field giving the element number containing the solid point
- **[get_solid_points_from_prepro]** (*type: flag*) get IBM solid points from prepro.
- **[get_solid_elems_from_prepro]** (*type: flag*) get IBM solid elems from prepro.
- **[get_is_dirichlet_from_prepro]** (*type: flag*) get IBM is_dirichlet from prepro.
- **[get_corresp_elems_from_prepro]** (*type: flag*) get IBM corresp_elems from prepro.
- **[impr]** (*type: flag*) To print IBM-related data
- **[nb_histo_boxes_impr]** (*type: int*) number of histogram boxes for printed data

1.13 Keywords derived from interprete

interprete

Basic class for interpreting a data file. Interpreters allow some operations to be carried out on objects.

- [Keywords derived from analyse_angle](#)
- [Keywords derived from extract_2d_from_3d](#)
- [Keywords derived from extruder](#)
- [Keywords derived from imprimer_flux](#)
- [Keywords derived from interprete_geometrique_base](#)
- [Keywords derived from mailer](#)
- [Keywords derived from raffiner_isotrope](#)
- [Keywords derived from read_file](#)
- [Keywords derived from read_med](#)
- [Keywords derived from regroupebord](#)
- [Keywords derived from scatter](#)
- [Keywords derived from tetraedriser](#)
- [Keywords derived from transformer](#)
- [Keywords derived from triangulate](#)
- [Keywords derived from write_file](#)

associate

Synonyms: associer

Inherits from: [interprete](#)

This interpreter allows one object to be associated with another. The order of the two objects in this instruction is not important. The object objet_2 is associated to objet_1 if this makes sense; if not either objet_1 is associated to objet_2 or the program exits with error because it cannot execute the Associate (Associer) instruction. For example, to calculate water flow in a pipe, a Pb_Hydraulique type object needs to be defined. But also a Domaine type object to represent the pipe, a time discretization object (Scheme_euler_explicit for ex.). These two objects must then be associated with the problem.

Parameters:

- **objet_1** (*type*: string) Objet_1
 - **objet_2** (*type*: string) Objet_2
-

axi

Inherits from: [interprete](#)

This keyword allows a 3D calculation to be executed using cylindrical coordinates (R,\$\vartheta\$,Z). If this instruction is not included, calculations are carried out using Cartesian coordinates.

bidim_axi

Inherits from: [interprete](#)

Keyword allowing a 2D calculation to be executed using axisymmetric coordinates (R, Z). If this instruction is not included, calculations are carried out using Cartesian coordinates.

calculer_moments

Inherits from: [interprete](#)

Calculates and prints the torque (moment of force) exerted by the fluid on each boundary in output files (.out) of the domain nom_dom.

Parameters:

- **nom_dom** (*type*: string) Name of domain.
 - **mot** (*type*: [lecture_bloc_moment_base](#)) Keyword.
-

cgns_to_lata

Synonyms: cgns_2_lata

Inherits from: [interprete](#)

To convert results file written with CGNS format to a single LATA file.

Parameters:

- **file_cgns** (*type*: string) CGNS file to convert to the new format.
 - **file_lata** (*type*: string) Name of the single LATA file.
-

criteres_convergence

Inherits from: [interprete](#)

convergence criteria

Parameters:

- **aco** (*type*: string into ['{'] Opening curly bracket.
 - **[inco]** (*type*: string) Unknown (i.e: alpha, temperature, velocity and pressure)
 - **[val]** (*type*: float) Convergence threshold
 - **acof** (*type*: string into ['}'] Closing curly bracket.
-

debog

Inherits from: [interprete](#)

Class to debug some differences between two TRUST versions on a same data file.

If you want to compare the results of the same code in sequential and parallel calculation, first run (mode=0) in sequential mode (the files fichier1 and fichier2 will be written first) then the second run in parallel calculation (mode=1).

During the first run (mode=0), it prints into the file DEBOG, values at different points of the code thanks to the C++ instruction call. see for example in Kernel/Framework/Resoudre.cpp file the instruction: Debug::verifier(msg,value); Where msg is a string and value may be a double, an integer or an array.

During the second run (mode=1), it prints into a file Err_Debog.dbg the same messages than in the DEBOG file and checks if the differences between results from both codes are less than a given value (error). If not, it prints Ok else show the differences and the lines where it occurred.

Parameters:

- **pb** (*type*: string) Name of the problem to debug.
 - **fichier1** | **file1** (*type*: string) Name of the file where domain will be written in sequential calculation.
 - **fichier2** | **file2** (*type*: string) Name of the file where faces will be written in sequential calculation.
 - **seuil** (*type*: float) Minimal value (by default 1.e-20) for the differences between the two codes.
 - **mode** (*type*: int) By default -1 (nothing is written in the different files), you will set 0 for the sequential run, and 1 for the parallel run.
-

declarer_bord_perio**Synonyms:** `corriger_frontiere_periodique`**Inherits from:** `interprete`

The `Declarer_bord_perio` keyword is mandatory to first define the periodic boundaries, to reorder the faces and eventually fix unaligned nodes of these boundaries. Faces on one side of the periodic domain are put first, then the faces on the opposite side, in the same order. It must be run in sequential before mesh splitting.

Parameters:

- **domaine** (*type*: string) Name of domain.
- **bord** (*type*: string) the name of the boundary (which must contain two opposite sides of the domain)
- **[direction]** (*type*: list of float) defines the periodicity direction vector (a vector that points from one node on one side to the opposite node on the other side). This vector must be given if the automatic algorithm fails, that is: - when the node coordinates are not perfectly periodic - when the periodic direction is not aligned with the normal vector of the boundary faces
- **[fichier_post]** (*type*: string) .
- **[declare_only]** (*type*: flag) flag if added will only add the periodic boundary without calling `corriger_frontiere_periodique`

decoupebord**Synonyms:** `decoupebord_pour_rayonnement`**Inherits from:** `interprete`

To subdivide the external boundary of a domain into several parts (may be useful for better accuracy when using radiation model in transparent medium). To specify the boundaries of the `fine_domain_name` domain to be splitted. These boundaries will be cut according the coarse mesh defined by either the keyword `domaine_grossier` (each boundary face of the coarse mesh `coarse_domain_name` will be used to group boundary faces of the fine mesh to define a new boundary), either by the keyword `nb_parts_naif` (each boundary of the fine mesh is splitted into a partition with `nx*ny*nz` elements), either by a geometric condition given by a formulae with the keyword `condition_geometrique`. If used, the `coarse_domain_name` domain should have the same boundaries name of the `fine_domain_name` domain.

A mesh file (ASCII format, except if `binaire` option is specified) named by default `newgeom` (or specified by the `nom_fichier_sortie` keyword) will be created and will contain the `fine_domain_name` domain with the splitted boundaries named `boundary_name1` (where 1 is between from 0 and `n-1`). Furthermore, several files named `boundary_name1` and `boundary_name_xv` will be created, containing the definition of the subdivided boundaries. `newgeom` will be used to calculate view factors with `geom2ansys` script whereas only the `boundary_name_xv` files will be necessary for the radiation calculation. The file `listb` will contain the list of the boundaries `boundary_name1`.

Parameters:

- **domaine** (*type*: string) not_set
- **[domaine_grossier]** (*type*: string) not_set
- **[nb_parts_naif]** (*type*: list of int) not_set
- **[nb_parts_geom]** (*type*: list of int) not_set
- **[condition_geometrique]** (*type*: list of str) not_set
- **bords_a_decouper** (*type*: list of str) not_set
- **[nom_fichier_sortie]** (*type*: string) not_set
- **[binaire]** (*type*: int) not_set

decouper_bord_coincident**Inherits from:** [interprete](#)

In case of non-coincident meshes and a paroi_contact condition, run is stopped and two external files are automatically generated in VEF (connectivity_failed_boundary_name and connectivity_failed_pb_name.med). In 2D, the keyword Decouper_bord_coincident associated to the connectivity_failed_boundary_name file allows to generate a new coincident mesh.

Parameters:

- **domain_name** (*type*: string) Name of domain.
 - **bord** (*type*: string) connectivity_failed_boundary_name
-

dilate**Inherits from:** [interprete](#)

Keyword to multiply the whole coordinates of the geometry.

Parameters:

- **domain_name** (*type*: string) Name of domain.
 - **alpha** (*type*: float) Value of dilatation coefficient.
-

dimension**Inherits from:** [interprete](#)

Keyword allowing calculation dimensions to be set (2D or 3D), where dim is an integer set to 2 or 3. This instruction is mandatory.

Parameters:

- **dim** (*type*: int into [2, 3]) Number of dimensions.
-

disable_tu**Inherits from:** [interprete](#)

Flag to disable the writing of the .TU files

discretiser_domaine**Inherits from:** [interprete](#)

Useful to discretize the domain domain_name (faces will be created) without defining a problem.

Parameters:

- **domain_name** (*type*: string) Name of the domain.
-

discretize**Synonyms:** discretiser**Inherits from:** [interprete](#)

Keyword to discretise a problem problem_name according to the discretization dis.

IMPORTANT: A number of objects must be already associated (a domain, time scheme, central object) prior to invoking the Discretize (Discretiser) keyword. The physical properties of this central object must also have been read.

Parameters:

- **problem_name** (*type*: string) Name of problem.
 - **dis** (*type*: string) Name of the discretization object.
-

distance_pari**Inherits from:** [interpret](#)

Class to generate external file `Wall_length.xyz` devoted for instance, for mixing length modelling. In this file, are saved the coordinates of each element (center of gravity) of dom domain and minimum distance between this point and boundaries (specified bords) that user specifies in data file (typically, those associated to walls). A field `Distance_pari` is available to post process the distance to the wall.

Parameters:

- **dom** (*type*: string) Name of domain.
 - **bords** (*type*: list of str) Boundaries.
 - **format** (*type*: string into ['binaire', 'formatte']) Value for format may be binaire (a binary file `Wall_length.xyz` is written) or formatte (moreover, a formatted file `Wall_length_formatted.xyz` is written).
-

ecrire_champ_med**Inherits from:** [interpret](#)

Keyword to write a field to MED format into a file.

Parameters:

- **nom_dom** (*type*: string) domain name
 - **nom_chp** (*type*: string) field name
 - **file** (*type*: string) file name
-

ecrire_fichier_xyz_valeur**Inherits from:** [interpret](#)

This keyword is used to write the values of a field only for some boundaries in a text file with the following format:

`n_valeur`

`x_1 y_1 [z_1] val_1`

...

`x_n y_n [z_n] val_n`

The created files are named : `pbname_fieldname_[boundaryname]_time.dat`

Parameters:

- **[binary_file]** (*type*: flag) To write file in binary format
 - **[dt]** (*type*: float) File writing frequency
 - **[fields]** (*type*: list of str) Names of the fields we want to write
 - **[boundaries]** (*type*: list of str) Names of the boundaries on which to write fields
-

ecrire_med**Synonyms:** `write_med`**Inherits from:** [interpret](#)

Write a domain to MED format into a file.

Parameters:

- **nom_dom** (*type*: string) Name of domain.
 - **file** (*type*: string) Name of file.
-

ecriturelecturespecial**Inherits from:** [interprete](#)

Class to write or not to write a .xyz file on the disk at the end of the calculation.

Parameters:

- **type** (*type*: string) If set to 0 (the default), no xyz file is created. If set to 1, the .xyz file is written at the end of the computation.
-

execute_parallel**Inherits from:** [interprete](#)

This keyword allows to run several computations in parallel on processors allocated to TRUST. The set of processors is split in N subsets and each subset will read and execute a different data file. Error messages usually written to stderr and stdout are redirected to .log files (journaling must be activated).

Parameters:

- **liste_cas** (*type*: list of str) N datafile1 ... datafileN. datafileX the name of a TRUST data file without the .data extension.
 - **[nb_procs]** (*type*: list of int) nb_procs is the number of processors needed to run each data file. If not given, TRUST assumes that computations are sequential.
-

extraire_domaine**Inherits from:** [interprete](#)

Keyword to create a new domain built with the domain elements of the pb_name problem verifying the two conditions given by Condition_elements. The problem pb_name should have been discretized.

Parameters:

- **domaine** (*type*: string) Domain in which faces are saved
 - **probleme** (*type*: string) Problem from which faces should be extracted
 - **[condition_elements]** (*type*: string) not_set
 - **[sous_domaine | sous_zone]** (*type*: string) not_set
-

extraire_plan**Inherits from:** [interprete](#)

This keyword extracts a plane mesh named domain_name (this domain should have been declared before) from the mesh of the pb_name problem. The plane can be either a triangle (defined by the keywords Origine, Point1, Point2 and Triangle), either a regular quadrangle (with keywords Origine, Point1 and Point2), or either a generalized quadrangle (with keywords Origine, Point1, Point2, Point3). The keyword Epaisseur specifies the thickness of volume around the plane which contains the faces of the extracted mesh. The keyword via_extraire_surface will create a plan and use Extraire_surface algorithm. Inverse_condition_element keyword then will be used in the case where the plane is a boundary not well oriented, and avec_certaines_bords_pour_extraire_surface is the option related to the Extraire_surface option named avec_certaines_bords. Point1, Point2 and Triangle), either a regular quadrangle (with keywords Origine, Point1 and Point2), or either a generalized quadrangle (with keywords Origine, Point1, Point2, Point3). The keyword Epaisseur specifies the thickness of volume around the plane which contains the faces of the extracted mesh. The keyword via_extraire_surface will create a plan and use Extraire_surface algorithm. Inverse_condition_element keyword then will be used in the case where the plane is a boundary not well oriented, and avec_certaines_bords_pour_extraire_surface is the option related to the Extraire_surface option named avec_certaines_bords.

Parameters:

- **domaine** (*type*: string) domain name
 - **probleme** (*type*: string) pb_name
-

- **origine** (*type*: list of float) not_set
- **point1** (*type*: list of float) not_set
- **point2** (*type*: list of float) not_set
- **[point3]** (*type*: list of float) not_set
- **[triangle]** (*type*: flag) not_set
- **epaisseur** (*type*: float) thickness
- **[via_extraire_surface]** (*type*: flag) not_set
- **[inverse_condition_element]** (*type*: flag) not_set
- **[avec_certains_bords_pour_extraire_surface]** (*type*: list of str) name of boundaries to include when extracting plan

extraire_surface

Inherits from: [interprete](#)

This keyword extracts a surface mesh named `domain_name` (this domain should have been declared before) from the mesh of the `pb_name` problem. The surface mesh is defined by one or two conditions. The first condition is about elements with `Condition_elements`. For example: `Condition_elements x*x+y*y+z*z<1`

Will define a surface mesh with external faces of the mesh elements inside the sphere of radius 1 located at (0,0,0). The second condition `Condition_faces` is useful to give a restriction.

By default, the faces from the boundaries are not added to the surface mesh excepted if option `avec_les_bords` is given (all the boundaries are added), or if the option `avec_certains_bords` is used to add only some boundaries.

The first condition is about elements with `Condition_elements`. For example: `Condition_elements x*x+y*y+z*z<1` Will define a surface mesh with external faces of the mesh elements inside the sphere of radius 1 located at (0,0,0).

The second condition `Condition_faces` is useful to give a restriction.

By default, the faces from the boundaries are not added to the surface mesh excepted if option `avec_les_bords` is given (all the boundaries are added), or if the option `avec_certains_bords` is used to add only some boundaries.

Parameters:

- **domaine** (*type*: string) Domain in which faces are saved
- **probleme** (*type*: string) Problem from which faces should be extracted
- **[condition_elements]** (*type*: string) condition on center of elements
- **[condition_faces]** (*type*: string) not_set
- **[avec_les_bords]** (*type*: flag) not_set
- **[avec_certains_bords]** (*type*: list of str) not_set

extrudebord

Inherits from: [interprete](#)

Class to generate an extruded mesh from a boundary of a tetrahedral or an hexahedral mesh.

Warning: If the initial domain is a tetrahedral mesh, the boundary will be moved in the XY plane then extrusion will be applied (you should maybe use the `Transformer` keyword on the final domain to have the domain you really want). You can use the keyword `Postraiter_domaine` to generate a `lata|med|...` file to visualize your initial and final meshes.

This keyword can be used for example to create a periodic box extracted from a boundary of a tetrahedral or a hexahedral mesh. This periodic box may be used then to engender turbulent inlet flow condition for the main domain.

Note that `ExtrudeBord` in VEF generates 3 or 14 tetrahedra from extruded prisms.

Parameters:

- **domaine_init** (*type*: string) Initial domain with hexaedras or tetrahedras.

- **direction** (*type*: list of float) Directions for the extrusion.
- **nb_tranches** (*type*: int) Number of elements in the extrusion direction.
- **domaine_final** (*type*: string) Extruded domain.
- **nom_bord** (*type*: string) Name of the boundary of the initial domain where extrusion will be applied.
- **[hexa_old]** (*type*: flag) Old algorithm for boundary extrusion from a hexahedral mesh.
- **[trois_tetra]** (*type*: flag) To extrude in 3 tetrahedras instead of 14 tetrahedras.
- **[vingt_tetra]** (*type*: flag) To extrude in 20 tetrahedras instead of 14 tetrahedras.
- **[sans_passer_par_le2d]** (*type*: int) Only for non-regression

extrudeparoi

Inherits from: [interprete](#)

Keyword dedicated in 3D (VEF) to create prismatic layer at wall. Each prism is cut into 3 tetraedra.

Parameters:

- **domaine** (*type*: string) Name of the domain.
- **nom_bord** (*type*: string) Name of the (no-slip) boundary for creation of prismatic layers.
- **[epaisseur]** (*type*: list of float) n r1 r2 rn : (relative or absolute) width for each layer.
- **[critere_absolu]** (*type*: flag) use absolute width for each layer instead of relative.
- **[projection_normale_bord]** (*type*: flag) keyword to project layers on the same plane that contiguous boundaries. default values are : epaisseur_relative 1 0.5 projection_normale_bord 1

extruder_en20

Inherits from: [interprete](#)

It does the same task as Extruder except that a prism is cut into 20 tetraedra instead of

1. The name of the boundaries will be devant (front) and derriere (back). But you can change these names with the keyword RegroupeBord.

Parameters:

- **domaine** | **domain_name** (*type*: string) Name of the domain.
- **nb_tranches** (*type*: int) Number of elements in the extrusion direction.
- **[direction]** (*type*: [troisf](#)) 0 Direction of the extrude operation.

facsec_expert

Inherits from: [interprete](#)

To parameter the safety factor for the time step during the simulation.

Parameters:

- **[facsec_ini]** (*type*: float) Initial facsec taken into account at the beginning of the simulation.
- **[facsec_max]** (*type*: float) Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec_max value. Warning: Some implicit schemes do not permit high facsec↔_max, example Schema_Adams_Moulton_order_3 needs facsec=facsec_max=1. Advice: The calculation may start with a facsec specified by the user and increased by the algorithm up to the facsec_max limit. But the user can also choose to specify a constant facsec (facsec_max will be set to facsec value then). Faster convergence has been seen and depends on the kind of calculation: -Hydraulic only or thermal hydraulic

with forced convection and low coupling between velocity and temperature (Boussinesq value beta low), facsec between 20-30-Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value beta high), facsec between 90-100 -Thermohydraulic with natural convection, facsec around 300 -Conduction only, facsec can be set to a very high value (1e8) as if the scheme was unconditionally stable. These values can also be used as rule of thumb for initial facsec with a facsec_max limit higher.

- **[rapport_residus]** (*type*: float) Ratio between the residual at time n and the residual at time n+1 above which the facsec is increased by multiplying by sqrt(rapport_residus) (1.2 by default).
- **[nb_ite_sans_accel_max]** (*type*: int) Maximum number of iterations without facsec increases (20000 by default): if facsec does not increase with the previous condition (ratio between 2 consecutive residuals too high), we increase it by force after nb_ite_sans_accel_max iterations.

fin

Synonyms: end

Inherits from: [interpret](#)

Keyword which must complete the data file. The execution of the data file stops when reaching this keyword.

integrer_champ_med

Inherits from: [interpret](#)

This keyword is used to calculate a flow rate from a velocity MED field read before. The method is either debit_total to calculate the flow rate on the whole surface, either integrale_en_z to calculate flow rates between z=zmin and z=zmax on nb_tranche surfaces. The output file indicates first the flow rate for the whole surface and then lists for each tranche : the height z, the surface average value, the surface area and the flow rate. For the debit_total method, only one tranche is considered.

file :z Sum(u.dS)/Sum(dS) Sum(dS) Sum(u.dS)

Parameters:

- **champ_med** (*type*: string) not_set
- **methode** (*type*: string into ['integrale_en_z', 'debit_total']) to choose between the integral following z or over the entire height (debit_total corresponds to zmin=-DMAXFLOAT, ZMax=DMAXFLOAT, nb_tranche=1)
- **[zmin]** (*type*: float) not_set
- **[zmax]** (*type*: float) not_set
- **[nb_tranche]** (*type*: int) not_set
- **[fichier_sortie]** (*type*: string) name of the output file, by default: integrale.

lata_to_cgns

Synonyms: lata_2_cgns

Inherits from: [interpret](#)

To convert results file written with LATA format to CGNS file. Warning: Fields located on faces are not supported yet.

Parameters:

- **file** (*type*: string) LATA file to convert to the new format.
- **file_cgns** (*type*: string) Name of the CGNS file.

lata_to_med

Synonyms: lata_2_med

Inherits from: [interpret](#)

To convert results file written with LATA format to MED file. Warning: Fields located on faces are not supported yet.

Parameters:

- **[format]** (*type:* [format_lata_to_med](#)) generated file post_med.data use format (MED or LATA or LML keyword).
 - **file** (*type:* string) LATA file to convert to the new format.
 - **file_med** (*type:* string) Name of the MED file.
-

lata_to_other

Synonyms: lata_2_other

Inherits from: [interpret](#)

To convert results file written with LATA format to CGNS, MED or LML format. Warning: Fields located at faces are not supported yet.

Parameters:

- **[format]** (*type:* string into ['lml', 'lata', 'lata_v2', 'med', 'cgns']) Results format (CGNS, MED or LATA or LML keyword).
 - **file** (*type:* string) LATA file to convert to the new format.
 - **file_post** (*type:* string) Name of file post.
-

lire_ideas

Inherits from: [interpret](#)

Read a geom in a unv file. 3D tetra mesh elements only may be read by TRUST.

Parameters:

- **nom_dom** (*type:* string) Name of domain.
 - **file** (*type:* string) Name of file.
-

lml_to_lata

Synonyms: lml_2_lata

Inherits from: [interpret](#)

To convert results file written with LML format to a single LATA file.

Parameters:

- **file_lml** (*type:* string) LML file to convert to the new format.
 - **file_lata** (*type:* string) Name of the single LATA file.
-

maillerparallel

Inherits from: [interpret](#)

creates a parallel distributed hexaedral mesh of a parallelepipedic box. It is equivalent to creating a mesh with a single Pave, splitting it with Decouper and reloading it in parallel with Scatter. It only works in 3D at this time. It can also be used for a sequential computation (with all NPARTS=1)}

Parameters:

- **domain** (*type:* string) the name of the domain to mesh (it must be an empty domain object).
-

- **nb_nodes** (*type*: list of int) dimension defines the spatial dimension (currently only dimension=3 is supported), and nX, nY and nZ defines the total number of nodes in the mesh in each direction.
- **splitting** (*type*: list of int) dimension is the spatial dimension and npartsX, npartsY and npartsZ are the number of parts created. The product of the number of parts must be equal to the number of processors used for the computation.
- **ghost_thickness** (*type*: int) the number of ghost cells (equivalent to the `epaisseur_joint` parameter of `Decouper`).
- **[perio_x]** (*type*: flag) change the splitting method to provide a valid mesh for periodic boundary conditions. Register the corresponding boundary in the list of periodic boundaries for the domain.
- **[perio_y]** (*type*: flag) change the splitting method to provide a valid mesh for periodic boundary conditions. Register the corresponding boundary in the list of periodic boundaries for the domain.
- **[perio_z]** (*type*: flag) change the splitting method to provide a valid mesh for periodic boundary conditions. Register the corresponding boundary in the list of periodic boundaries for the domain.
- **[function_coord_x]** (*type*: string) By default, the meshing algorithm creates nX nY nZ coordinates ranging between 0 and 1 (eg a unity size box). If `function_coord_x` is specified, it is used to transform the [0,1] segment to the coordinates of the nodes. `funcX` must be a function of the x variable only.
- **[function_coord_y]** (*type*: string) like `function_coord_x` for y
- **[function_coord_z]** (*type*: string) like `function_coord_x` for z
- **[file_coord_x]** (*type*: string) Keyword to read the Nx floating point values used as nodes coordinates in the file.
- **[file_coord_y]** (*type*: string) idem `file_coord_x` for y
- **[file_coord_z]** (*type*: string) idem `file_coord_x` for z
- **[boundary_xmin]** (*type*: string) the name of the boundary at the minimum X direction. If it not provided, the default boundary names are xmin, xmax, ymin, ymax, zmin and zmax. If the mesh is periodic in a given direction, only the MIN boundary name is used, for both sides of the box.
- **[boundary_xmax]** (*type*: string) not_set
- **[boundary_ymin]** (*type*: string) not_set
- **[boundary_ymax]** (*type*: string) not_set
- **[boundary_zmin]** (*type*: string) not_set
- **[boundary_zmax]** (*type*: string) not_set

mass_source

Inherits from: [interpret](#)

Mass source used in a dilatable simulation to add/reduce a mass at the boundary (volumetric source in the first cell of a given boundary).

Parameters:

- **bord** (*type*: string) Name of the boundary where the source term is applied
 - **surfacic_flux** (*type*: [front_field_base](#)) The boundary field that the user likes to apply: for example, `champ↔_front_uniforme`, `ch_front_input_uniform` or `champ_front_fonc_t`
-

merge_med

Inherits from: [interprete](#)

This keyword allows to merge multiple MED files produced during a parallel computation into a single MED file.

Parameters:

- **med_files_base_name** (*type*: string) Base name of multiple med files that should appear as base_name_↵_xxxxx.med, where xxxxx denotes the MPI rank number. If you specify NOM_DU_CAS, it will automatically take the basename from your datafile's name.
 - **time_iterations** (*type*: string into ['all_times', 'last_time']) Identifies whether to merge all time iterations present in the MED files or only the last one.
-

mkdir

Inherits from: [interprete](#)

equivalent to system mkdir

Parameters:

- **directory** (*type*: string) directory to create
-

modif_bord_to_raccord

Inherits from: [interprete](#)

Keyword to convert a boundary of domain_name domain of kind Bord to a boundary of kind Raccord (named boundary_name). It is useful when using meshes with boundaries of kind Bord defined and to run a coupled calculation.

Parameters:

- **domaine | domain** (*type*: string) Name of domain
 - **nom_bord** (*type*: string) Name of the boundary to transform.
-

modifydomaineaxi1d

Synonyms: convert_1d_to_1daxi

Inherits from: [interprete](#)

Convert a 1D mesh to 1D axisymmetric mesh

Parameters:

- **dom** (*type*: string) not_set
 - **bloc** (*type*: [bloc_lecture](#)) not_set
-

moyenne_volumique

Inherits from: [interprete](#)

This keyword should be used after Resoudre keyword. It computes the convolution product of one or more fields with a given filtering function.

Parameters:

- **nom_pb** (*type*: string) name of the problem where the source fields will be searched.
 - **nom_domaine** (*type*: string) name of the destination domain (for example, it can be a coarser mesh, but for optimal performance in parallel, the domain should be split with the same algorithm as the computation mesh, eg, same tranche parameters for example)
 - **noms_champs** (*type*: list of str) name of the source fields (these fields must be accessible from the post-treatment) N source_field1 source_field2 ... source_fieldN
 - **[format_post]** (*type*: string) gives the fileformat for the result (by default : lata)
-

- **[nom_fichier_post]** (*type*: string) indicates the filename where the result is written
- **fonction_filtre** (*type*: [bloc_lecture](#)) to specify the given filter `Fonction_filtre { type filter_type demie-largeur l [ omega w ] [ expression string ] }` *type filter_type* : This parameter specifies the filtering function. Valid *filter_type* are: Boite is a box filter, $f(x,y,z)=(abs(x)<l)*(abs(y)<l)*(abs(z)<l)/(8\ l^3)$ Chapeau is a hat filter (product of hat filters in each direction) centered on the origin, the half-width of the filter being *l* and its integral being 1. Quadra is a 2nd order filter. Gaussienne is a normalized gaussian filter of standard deviation *sigma* in each direction (all field elements outside a cubic box defined by *clipping_half_width* are ignored, hence, taking *clipping_half_width*=2.5**sigma* yields an integral of 0.99 for a uniform unity field). Parser allows a user defined function of the *x,y,z* variables. All elements outside a cubic box defined by *clipping_half_width* are ignored. The parser is much slower than the equivalent c++ coded function... *demie-largeur l* : This parameter specifies the half width of the filter *[omega w]* : This parameter must be given for the gaussienne filter. It defines the standard deviation of the gaussian filter. *[expression string]* : This parameter must be given for the parser filter type. This expression will be interpreted by the math parser with the predefined variables *x*, *y* and *z*.
- **[localisation]** (*type*: string into ['som', 'elem']) indicates where the convolution product should be computed: either on the elements or on the nodes of the destination domain.

multigrid_solver

Inherits from: [interprete](#)

Object defining a multigrid solver in IJK discretization

Parameters:

- **[coarsen_operators]** (*type*: list of `Coarsen_operator_uniform`) *not_set*
- **[ghost_size]** (*type*: int) Number of ghost cells known by each processor in each of the three directions
- **[relax_jacobi]** (*type*: list of float) Parameter between 0 and 1 that will be used in the Jacobi method to solve equation on each grid. Should be around 0.7
- **[pre_smooth_steps]** (*type*: list of int) First integer of the list indicates the numbers of integers that has to be read next. Following integers define the numbers of iterations done before solving the equation on each grid. For example, 2 7 8 means that we have a list of 2 integers, the first one tells us to perform 7 pre-smooth steps on the first grid, the second one tells us to perform 8 pre-smooth steps on the second grid. If there are more than 2 grids in the solver, then the remaining ones will have as many pre-smooth steps as the last mentioned number (here, 8)
- **[smooth_steps]** (*type*: list of int) First integer of the list indicates the numbers of integers that has to be read next. Following integers define the numbers of iterations done after solving the equation on each grid. Same behavior as *pre_smooth_steps*
- **[nb_full_mg_steps]** (*type*: list of int) Number of multigrid iterations at each level
- **[solveur_grossier]** (*type*: [solveur_sys_base](#)) Name of the iterative solver that will be used to solve the system on the coarsest grid. This resolution must be more precise than the ones occurring on the fine grids. The threshold of this solver must therefore be lower than *seuil* defined above.
- **[seuil]** (*type*: float) Define an upper bound on the norm of the final residue (i.e. the one obtained after applying the multigrid solver). With hybrid precision, as long as we have not obtained a residue whose norm is lower than the imposed threshold, we keep applying the solver
- **[impr]** (*type*: flag) Flag to display some info on the resolution on each grid
- **[solver_precision]** (*type*: string into ['mixed', 'double']) Precision with which the variables at stake during the resolution of the system will be stored. We can have a simple or double precision or both. In the case of a hybrid precision, the multigrid solver is launched in simple precision, but the residual is calculated in double precision.
- **[iterations_mixed_solver]** (*type*: int) Define the maximum number of iterations in mixed precision solver

multiplefiles**Inherits from:** [interpreté](#)

Change MPI rank limit for multiple files during I/O

Parameters:

- **type** (*type*: int) New MPI rank limit
-

my_comm_group**Inherits from:** [interpreté](#)

This keyword allows to create a user MPI Comm Group of size N using the processors allocated to TRUST. The set of processors is split in N subsets.

Parameters:

- **group_nb** (*type*: int) Number of groups to define in your Comm Group.
-

nettoiepasnoeuds**Inherits from:** [interpreté](#)

Keyword NettoiePasNoeuds does not delete useless nodes (nodes without elements) from a domain.

Parameters:

- **domain_name** (*type*: string) Name of domain.
-

op_conv_ef_stab_dg_elem**Inherits from:** [interpreté](#)

Class Op_Conv_EF_Stab_DG_Elem

op_conv_ef_stab_polymac_cdo_face**Inherits from:** [interpreté](#)

Class Op_Conv_EF_Stab_PolyMAC_CDO_Face_PolyMAC_CDO

Parameters:

- **[alpha]** (*type*: float) parametre ajustant la stabilisation de 0 (schema centre) a 1 (schema amont)
-

op_conv_ef_stab_polymac_hfv_elem**Synonyms:** op_conv_ef_stab_polymac_mpfa_elem**Inherits from:** [interpreté](#)

Class Op_Conv_EF_Stab_PolyMAC_HFV_Elem

Parameters:

- **[alpha]** (*type*: float) parametre ajustant la stabilisation de 0 (schema centre) a 1 (schema amont)
-

op_conv_ef_stab_polymac_hfv_face**Inherits from:** [interpreté](#)

Class Op_Conv_EF_Stab_PolyMAC_HFV_Face

op_conv_ef_stab_polymac_mpfa_face**Inherits from:** [interpreté](#)

Class Op_Conv_EF_Stab_PolyMAC_MPFA_Face

option_cgns**Inherits from:** [interpret](#)

Class for CGNS options.

Parameters:

- **[parallel_over_zone]** (*type*: flag) If used, data will be written in separate zones (ie: one zone per processor). This is not so performant but easier to read later ...
- **[use_links]** (*type*: flag) If used, data will be written in separate files; one file for mesh, and then one file for solution time. Links will be used.
- **[linked_files_per_comm_group]** (*type*: flag) If used, data will be written (at each comm group) in separate files; one file for mesh, and then one file for solution time. Links will be used.
- **[single_file_per_comm_group]** (*type*: flag) If used, data will be written (at each comm group) in a single file.
- **[keep_files_before_reset_time]** (*type*: flag) If used with resetTime, CGNS files will be kept. Otherwise the files are overwritten.
- **[close_every_n]** (*type*: int) Used to fix the opening/closing frequency when writing in a single CGNS file (choice by default).
- **[flush_every_n]** (*type*: int) Used to fix the flush-to-disc frequency when writing in a single CGNS file (choice by default).

option_dg**Inherits from:** [interpret](#)

Class for DG options.

Parameters:

- **[order]** (*type*: int) global order for the DG unknowns (1 by default)
- **[velocity_order]** (*type*: int) optional order for DG velocity unknown
- **[pressure_order]** (*type*: int) optional order for DG pressure unknown
- **[temperature_order]** (*type*: int) optional order for DG temperature unknown
- **[gram_schmidt]** (*type*: int) Gram Schmidt orthogonalization (1 by default)

option_ijk**Inherits from:** [interpret](#)

Class of IJK options.

Parameters:

- **[check_divergence]** (*type*: flag) Flag to compute and print the value of $\text{div}(\mathbf{u})$ after each pressure-correction
- **[disable_diphasique]** (*type*: flag) Disable all calculations related to interfaces (phase properties, interfacial force, ...)

option_interpolation**Inherits from:** [interpret](#)

Class for interpolation fields using MEDCoupling.

Parameters:

- **[sans_dec | without_dec]** (*type*: flag) Use remapper even for a parallel calculation
- **[sharing_algo]** (*type*: int) Setting the DEC sharing algo : 0,1,2

option_polymac_family

Synonyms: option_polymac

Inherits from: [interpret](#)

Class of PolyMAC options.

Parameters:

- **[use_osqp]** (*type*: flag) Flag to use the old formulation of the M2 matrix provided by the OSQP library. Only useful for PolyMAC_CDO version.
- **[maillage_vdf | vdf_mesh]** (*type*: flag) Flag used to force the calculation of the equiv tab.
- **[interp_ve1]** (*type*: flag) Flag to enable a first-order face-to-element velocity interpolation. By default, it is not activated which means a second order interpolation. Only useful for PolyMAC_MPFA version.
- **[traitement_axi]** (*type*: flag) Flag used to relax the time-step stability criterion in case of a thin slice geometry while modelling an axi-symetrical case. Only useful for PolyMAC_MPFA version.

option_vdf

Inherits from: [interpret](#)

Class of VDF options.

Parameters:

- **[traitement_coins]** (*type*: string into ['oui', 'non']) Treatment of corners (yes or no). This option modifies slightly the calculations at the outlet of the plane channel. It supposes that the boundary continues after channel outlet (i.e. velocity vector remains parallel to the boundary).
- **[traitement_gradients]** (*type*: string into ['oui', 'non']) Treatment of gradient calculations (yes or no). This option modifies slightly the gradient calculation at the corners and activates also the corner treatment option.
- **[p_imposee_aux_faces]** (*type*: string into ['oui', 'non']) Pressure imposed at the faces (yes or no).
- **[deactivate_arete_mixte]** (*type*: flag) Deactivate the arete_mixte contribution in the conv op of the momentum equation.
- **[all_options | toutes_les_options]** (*type*: flag) Activates all Option_VDF options. If used, must be used alone without specifying the other options, nor combinations.

orientefacesbord

Inherits from: [interpret](#)

Keyword to modify the order of the boundary vertices included in a domain, such that the surface normals are outer pointing.

Parameters:

- **domain_name** (*type*: string) Name of domain.

parallel_io_parameters

Inherits from: [interpret](#)

Object to handle parallel files in IJK discretization

Parameters:

- **[block_size_bytes]** (*type*: int) File writes will be performed by chunks of this size (in bytes). This parameter will not be taken into account if block_size_megabytes has been defined
- **[block_size_megabytes]** (*type*: int) File writes will be performed by chunks of this size (in megabytes). The size should be a multiple of the GPFS block size or lustre stripping size (typically several megabytes)
- **[writing_processes]** (*type*: int) This is the number of processes that will write concurrently to the file system (this must be set according to the capacity of the filesystem, set to 1 on small computers, can be up to 64 or 128 on very large systems).

- **[bench_ijk_splitting_write]** (*type*: string) Name of the splitting object we want to use to run a parallel write bench (optional parameter)
- **[bench_ijk_splitting_read]** (*type*: string) Name of the splitting object we want to use to run a parallel read bench (optional parameter)

partition

Synonyms: `decouper`

Inherits from: [interprete](#)

Class for parallel calculation to cut a domain for each processor. By default, this keyword is commented in the reference test cases.

Parameters:

- **domaine** (*type*: string) Name of the domain to be cut.
- **bloc_decouper** (*type*: [bloc_decouper](#)) Description how to cut a domain.

partition_multi

Synonyms: `decouper_multi`

Inherits from: [interprete](#)

allows to partition multiple domains in contact with each other in parallel: necessary for resolution monolithique in implicit schemes and for all coupled problems using PolyMAC_HFV. By default, this keyword is commented in the reference test cases.

Parameters:

- **aco** (*type*: string into ['{'] Opening curly bracket.
- **domaine1** (*type*: string into ['domaine']) not set.
- **dom** (*type*: string) Name of the first domain to be cut.
- **blocdecouppdom1** (*type*: [bloc_decouper](#)) Partition bloc for the first domain.
- **domaine2** (*type*: string into ['domaine']) not set.
- **dom2** (*type*: string) Name of the second domain to be cut.
- **blocdecouppdom2** (*type*: [bloc_decouper](#)) Partition bloc for the second domain.
- **acof** (*type*: string into ['}'] Closing curly bracket.

pilote_icoco

Inherits from: [interprete](#)

`not_set`

Parameters:

- **pb_name** (*type*: string) `not_set`
- **main** (*type*: string) `not_set`
- **[nb_pas_dt_reset]** (*type*: int) number of time steps before a single call to `resetTime(0)` like the Cathare directive (default value 1000000000)
- **[nstep_stabilized]** (*type*: int) number of first iterations in which `resetTime(0)` is called as a Cathare stabilized transient (default value 0)
- **[sortie_root_directory]** (*type*: int) if enable, no save post before the `resetTime(0)` call (default value 0)

polyedriser**Inherits from:** [interprete](#)

cast hexahedra into polyhedra so that the indexing of the mesh vertices is compatible with PolyMAC_HFV discretization. Must be used in PolyMAC_HFV discretization if a hexahedral mesh has been produced with TRUST's internal mesh generator.

Parameters:

- **domain_name** (*type*: string) Name of domain.

postraiter_domaine**Inherits from:** [interprete](#)

To write one or more domains in a file with a specified format (MED,LML,LATA,SINGLE_LATA,CGNS).

Parameters:

- **format** (*type*: string into ['lml', 'lata', 'single_lata', 'lata_v2', 'med', 'cgns']) File format.
- **[binaire]** (*type*: int into [0, 1]) Binary (binaire 1) or ASCII (binaire 0) may be used. By default, it is 0 for LATA and only ASCII is available for LML and only binary is available for MED.
- **[ecrire_frontiere]** (*type*: int into [0, 1]) This option will write (if set to 1, the default) or not (if set to 0) the boundaries as fields into the file (it is useful to not add the boundaries when writing a domain extracted from another domain)
- **[dual]** (*type*: int into [0, 1]) This option indicates whether the original mesh (default) or the dual one (the one used for postprocessing of field faces) is to be written.
- **[fichier | file]** (*type*: string) The file name can be changed with the fichier option.
- **[joints_non_postraites]** (*type*: int into [0, 1]) The joints_non_postraites (1 by default) will not write the boundaries between the partitioned mesh.
- **[domaine | domain]** (*type*: string) Name of domain
- **[domaines]** (*type*: [bloc_lecture](#)) Names of domains : { name1 name2 }

precisiongeom**Inherits from:** [interprete](#)

Class to change the way floating-point number comparison is done. By default, two numbers are equal if their absolute difference is smaller than 1e-10. The keyword is useful to modify this value. Moreover, nodes coordinates will be written in .geom files with this same precision.

Parameters:

- **precision** (*type*: float) New value of precision.

raffiner_anisotrope**Inherits from:** [interprete](#)

Only for VEF discretizations, allows to cut triangle elements in 3, or tetrahedra in 4 parts, by defining a new summit located at the center of the element:

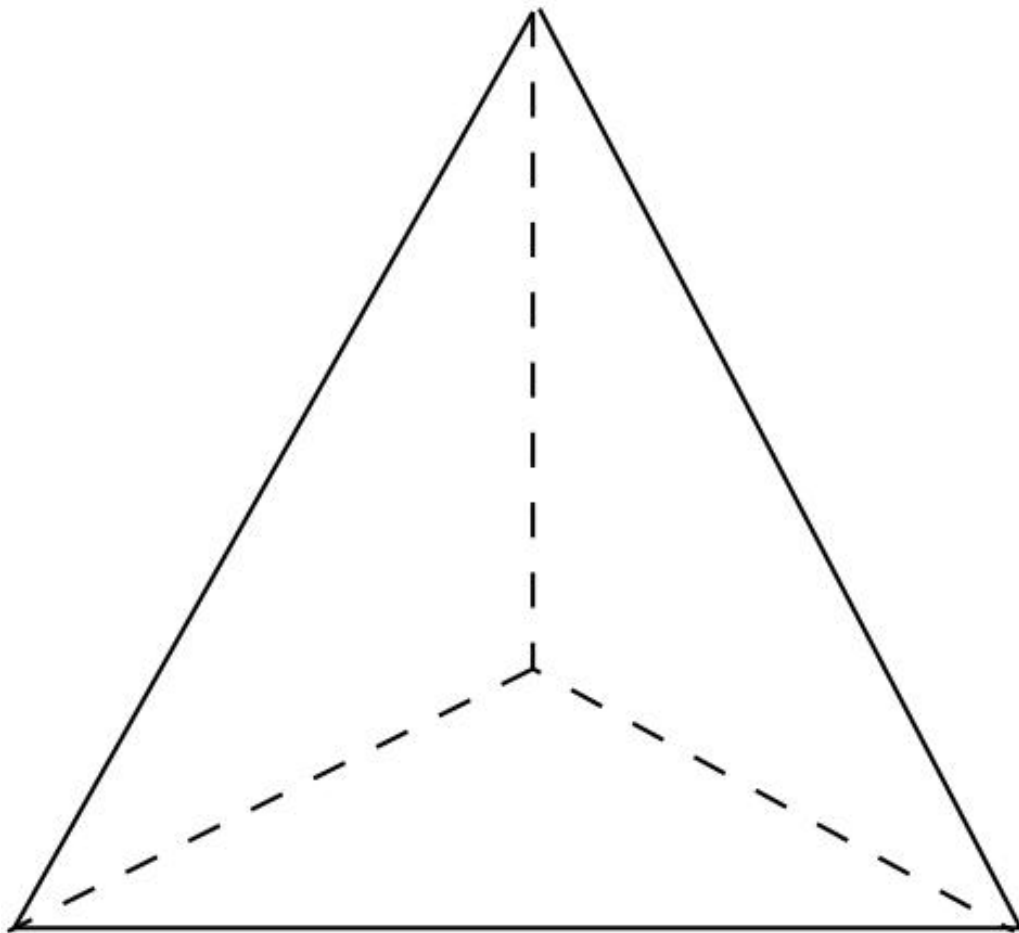


Figure 1 raffineranisotri

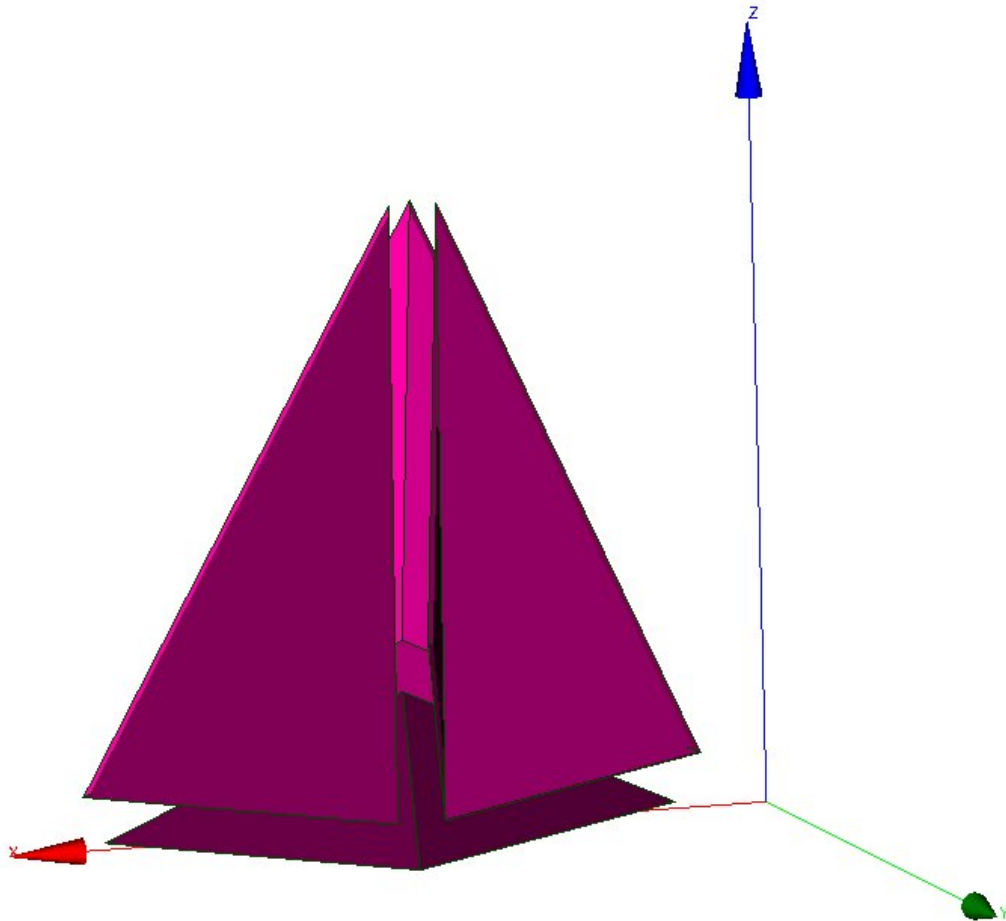


Figure 2 raffineranisotetra

Note that such a cut creates flat elements (anisotropic).

Parameters:

- **domain_name** (*type*: string) Name of domain.

raffiner_isotrope_parallelele

Inherits from: [interprete](#)

Refine parallel mesh in parallel

Parameters:

- **name_of_initial_domaines** | **name_of_initial_zones** (*type*: string) name of initial Domaines
- **name_of_new_domaines** | **name_of_new_zones** (*type*: string) name of new Domaines
- **[ascii]** (*type*: flag) writing Domaines in ascii format
- **[single_hdf]** (*type*: flag) writing Domaines in hdf format

read

Synonyms: lire

Inherits from: [interprete](#)

The 'read' instruction in a TRUST dataset. Overriden from the automatic generation to make the second argument a `Objet_u`. See also `Read_Parser` class in `base.py` module.

Parameters:

- **identifier** (*type*: string) Identifier of the class being read. Must match a previous forward Declaration.
- **obj** (*type*: `objet_u`) The object being read.

read_tgrid**Synonyms:** lire_tgrid**Inherits from:** [interpret](#)

Keyword to read Tgrid/Gambit mesh files. 2D (triangles or quadrangles) and 3D (tetra or hexa elements) meshes, may be read by TRUST.

Parameters:

- **dom** (*type:* string) Name of domain.
- **filename** (*type:* string) Name of file containing the mesh.

rectify_mesh**Synonyms:** orienter_simplexes**Inherits from:** [interpret](#)

Keyword to refine a mesh

Parameters:

- **domain_name** (*type:* string) Name of domain.

redresser_hexaedres_vdf**Inherits from:** [interpret](#)

Keyword to convert a domain (named domain_name) with quadrilaterals/VEF hexaedras which looks like rectangles/VDF hexaedras into a domain with real rectangles/VDF hexaedras.

Parameters:

- **domain_name** (*type:* string) Name of domain to resequence.

refine_mesh**Inherits from:** [interpret](#)

not_set

Parameters:

- **domaine** (*type:* string) not_set

remove_elem**Inherits from:** [interpret](#)

Keyword to remove element from a VDF mesh (named domaine_name), either from an explicit list of elements or from a geometric condition defined by a condition $f(x,y) > 0$ in 2D and $f(x,y,z) > 0$ in 3D. All the new borders generated are gathered in one boundary called : newBord (to rename it, use RegroupeBord keyword. To split it to different boundaries, use decoupebord keyword). Example of a removed zone of radius 0.2 centered at $(x,y)=(0.5,0.5)$:

Remove_elem dom { fonction $0.2 * 0.2 - (x - 0.5)^2 - (y - 0.5)^2 > 0$ }

Warning : the thickness of removed zone has to be large enough to avoid singular nodes as described below : 

Parameters:

- **domaine** | **domain** (*type:* string) Name of domain
- **bloc** (*type:* [remove_elem_bloc](#)) not_set

remove_invalid_internal_boundaries**Inherits from:** [interpret](#)

Keyword to suppress an internal boundary of the domain_name domain. Indeed, some mesh tools may define internal boundaries (eg: for post processing task after the calculation) but TRUST does not support it yet.

Parameters:

- **domain_name** (*type:* string) Name of domain.

reorienter_tetraedres**Inherits from:** [interprete](#)

This keyword is mandatory for front-tracking computations with the VEF discretization. For each tetrahedral element of the domain, it checks if it has a positive volume. If the volume (determinant of the three vectors) is negative, it swaps two nodes to reverse the orientation of this tetrahedron.

Parameters:

- **domain_name** (*type*: string) Name of domain.

reorienter_triangles**Inherits from:** [interprete](#)

not_set

Parameters:

- **domain_name** (*type*: string) Name of domain.

resequencing**Synonyms:** reordonner**Inherits from:** [interprete](#)

The Reordonner interpreter is required sometimes for a VDF mesh which is not produced by the internal mesher. Example where this is used:

Read_file dom fichier.geom

Reordonner dom

Observations: This keyword is redundant when the mesh that is read is correctly sequenced in the TRUST sense. This significant mesh operation may take some time... The message returned by TRUST is not explicit when the Reordonner (Resequencing) keyword is required but not included in the data set...

Parameters:

- **domain_name** (*type*: string) Name of domain to resequence.

residuals**Inherits from:** [interprete](#)

To specify how the residuals will be computed.

Parameters:

- **[norm]** (*type*: string into ['l2', 'max']) allows to choose the norm we want to use (max norm by default). Possible to specify L2-norm.
- **[relative]** (*type*: string into ['0', '1', '2']) This is the old keyword `seuil_statio_relatif_deconseille`. If it is set to 1, it will normalize the residuals with the residuals of the first 5 timesteps (default is 0). if set to 2, residual will be computed as $R/(\max-\min)$.

rotation**Inherits from:** [interprete](#)

Keyword to rotate the geometry of an arbitrary angle around an axis aligned with Ox, Oy or Oz axis.

Parameters:

- **domain_name** (*type*: string) Name of domain to which the transformation is applied.
- **dir** (*type*: string into ['x', 'y', 'z']) X, Y or Z to indicate the direction of the rotation axis
- **coord1** (*type*: float) coordinates of the center of rotation in the plane orthogonal to the rotation axis. These coordinates must be specified in the direct triad sense.
- **coord2** (*type*: float) not_set
- **angle** (*type*: float) angle of rotation (in degrees)

solve**Synonyms:** resoudre**Inherits from:** [interpret](#)

Interpreter to start calculation with TRUST.

Parameters:

- **pb** (*type:* string) Name of problem to be solved.

stat_per_proc_perf_log**Inherits from:** [interpret](#)

Keyword allowing to activate the detailed statistics per processor (by default this is false, and only the master proc will produce stats).

Parameters:

- **flg** (*type:* int) A flag that can be either 0 or 1 to turn off (default) or on the detailed stats.

supprime_bord**Inherits from:** [interpret](#)

Keyword to remove boundaries (named Boundary_name1 Boundary_name2) of the domain named domain_name.

Parameters:

- **domaine** | **domain** (*type:* string) Name of domain
- **bords** (*type:* list of Nom_anonyme) List of name.

system**Inherits from:** [interpret](#)To run Unix commands from the data file. Example: System 'echo The End | mail trust@cea.fr'**Parameters:**

- **cmd** (*type:* string) command to execute.

test_solveur**Inherits from:** [interpret](#)

To test several solvers

Parameters:

- **[fichier_secmem]** (*type:* string) Filename containing the second member B
- **[fichier_matrice]** (*type:* string) Filename containing the matrix A
- **[fichier_solution]** (*type:* string) Filename containing the solution x
- **[nb_test]** (*type:* int) Number of tests to measure the time resolution (one preconditionnement)
- **[impr]** (*type:* flag) To print the convergence solver
- **[solveur]** (*type:* [solveur_sys_base](#)) To specify a solver
- **[fichier_solveur]** (*type:* string) To specify a file containing a list of solvers
- **[genere_fichier_solveur]** (*type:* float) To create a file of the solver with a threshold convergence
- **[seuil_verification]** (*type:* float) Check if the solution satisfy $\|Ax-B\| < \text{precision}$
- **[pas_de_solution_initiale]** (*type:* flag) Resolution isn't initialized with the solution x
- **[ascii]** (*type:* flag) Ascii files

test_sse_kernels**Inherits from:** [interpreté](#)

Object to test the different kernel methods used in the multigrid solver in IJK discretization

Parameters:

- **[nmax]** (*type*: int) Number of tests we want to perform
-

testeur**Inherits from:** [interpreté](#)

not_set

Parameters:

- **data** (*type*: [bloc_lecture](#)) not_set
-

testeur_medcoupling**Inherits from:** [interpreté](#)

not_set

Parameters:

- **pb_name** (*type*: string) Name of domain.
 - **field_name** | **filed_name** (*type*: string) Name of domain.
-

verifier_qualite_raffinements**Inherits from:** [interpreté](#)

not_set

Parameters:

- **domain_names** (*type*: list of Nom_anonyme) Vect of name.
-

verifier_simplexes**Inherits from:** [interpreté](#)

Keyword to raffine a simplexes

Parameters:

- **domain_name** (*type*: string) Name of domain.
-

verifiercoin**Inherits from:** [interpreté](#)

This keyword subdivides inconsistent 2D/3D cells used with VEFPreP1B discretization. Must be used before the mesh is discretized. The Read_file option can be used only if the file.decoupage_som was previously created by TRUST. This option, only in 2D, reverses the common face at two cells (at least one is inconsistent), through the nodes opposed. In 3D, the option has no effect.

Parameters:

- **domain_name** | **dom** (*type*: string) Name of the domaine
 - **bloc** (*type*: [verifiercoin_bloc](#)) not_set
-

write**Synonyms:** ecrire**Inherits from:** [interpret](#)

Keyword to write the object of name name_obj to a standard outlet.

Parameters:

- **name_obj** (*type*: string) Name of the object to be written.

1.13.1 Keywords derived from analyse_angle**analyse_angle****Inherits from:** [interpret](#)

Keyword Analyse_angle prints the histogram of the largest angle of each mesh elements of the domain named name_domain. nb_histo is the histogram number of bins. It is called by default during the domain discretization with nb_histo set to 18. Useful to check the number of elements with angles above 90 degrees.

Parameters:

- **domain_name** (*type*: string) Name of domain to resequence.
- **nb_histo** (*type*: int) not_set

analyse_angle_64**Inherits from:** [analyse_angle](#)

Analyse_angle for big (64b) domain.

Parameters:

- **domain_name** (*type*: string) Name of domain to resequence.
- **nb_histo** (*type*: int) not_set

1.13.2 Keywords derived from extract_2d_from_3d**extract_2d_from_3d****Inherits from:** [interpret](#)

Keyword to extract a 2D mesh by selecting a boundary of the 3D mesh. To generate a 2D axisymmetric mesh prefer Extract_2Daxi_from_3D keyword.

Parameters:

- **dom3d** (*type*: string) Domain name of the 3D mesh
- **bord** (*type*: string) Boundary name. This boundary becomes the new 2D mesh and all the boundaries, in 3D, attached to the selected boundary, give their name to the new boundaries, in 2D.
- **dom2d** (*type*: string) Domain name of the new 2D mesh

extract_2daxi_from_3d**Inherits from:** [extract_2d_from_3d](#)

Keyword to extract a 2D axisymmetric mesh by selecting a boundary of the 3D mesh.

Parameters:

- **dom3d** (*type*: string) Domain name of the 3D mesh
- **bord** (*type*: string) Boundary name. This boundary becomes the new 2D mesh and all the boundaries, in 3D, attached to the selected boundary, give their name to the new boundaries, in 2D.
- **dom2d** (*type*: string) Domain name of the new 2D mesh

1.13.3 Keywords derived from extruder

extruder

Inherits from: [interprete](#)

Class to create a 3D tetrahedral/hexahedral mesh (a prism is cut in 14) from a 2D triangular/quadrangular mesh.

Parameters:

- **domaine** | **domain_name** (*type*: string) Name of the domain.
- **nb_tranches** (*type*: int) Number of elements in the extrusion direction.
- **direction** (*type*: [troisf](#)) Direction of the extrude operation.

extruder_en3

Inherits from: [extruder](#)

Class to create a 3D tetrahedral/hexahedral mesh (a prism is cut in 3) from a 2D triangular/quadrangular mesh. The names of the boundaries (by default, devant (front) and derriere (back)) may be edited by the keyword `nom_cl_devant` and `nom_cl_derriere`. If 'null' is written for `nom_cl`, then no boundary condition is generated at this place.

Recommendation : to ensure conformity between meshes (in case of fluid/solid coupling) it is recommended to extrude all the domains at the same time.

Parameters:

- **domaine** | **domain_name** (*type*: list of str) List of the domains
- **[nom_cl_devant]** (*type*: string) New name of the first boundary.
- **[nom_cl_derriere]** (*type*: string) New name of the second boundary.
- **nb_tranches** (*type*: int) Number of elements in the extrusion direction.
- **direction** (*type*: [troisf](#)) Direction of the extrude operation.

1.13.4 Keywords derived from imprimer_flux

imprimer_flux

Inherits from: [interprete](#)

This keyword prints the flux per face at the specified domain boundaries in the data set. The fluxes are written to the .face files at a frequency defined by `dt_impr`, the evaluation printing frequency (refer to time scheme keywords). By default, fluxes are incorporated onto the edges before being displayed.

Parameters:

- **domain_name** (*type*: string) Name of the domain.
- **noms_bord** (*type*: [bloc_lecture](#)) List of boundaries, for ex: { Bord1 Bord2 }

imprimer_flux_sum

Inherits from: [imprimer_flux](#)

This keyword prints the sum of the flux per face at the domain boundaries defined by the user in the data set. The fluxes are written into the .out files at a frequency defined by `dt_impr`, the evaluation printing frequency (refer to time scheme keywords).

Parameters:

- **domain_name** (*type*: string) Name of the domain.
- **noms_bord** (*type*: [bloc_lecture](#)) List of boundaries, for ex: { Bord1 Bord2 }

1.13.5 Keywords derived from interprete_geometrique_base

interprete_geometrique_base

Inherits from: [interprete](#)

Class for interpreting a data file

create_domain_from_sub_domain

Synonyms: create_domain_from_sous_zone, create_domain_from_sub_domains

Inherits from: [interprete_geometrique_base](#)

This keyword fills the domain `domaine_final` with the subdomaine `par_sous_zone` from the domain `domaine_init`. It is very useful when meshing several mediums with Gmsh. Each medium will be defined as a subdomaine into Gmsh. A MED mesh file will be saved from Gmsh and read with Lire_Med keyword by the TRUST data file. And with this keyword, a domain will be created for each medium in the TRUST data file.

Parameters:

- **[domaine_final]** (*type*: string) new domain in which faces are stored
- **[par_sous_zone | par_sous_dom]** (*type*: string) a sub-area (a group in a MED file) allowing to choose the elements
- **domaine_init** (*type*: string) initial domain

1.13.6 Keywords derived from mailler

mailler

Inherits from: [interprete](#)

The Mailler (Mesh) interpreter allows a Domain type object `domaine` to be meshed with objects `objet_1`, `objet_2`, etc...

Parameters:

- **domaine** (*type*: string) Name of domain.
 - **bloc** (*type*: list of Mailler_base) List of block mesh.
-

mailler_64

Inherits from: [mailler](#)

The Mailler (Mesh) interpreter allows a big (64b) domain type object `domaine` to be meshed with objects `objet_1`, `objet_2`, etc...

Parameters:

- **domaine** (*type*: string) Name of domain.
- **bloc** (*type*: list of Mailler_base) List of block mesh.

1.13.7 Keywords derived from raffiner_isotrope

raffiner_isotrope

Synonyms: raffiner_simplexes

Inherits from: [interprete](#)

For VDF and VEF discretizations, allows to cut triangles/quadrangles or tetrahedral/hexaedras elements respectively in 4 or 8 new ones by defining new summits located at the middle of edges (and center of faces and elements for quadrangles and hexaedra). Such a cut preserves the shape of original elements (isotropic). For 2D elements:



For 3D elements: 



Parameters:

- **domain_name** (*type*: string) Name of domain.
-

raffiner_simplexes_64**Inherits from:** [raffiner_isotrope](#)

Same as Raffiner_isotrope and Raffiner_simplexes for big (64b) domain

Parameters:

- **domain_name** (*type*: string) Name of domain.

1.13.8 Keywords derived from read_file**read_file****Synonyms:** lire_fichier**Inherits from:** [interpret](#)

Keyword to read the object name_obj contained in the file filename.

This is notably used when the calculation domain has already been meshed and the mesh contains the file filename, simply write read_file dom filename (where dom is the name of the meshed domain).

If the filename is ;, is to execute a data set given in the file of name name_obj (a space must be entered between the semi-colon and the file name).

Parameters:

- **name_obj** (*type*: string) Name of the object to be read.
 - **filename** (*type*: string) Name of the file.
-

read_file_bin**Synonyms:** lire_fichier_bin, read_file_binary**Inherits from:** [read_file](#)

Keyword to read an object name_obj in the unformatted type file filename.

Parameters:

- **name_obj** (*type*: string) Name of the object to be read.
 - **filename** (*type*: string) Name of the file.
-

read_unsupported_ascii_file_from_icem**Inherits from:** [read_file](#)

not_set

Parameters:

- **name_obj** (*type*: string) Name of the object to be read.
 - **filename** (*type*: string) Name of the file.
-

1.13.9 Keywords derived from read_med**read_med****Synonyms:** lire_med**Inherits from:** [interpret](#)

Keyword to read MED mesh files where 'domain' corresponds to the domain name, 'file' corresponds to the file (written in the MED format) containing the mesh named mesh_name.

Note about naming boundaries: When reading 'file', TRUST will detect boundaries between domains (Raccord) when the name of the boundary begins by 'type_raccord_'. For example, a boundary named type_raccord_wall in 'file' will be considered by TRUST as a boundary named 'wall' between two domains.

NB: To read several domains from a mesh issued from a MED file, use Read_Med to read the mesh then use Create_domain_from_sub_domain keyword.

NB: If the MED file contains one or several subdomaine defined as a group of volumes, then Read_MED will read it and will create two files domain_name_ssz.geo and domain_name_ssz_par.geo defining the subdomaines for

sequential and/or parallel calculations. These subdomains will be read in sequential in the datafile by including (after Read_Med keyword) something like:

Read_Med

Read_file domain_name_ssz.geo ;

During the parallel calculation, you will include something:

Scatter { ... }

Read_file domain_name_ssz_par.geo ;

Parameters:

- **[convertalltopoly]** (*type*: flag) Option to convert mesh with mixed cells into polyhedral/polygonal cells
- **domain | domaine** (*type*: string) Corresponds to the domain name.
- **file | fichier** (*type*: string) File (written in the MED format, with extension '.med') containing the mesh
- **[mesh | maillage]** (*type*: string) Name of the mesh in med file. If not specified, the first mesh will be read.
- **[exclude_groups | exclure_groupes]** (*type*: list of str) List of face groups to skip in the MED file.
- **[sub_zones | sous_zones]** (*type*: list of str) List of subzones to keep in the MED file and write directly in the .geo
- **[include_additional_face_groups | inclure_groupes_faces_additionnels]** (*type*: list of str) List of face groups to read and register in the MED file.

lire_med_64

Inherits from: [read_med](#)

Did the same thing as Read_MED for big (64b) domain

Parameters:

- **[convertalltopoly]** (*type*: flag) Option to convert mesh with mixed cells into polyhedral/polygonal cells
- **domain | domaine** (*type*: string) Corresponds to the domain name.
- **file | fichier** (*type*: string) File (written in the MED format, with extension '.med') containing the mesh
- **[mesh | maillage]** (*type*: string) Name of the mesh in med file. If not specified, the first mesh will be read.
- **[exclude_groups | exclure_groupes]** (*type*: list of str) List of face groups to skip in the MED file.
- **[sub_zones | sous_zones]** (*type*: list of str) List of subzones to keep in the MED file and write directly in the .geo
- **[include_additional_face_groups | inclure_groupes_faces_additionnels]** (*type*: list of str) List of face groups to read and register in the MED file.

1.13.10 Keywords derived from regroupebord

regroupebord

Inherits from: [interprete](#)

Keyword to build one boundary new_bord with several boundaries of the domain named domaine.

Parameters:

- **domaine | domain** (*type*: string) Name of domain
- **new_bord** (*type*: string) Name of the new boundary
- **bords** (*type*: [bloc_lecture](#)) { Bound1 Bound2 }

regroupebord_64**Inherits from:** [regroupebord](#)

Keyword to build one boundary new_bord with several boundaries of the big (64b) domain named domaine.

Parameters:

- **domaine** | **domain** (*type*: string) Name of domain
- **new_bord** (*type*: string) Name of the new boundary
- **bords** (*type*: [bloc_lecture](#)) { Bound1 Bound2 }

1.13.11 Keywords derived from scatter**scatter****Inherits from:** [interprete](#)

Class to read a partitioned mesh from the files during a parallel calculation. The files are in binary format.

Parameters:

- **file** (*type*: string) Name of file.
- **domaine** (*type*: string) Name of domain.

scattermed**Inherits from:** [scatter](#)

This keyword will read the partition of the domain_name domain into a the MED format files file.med created by Medsplitter.

Parameters:

- **file** (*type*: string) Name of file.
- **domaine** (*type*: string) Name of domain.

1.13.12 Keywords derived from tetraedriser**tetraedriser****Inherits from:** [interprete](#)To achieve a tetrahedral mesh based on a mesh comprising blocks, the Tetraedriser (Tetrahedralise) interpreter is used in VEF discretization. Initial block is divided in 6 tetrahedra: **Parameters:**

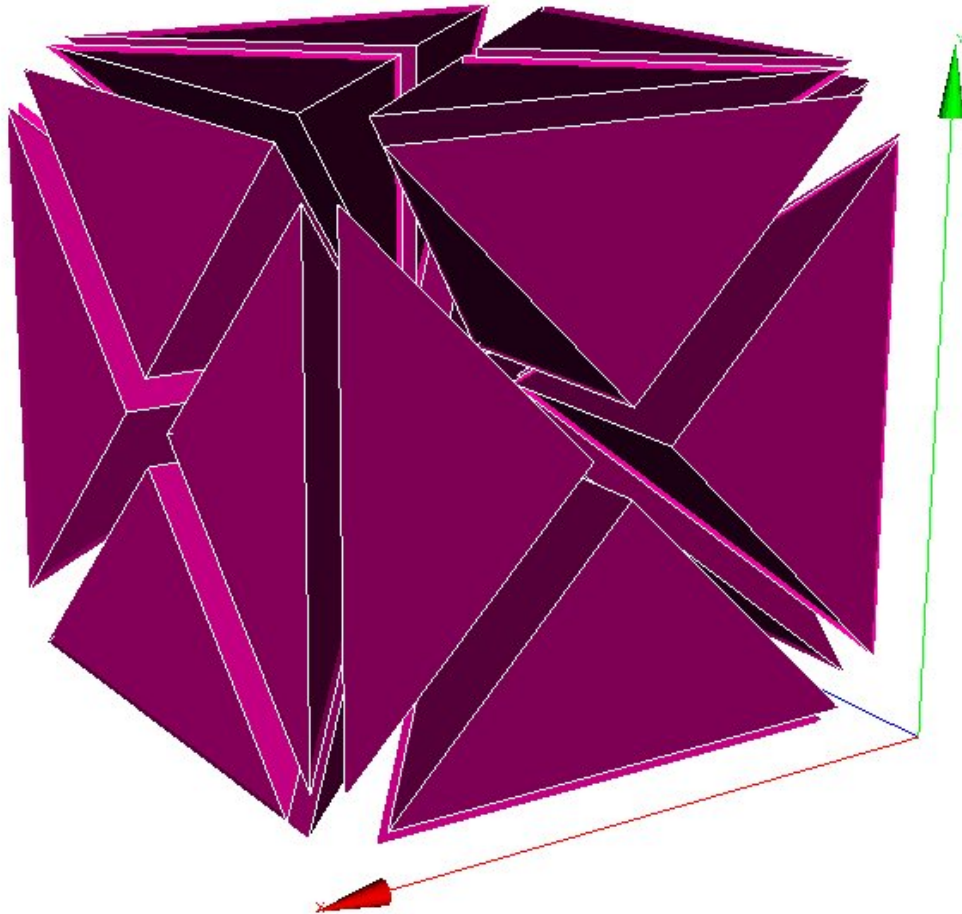
- **domain_name** (*type*: string) Name of domain.

tetraedriser_homogene**Inherits from:** [tetraedriser](#)Use the Tetraedriser_homogene (Homogeneous_Tetrahedralisation) interpreter in VEF discretization to mesh a block in tetrahedrals. Each block hexahedral is no longer divided into 6 tetrahedrals (keyword Tetraedriser (Tetrahedralise)), it is now broken down into 40 tetrahedrals. Thus a block defined with 11 nodes in each X, Y, Z direction will contain $10 \times 10 \times 10 \times 40 = 40,000$ tetrahedrals. This also allows problems in the mesh corners with the P1NC/ $\leftrightarrow$ P1iso/P1bulle or P1/P1 discretization items to be avoided. Initial block is divided in 40 tetrahedra: **Parameters:**

- **domain_name** (*type*: string) Name of domain.

tetraedriser_homogene_compact**Inherits from:** [tetraedriser](#)

This new discretization generates tetrahedral elements from cartesian or non-cartesian hexahedral elements. The process cut each hexahedral in 6 pyramids, each of them being cut then in 4 tetrahedral. So, in comparison with `tetra_homogene`, less elements (*24 instead of*40) with more homogeneous volumes are generated. Moreover, this process is done in a faster way. Initial block is divided in 24 tetrahedra:

**Figure 3 tetraedriserhomogenecompact****Parameters:**

- **domain_name** (*type: string*) Name of domain.

tetraedriser_homogene_fin**Inherits from:** [tetraedriser](#)

`Tetraedriser_homogene_fin` is the recommended option to tetrahedralise blocks. As an extension (subdivision) of `Tetraedriser_homogene_compact`, this last one cut each initial block in 48 tetrahedra (against 24, previously). This cutting ensures :

- a correct cutting in the corners (in respect to pressure discretization PreP1B),
- a better isotropy of elements than with `Tetraedriser_homogene_compact`,
- a better alignment of summits (this could have a benefit effect on calculation near walls since first elements in contact with it are all contained in the same constant thickness and ii/ by the way, a 3D cartesian grid based on summits can be engendered and used to realise spectral analysis in HIT for instance). Initial block is divided in 48 tetrahedra: 

Parameters:

- **domain_name** (*type*: string) Name of domain.

tetraedriser_par_prisme**Inherits from:** [tetraedriser](#)

Tetraedriser_par_prisme generates 6 iso-volume tetrahedral element from primary hexahedral one (contrarily to the 5 elements ordinarily generated by tetraedriser). This element is suitable for calculation of gradients at the summit (coincident with the gravity centre of the jointed elements related with) and spectra (due to a better alignment of the points).

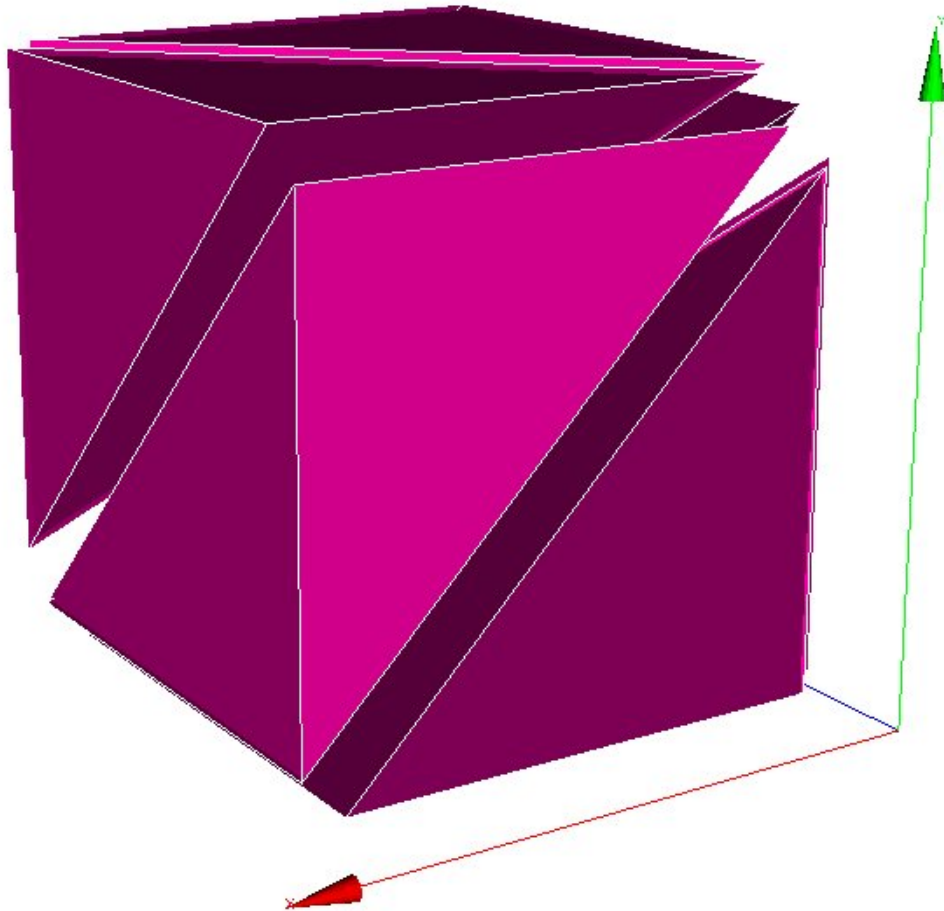


Figure 4 tetraedriserparprisme

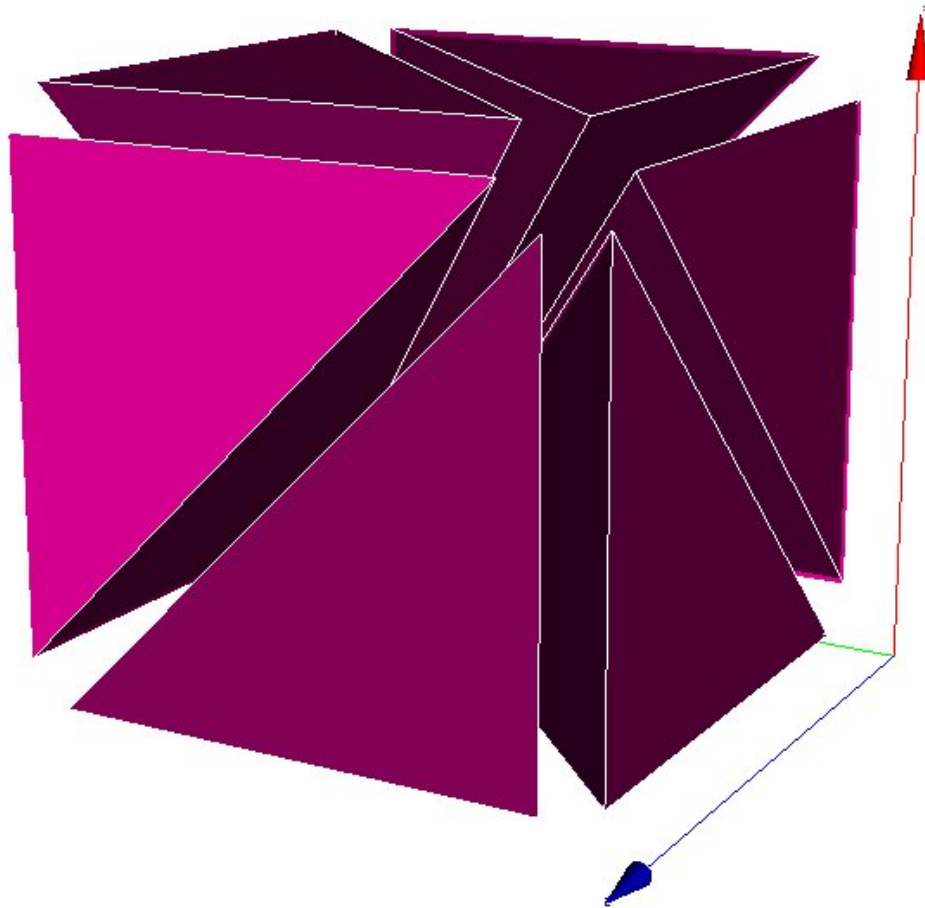


Figure 5 tetraedriserparprisme2

Initial block is divided in 6 prisms.

Parameters:

- **domain_name** (*type*: string) Name of domain.

1.13.13 Keywords derived from transformer

transformer

Inherits from: [interpret](#)

Keyword to transform the coordinates of the geometry.

Exemple to rotate your mesh by a 90o rotation and to scale the z coordinates by a factor 2: Transformer domain_↔ name -y -x 2*z

Parameters:

- **domain_name** (*type*: string) Name of domain.
- **formule** (*type*: list of str) Function_for_x Function_for_y [Function_for z]

transformer_64

Inherits from: [transformer](#)

Transformer keyword applied on a big mesh

Parameters:

- **domain_name** (*type*: string) Name of domain.
- **formule** (*type*: list of str) Function_for_x Function_for_y [Function_for z]

1.13.14 Keywords derived from triangulate

triangulate

Synonyms: `triangler`

Inherits from: `interpret`

To achieve a triangular mesh from a mesh comprising rectangles (2 triangles per rectangle). Should be used in VEF discretization. Principle:

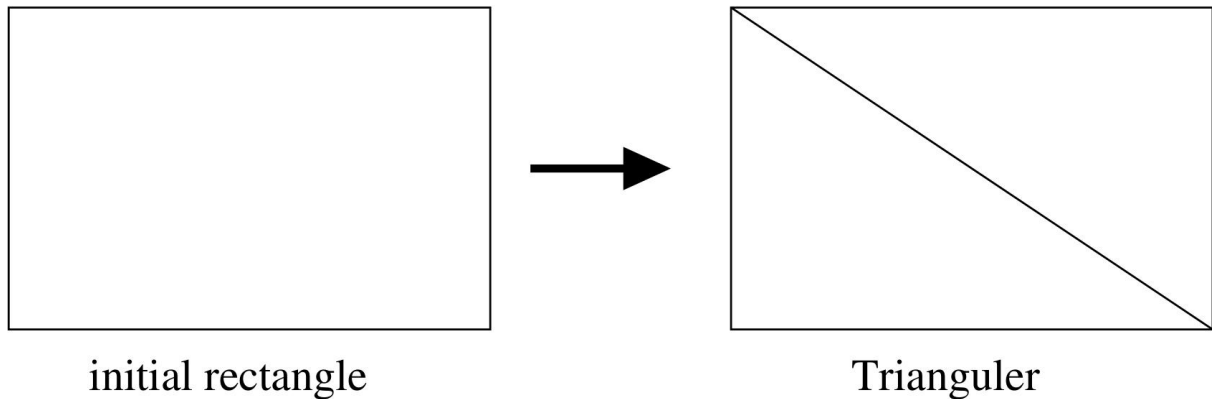


Figure 6 triangler

Parameters:

- **domain_name** (*type: string*) Name of domain.

triangler_fin

Inherits from: `triangulate`

`Triangler_fin` is the recommended option to triangulate rectangles.

As an extension (subdivision) of `Triangler_h` option, this one cut each initial rectangle in 8 triangles (against 4, previously). This cutting ensures :

- a correct cutting in the corners (in respect to pressure discretization PreP1B).
- a better isotropy of elements than with `Triangler_h` option.
- a better alignment of summits (this could have a benefit effect on calculation near walls since first elements in contact with it are all contained in the same constant thickness, and, by this way, a 2D cartesian grid based on summits can be engendered and used to realize statistical analysis in plane channel configuration for instance). Principle:

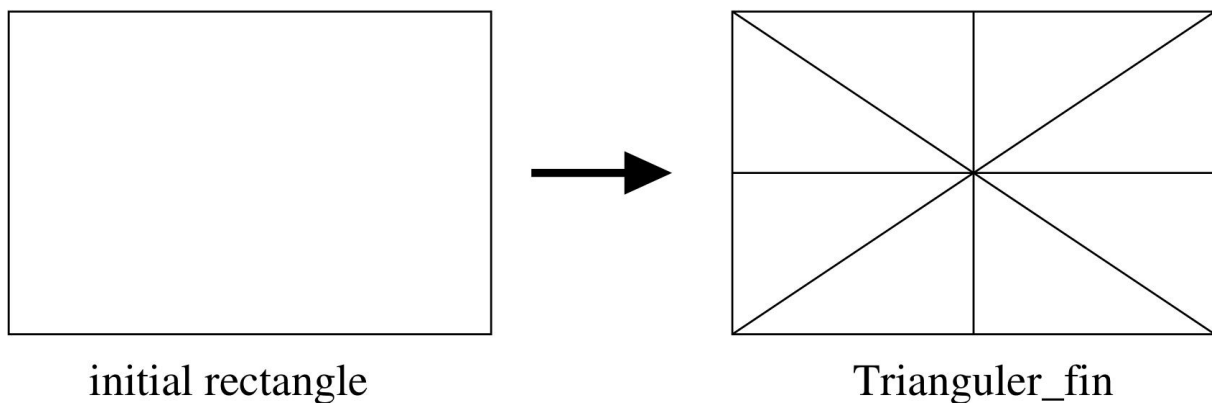


Figure 7 trianglerfin

Parameters:

- **domain_name** (*type*: string) Name of domain.

trianguler_h

Inherits from: [triangulate](#)

To achieve a triangular mesh from a mesh comprising rectangles (4 triangles per rectangle). Should be used in VEF discretization. Principle:

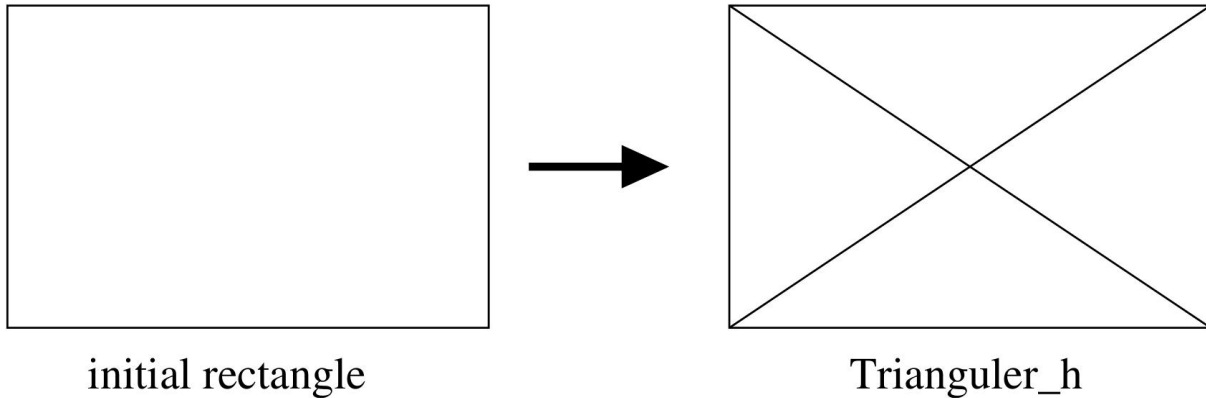


Figure 8 triangulerh

Parameters:

- **domain_name** (*type*: string) Name of domain.

1.13.15 Keywords derived from write_file

write_file

Synonyms: `ecrire_fichier`, `ecrire_fichier_bin`

Inherits from: [interpret](#)

Keyword to write the object of name `name_obj` to a file `filename`. Since the v1.6.3, the default format is now binary format file.

Parameters:

- **name_obj** (*type*: string) Name of the object to be written.
- **filename** (*type*: string) Name of the file.

ecrire_fichier_formatte

Inherits from: [write_file](#)

Keyword to write the object of name `name_obj` to a file `filename` in ASCII format.

Parameters:

- **name_obj** (*type*: string) Name of the object to be written.
- **filename** (*type*: string) Name of the file.

1.14 Keywords derived from loi_etat_base

loi_etat_base

Basic class for state laws used with a dilatable fluid.

- [Keywords derived from loi_etat_gaz_parfait_base](#)
- [Keywords derived from loi_etat_gaz_reel_base](#)
- [Keywords derived from loi_etat_tppi_base](#)

1.14.1 Keywords derived from `loi_etat_gaz_parfait_base`

`loi_etat_gaz_parfait_base`

Inherits from: [loi_etat_base](#)

Basic class for perfect gases state laws used with a dilatable fluid.

`binaire_gaz_parfait_qc`

Inherits from: [loi_etat_gaz_parfait_base](#)

Class for perfect gas binary mixtures state law used with a quasi-compressible fluid under the iso-thermal and iso-bar assumptions.

Parameters:

- **molar_mass1** (*type*: float) Molar mass of species 1 (in kg/mol).
 - **molar_mass2** (*type*: float) Molar mass of species 2 (in kg/mol).
 - **mu1** (*type*: float) Dynamic viscosity of species 1 (in kg/m.s).
 - **mu2** (*type*: float) Dynamic viscosity of species 2 (in kg/m.s).
 - **temperature** (*type*: float) Temperature (in Kelvin) which will be constant during the simulation since this state law only works for iso-thermal conditions.
 - **diffusion_coeff** (*type*: float) Diffusion coefficient assumed the same for both species (in m²/s).
-

`binaire_gaz_parfait_wc`

Inherits from: [loi_etat_gaz_parfait_base](#)

Class for perfect gas binary mixtures state law used with a weakly-compressible fluid under the iso-thermal and iso-bar assumptions.

Parameters:

- **molar_mass1** (*type*: float) Molar mass of species 1 (in kg/mol).
 - **molar_mass2** (*type*: float) Molar mass of species 2 (in kg/mol).
 - **mu1** (*type*: float) Dynamic viscosity of species 1 (in kg/m.s).
 - **mu2** (*type*: float) Dynamic viscosity of species 2 (in kg/m.s).
 - **temperature** (*type*: float) Temperature (in Kelvin) which will be constant during the simulation since this state law only works for iso-thermal conditions.
 - **diffusion_coeff** (*type*: float) Diffusion coefficient assumed the same for both species (in m²/s).
-

`multi_gaz_parfait_qc`

Inherits from: [loi_etat_gaz_parfait_base](#)

Class for perfect gas multi-species mixtures state law used with a quasi-compressible fluid.

Parameters:

- **sc** (*type*: float) Schmidt number of the gas $Sc = \nu/D$ (D : diffusion coefficient of the mixing).
 - **prandtl** (*type*: float) Prandtl number of the gas $Pr = \mu * Cp / \lambda$
 - **[cp]** (*type*: float) Specific heat at constant pressure of the gas C_p .
 - **[dtol_fraction]** (*type*: float) Delta tolerance on mass fractions for check testing (default value 1.e-6).
 - **[correction_fraction]** (*type*: flag) To force mass fractions between 0. and 1.
 - **[ignore_check_fraction]** (*type*: flag) Not to check if mass fractions between 0. and 1.
-

multi_gaz_parfait_wc**Inherits from:** [loi_etat_gaz_parfait_base](#)

Class for perfect gas multi-species mixtures state law used with a weakly-compressible fluid.

Parameters:

- **species_number** (*type*: int) Number of species you are considering in your problem.
 - **diffusion_coeff** (*type*: [field_base](#)) Diffusion coefficient of each species, defined with a Champ_uniforme of dimension equals to the species_number.
 - **molar_mass** (*type*: [field_base](#)) Molar mass of each species, defined with a Champ_uniforme of dimension equals to the species_number.
 - **mu** (*type*: [field_base](#)) Dynamic viscosity of each species, defined with a Champ_uniforme of dimension equals to the species_number.
 - **cp** (*type*: [field_base](#)) Specific heat at constant pressure of the gas Cp, defined with a Champ_uniforme of dimension equals to the species_number..
 - **prandtl** (*type*: float) Prandtl number of the gas $Pr = \mu * Cp / \lambda$.
-

perfect_gaz_qc**Synonyms:** [gaz_parfait_qc](#)**Inherits from:** [loi_etat_gaz_parfait_base](#)

Class for perfect gas state law used with a quasi-compressible fluid.

Parameters:

- **cp** (*type*: float) Specific heat at constant pressure (J/kg/K).
 - **[cv]** (*type*: float) Specific heat at constant volume (J/kg/K).
 - **[gamma]** (*type*: float) Cp/Cv
 - **prandtl** (*type*: float) Prandtl number of the gas $Pr = \mu * Cp / \lambda$
 - **[rho_constant_pour_debug]** (*type*: [field_base](#)) For developers to debug the code with a constant rho.
-

perfect_gaz_wc**Synonyms:** [gaz_parfait_wc](#)**Inherits from:** [loi_etat_gaz_parfait_base](#)

Class for perfect gas state law used with a weakly-compressible fluid.

Parameters:

- **cp** (*type*: float) Specific heat at constant pressure (J/kg/K).
 - **[cv]** (*type*: float) Specific heat at constant volume (J/kg/K).
 - **[gamma]** (*type*: float) Cp/Cv
 - **prandtl** (*type*: float) Prandtl number of the gas $Pr = \mu * Cp / \lambda$
-

rho_t_gaz_parfait_qc**Inherits from:** [loi_etat_gaz_parfait_base](#)Class for perfect gas used with a quasi-compressible fluid where the state equation is defined as $\rho = f(T)$.**Parameters:**

- **cp** (*type*: float) Specific heat at constant pressure of the gas Cp.
- **[prandtl]** (*type*: float) Prandtl number of the gas $Pr = \mu * Cp / \lambda$
- **[rho_xyz]** (*type*: [field_base](#)) Defined with a Champ_Fonc_xyz to define a constant rho with time (space dependent)
- **[rho_t]** (*type*: string) Expression of T used to calculate rho. This can lead to a variable rho, both in space and in time.
- **[tmin_for_exit]** (*type*: float) If temperature goes below Tmin_for_exit (default value -1000), computation will stop.

1.14.2 Keywords derived from loi_etat_gaz_reel_base**loi_etat_gaz_reel_base****Inherits from:** [loi_etat_base](#)

Basic class for real gases state laws used with a dilatable fluid.

rho_t_gaz_reel_qc**Inherits from:** [loi_etat_gaz_reel_base](#)

Class for real gas state law used with a quasi-compressible fluid.

Parameters:

- **bloc** (*type*: [bloc_lecture](#)) Description.

1.14.3 Keywords derived from loi_etat_tppi_base**loi_etat_tppi_base****Inherits from:** [loi_etat_base](#)

Basic class for thermo-physical properties interface (TPPI) used for dilatable problems

coolprop_qc**Inherits from:** [loi_etat_tppi_base](#)

Class for using CoolProp with QC problem

Parameters:

- **cp** (*type*: float) Specific heat at constant pressure (J/kg/K).
- **fluid** (*type*: string) Fluid name in the CoolProp model
- **model** (*type*: string) CoolProp model name

coolprop_wc**Inherits from:** [loi_etat_tppi_base](#)

Class for using CoolProp with WC problem

Parameters:

- **cp** (*type*: float) Specific heat at constant pressure (J/kg/K).
- **fluid** (*type*: string) Fluid name in the CoolProp model
- **model** (*type*: string) CoolProp model name

eos_qc**Inherits from:** [loi_etat_tppi_base](#)

Class for using EOS with QC problem

Parameters:

- **cp** (*type*: float) Specific heat at constant pressure (J/kg/K).
 - **fluid** (*type*: string) Fluid name in the EOS model
 - **model** (*type*: string) EOS model name
-

eos_wc**Inherits from:** [loi_etat_tppi_base](#)

Class for using EOS with WC problem

Parameters:

- **cp** (*type*: float) Specific heat at constant pressure (J/kg/K).
- **fluid** (*type*: string) Fluid name in the EOS model
- **model** (*type*: string) EOS model name

1.15 Keywords derived from loi_fermeture_base**loi_fermeture_base**

Class for appends fermeture to problem

loi_fermeture_test**Inherits from:** [loi_fermeture_base](#)

Loi for test only

Parameters:

- **[coef]** (*type*: float) coefficient

1.16 Keywords derived from loi_horaire**loi_horaire**

to define the movement with a time-dependant law for the solid interface.

Parameters:

- **[position]** (*type*: list of str) Vecteur position (default: zero vector of length `dimension`)
- **[vitesse]** (*type*: list of str) Vecteur vitesse (default: zero vector of length `dimension`)
- **[rotation]** (*type*: list of str) Matrice de passage
- **[derivee_rotation]** (*type*: list of str) Derivee matrice de passage
- **[verification_derivee]** (*type*: int) not_set
- **[impr]** (*type*: int) Whether to print output

1.17 Keywords derived from milieu_base

milieu_base

Basic class for medium (physics properties of medium).

Parameters:

- **[rho]** (*type*: [field_base](#)) Density (kg.m-3).
- **[lambda_ | lambda]** (*type*: [field_base](#)) Conductivity (W.m-1.K-1).
- **[cp]** (*type*: [field_base](#)) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (*type*: [field_base](#)) Hydraulic diameter field (optional).
- **[porosites_champ]** (*type*: [field_base](#)) The porosity is given at each element and the porosity at each face, $\Psi(\text{face})$, is calculated by the average of the porosities of the two neighbour elements $\Psi(\text{elem1})$, $\Psi(\text{elem2})$: $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$. This keyword is optional.
- **[porosites]** (*type*: [porosites](#)) Porosities.
- **[gravite]** (*type*: [field_base](#)) Gravity field (optional).

-
- [Keywords derived from fluide_base](#)

constituant

Inherits from: [milieu_base](#)

Constituent.

Parameters:

- **coefficient_diffusion** (*type*: [field_base](#)) Constituent diffusion coefficient value (m².s-1). If a multi-constituent problem is being processed, the diffusivity will be a vectorial and each components will be the diffusion of the constituent.
- **[is_multi_scalar | is_multi_scalar_diffusion]** (*type*: flag) Flag to activate the multi_scalar diffusion operator
- **[rho]** (*type*: [field_base](#)) Density (kg.m-3).
- **[lambda_ | lambda]** (*type*: [field_base](#)) Conductivity (W.m-1.K-1).
- **[cp]** (*type*: [field_base](#)) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (*type*: [field_base](#)) Hydraulic diameter field (optional).
- **[porosites_champ]** (*type*: [field_base](#)) The porosity is given at each element and the porosity at each face, $\Psi(\text{face})$, is calculated by the average of the porosities of the two neighbour elements $\Psi(\text{elem1})$, $\Psi(\text{elem2})$: $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$. This keyword is optional.
- **[porosites]** (*type*: [porosites](#)) Porosities.
- **[gravite]** (*type*: [field_base](#)) Gravity field (optional).

espece

Inherits from: [milieu_base](#)

not_set

Parameters:

- **mu** (*type*: [field_base](#)) Species dynamic viscosity value (kg.m-1.s-1).
- **cp** (*type*: [field_base](#)) Species specific heat value (J.kg-1.K-1).
- **masse_molaire** (*type*: float) Species molar mass.
- **[lambda_ | lambda]** (*type*: [field_base](#)) Conductivity (W.m-1.K-1).

milieu_elasticite**Inherits from:** milieu_base

Milieu_Elasticite.

Parameters:

- **[rho]** (type: field_base) Density (kg.m-3).
- **[e]** (type: field_base) Field E
- **[nu]** (type: field_base) Field nu.
- **[alpha]** (type: field_base) Coeff dilatation.
- **[lambda_ | lambda]** (type: field_base) Conductivity (W.m-1.K-1).
- **[cp]** (type: field_base) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (type: field_base) Hydraulic diameter field (optional).
- **[porosites_champ]** (type: field_base) The porosity is given at each element and the porosity at each face, Psi(face), is calculated by the average of the porosities of the two neighbour elements Psi(elem1), Psi(elem2) : $\text{Psi}(\text{face}) = 2 / (1/\text{Psi}(\text{elem1}) + 1/\text{Psi}(\text{elem2}))$. This keyword is optional.
- **[porosites]** (type: porosites) Porosities.
- **[gravite]** (type: field_base) Gravity field (optional).

solide**Inherits from:** milieu_base

Solid with cp and/or rho non-uniform.

Parameters:

- **[rho]** (type: field_base) Density (kg.m-3).
- **[cp]** (type: field_base) Specific heat (J.kg-1.K-1).
- **[lambda_ | lambda]** (type: field_base) Conductivity (W.m-1.K-1).
- **[user_field]** (type: field_base) user defined field.
- **[diametre_hyd_champ]** (type: field_base) Hydraulic diameter field (optional).
- **[porosites_champ]** (type: field_base) The porosity is given at each element and the porosity at each face, Psi(face), is calculated by the average of the porosities of the two neighbour elements Psi(elem1), Psi(elem2) : $\text{Psi}(\text{face}) = 2 / (1/\text{Psi}(\text{elem1}) + 1/\text{Psi}(\text{elem2}))$. This keyword is optional.
- **[porosites]** (type: porosites) Porosities.
- **[gravite]** (type: field_base) Gravity field (optional).

1.17.1 Keywords derived from fluide_base**fluide_base****Inherits from:** milieu_base

Basic class for fluids.

Parameters:

- **[indice]** (type: field_base) Refractivity of fluid.
- **[kappa]** (type: field_base) Absorptivity of fluid (m-1).
- **[rho]** (type: field_base) Density (kg.m-3).

- **[lambda_ | lambda]** (type: [field_base](#)) Conductivity (W.m-1.K-1).
- **[cp]** (type: [field_base](#)) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (type: [field_base](#)) Hydraulic diameter field (optional).
- **[porosites_champ]** (type: [field_base](#)) The porosity is given at each element and the porosity at each face, $\Psi(\text{face})$, is calculated by the average of the porosities of the two neighbour elements $\Psi(\text{elem1})$, $\Psi(\text{elem2})$: $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$. This keyword is optional.
- **[porosites]** (type: [porosites](#)) Porosities.
- **[gravite]** (type: [field_base](#)) Gravity field (optional).

- [Keywords derived from fluide_incompressible](#)

fluide_quasi_compressible

Inherits from: [fluide_base](#)

Quasi-compressible flow with a low mach number assumption; this means that the thermo- dynamic pressure (used in state law) is uniform in space.

Parameters:

- **[sutherland]** (type: [bloc_sutherland](#)) Sutherland law for viscosity and for conductivity.
- **[pression]** (type: float) Initial thermo-dynamic pressure used in the associated state law.
- **[loi_etat]** (type: [loi_etat_base](#)) The state law that will be associated to the Quasi-compressible fluid.
- **[traitement_pth]** (type: string into ['edo', 'constant', 'conservation_masse']) Particular treatment for the thermodynamic pressure Pth ; there are three possibilities: 1) with the keyword 'edo' the code computes Pth solving an O.D.E. ; in this case, the mass is not strictly conserved (it is the default case for quasi compressible computation): 2) the keyword 'conservation_masse' forces the conservation of the mass (closed geometry or with periodic boundaries condition) 3) the keyword 'constant' makes it possible to have a constant Pth ; it's the good choice when the flow is open (e.g. with pressure boundary conditions). It is possible to monitor the volume averaged value for temperature and density, plus Pth evolution in the .evol_glob file.
- **[traitement_rho_gravite]** (type: string into ['standard', 'moins_rho_moyen']) It may be :1) 'standard' : the gravity term is evaluated with $\rho * g$ (It is the default). 2) 'moins_rho_moyen' : the gravity term is evaluated with $(\rho - \rho_{\text{moy}}) * g$. *Unknown pressure is then $P = P + \rho_{\text{moy}} * g * z$.* It is useful when you apply uniforme pressure boundary condition like $P^* = 0$.
- **[temps_debut_prise_en_compte_drho_dt]** (type: float) While time < value, $d\rho/dt$ is set to zero (Rho, volumic mass). Useful for some calculation during the first time steps with big variation of temperature and volumic mass.
- **[omega_relaxation_drho_dt]** (type: float) Optional option to have a relaxed algorithm to solve the mass equation. value is used (1 per default) to specify omega.
- **[lambda_ | lambda]** (type: [field_base](#)) Conductivity (W.m-1.K-1).
- **[mu]** (type: [field_base](#)) Dynamic viscosity (kg.m-1.s-1).
- **[indice]** (type: [field_base](#)) Refractivity of fluid.
- **[kappa]** (type: [field_base](#)) Absorptivity of fluid (m-1).
- **[rho]** (type: [field_base](#)) Density (kg.m-3).
- **[cp]** (type: [field_base](#)) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (type: [field_base](#)) Hydraulic diameter field (optional).
- **[porosites_champ]** (type: [field_base](#)) The porosity is given at each element and the porosity at each face, $\Psi(\text{face})$, is calculated by the average of the porosities of the two neighbour elements $\Psi(\text{elem1})$, $\Psi(\text{elem2})$: $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$. This keyword is optional.

- **[porosites]** (type: [porosites](#)) Porosities.
- **[gravite]** (type: [field_base](#)) Gravity field (optional).

fluide_sodium_gaz

Inherits from: [fluide_base](#)

Class for Fluide_sodium_gaz (real fluid)

Parameters:

- **[p_ref]** (type: float) Use to set the pressure value in the closure law. If not specified, the value of the pressure unknown will be used
- **[t_ref]** (type: float) Use to set the temperature value in the closure law. If not specified, the value of the temperature unknown will be used
- **[indice]** (type: [field_base](#)) Refractivity of fluid.
- **[kappa]** (type: [field_base](#)) Absorptivity of fluid (m-1).
- **[rho]** (type: [field_base](#)) Density (kg.m-3).
- **[lambda_ | lambda]** (type: [field_base](#)) Conductivity (W.m-1.K-1).
- **[cp]** (type: [field_base](#)) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (type: [field_base](#)) Hydraulic diameter field (optional).
- **[porosites_champ]** (type: [field_base](#)) The porosity is given at each element and the porosity at each face, $\Psi(\text{face})$, is calculated by the average of the porosities of the two neighbour elements $\Psi(\text{elem1})$, $\Psi(\text{elem2})$: $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$. This keyword is optional.
- **[porosites]** (type: [porosites](#)) Porosities.
- **[gravite]** (type: [field_base](#)) Gravity field (optional).

fluide_sodium_liquide

Inherits from: [fluide_base](#)

Class for Fluide_sodium_liquide (real fluid)

Parameters:

- **[p_ref]** (type: float) Use to set the pressure value in the closure law. If not specified, the value of the pressure unknown will be used
- **[t_ref]** (type: float) Use to set the temperature value in the closure law. If not specified, the value of the temperature unknown will be used
- **[indice]** (type: [field_base](#)) Refractivity of fluid.
- **[kappa]** (type: [field_base](#)) Absorptivity of fluid (m-1).
- **[rho]** (type: [field_base](#)) Density (kg.m-3).
- **[lambda_ | lambda]** (type: [field_base](#)) Conductivity (W.m-1.K-1).
- **[cp]** (type: [field_base](#)) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (type: [field_base](#)) Hydraulic diameter field (optional).
- **[porosites_champ]** (type: [field_base](#)) The porosity is given at each element and the porosity at each face, $\Psi(\text{face})$, is calculated by the average of the porosities of the two neighbour elements $\Psi(\text{elem1})$, $\Psi(\text{elem2})$: $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$. This keyword is optional.
- **[porosites]** (type: [porosites](#)) Porosities.
- **[gravite]** (type: [field_base](#)) Gravity field (optional).

fluide_stiffened_gas**Inherits from:** [fluide_base](#)

Class for Stiffened Gas

Parameters:

- **[gamma]** (*type:* float) Heat capacity ratio (Cp/Cv)
- **[pinf]** (*type:* float) Stiffened gas pressure constant (if set to zero, the state law becomes identical to that of perfect gases)
- **[mu]** (*type:* float) Dynamic viscosity
- **[lambda_ | lambda]** (*type:* float) Thermal conductivity
- **[cv]** (*type:* float) Thermal capacity at constant volume
- **[q]** (*type:* float) Reference energy
- **[q_prim]** (*type:* float) Model constant
- **[indice]** (*type:* [field_base](#)) Refractivity of fluid.
- **[kappa]** (*type:* [field_base](#)) Absorptivity of fluid (m-1).
- **[rho]** (*type:* [field_base](#)) Density (kg.m-3).
- **[cp]** (*type:* [field_base](#)) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (*type:* [field_base](#)) Hydraulic diameter field (optional).
- **[porosites_champ]** (*type:* [field_base](#)) The porosity is given at each element and the porosity at each face, Psi(face), is calculated by the average of the porosities of the two neighbour elements Psi(elem1), Psi(elem2) : Psi(face)=2/(1/Psi(elem1)+1/Psi(elem2)). This keyword is optional.
- **[porosites]** (*type:* [porosites](#)) Porosities.
- **[gravite]** (*type:* [field_base](#)) Gravity field (optional).

fluide_weakly_compressible**Inherits from:** [fluide_base](#)

Weakly-compressible flow with a low mach number assumption; this means that the thermo- dynamic pressure (used in state law) can vary in space.

Parameters:

- **[loi_etat]** (*type:* [loi_etat_base](#)) The state law that will be associated to the Weakly-compressible fluid.
- **[sutherland]** (*type:* [bloc_sutherland](#)) Sutherland law for viscosity and for conductivity.
- **[traitement_pth]** (*type:* string into ['constant']) Particular treatment for the thermodynamic pressure Pth ; there is currently one possibility: 1) the keyword 'constant' makes it possible to have a constant Pth but not uniform in space ; it's the good choice when the flow is open (e.g. with pressure boundary conditions).
- **[lambda_ | lambda]** (*type:* [field_base](#)) Conductivity (W.m-1.K-1).
- **[mu]** (*type:* [field_base](#)) Dynamic viscosity (kg.m-1.s-1).
- **[pression_thermo]** (*type:* float) Initial thermo-dynamic pressure used in the associated state law.
- **[pression_xyz]** (*type:* [field_base](#)) Initial thermo-dynamic pressure used in the associated state law. It should be defined with as a Champ_Fonc_xyz.
- **[use_total_pressure]** (*type:* int) Flag (0 or 1) used to activate and use the total pressure in the associated state law. The default value of this Flag is 0.

- **[use_hydrostatic_pressure]** (*type: int*) Flag (0 or 1) used to activate and use the hydro-static pressure in the associated state law. The default value of this Flag is 0.
- **[use_grad_pression_eos]** (*type: int*) Flag (0 or 1) used to specify whether or not the gradient of the thermodynamic pressure will be taken into account in the source term of the temperature equation (case of a non-uniform pressure). The default value of this Flag is 1 which means that the gradient is used in the source.
- **[time_activate_ptot]** (*type: float*) Time (in seconds) at which the total pressure will be used in the associated state law.
- **[indice]** (*type: field_base*) Refractivity of fluid.
- **[kappa]** (*type: field_base*) Absorptivity of fluid (m-1).
- **[rho]** (*type: field_base*) Density (kg.m-3).
- **[cp]** (*type: field_base*) Specific heat (J.kg-1.K-1).
- **[diametre_hyd_champ]** (*type: field_base*) Hydraulic diameter field (optional).
- **[porosites_champ]** (*type: field_base*) The porosity is given at each element and the porosity at each face, $\Psi(\text{face})$, is calculated by the average of the porosities of the two neighbour elements $\Psi(\text{elem1})$, $\Psi(\text{elem2})$: $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$. This keyword is optional.
- **[porosites]** (*type: porosites*) Porosities.
- **[gravite]** (*type: field_base*) Gravity field (optional).

1.17.1.1 Keywords derived from fluide_incompressible

fluide_incompressible

Inherits from: [fluide_base](#)

Class for non-compressible fluids.

Parameters:

- **[beta_th]** (*type: field_base*) Thermal expansion (K-1).
- **[mu]** (*type: field_base*) Dynamic viscosity (kg.m-1.s-1).
- **[beta_co]** (*type: field_base*) Volume expansion coefficient values in concentration.
- **[rho]** (*type: field_base*) Density (kg.m-3).
- **[cp]** (*type: field_base*) Specific heat (J.kg-1.K-1).
- **[lambda_ | lambda]** (*type: field_base*) Conductivity (W.m-1.K-1).
- **[porosites]** (*type: bloc_lecture*) Porosity (optional)
- **[indice]** (*type: field_base*) Refractivity of fluid.
- **[kappa]** (*type: field_base*) Absorptivity of fluid (m-1).
- **[diametre_hyd_champ]** (*type: field_base*) Hydraulic diameter field (optional).
- **[porosites_champ]** (*type: field_base*) The porosity is given at each element and the porosity at each face, $\Psi(\text{face})$, is calculated by the average of the porosities of the two neighbour elements $\Psi(\text{elem1})$, $\Psi(\text{elem2})$: $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$. This keyword is optional.
- **[gravite]** (*type: field_base*) Gravity field (optional).

fluide_ostwald

Inherits from: [fluide_incompressible](#)

Non-Newtonian fluids governed by Ostwald's law. The law applicable to stress tensor is:

$$\tau = K(T) * (D:D/2)^{n/2} * D$$

Where:

D refers to the deformation tensor

K refers to fluid consistency (may be a function of the temperature T)

n refers to the fluid structure index $n=1$ for a Newtonian fluid, $n<1$ for a rheofluidifier fluid, $n>1$ for a rheothickening fluid.

Parameters:

- **[k]** (type: [field_base](#)) Fluid consistency.
- **[n]** (type: [field_base](#)) Fluid structure index.
- **[beta_th]** (type: [field_base](#)) Thermal expansion (K-1).
- **[mu]** (type: [field_base](#)) Dynamic viscosity (kg.m-1.s-1).
- **[beta_co]** (type: [field_base](#)) Volume expansion coefficient values in concentration.
- **[rho]** (type: [field_base](#)) Density (kg.m-3).
- **[cp]** (type: [field_base](#)) Specific heat (J.kg-1.K-1).
- **[lambda_ | lambda]** (type: [field_base](#)) Conductivity (W.m-1.K-1).
- **[porosites]** (type: [bloc_lecture](#)) Porosity (optional)
- **[indice]** (type: [field_base](#)) Refractivity of fluid.
- **[kappa]** (type: [field_base](#)) Absorptivity of fluid (m-1).
- **[diametre_hyd_champ]** (type: [field_base](#)) Hydraulic diameter field (optional).
- **[porosites_champ]** (type: [field_base](#)) The porosity is given at each element and the porosity at each face, Psi(face), is calculated by the average of the porosities of the two neighbour elements Psi(elem1), Psi(elem2) : Psi(face)=2/(1/Psi(elem1)+1/Psi(elem2)). This keyword is optional.
- **[gravite]** (type: [field_base](#)) Gravity field (optional).

1.18 Keywords derived from modele_turbulence_scal_base

modele_turbulence_scal_base

Basic class for turbulence model for energy equation.

Parameters:

- **[dt_impr_nusselt]** (type: float) Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the _Nusselt.face file each dt_impr_nusselt time period. The local Nusselt expression is as follows : $Nu = ((\lambda + \lambda_t)/\lambda) * d_{wall}/d_{eq}$ where d_{wall} is the distance from the first mesh to the wall and d_{eq} is given by the wall law. This option also gives the value of d_{eq} and $h = (\lambda + \lambda_t)/d_{eq}$ and the fluid temperature of the first mesh near the wall. For the Neumann boundary conditions (flux_impose), the <<equivalent>> wall temperature given by the wall law is also printed (Tparoi equiv.) preceded for VEF calculation by the edge temperature <<T face de bord>>.
- **[dt_impr_nusselt_mean_only]** (type: [dt_impr_nusselt_mean_only](#)) This keyword is used to print the mean values of Nusselt (obtained with the wall laws) on each boundary, into a file named datafile_ProblemName<-_nusselt_mean_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb_boundaries which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.
- **[turbulence_paroi]** (type: [turbulence_paroi_scalaire_base](#)) Keyword to set the wall law.

modele_turbulence_scal_null**Synonyms:** null**Inherits from:** [modele_turbulence_scal_base](#)

Null scalar turbulence model (turbulent diffusivity = 0) which can be used with a turbulent problem.

Parameters:

- **[dt_impr_nusselt]** (*type:* float) Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the _Nusselt.face file each dt_impr_nusselt time period. The local Nusselt expression is as follows : $Nu = ((\lambda + \lambda_t)/\lambda) * d_{wall}/d_{eq}$ where d_{wall} is the distance from the first mesh to the wall and d_{eq} is given by the wall law. This option also gives the value of d_{eq} and $h = (\lambda + \lambda_t)/d_{eq}$ and the fluid temperature of the first mesh near the wall. For the Neumann boundary conditions (flux_impose), the <<equivalent>> wall temperature given by the wall law is also printed (Tparoi equiv.) preceded for VEF calculation by the edge temperature <<T face de bord>>.
- **[dt_impr_nusselt_mean_only]** (*type:* [dt_impr_nusselt_mean_only](#)) This keyword is used to print the mean values of Nusselt (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName↵_nusselt_mean_only.out`. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb_boundaries which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.

prandtl**Inherits from:** [modele_turbulence_scal_base](#)

The Prandtl model. For the scalar equations, only the model based on Reynolds analogy is available. If K_Epsilon was selected in the hydraulic equation, Prandtl must be selected for the convection-diffusion temperature equation coupled to the hydraulic equation and Schmidt for the concentration equations.

Parameters:

- **[prdt]** (*type:* string) Keyword to modify the constant (Prdt) of Prandtl model : $\text{Alphat} = \text{Nut}/\text{Prdt}$ Default value is 0.9
- **[prandt_turbulent_fonction_nu_t_alpha]** (*type:* string) Optional keyword to specify turbulent diffusivity (by default, $\alpha_t = \text{nu}_t/\text{Prt}$) with another formulae, for example: $\alpha_t = \text{nu}_t^2/(0.7 * \alpha + 0.85 * \text{nu}_t)$ with the string $\text{nu}_t * \text{nu}_t/(0.7 * \alpha + 0.85 * \text{nu}_t)$ where alpha is the thermal diffusivity.
- **[dt_impr_nusselt]** (*type:* float) Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the _Nusselt.face file each dt_impr_nusselt time period. The local Nusselt expression is as follows : $Nu = ((\lambda + \lambda_t)/\lambda) * d_{wall}/d_{eq}$ where d_{wall} is the distance from the first mesh to the wall and d_{eq} is given by the wall law. This option also gives the value of d_{eq} and $h = (\lambda + \lambda_t)/d_{eq}$ and the fluid temperature of the first mesh near the wall. For the Neumann boundary conditions (flux_impose), the <<equivalent>> wall temperature given by the wall law is also printed (Tparoi equiv.) preceded for VEF calculation by the edge temperature <<T face de bord>>.
- **[dt_impr_nusselt_mean_only]** (*type:* [dt_impr_nusselt_mean_only](#)) This keyword is used to print the mean values of Nusselt (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName↵_nusselt_mean_only.out`. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb_boundaries which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.
- **[turbulence_paroil]** (*type:* [turbulence_paroil_scalaire_base](#)) Keyword to set the wall law.

schmidt**Inherits from:** [modele_turbulence_scal_base](#)

The Schmidt model. For the scalar equations, only the model based on Reynolds analogy is available. If K_Epsilon was selected in the hydraulic equation, Schmidt must be selected for the convection-diffusion temperature equation coupled to the hydraulic equation and Schmidt for the concentration equations.

Parameters:

- **[scturb]** (*type: float*) Keyword to modify the constant (Sct) of Schmidt model : $Dt = \text{Nut} / \text{Sct}$ Default value is 0.7.
- **[dt_impr_nusselt]** (*type: float*) Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the `_Nusselt.face` file each `dt_impr_nusselt` time period. The local Nusselt expression is as follows : $Nu = ((\lambda + \lambda_t) / \lambda) * d_{\text{wall}} / d_{\text{eq}}$ where d_{wall} is the distance from the first mesh to the wall and d_{eq} is given by the wall law. This option also gives the value of d_{eq} and $h = (\lambda + \lambda_t) / d_{\text{eq}}$ and the fluid temperature of the first mesh near the wall. For the Neumann boundary conditions (flux_impose), the `<<equivalent>>` wall temperature given by the wall law is also printed (Tparoi equiv.) preceded for VEF calculation by the edge temperature `<<T face de bord>>`.
- **[dt_impr_nusselt_mean_only]** (*type: dt_impr_nusselt_mean_only*) This keyword is used to print the mean values of Nusselt (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_↔_nusselt_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.
- **[turbulence_paroil]** (*type: turbulence_paroil_scalaire_base*) Keyword to set the wall law.

1.19 Keywords derived from mor_eqn

mor_eqn

Class of equation pieces (morceaux d'équation).

- [Keywords derived from eqn_base](#)

1.19.1 Keywords derived from eqn_base

eqn_base

Inherits from: [mor_eqn](#)

Basic class for equations.

Parameters:

- **[disable_equation_residual]** (*type: int*) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type: bloc_convection*) Keyword to alter the convection scheme.
- **[diffusion]** (*type: bloc_diffusion*) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type: list of Condlimlu*) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type: list of Condinit*) Initial conditions.
- **[sources]** (*type: list of Source_base*) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type: ecrire_fichier_xyz_valeur*) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type: parametre_equation_base*) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type: string*) The equation will not be solved while condition(t) is verified if `equation_non_resolue` keyword is used. Exemple: The Navier-Stokes equations are not solved between time `t0` and `t1`. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`

- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

- [Keywords derived from conduction](#)
- [Keywords derived from cons_euler](#)
- [Keywords derived from convection_diffusion_chaleur_qc](#)
- [Keywords derived from convection_diffusion_concentration](#)
- [Keywords derived from convection_diffusion_espece_binaire_qc](#)
- [Keywords derived from convection_diffusion_temperature](#)
- [Keywords derived from navier_stokes_standard](#)

aire_interfaciale

Synonyms: [interfacial_area](#)

Inherits from: [eqn_base](#)

Interfacial-area transport equation for a multi-phase problem (available in TrioCFD)

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_chaleur_wc

Inherits from: [eqn_base](#)

Temperature equation for a weakly-compressible fluid.

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.

- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_espece_binaire_wc

Inherits from: [eqn_base](#)

Species conservation equation for a binary weakly-compressible fluid.

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limtes | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_espece_multi_qc

Inherits from: [eqn_base](#)

Species conservation equation for a multi-species quasi-compressible fluid.

Parameters:

- **[espece]** (*type*: [espece](#)) Associate a species (with its properties) to the equation
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limtes | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.

- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_espece_multi_turbulent_qc

Inherits from: [eqn_base](#)

not_set

Parameters:

- **[modele_turbulence]** (*type*: [modele_turbulence_scal_base](#)) Turbulence model to be used.
- **[espece]** (*type*: [espece](#)) not_set
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_espece_multi_wc

Inherits from: [eqn_base](#)

Species conservation equation for a multi-species weakly-compressible fluid.

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.

- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_temperature_ibm_turbulent

Inherits from: [eqn_base](#)

IBM Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.

Parameters:

- **[modele_turbulence]** (*type*: [modele_turbulence_scal_base](#)) Turbulence model for the energy equation.
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_temperature_turbulent

Inherits from: [eqn_base](#)

Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.

Parameters:

- **[modele_turbulence]** (*type*: [modele_turbulence_scal_base](#)) Turbulence model for the energy equation.
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.

- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

echelle_temporelle_turbulente

Inherits from: [eqn_base](#)

Turbulent Dissipation time scale equation for a turbulent mono/multi-phase problem (available in TrioCFD)

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limtes | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

energie_cinetique_turbulente

Inherits from: [eqn_base](#)

Turbulent kinetic Energy conservation equation for a turbulent mono/multi-phase problem (available in TrioCFD)

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limtes | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file

- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

energie_cinetique_turbulente_wit

Inherits from: [eqn_base](#)

Bubble Induced Turbulent kinetic Energy equation for a turbulent multi-phase problem (available in TrioCFD)

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

energie_multiphase

Inherits from: [eqn_base](#)

Internal energy conservation equation for a multi-phase problem where the unknown is the temperature

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation

- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

energie_multiphase_enthalpie

Synonyms: energie_multiphase_h

Inherits from: eqn_base

Internal energy conservation equation for a multi-phase problem where the unknown is the enthalpy

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: bloc_convection) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: bloc_diffusion) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: ecrire_fichier_xyz_valeur) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: parametre_equation_base) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

equation_navier_cauchy

Inherits from: eqn_base

Equation_Navier_Cauchy equation

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: bloc_convection) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: bloc_diffusion) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: ecrire_fichier_xyz_valeur) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: parametre_equation_base) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

masse_multiphase

Inherits from: [eqn_base](#)

Mass conservation equation for a multi-phase problem where the unknown is the alpha (void fraction)

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

momentum_euler

Synonyms: qdm_euler

Inherits from: [eqn_base](#)

Momentum conservation equation for a multi-phase Euler problem where the unknown is the velocity

Parameters:

- **[termes_non_conservatifs | non_conservative_terms]** (*type*: [bloc_op_non_conservatifs](#)) Keyword to alter the non-conservative scheme.
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

qdm_multiphase**Inherits from:** [eqn_base](#)

Momentum conservation equation for a multi-phase problem where the unknown is the velocity

Parameters:

- **[solveur_pression]** (*type:* [solveur_sys_base](#)) Linear pressure system resolution method.
- **[evanescence]** (*type:* [bloc_lecture](#)) Management of the vanishing phase (when alpha tends to 0 or 1)
- **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type:* list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type:* list of [Condinit](#)) Initial conditions.
- **[sources]** (*type:* list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type:* string) Rename the equation with a specific name.

taux_dissipation_turbulent**Inherits from:** [eqn_base](#)

Turbulent Dissipation frequency equation for a turbulent mono/multi-phase problem (available in TrioCFD)

Parameters:

- **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type:* list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type:* list of [Condinit](#)) Initial conditions.
- **[sources]** (*type:* list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type:* string) Rename the equation with a specific name.

1.19.1.1 Keywords derived from conduction

conduction

Inherits from: [eqn_base](#)

Heat equation.

Parameters:

- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
 - **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
 - **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
 - **[conditions_limites | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
 - **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
 - **[sources]** (*type*: list of [Source_base](#)) The sources.
 - **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
 - **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
 - **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
 - **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.
-

conduction_ibm

Inherits from: [conduction](#)

IBM Heat equation.

Parameters:

- **[correction_variable_initiale]** (*type*: int) Modify initial variable
 - **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
 - **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
 - **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
 - **[conditions_limites | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
 - **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
 - **[sources]** (*type*: list of [Source_base](#)) The sources.
 - **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
 - **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
 - **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
 - **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.
-

1.19.1.2 Keywords derived from cons_euler

cons_euler**Inherits from:** [eqn_base](#)

Base class equation for a multi-phase Euler conservation equations

Parameters:

- **[termes_non_conservatifs | non_conservative_terms]** (*type:* [bloc_op_non_conservativtifs](#)) Keyword to alter the non-conservative scheme.
- **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type:* list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type:* list of [Condinit](#)) Initial conditions.
- **[sources]** (*type:* list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type:* string) Rename the equation with a specific name.

energy_euler**Synonyms:** energie_euler**Inherits from:** [cons_euler](#)

Internal energy conservation equation for a multi-phase Euler problem where the unknown is the temperature

Parameters:

- **[termes_non_conservatifs | non_conservative_terms]** (*type:* [bloc_op_non_conservativtifs](#)) Keyword to alter the non-conservative scheme.
- **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type:* list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type:* list of [Condinit](#)) Initial conditions.
- **[sources]** (*type:* list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type:* string) Rename the equation with a specific name.

fraction_euler**Inherits from:** [cons_euler](#)

Void fraction conservation equation for a multi-phase Euler problem where the unknown is the temperature

Parameters:

- **[termes_non_conservatifs | non_conservative_terms]** (*type:* [bloc_op_non_conservatifs](#)) Keyword to alter the non-conservative scheme.
 - **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
 - **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
 - **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
 - **[conditions_limtes | boundary_conditions]** (*type:* list of [Condlimlu](#)) Boundary conditions.
 - **[conditions_initiales | initial_conditions]** (*type:* list of [Condinit](#)) Initial conditions.
 - **[sources]** (*type:* list of [Source_base](#)) The sources.
 - **[ecrire_fichier_xyz_valeur]** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
 - **[parametre_equation]** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
 - **[equation_non_resolue]** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
 - **[renommer_equation | rename_equation]** (*type:* string) Rename the equation with a specific name.
-

masse_euler**Synonyms:** [density_euler](#)**Inherits from:** [cons_euler](#)

Mass convection equation for a multi-phase Euler problem where the unknown is the alpha (void fraction)

Parameters:

- **[termes_non_conservatifs | non_conservative_terms]** (*type:* [bloc_op_non_conservatifs](#)) Keyword to alter the non-conservative scheme.
 - **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
 - **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
 - **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
 - **[conditions_limtes | boundary_conditions]** (*type:* list of [Condlimlu](#)) Boundary conditions.
 - **[conditions_initiales | initial_conditions]** (*type:* list of [Condinit](#)) Initial conditions.
 - **[sources]** (*type:* list of [Source_base](#)) The sources.
 - **[ecrire_fichier_xyz_valeur]** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
 - **[parametre_equation]** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
 - **[equation_non_resolue]** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
 - **[renommer_equation | rename_equation]** (*type:* string) Rename the equation with a specific name.
-

1.19.1.3 Keywords derived from convection_diffusion_chaleur_qc

convection_diffusion_chaleur_qc

Inherits from: [eqn_base](#)

Temperature equation for a quasi-compressible fluid.

Parameters:

- **[mode_calcul_convection]** (*type*: string into ['ancien', 'divut_moins_tdivu', 'divrhout_moins_tdivrhout']) Option to set the form of the convective operator $\text{divrhout}_T - \text{Tdivrhout}$ (the default since 1.6.8): $\rho u \cdot \text{grad} T = \text{div}(\rho u \cdot T) - T \text{div}(\rho u \cdot 1)$ ancien: $u \cdot \text{grad} T = \text{div}(u \cdot T) - T \text{div}(u)$ $\text{divu}_T - \text{Tdivu}$: $u \cdot \text{grad} T = \text{div}(u \cdot T) - T \text{div}(u \cdot 1)$
- **[t_min]** (*type*: float) Specifies $T_{\min}$ (and $T_{\max}$) to keep temperature within $[T_{\min}, T_{\max}]$ and avoid excessive excursions beyond physical limits in QC simulations. Use with caution.
- **[t_max]** (*type*: float) Should be set when $T_{\min}$ set in the datafile.
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_chaleur_turbulent_qc

Inherits from: [convection_diffusion_chaleur_qc](#)

Temperature equation for a quasi-compressible fluid as well as the associated turbulence model equations.

Parameters:

- **[modele_turbulence]** (*type*: [modele_turbulence_scal_base](#)) Turbulence model for the temperature (energy) conservation equation.
- **[mode_calcul_convection]** (*type*: string into ['ancien', 'divut_moins_tdivu', 'divrhout_moins_tdivrhout']) Option to set the form of the convective operator $\text{divrhout}_T - \text{Tdivrhout}$ (the default since 1.6.8): $\rho u \cdot \text{grad} T = \text{div}(\rho u \cdot T) - T \text{div}(\rho u \cdot 1)$ ancien: $u \cdot \text{grad} T = \text{div}(u \cdot T) - T \text{div}(u)$ $\text{divu}_T - \text{Tdivu}$: $u \cdot \text{grad} T = \text{div}(u \cdot T) - T \text{div}(u \cdot 1)$
- **[t_min]** (*type*: float) Specifies $T_{\min}$ (and $T_{\max}$) to keep temperature within $[T_{\min}, T_{\max}]$ and avoid excessive excursions beyond physical limits in QC simulations. Use with caution.
- **[t_max]** (*type*: float) Should be set when $T_{\min}$ set in the datafile.
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step

- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limtes | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

1.19.1.4 Keywords derived from convection_diffusion_concentration

convection_diffusion_concentration

Inherits from: [eqn_base](#)

Constituent transport vectorial equation (concentration diffusion convection).

Parameters:

- **[nom_inconnue]** (*type*: string) Keyword Nom_inconnue will rename the unknown of this equation with the given name. In the postprocessing part, the concentration field will be accessible with this name. This is usefull if you want to track more than one concentration (otherwise, only the concentration field in the first concentration equation can be accessed).
- **[alias]** (*type*: string) not_set
- **[masse_molaire]** (*type*: float) not_set
- **[is_multi_scalar | is_multi_scalar_diffusion]** (*type*: flag) Flag to activate the multi_scalar diffusion operator
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limtes | boundary_conditions]** (*type*: list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of [Condinit](#)) Initial conditions.
- **[sources]** (*type*: list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_concentration_turbulent**Inherits from:** [convection_diffusion_concentration](#)

Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.

Parameters:

- **[modele_turbulence]** (*type:* [modele_turbulence_scal_base](#)) Turbulence model to be used in the constituent transport equations. The only model currently available is Schmidt.
- **[nom_inconnue]** (*type:* string) Keyword Nom_inconnue will rename the unknown of this equation with the given name. In the postprocessing part, the concentration field will be accessible with this name. This is useful if you want to track more than one concentration (otherwise, only the concentration field in the first concentration equation can be accessed).
- **[alias]** (*type:* string) not_set
- **[masse_molaire]** (*type:* float) not_set
- **[is_multi_scalar | is_multi_scalar_diffusion]** (*type:* flag) Flag to activate the multi_scalar diffusion operator
- **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type:* list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type:* list of Condinit) Initial conditions.
- **[sources]** (*type:* list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type:* string) Rename the equation with a specific name.

1.19.1.5 Keywords derived from convection_diffusion_espece_binaire_qc**convection_diffusion_espece_binaire_qc****Inherits from:** [eqn_base](#)

Species conservation equation for a binary quasi-compressible fluid.

Parameters:

- **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type:* list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type:* list of Condinit) Initial conditions.
- **[sources]** (*type:* list of Source_base) The sources.

- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_espece_binaire_turbulent_qc

Inherits from: [convection_diffusion_espece_binaire_qc](#)

Species conservation equation for a binary quasi-compressible fluid as well as the associated turbulence model equations.

Parameters:

- **[modele_turbulence]** (*type*: [modele_turbulence_scal_base](#)) Turbulence model for the species conservation equation.
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

1.19.1.6 Keywords derived from convection_diffusion_temperature

convection_diffusion_temperature

Inherits from: [eqn_base](#)

Energy equation (temperature diffusion convection).

Parameters:

- **[penalisation_l2_ftd]** (*type*: [bloc_lecture](#)) to activate or not (the default is Direct Forcing method) the Penalized Direct Forcing method to impose the specified temperature on the solid-fluid interface.
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.

- **[conditions_limites | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

convection_diffusion_temperature_ibm

Inherits from: [convection_diffusion_temperature](#)

IBM Energy equation (temperature diffusion convection).

Parameters:

- **[correction_variable_initiale]** (*type*: int) Modify initial variable
- **[penalisation_l2_ftd]** (*type*: [bloc_lecture](#)) to activate or not (the default is Direct Forcing method) the Penalized Direct Forcing method to impose the specified temperature on the solid-fluid interface.
- **[disable_equation_residual]** (*type*: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type*: [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type*: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limites | boundary_conditions]** (*type*: list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type*: list of Condinit) Initial conditions.
- **[sources]** (*type*: list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type*: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type*: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type*: string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (*type*: string) Rename the equation with a specific name.

1.19.1.7 Keywords derived from navier_stokes_standard

navier_stokes_standard

Inherits from: [eqn_base](#)

Navier-Stokes equations.

Parameters:

- **[solveur_pression]** (*type*: [solveur_sys_base](#)) Linear pressure system resolution method.
- **[dt_projection]** (*type*: [deuxmots](#)) nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.

- **[traitement_particulier]** (*type*: [traitement_particulier](#)) Keyword to post-process particular values.
 - **[seuil_divu]** (*type*: [floatfloat](#)) value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur_pression) is dynamically adapted according to the mass conservation. At t_n , the linear system $Ax=B$ is considered as solved if the residual $\|Ax-B\| < \text{seuil}(t_n)$. For t_{n+1} , the threshold value $\text{seuil}(t_{n+1})$ will be evaluated as follows : If ($|\max(\text{Div}U) \cdot dt| < \text{value}$) $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$ Else $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$ Endif The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10% per timestep). Investigations has to be lead to know more about the effects of these two last parameters on the behaviour of the simulations.
 - ****[solveur_bar]**** (**type**: [solveur_sys_base](#)) This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source_Qdm_lambdaup). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
 - ****[projection_initiale]**** (**type**: [int](#)) Keyword to suppress, if boolean equals 0, the initial projection which checks $\text{Div}U=0$. By default, boolean equals 1.
 - ****[postraiter_gradient_pression_sans_masse]**** (**type**: [flag](#)) Avoid mass matrix multiplication for the gradient postprocessing
 - ****[methode_calcul_pression_initiale]**** (**type**: [string](#) into ['avec_les_cl', 'avec_sources', 'avec_sources_et_operateurs', 'sans_rien']) Keyword to select an option for the pressure calculation before the first time step. Options are : avec_les_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec_sources_et_operateurs (lapP=f is solved as with the previous option avec_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
 - ****[disable_equation_residual]**** (**type**: [int](#)) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
 - ****[convection]**** (**type**: [bloc_convection](#)) Keyword to alter the convection scheme.
 - ****[diffusion]**** (**type**: [bloc_diffusion](#)) Keyword to specify the diffusion operator.
 - ****[conditions_limite | boundary_conditions]**** (**type**: [list of Condlimlu](#)) Boundary conditions.
 - ****[conditions_initiales | initial_conditions]**** (**type**: [list of Condinit](#)) Initial conditions.
 - ****[sources]**** (**type**: [list of Source_base](#)) The sources.
 - ****[ecrire_fichier_xyz_valeur]**** (**type**: [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
 - ****[parametre_equation]**** (**type**: [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
 - ****[equation_non_resolue]**** (**type**: [string](#)) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t_0 and t_1 . Navier_Sokes_Standard { equation_non_resolue ($t > t_0$)*($t < t_1$) }
 - **[renommer_equation | rename_equation]** (*type*: [string](#)) Rename the equation with a specific name.
-
- [Keywords derived from navier_stokes_turbulent](#)

navier_stokes_ibm**Inherits from:** [navier_stokes_standard](#)

IBM Navier-Stokes equations.

Parameters:

- **[correction_matrice_projection_initiale]** (*type*: int) (IBM advanced) fix matrix of initial projection for PDF
- **[correction_calcul_pression_initiale]** (*type*: int) (IBM advanced) fix initial pressure computation for PDF
- **[correction_vitesse_projection_initiale]** (*type*: int) (IBM advanced) fix initial velocity computation for PDF
- **[correction_matrice_pression]** (*type*: int) (IBM advanced) fix pressure matrix for PDF
- **[matrice_pression_penalisee_h1]** (*type*: int) (IBM advanced) fix pressure matrix for PDF
- **[correction_vitesse_modifie]** (*type*: int) (IBM advanced) fix velocity for PDF
- **[correction_pression_modifie]** (*type*: int) (IBM advanced) fix pressure for PDF
- **[gradient_pression_qdm_modifie]** (*type*: int) (IBM advanced) fix pressure gradient
- **[correction_variable_initiale]** (*type*: int) Modify initial variable
- **[solveur_pression]** (*type*: [solveur_sys_base](#)) Linear pressure system resolution method.
- **[dt_projection]** (*type*: [deuxmots](#)) nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **[traitement_particulier]** (*type*: [traitement_particulier](#)) Keyword to post-process particular values.
- **[seuil_divu]** (*type*: [floatfloat](#)) value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur_pression) is dynamically adapted according to the mass conservation. At t_n , the linear system $Ax=B$ is considered as solved if the residual $\|Ax-B\| < \text{seuil}(t_n)$. For t_{n+1} , the threshold value $\text{seuil}(t_{n+1})$ will be evaluated as $\leftarrow$: If ($|\max(\text{Div}U)*dt| < \text{value}$) $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n)*\text{factor}$ Else $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n)*\text{factor}$ Endif The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10% per timestep). Investigations has to be lead to know more about the effects of these two last parameters on the behaviour of the simulations.
- ****[solveur_bar]**** (**type**: [solveur_sys_base](#)) This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source_Qdm_lambdaup). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- ****[projection_initiale]**** (**type**: int) Keyword to suppress, if boolean equals 0, the initial projection which checks $\text{Div}U=0$. By default, boolean equals 1.
- ****[postraiter_gradient_pression_sans_masse]**** (**type**: flag) Avoid mass matrix multiplication for the gradient postprocessing
- ****[methode_calcul_pression_initiale]**** (**type**: string into ['avec_les_cl', 'avec_sources', 'avec_sources_et_operateurs', 'sans_rien']) Keyword to select an option for the pressure calculation before the first time step. Options are : avec_les_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec_sources_et_operateurs (lapP=f is solved as with the previous option avec_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- ****[disable_equation_residual]**** (**type**: int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- ****[convection]**** (**type**: [bloc_convection](#)) Keyword to alter the convection scheme.
- ****[diffusion]**** (**type**: [bloc_diffusion](#)) Keyword to specify the diffusion operator.

- **[conditions_limtes | boundary_conditions]**** (*type:* list of `Condlimlu`) Boundary conditions.
- **[conditions_initiales | initial_conditions]**** (*type:* list of `Condinit`) Initial conditions.
- **[sources]**** (*type:* list of `Source_base`) The sources.
- **[ecrire_fichier_xyz_valeur]**** (*type:* `ecrire_fichier_xyz_valeur`) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]**** (*type:* `parametre_equation_base`) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]**** (*type:* string) The equation will not be solved while condition(t) is verified if `equation_non_resolue` keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (type: string) Rename the equation with a specific name.

navier_stokes_ibm_turbulent

Inherits from: `navier_stokes_standard`

IBM Navier-Stokes equations as well as the associated turbulence model equations.

Parameters:

- **[modele_turbulence]** (type: `modele_turbulence_hyd_deriv`) Turbulence model for Navier-Stokes equations.
- **[solveur_pression]** (type: `solveur_sys_base`) Linear pressure system resolution method.
- **[dt_projection]** (type: `deuxmots`) nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **[traitement_particulier]** (type: `traitement_particulier`) Keyword to post-process particular values.
- **[seuil_divu]** (type: `floatfloat`) value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in `solveur_pression`) is dynamically adapted according to the mass conservation. At t_n , the linear system $Ax=B$ is considered as solved if the residual $||Ax-B|| < \text{seuil}(t_n)$. For t_{n+1} , the threshold value $\text{seuil}(t_{n+1})$ will be evaluated as: If ($| \max(\text{Div}U) * dt | < \text{value}$) $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) * \text{factor}$ Else $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) * \text{factor}$ Endif The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10% per timestep). Investigations has to be lead to know more about the effects of these two last parameters on the behaviour of the simulations.
- **[solveur_bar]**** (*type:* `solveur_sys_base`) This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and `Source_Qdm_lambdaup`). A file (`solveur.bar`) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **[projection_initiale]**** (*type:* int) Keyword to suppress, if boolean equals 0, the initial projection which checks $\text{Div}U=0$. By default, boolean equals 1.
- **[postraiter_gradient_pression_sans_masse]**** (*type:* flag) Avoid mass matrix multiplication for the gradient postprocessing
- **[methode_calcul_pression_initiale]**** (*type:* string into ['avec_les_cl', 'avec_sources', 'avec_sources_et_operateurs', 'sans_rien']) Keyword to select an option for the pressure calculation before the first time step. Options are : `avec_les_cl` (default option $\text{lap}P=0$ is solved with Neuman boundary conditions on pressure if any), `avec_sources` ($\text{lap}P=f$ is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and `avec_sources_et_operateurs` ($\text{lap}P=f$ is solved as with the previous option `avec_sources` but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **[disable_equation_residual]**** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step

- **[convection]**** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]**** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]**** (*type:* list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]**** (*type:* list of [Condit](#)) Initial conditions.
- **[sources]**** (*type:* list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]**** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]**** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]**** (*type:* string) The equation will not be solved while condition(t) is verified if [equation_non_resolue](#) keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (type: string) Rename the equation with a specific name.

navier_stokes_qc

Inherits from: [navier_stokes_standard](#)

Navier-Stokes equation for a quasi-compressible fluid.

Parameters:

- **[solveur_pression]** (type: [solveur_sys_base](#)) Linear pressure system resolution method.
- **[dt_projection]** (type: [deuxmots](#)) nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **[traitement_particulier]** (type: [traitement_particulier](#)) Keyword to post-process particular values.
- **[seuil_divu]** (type: [floatfloat](#)) value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in [solveur_pression](#)) is dynamically adapted according to the mass conservation. At tn , the linear system $Ax=B$ is considered as solved if the residual $||Ax-B|| < \text{seuil}(tn)$. For tn+1, the threshold value $\text{seuil}(tn+1)$ will be evaluated as follows : If ($|\max(\text{DivU}) \cdot dt| < \text{value}$) $\text{Seuil}(tn+1) = \text{Seuil}(tn) \cdot \text{factor}$ Else $\text{Seuil}(tn+1) = \text{Seuil}(tn) \cdot \text{factor}$ Endif The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10% per timestep). Investigations has to be lead to know more about the effects of these two last parameters on the behaviour of the simulations.
- **[solveur_bar]**** (*type:* [solveur_sys_base](#)) This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and [Source_Qdm_lambdaup](#)). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **[projection_initiale]**** (*type:* int) Keyword to suppress, if boolean equals 0, the initial projection which checks $\text{DivU}=0$. By default, boolean equals 1.
- **[postraiter_gradient_pression_sans_masse]**** (*type:* flag) Avoid mass matrix multiplication for the gradient postprocessing
- **[methode_calcul_pression_initiale]**** (*type:* string into ['avec_les_cl', 'avec_sources', 'avec_sources_et_operateurs', 'sans_rien']) Keyword to select an option for the pressure calculation before the first time step. Options are : [avec_les_cl](#) (default option $\text{lapP}=0$ is solved with Neuman boundary conditions on pressure if any), [avec_sources](#) ($\text{lapP}=f$ is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and [avec_sources_et_operateurs](#) ($\text{lapP}=f$ is solved as with the previous option [avec_sources](#) but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.

- **[disable_equation_residual]**** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]**** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]**** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]**** (*type:* list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]**** (*type:* list of [Condinit](#)) Initial conditions.
- **[sources]**** (*type:* list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]**** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]**** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]**** (*type:* string) The equation will not be solved while condition(t) is verified if [equation_non_resolue](#) keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (type: string) Rename the equation with a specific name.

navier_stokes_wc

Inherits from: [navier_stokes_standard](#)

Navier-Stokes equation for a weakly-compressible fluid.

Parameters:

- **[mass_source]** (type: [mass_source](#)) Mass source used in a dilatable simulation to add/reduce a mass at the boundary (volumetric source in the first cell of a given boundary).
- **[solveur_pression]** (type: [solveur_sys_base](#)) Linear pressure system resolution method.
- **[dt_projection]** (type: [deuxmots](#)) nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **[traitement_particulier]** (type: [traitement_particulier](#)) Keyword to post-process particular values.
- **[seuil_divu]** (type: [floatfloat](#)) value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in [solveur_pression](#)) is dynamically adapted according to the mass conservation. At t_n , the linear system $Ax=B$ is considered as solved if the residual $||Ax-B|| < \text{seuil}(t_n)$. For t_{n+1} , the threshold value $\text{seuil}(t_{n+1})$ will be evaluated as follows: If ($|\max(\text{Div}U)| \cdot dt < \text{value}$) $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$ Else $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$ Endif The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10% per timestep). Investigations has to be lead to know more about the effects of these two last parameters on the behaviour of the simulations.
- **[solveur_bar]**** (*type:* [solveur_sys_base](#)) This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and [Source_Qdm_lambdaup](#)). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **[projection_initiale]**** (*type:* int) Keyword to suppress, if boolean equals 0, the initial projection which checks $\text{Div}U=0$. By default, boolean equals 1.
- **[postraiter_gradient_pression_sans_masse]**** (*type:* flag) Avoid mass matrix multiplication for the gradient postprocessing

- **[methode_calcul_pression_initiale]**** (*type:* string into ['avec_les_cl', 'avec_sources', 'avec_sources_et_operateurs', 'sans_rien']) Keyword to select an option for the pressure calculation before the first time step. Options are : avec_les_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec_sources_et_operateurs (lapP=f is solved as with the previous option avec_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **[disable_equation_residual]**** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]**** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]**** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]**** (*type:* list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]**** (*type:* list of [Condit](#)) Initial conditions.
- **[sources]**** (*type:* list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]**** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]**** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]**** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (type: string) Rename the equation with a specific name.

1.19.1.7.1 Keywords derived from navier_stokes_turbulent

navier_stokes_turbulent

Inherits from: [navier_stokes_standard](#)

Navier-Stokes equations as well as the associated turbulence model equations.

Parameters:

- **[modele_turbulence]** (type: [modele_turbulence_hyd_deriv](#)) Turbulence model for Navier-Stokes equations.
- **[solveur_pression]** (type: [solveur_sys_base](#)) Linear pressure system resolution method.
- **[dt_projection]** (type: [deuxmots](#)) nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **[traitement_particulier]** (type: [traitement_particulier](#)) Keyword to post-process particular values.
- **[seuil_divu]** (type: [floatfloat](#)) value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur_pression) is dynamically adapted according to the mass conservation. At tn , the linear system $Ax=B$ is considered as solved if the residual $||Ax-B|| < \text{seuil}(tn)$. For tn+1, the threshold value $\text{seuil}(tn+1)$ will be evaluated as: If $(| \max(\text{DivU}) * dt | < \text{value})$ $\text{Seuil}(tn+1) = \text{Seuil}(tn) * \text{factor}$ Else $\text{Seuil}(tn+1) = \text{Seuil}(tn) * \text{factor}$ Endif The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10% per timestep). Investigations has to be lead to know more about the effects of these two last parameters on the behaviour of the simulations.
- **[solveur_bar]**** (*type:* [solveur_sys_base](#)) This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and [Source_Qdm_lambdaup](#)). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).

- **[projection_initiale]**** (*type:* int) Keyword to suppress, if boolean equals 0, the initial projection which checks $\text{Div}U=0$. By default, boolean equals 1.
- **[postraiter_gradient_pression_sans_masse]**** (*type:* flag) Avoid mass matrix multiplication for the gradient postprocessing
- **[methode_calcul_pression_initiale]**** (*type:* string into ['avec_les_cl', 'avec_sources', 'avec_sources_↔ et_operateurs', 'sans_rien']) Keyword to select an option for the pressure calculation before the first time step. Options are : avec_les_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec_sources_et_operateurs (lapP=f is solved as with the previous option avec_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **[disable_equation_residual]**** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]**** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]**** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]**** (*type:* list of [Condlimlu](#)) Boundary conditions.
- **[conditions_initiales | initial_conditions]**** (*type:* list of [Condinit](#)) Initial conditions.
- **[sources]**** (*type:* list of [Source_base](#)) The sources.
- **[ecrire_fichier_xyz_valeur]**** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]**** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]**** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. `Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }`
- **[renommer_equation | rename_equation]** (type: string) Rename the equation with a specific name.

navier_stokes_turbulent_qc

Inherits from: [navier_stokes_turbulent](#)

Navier-Stokes equations under low Mach number as well as the associated turbulence model equations.

Parameters:

- **[modele_turbulence]** (type: [modele_turbulence_hyd_deriv](#)) Turbulence model for Navier-Stokes equations.
- **[solveur_pression]** (type: [solveur_sys_base](#)) Linear pressure system resolution method.
- **[dt_projection]** (type: [deuxmots](#)) nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **[traitement_particulier]** (type: [traitement_particulier](#)) Keyword to post-process particular values.
- **[seuil_divu]** (type: [floatfloat](#)) value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur_pression) is dynamically adapted according to the mass conservation. At t_n , the linear system $Ax=B$ is considered as solved if the residual $\|Ax-B\| < \text{seuil}(t_n)$. For t_{n+1} , the threshold value $\text{seuil}(t_{n+1})$ will be evaluated as↔ : If ($\|\max(\text{Div}U)*dt\| < \text{value}$) $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n)*\text{factor}$ Else $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n)*\text{factor}$ Endif The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10% per timestep). Investigations has to be lead to know more about the effects of these two last parameters on the behaviour of the simulations.

- **[solveur_bar]** (*type:* [solveur_sys_base](#)) This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source_Qdm_lambdaup). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **[projection_initiale]** (*type:* int) Keyword to suppress, if boolean equals 0, the initial projection which checks DivU=0. By default, boolean equals 1.
- **[postraiter_gradient_pression_sans_masse]** (*type:* flag) Avoid mass matrix multiplication for the gradient postprocessing
- **[methode_calcul_pression_initiale]** (*type:* string into ['avec_les_cl', 'avec_sources', 'avec_sources_et_operateurs', 'sans_rien']) Keyword to select an option for the pressure calculation before the first time step. Options are : avec_les_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec_sources_et_operateurs (lapP=f is solved as with the previous option avec_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **[disable_equation_residual]** (*type:* int) The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **[convection]** (*type:* [bloc_convection](#)) Keyword to alter the convection scheme.
- **[diffusion]** (*type:* [bloc_diffusion](#)) Keyword to specify the diffusion operator.
- **[conditions_limite | boundary_conditions]** (*type:* list of Condlimlu) Boundary conditions.
- **[conditions_initiales | initial_conditions]** (*type:* list of Condinit) Initial conditions.
- **[sources]** (*type:* list of Source_base) The sources.
- **[ecrire_fichier_xyz_valeur]** (*type:* [ecrire_fichier_xyz_valeur](#)) This keyword is used to write the values of a field only for some boundaries in a text file
- **[parametre_equation]** (*type:* [parametre_equation_base](#)) Keyword used to specify additional parameters for the equation
- **[equation_non_resolue]** (*type:* string) The equation will not be solved while condition(t) is verified if equation_non_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1. Navier_Sokes_Standard { equation_non_resolue (t>t0)*(t<t1) }
- **[renommer_equation | rename_equation]** (type: string) Rename the equation with a specific name.

1.20 Keywords derived from moyenne_imposee_deriv

moyenne_imposee_deriv

not_set

moyenne_imposee_connexion_approchee

Synonyms: connexion_approchee

Inherits from: [moyenne_imposee_deriv](#)

To read the imposed field from a file where positions and values are given (it is not necessary that the coordinates of points match the coordinates of the boundary faces, indeed, the nearest point of each face of the boundary will be used).

Parameters:

- **fichier** (type: string into ['fichier']) not_set
- **file1** (type: string) filename. The format of the file is: N x(1) y(1) [z(1)] valx(1) valy(1) [valz(1)] x(2) y(2) [z(2)] valx(2) valy(2) [valz(2)] ... x(N) y(N) [z(N)] valx(N) valy(N) [valz(N)]

moyenne_imposee_connexion_exacte

Synonyms: connexion_exacte

Inherits from: [moyenne_imposee_deriv](#)

To read the imposed field from two files.

Parameters:

- **fichier** (*type*: string into ['fichier']) not_set
- **file1** (*type*: string) first file, contains the points coordinates (which should be the same as the coordinates of the boundary faces). The format of this file is: N 1 x(1) y(1) [z(1)] 2 x(2) y(2) [z(2)] ... N x(N) y(N) [z(N)]
- **[file2]** (*type*: string) second file, contains the mean values. The format of this file is: N 1 valx(1) valy(1) [valz(1)] 2 valx(2) valy(2) [valz(2)] ... N valx(N) valy(N) [valz(N)]

moyenne_imposee_interpolation

Synonyms: champ_post_interpolation, interpolation

Inherits from: [moyenne_imposee_deriv](#)

To create an imposed field built by interpolation of values read from a file. The imposed field is applied on the direction given by the keyword `direction_anisotrope` (the field is zero for the other directions).

Parameters:

- **fichier** (*type*: string into ['fichier']) The format of the file is: pos(1) val(1) pos(2) val(2) ... pos(N) val(N) If direction given by `direction_anisotrope` is 1 (or 2 or 3), then pos will be X (or Y or Z) coordinate and val will be X value (or Y value, or Z value) of the imposed field.
- **file1** (*type*: string) name of `geom_face_perio`

moyenne_imposee_logarithmique

Synonyms: logarithmique

Inherits from: [moyenne_imposee_deriv](#)

To specify the imposed field (in this case, velocity) by an analytical logarithmic law of the wall:

$g(x,y,z) = u_tau * (\log(0.5*diametre*u_tau/visco_cin)/Kappa + 5.1)$

with $g(x,y,z)=u(x,y,z)$ if direction is set to 1, $g=v(x,y,z)$ if direction is set to 2 and $g=w(x,y,z)$ if it is set to 3

Parameters:

- **diametre** (*type*: string into ['diametre']) not_set
- **val** (*type*: float) diameter
- **u_tau** (*type*: string into ['u_tau']) not_set
- **val_u_tau** | **val_u_tau1** (*type*: float) value of `u_tau`
- **visco_cin** (*type*: string into ['visco_cin']) not_set
- **val_visco_cin** (*type*: float) value of `visco_cin`
- **direction** (*type*: string into ['direction']) not_set
- **val_direction** (*type*: int) direction

moyenne_imposee_profil

Synonyms: profil

Inherits from: [moyenne_imposee_deriv](#)

To specify analytic profile for the imposed `g` field.

Parameters:

- **profile** (*type*: list of str) specifies the analytic profile: 2|3 valx(x,y,z,t) valy(x,y,z,t) [valz(x,y,z,t)]

1.21 Keywords derived from nom

nom

Class to name the TRUST objects.

Parameters:

- **[mot]** (*type*: string) Chain of characters.
-

nom_anonyme

Inherits from: [nom](#)

not_set

Parameters:

- **[mot]** (*type*: string) Chain of characters.

1.22 Keywords derived from objet_lecture

objet_lecture

Auxiliary class for reading.

- [Keywords derived from bloc_lecture](#)
- [Keywords derived from bord_base](#)
- [Keywords derived from convection_deriv](#)
- [Keywords derived from defbord](#)
- [Keywords derived from diffusion_deriv](#)
- [Keywords derived from format_file_base](#)
- [Keywords derived from lecture_bloc_moment_base](#)
- [Keywords derived from mailer_base](#)
- [Keywords derived from modele_turbulence_hyd_deriv](#)
- [Keywords derived from op_non_conservativtifs_deriv](#)
- [Keywords derived from parametre_equation_base](#)
- [Keywords derived from postraitemnt_base](#)
- [Keywords derived from sonde_base](#)
- [Keywords derived from spec_pdc_r_base](#)
- [Keywords derived from stat_post_deriv](#)
- [Keywords derived from traitement_particulier_base](#)
- [Keywords derived from type_diffusion_turbulente_multiphase_deriv](#)
- [Keywords derived from type_perte_charge_deriv](#)

bloc_convection

not_set

Parameters:

- **aco** (*type*: string into ['{']) Opening curly bracket.
 - **opérateur** (*type*: [convection_deriv](#)) not_set
 - **acof** (*type*: string into ['}']) Closing curly bracket.
-

bloc_couronne

Class to create a couronne (2D).

Parameters:

- **name** (*type*: string into ['origine']) Keyword to define the center of the circle.
- **origin | origine** (*type*: list of float) Center of the circle.
- **name3** (*type*: string into ['ri']) Keyword to define the interior radius.
- **ri** (*type*: float) Interior radius.
- **name4** (*type*: string into ['re']) Keyword to define the exterior radius.
- **re** (*type*: float) Exterior radius.

bloc_decouper

Auxiliary class to cut a domain.

Parameters:

- **[partitionneur | partition_tool]** (*type*: [partitionneur_deriv](#)) Defines the partitionning algorithm (the effective C++ object used is '\Partitionneur_ALGORITHM_NAME').
- **[larg_joint]** (*type*: int) This keyword specifies the thickness of the virtual ghost domaine (data known by one processor though not owned by it). The default value is 1 and is generally correct for all algorithms except the QUICK convection scheme that require a thickness of 2. Since the 1.5.5 version, the VEF discretization imply also a thickness of 2 (except VEF P0). Any non-zero positive value can be used, but the amount of data to store and exchange between processors grows quickly with the thickness.
- **[nom_zones | zones_name]** (*type*: string) Name of the files containing the different partition of the domain. The files will be : name_0001.Zones name_0002.Zones ... name_000n.Zones. If this keyword is not specified, the geometry is not written on disk (you might just want to generate a '\ecrire_decoupage\' or '\ecrire_lata\').
- **[ecrire_decoupage]** (*type*: string) After having called the partitionning algorithm, the resulting partition is written on disk in the specified filename. See also partitionneur Fichier_Decoupage. This keyword is useful to change the partition numbers: first, you write the partition into a file with the option `ecrire_decoupage`. This file contains the domaine number for each element's mesh. Then you can easily permute domaine numbers in this file. Then read the new partition to create the .Zones files with the Fichier_Decoupage keyword.
- **[ecrire_lata]** (*type*: string) Save the partition field in a LATA format file for visualization
- **[ecrire_med]** (*type*: string) Save the partition field in a MED format file for visualization
- **[nb_parts_tot]** (*type*: int) Keyword to generates N .Domaine files, instead of the default number M obtained after the partitionning algorithm. N must be greater or equal to M. This option might be used to perform coupled parallel computations. Supplemental empty domaines from M to N-1 are created. This keyword is used when you want to run a parallel calculation on several domains with for example, 2 processors on a first domain and 10 on the second domain because the first domain is very small compare to second one. You will write Nb_parts 2 and Nb_parts_tot 10 for the first domain and Nb_parts 10 for the second domain.
- **[reorder]** (*type*: int) If this option is set to 1 (0 by default), the partition is renumbered in order that the processes which communicate the most are nearer on the network. This may slightly improves parallel performance.
- **[single_hdf]** (*type*: flag) Optional keyword to enable you to write the partitioned domaines in a single file in hdf5 format.
- **[print_more_infos]** (*type*: int) If this option is set to 1 (0 by default), print infos about number of remote elements (ghosts) and additional infos about the quality of partitionning. Warning, it slows down the cutting operations.

bloc_diffusion

not_set

Parameters:

- **aco** (*type*: string into ['{'] Opening curly bracket.
- **[opérateur]** (*type*: [diffusion_deriv](#)) if none is specified, the diffusive scheme used is a 2nd-order scheme.
- **[op_implicite]** (*type*: [op_implicite](#)) To have diffusive implicitation, it use Uzawa algorithm. Very useful when viscosity has large variations.
- **acof** (*type*: string into ['] Closing curly bracket.

bloc_diffusion_standard

grad_Ubar 1 makes the gradient calculated through the filtered values of velocity (P1-conform).

nu 1 (respectively nut 1) takes the molecular viscosity (eddy viscosity) into account in the velocity gradient part of the diffusion expression.

nu_transp 1 (respectively nut_transp 1) takes the molecular viscosity (eddy viscosity) into account according in the TRANSPOSED velocity gradient part of the diffusion expression.

filtrer_resu 1 allows to filter the resulting diffusive fluxes contribution.

Parameters:

- **mot1** (*type*: string into ['grad_ubar', 'nu', 'nut', 'nu_transp', 'nut_transp', 'filtrer_resu']) not_set
- **val1** (*type*: int into [0, 1]) not_set
- **mot2** (*type*: string into ['grad_ubar', 'nu', 'nut', 'nu_transp', 'nut_transp', 'filtrer_resu']) not_set
- **val2** (*type*: int into [0, 1]) not_set
- **mot3** (*type*: string into ['grad_ubar', 'nu', 'nut', 'nu_transp', 'nut_transp', 'filtrer_resu']) not_set
- **val3** (*type*: int into [0, 1]) not_set
- **mot4** (*type*: string into ['grad_ubar', 'nu', 'nut', 'nu_transp', 'nut_transp', 'filtrer_resu']) not_set
- **val4** (*type*: int into [0, 1]) not_set
- **mot5** (*type*: string into ['grad_ubar', 'nu', 'nut', 'nu_transp', 'nut_transp', 'filtrer_resu']) not_set
- **val5** (*type*: int into [0, 1]) not_set
- **mot6** (*type*: string into ['grad_ubar', 'nu', 'nut', 'nu_transp', 'nut_transp', 'filtrer_resu']) not_set
- **val6** (*type*: int into [0, 1]) not_set

bloc_ef

not_set

Parameters:

- **mot1** (*type*: string into ['transportant_bar', 'transporte_bar', 'filtrer_resu', 'antisym']) not_set
- **val1** (*type*: int into [0, 1]) not_set
- **mot2** (*type*: string into ['transportant_bar', 'transporte_bar', 'filtrer_resu', 'antisym']) not_set
- **val2** (*type*: int into [0, 1]) not_set
- **mot3** (*type*: string into ['transportant_bar', 'transporte_bar', 'filtrer_resu', 'antisym']) not_set
- **val3** (*type*: int into [0, 1]) not_set
- **mot4** (*type*: string into ['transportant_bar', 'transporte_bar', 'filtrer_resu', 'antisym']) not_set
- **val4** (*type*: int into [0, 1]) not_set

bloc_fichier

Block containing the name of the file

Parameters:

- **fichier** | **file** (*type*: string) File name

bloc_lec_champ_init_canal_sinal

Parameters for the class champ_init_canal_sinal.

in 2D:

$$U = u_{cent} * y(2h - y) / h / h$$

$$V = ampli_bruit * rand + ampli_sin * \sin(\omega * x)$$

rand: unpredictable value between -1 and 1.

in 3D:

$$U = u_{cent} * y(2h - y) / h / h$$

$$V = ampli_bruit * rand1 + ampli_sin * \sin(\omega * x)$$

$$W = ampli_bruit * rand2$$

rand1 and rand2: unpredictable values between -1 and 1.

Parameters:

- **ucent** (*type*: float) Velocity value at the center of the channel.
- **h** (*type*: float) Half henght of the channel.
- **ampli_bruit** (*type*: float) Amplitude for the disturbance.
- **[ampli_sin]** (*type*: float) Amplitude for the sinusoidal disturbance (by default equals to ucent/10).
- **omega** (*type*: float) Value of pulsation for the of the sinusoidal disturbance.
- **[dir_flow]** (*type*: int into [0, 1, 2]) Flow direction for the initialization of the flow in a channel. - if dir_flow=0, the flow direction is X - if dir_flow=1, the flow direction is Y - if dir_flow=2, the flow direction is Z Default value for dir_flow is 0
- **[dir_wall]** (*type*: int into [0, 1, 2]) Wall direction for the initialization of the flow in a channel. - if dir_wall=0, the normal to the wall is in X direction - if dir_wall=1, the normal to the wall is in Y direction - if dir_wall=2, the normal to the wall is in Z direction Default value for dir_flow is 1
- **[min_dir_flow]** (*type*: float) Value of the minimum coordinate in the flow direction for the initialization of the flow in a channel. Default value for dir_flow is 0.
- **[min_dir_wall]** (*type*: float) Value of the minimum coordinate in the wall direction for the initialization of the flow in a channel. Default value for dir_flow is 0.

bloc_lecture_poro

Surface and volume porosity values.

Parameters:

- **volumique** (*type*: float) Volume porosity value.
- **surfactive** (*type*: list of float) Surface porosity values (in X, Y, Z directions).

bloc_op_non_conservativtifs

not_set

Parameters:

- **aco** (*type*: string into ['{'] Opening curly bracket.
- **operateur** (*type*: [op_non_conservativtifs_deriv](#)) not_set
- **acof** (*type*: string into [']}') Closing curly bracket.

bloc_origine_cotes

Class to create a rectangle (or a box).

Parameters:

- **name** (*type*: string into ['origine']) Keyword to define the origin of the rectangle (or the box).
 - **origin | origine** (*type*: list of float) Coordinates of the origin of the rectangle (or the box).
 - **name2** (*type*: string into ['cotes']) Keyword to define the length along the axes.
 - **cotes** (*type*: list of float) Length along the axes.
-

bloc_pave

Class to create a pave.

Parameters:

- **[origine]** (*type*: list of float) Keyword to define the pave (block) origin, that is to say one of the 8 block points (or 4 in a 2D coordinate system).
 - **[longueurs]** (*type*: list of float) Keyword to define the block dimensions, that is to say knowing the origin, length along the axes.
 - **[nombre_de_noeuds]** (*type*: list of int) Keyword to define the discretization (nodenumber) in each direction.
 - **[facteurs]** (*type*: list of float) Keyword to define stretching factors for mesh discretization in each direction. This is a real number which must be positive (by default 1.0). A stretching factor other than 1 allows refinement on one edge in one direction.
 - **[symx]** (*type*: flag) Keyword to define a block mesh that is symmetrical with respect to the YZ plane (respectively Y-axis in 2D) passing through the block centre.
 - **[symy]** (*type*: flag) Keyword to define a block mesh that is symmetrical with respect to the XZ plane (respectively X-axis in 2D) passing through the block centre.
 - **[symz]** (*type*: flag) Keyword defining a block mesh that is symmetrical with respect to the XY plane passing through the block centre.
 - **[xtanh]** (*type*: float) Keyword to generate mesh with tanh (hyperbolic tangent) variation in the X-direction.
 - **[xtanh_dilatation]** (*type*: int into [-1, 0, 1]) Keyword to generate mesh with tanh (hyperbolic tangent) variation in the X-direction. xtanh_dilatation: The value may be -1,0,1 (0 by default): 0: coarse mesh at the middle of the channel and smaller near the walls -1: coarse mesh at the left side of the channel and smaller at the right side 1: coarse mesh at the right side of the channel and smaller near the left side of the channel.
 - **[xtanh_taille_premiere_maille]** (*type*: float) Size of the first cell of the mesh with tanh (hyperbolic tangent) variation in the X-direction.
 - **[ytanh]** (*type*: float) Keyword to generate mesh with tanh (hyperbolic tangent) variation in the Y-direction.
 - **[ytanh_dilatation]** (*type*: int into [-1, 0, 1]) Keyword to generate mesh with tanh (hyperbolic tangent) variation in the Y-direction. ytanh_dilatation: The value may be -1,0,1 (0 by default): 0: coarse mesh at the middle of the channel and smaller near the walls -1: coarse mesh at the bottom of the channel and smaller near the top 1: coarse mesh at the top of the channel and smaller near the bottom.
 - **[ytanh_taille_premiere_maille]** (*type*: float) Size of the first cell of the mesh with tanh (hyperbolic tangent) variation in the Y-direction.
 - **[ztanh]** (*type*: float) Keyword to generate mesh with tanh (hyperbolic tangent) variation in the Z-direction.
 - **[ztanh_dilatation]** (*type*: int into [-1, 0, 1]) Keyword to generate mesh with tanh (hyperbolic tangent) variation in the Z-direction. ztanh_dilatation: The value may be -1,0,1 (0 by default): 0: coarse mesh at the middle of the channel and smaller near the walls -1: coarse mesh at the back of the channel and smaller near the front 1: coarse mesh at the front of the channel and smaller near the back.
 - **[ztanh_taille_premiere_maille]** (*type*: float) Size of the first cell of the mesh with tanh (hyperbolic tangent) variation in the Z-direction.
-

bloc_pdf_model

not_set

Parameters:

- **[moving_ib]** (*type*: flag) flag to activate moving_IB
 - **[use_pseudo_level_set_moving_pdf]** (*type*: flag) flag to activate use_pseudo_level_set_moving_PDF
 - **[ibm_spring_parameter]** (*type*: float) spring coefficient for moving IBM
 - **eta** (*type*: float) penalization coefficient
 - **[bilan_pdf]** (*type*: int) type de bilan du terme PDF (seul/avec temps/avec convection)
 - **[temps_relaxation_coefficient_pdf]** (*type*: float) time relaxation on the forcing term to help
 - **[echelle_relaxation_coefficient_pdf]** (*type*: float) time relaxation on the forcing term to help convergence
 - **[regularization_coefficient_pdf]** (*type*: float) regularization coefficient for the forcing term (dead cell)
 - **[local]** (*type*: flag) whether the prescribed velocity is expressed in the global or local basis
 - **[vitesse_imposee_data]** (*type*: [field_base](#)) Prescribed velocity as a field
 - **[vitesse_imposee_fonction]** (*type*: list of str) Prescribed velocity as a set of ananalytical component
 - **[variable_imposee_data]** (*type*: [field_base](#)) Prescribed variable as a field
 - **[variable_imposee_fonction]** (*type*: list of str) Prescribed variable as a set of ananalytical component
-

bloc_sutherland

Sutherland law for viscosity $\mu(T)=\mu_0*((T_0+C)/(T+C))*(T/T_0)**1.5$ and (optional) for conductivity $\lambda(T)=\mu_0*C_p/Prandtl*((T_0+Slambda)/(T+Slambda))*(T/T_0)**1.5$

Parameters:

- **problem_name** (*type*: string) Name of problem.
 - **mu0** (*type*: string into ['mu0']) not_set
 - **mu0_val** (*type*: float) not_set
 - **t0** (*type*: string into ['t0']) not_set
 - **t0_val** (*type*: float) not_set
 - **[slambda]** (*type*: string into ['slambda']) not_set
 - **[s]** (*type*: float) not_set
 - **c** (*type*: string into ['c']) not_set
 - **c_val** (*type*: float) not_set
-

bloc_tube

Class to create a tube (3D).

Parameters:

- **name** (*type*: string into ['origine']) Keyword to define the center of the tube.
 - **origin** | **origine** (*type*: list of float) Center of the tube.
 - **name2** (*type*: string into ['dir']) Keyword to define the direction of the main axis.
 - **direction** (*type*: string into ['x', 'y', 'z']) direction of the main axis X, Y or Z
 - **name3** (*type*: string into ['ri']) Keyword to define the interior radius.
 - **ri** (*type*: float) Interior radius.
 - **name4** (*type*: string into ['re']) Keyword to define the exterior radius.
 - **re** (*type*: float) Exterior radius.
 - **name5** (*type*: string into ['hauteur']) Keyword to define the height of the tube.
 - **h** (*type*: float) Height of the tube.
-

champ_a_post

Field to be post-processed.

Parameters:

- **champ** (*type*: string) Name of the post-processed field.
 - **[localisation]** (*type*: string into ['elem', 'som', 'faces', 'elem_dg']) Localisation of post-processed field values: The two available values are elem, som, or faces (LATA format only) used respectively to select field values at mesh centres (CHAMPMAILLE type field in the lml file) or at mesh nodes (CHAMPPOINT type field in the lml file). If no selection is made, localisation is set to som by default, elem_dg is specific for dg discretisation to interpolate value on one unique cell center value.
-

champs_posts

Field's write mode.

Parameters:

- **[format]** (*type*: string into ['binaire', 'formatte']) Type of file.
 - **[mot]** (*type*: string into ['dt_post', 'nb_pas_dt_post']) Keyword to set the kind of the field's write frequency. Either a time period or a time step period. it can be specified either here, or at the beginning of the postprocessing bloc.
 - **[period]** (*type*: string) Value of the period which can be like (2.*t).
 - **champs** | **fields** (*type*: list of Champ_a_post) Fields to be post-processed.
-

champs_posts_fichier

Fields read from file.

Parameters:

- **[format]** (*type*: string into ['binaire', 'formatte']) Type of file.
 - **[mot]** (*type*: string into ['dt_post', 'nb_pas_dt_post']) Keyword to set the kind of the field's write frequency. Either a time period or a time step period.
 - **[period]** (*type*: string) Value of the period which can be like (2.*t).
 - **fichier** | **file** (*type*: [bloc_fichier](#)) name of file
-

coarsen_operator_uniform

Object defining the uniform coarsening process of the given grid in IJK discretization

Parameters:

- **[coarsen_operator_uniform]** (*type*: string) not_set
 - **aco** (*type*: string into ['{']) opening curly brace
 - **[coarsen_i]** (*type*: string into ['coarsen_i']) not_set
 - **[coarsen_i_val]** (*type*: int) Integer indicating the number by which we will divide the number of elements in the I direction (in order to obtain a coarser grid)
 - **[coarsen_j]** (*type*: string into ['coarsen_j']) not_set
 - **[coarsen_j_val]** (*type*: int) Integer indicating the number by which we will divide the number of elements in the J direction (in order to obtain a coarser grid)
 - **[coarsen_k]** (*type*: string into ['coarsen_k']) not_set
 - **[coarsen_k_val]** (*type*: int) Integer indicating the number by which we will divide the number of elements in the K direction (in order to obtain a coarser grid)
 - **acof** (*type*: string into ['}']) closing curly brace
-

condinit

Initial condition.

Parameters:

- **nom** (*type*: string) Name of initial condition field.
 - **ch** (*type*: [field_base](#)) Type field and the initial values.
-

condlimlu

Boundary condition specified.

Parameters:

- **bord** (*type*: string) Name of the edge where the boundary condition applies.
 - **cl** (*type*: [condlim_base](#)) Boundary condition at the boundary called bord (edge).
-

definition_champ

Keyword to create new complex field for advanced postprocessing.

Parameters:

- **name** (*type*: string) The name of the new created field.
 - **champ_generique** (*type*: [champ_generique_base](#)) not_set
-

definition_champs_fichier

Keyword to read definition_champs from a file

Parameters:

- **fichier** | **file** (*type*: string) name of file
-

deuxentiers

Two integers.

Parameters:

- **int1** (*type*: int) First integer.
 - **int2** (*type*: int) Second integer.
-

deuxmots

Two words.

Parameters:

- **mot_1** (*type*: string) First word.
 - **mot_2** (*type*: string) Second word.
-

dt_impr_nusselt_mean_only

not_set

Parameters:

- **dt_impr** (*type*: float) not_set
 - **[boundaries]** (*type*: list of str) not_set
-

dt_impr_ustar_mean_only

not_set

Parameters:

- **dt_impr** (*type*: float) not_set
 - **[boundaries]** (*type*: list of str) not_set
-

entierfloat

An integer and a real.

Parameters:

- **the_int** (*type*: int) Integer.
 - **the_float** (*type*: float) Real.
-

floatfloat

Two reals.

Parameters:

- **a** (*type*: float) First real.
 - **b** (*type*: float) Second real.
-

fonction_champ_reprise

not_set

Parameters:

- **mot** (*type*: string into ['fonction']) not_set
 - **fonction** (*type*: list of str) n f1(val) f2(val) ... fn(val)] time
-

form_a_nb_points

The structure function is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.

Parameters:

- **nb** (*type*: int into [4]) Number of points.
 - **dir1** (*type*: int) First direction.
 - **dir2** (*type*: int) Second direction.
-

format_lata_to_med

not_set

Parameters:

- **not** (*type*: string into ['format_post_sup']) not_set
 - **[format]** (*type*: string into ['lml', 'lata', 'lata_v2', 'med']) generated file post_med.data use format (MED or LATA or LML keyword).
-

format_trustlata

Post-processing for STT

Parameters:

- **[fields_to_write]** (*type*: [bloc_lecture](#)) not_set
 - **[write_all_fields]** (*type*: flag) not_set
 - **[use_xyz_nodes]** (*type*: flag) not_set
 - **[use_xy_nodes]** (*type*: flag) not_set
-

info_med

not_set

Parameters:

- **file_med** (*type*: string) Name of the MED file.
 - **domaine** (*type*: string) Name of domain.
 - **pb_post** (*type*: [pb_post](#)) not_set
-

interface_posts

not set

Parameters:

- **[nom_interf]** (*type*: string) name of the interface to post process
 - **blocs** (*type*: list of Champ_a_post) Fields to be post-processed.
-

nom_postraitement

not_set

Parameters:

- **nom** (*type*: string) Name of the post-processing.
 - **post** (*type*: [postraitement_base](#)) the post
-

op_implicite

not_set

Parameters:

- **implicite** (*type*: string into ['implicite']) not_set
- **mot** (*type*: string into ['solveur']) not_set
- **solveur** (*type*: [solveur_sys_base](#)) not_set

quatremots

Three words.

Parameters:

- **mot_1** (*type*: string) First word.
- **mot_2** (*type*: string) Snd word.
- **mot_3** (*type*: string) Third word.
- **mot_4** (*type*: string) Fourth word.

reaction

Keyword to describe reaction:

$$w = K \text{ pow}(T, \text{beta}) \exp(-E_a / (R T)) \prod_i \text{pow}(\text{Reactif}_i, \text{activity}_i).$$
If $K_{\text{inv}} > 0$,
$$w = K \text{ pow}(T, \text{beta}) \exp(-E_a / (R T)) \left(\prod_i \text{pow}(\text{Reactif}_i, \text{activity}_i) - K_{\text{inv}} / \exp(-c_r E_a / (R T)) \prod_i \text{pow}(\text{Produit}_i, \text{activity}_i) \right)$$
Parameters:

- **reactifs** (*type*: string) LHS of equation (ex CH₄+2*O₂)
- **produits** (*type*: string) RHS of equation (ex CO₂+2*H₂O)
- **[constante_taux_reaction]** (*type*: float) constante of cinetic K
- **enthalpie_reaction** (*type*: float) DH
- **energie_activation** (*type*: float) Ea
- **exposant_beta** (*type*: float) Beta
- **[coefficients_activites]** (*type*: [bloc_lecture](#)) coefficients of activity (exemple { CH₄ 1 O₂ 2 })
- **[contre_reaction]** (*type*: float) K_{inv}
- **[contre_energie_activation]** (*type*: float) c_r Ea

remove_elem_bloc

not_set

Parameters:

- **[liste]** (*type*: list of int) not_set
- **[fonction]** (*type*: string) not_set

sonde

Keyword is used to define the probes. Observations: the probe coordinates should be given in Cartesian coordinates (X, Y, Z), including axisymmetric.

Parameters:

- **nom_sonde** (*type*: string) Name of the file in which the values taken over time will be saved. The complete file name is nom_sonde.son.
- **[special]** (*type*: string into ['grav', 'som', 'nodes', 'chsom', 'gravcl']) Option to change the positions of the probes. Several options are available: grav : each probe is moved to the nearest cell center of the mesh; som : each probe is moved to the nearest vertex of the mesh nodes : each probe is moved to the nearest face center of the mesh; chsom : only available for P1NC sampled field. The values of the probes are calculated according to P1-Conform corresponding field. gravcl : Extend to the domain face boundary a cell-located segment probe in order to have the boundary condition for the field. For this type the extreme probe point has to be on the face center of gravity.
- **nom_inco** (*type*: string) Name of the sampled field.
- **mperiode** (*type*: string into ['periode']) Keyword to set the sampled field measurement frequency.
- **prd** (*type*: float) Period value. Every prd seconds, the field value calculated at the previous time step is written to the nom_sonde.son file.
- **type** (*type*: [sonde_base](#)) Type of probe.

sondes_fichier

Keyword to read probes from a file

Parameters:

- **fichier** | **file** (*type*: string) name of file

sous_zone_valeur

Two words.

Parameters:

- **sous_zone** (*type*: string) sous zone
- **valeur** (*type*: float) value

stats_posts

Post-processing for statistics. Example:

```
Statistiques Dt_post dtst {
t_deb 0.1 t_fin 0.12
Moyenne Pression
Ecart_type Pression
Correlation Vitesse Vitesse
}
```

will write every dt_post the mean, standard deviation and correlation value:

if $t < t_{deb}$ or $t > t_{fin}$

$$\text{average: } \overline{P(t)} = 0$$

$$\text{std deviation: } \langle P(t) \rangle = 0$$

$$\text{correlation: } \langle U(t) \cdot V(t) \rangle = 0$$

if $t > t_{deb}$ and $t < t_{fin}$

$$\text{average: } \overline{P(t)} = \frac{1}{t - t_{deb}} \int_{t_{deb}}^t P(s) ds$$

$$\text{std deviation: } \langle P(t) \rangle = \sqrt{\frac{1}{t - t_{deb}} \int_{t_{deb}}^t [P(s) - \overline{P(t)}]^2 ds}$$

$$\text{correlation: } \langle U(t) \cdot V(t) \rangle = \frac{1}{t - t_{deb}} \int_{t_{deb}}^t [U(s) - \overline{U(t)}] \cdot [V(s) - \overline{V(t)}] ds$$

Parameters:

- **[mot]** (*type*: string into ['dt_post', 'nb_pas_dt_post']) Keyword to set the kind of the field's write frequency. Either a time period or a time step period.
- **[period]** (*type*: string) Value of the period which can be like (2.*t).
- **champs | fields** (*type*: list of Stat_post_deriv) Post-processing for statistics

stats_posts_fichier

Statistics read from file.. Example:

Statistiques Dt_post dtst {

t_deb 0.1 t_fin 0.12

Moyenne Pression

Ecart_type Pression

Correlation Vitesse Vitesse

}

will write every dt_post the mean, standard deviation and correlation value:

if $t < t_{deb}$ or $t > t_{fin}$

$$\text{average: } \overline{P(t)} = 0$$

$$\text{std deviation: } \langle P(t) \rangle = 0$$

$$\text{correlation: } \langle U(t) \cdot V(t) \rangle = 0$$

if $t > t_{deb}$ and $t < t_{fin}$

$$\text{average: } \overline{P(t)} = \frac{1}{t - t_{deb}} \int_{t_{deb}}^t P(s) ds$$

$$\text{std deviation: } \langle P(t) \rangle = \sqrt{\frac{1}{t - t_{deb}} \int_{t_{deb}}^t [P(s) - \overline{P(t)}]^2 ds}$$

$$\text{correlation: } \langle U(t) \cdot V(t) \rangle = \frac{1}{t - t_{deb}} \int_{t_{deb}}^t [U(s) - \overline{U(t)}] \cdot [V(s) - \overline{V(t)}] ds$$

Parameters:

- **[mot]** (*type*: string into ['dt_post', 'nb_pas_dt_post']) Keyword to set the kind of the field's write frequency. Either a time period or a time step period.
- **[period]** (*type*: string) Value of the period which can be like (2.*t).
- **fichier | file** (*type*: [bloc_fichier](#)) name of file

stats_serie_posts

This keyword is used to set the statistics. Average on dt_integr time interval is post- processed every dt_integr seconds. Example:

```
Statistiques_en_serie Dt_integr dtst {
Moyenne Pression
}
```

will calculate and write every dtst seconds the mean value:

$$(n + 1) dt_{integr} > t > n \cdot dt_{integr}, \quad \overline{P(t)} = \frac{1}{t - n \cdot dt_{integr}} \int_{n \cdot dt_{integr}}^t P(t) dt$$

Parameters:

- **mot** (*type*: string into ['dt_integr']) Keyword is used to set the statistics period of integration and write period.
- **dt_integr** (*type*: float) Average on dt_integr time interval is post-processed every dt_integr seconds.
- **stat** (*type*: list of Stat_post_deriv) Post-processing for statistics

stats_serie_posts_fichier

This keyword is used to set the statistics read from a file. Average on dt_integr time interval is post-processed every dt_integr seconds. Example:

```
Statistiques_en_serie Dt_integr dtst {
Moyenne Pression
}
```

will calculate and write every dtst seconds the mean value:

$$(n + 1) dt_{integr} > t > n \cdot dt_{integr}, \quad \overline{P(t)} = \frac{1}{t - n \cdot dt_{integr}} \int_{n \cdot dt_{integr}}^t P(t) dt$$

Parameters:

- **mot** (*type*: string into ['dt_integr']) Keyword is used to set the statistics period of integration and write period.
- **dt_integr** (*type*: float) Average on dt_integr time interval is post-processed every dt_integr seconds.
- **fichier** | **file** (*type*: [bloc_fichier](#)) name of file

traitement_particulier

Auxiliary class to post-process particular values.

Parameters:

- **aco** (*type*: string into ['{']) Opening curly bracket.
- **trait_part** (*type*: [traitement_particulier_base](#)) Type of traitement_particulier.
- **acof** (*type*: string into [']) Closing curly bracket.

troisf

Auxiliary class to extrude.

Parameters:

- **lx** (*type*: float) X direction of the extrude operation.
- **ly** (*type*: float) Y direction of the extrude operation.
- **lz** (*type*: float) Z direction of the extrude operation.

troismots

Three words.

Parameters:

- **mot_1** (*type*: string) First word.
 - **mot_2** (*type*: string) Snd word.
 - **mot_3** (*type*: string) Third word.
-

type_postraitement_ft_lata

not_set

Parameters:

- **type** (*type*: string into ['postraitement_ft_lata', 'postraitement_ftd']) not_set
 - **nom** (*type*: string) Name of the post-processing.
 - **bloc** (*type*: [bloc_lecture](#)) not_set
-

type_un_post

not_set

Parameters:

- **type** (*type*: string into ['postraitement', 'post_processing']) not_set
 - **post** (*type*: [un_postraitement](#)) not_set
-

un_pb

pour les groupes

Parameters:

- **mot** (*type*: string) the string
-

un_point

A point.

Parameters:

- **pos** (*type*: list of float) Point coordinates.
-

un_postraitement

An object of post-processing (with name).

Parameters:

- **nom** (*type*: string) Name of the post-processing.
 - **post** (*type*: [corps_postraitement](#)) Definition of the post-processing.
-

un_postraitement_spec

An object of post-processing (with type +name).

Parameters:

- **[type_un_post]** (*type*: [type_un_post](#)) not_set
 - **[type_postraitement_ft_lata]** (*type*: [type_postraitement_ft_lata](#)) not_set
-

verifiercoin_bloc

not_set

Parameters:

- **[read_file | filename | lire_fichier]** (*type*: string) name of the *.decoupage_som file

1.22.1 Keywords derived from bloc_lecture**bloc_lecture**

to read between two braces

Parameters:

- **bloc_lecture** (*type*: string) not_set
-

bloc_criteres_convergenceInherits from: [bloc_lecture](#)

Not set

Parameters:

- **bloc_lecture** (*type*: string) not_set
-

solveur_petsc_option_cliInherits from: [bloc_lecture](#)

solver

Parameters:

- **bloc_lecture** (*type*: string) not_set

1.22.2 Keywords derived from bord_base**bord_base**

Basic class for block sides. Block sides that are neither edges nor connectors are not specified. The duplicate nodes of two blocks in contact are automatically recognized and deleted.

bordInherits from: [bord_base](#)

The block side is not in contact with another block and boundary conditions are applied to it.

Parameters:

- **nom** (*type*: string) Name of block side.
 - **defbord** (*type*: [defbord](#)) Definition of block side.
-

groupe_facesInherits from: [bord_base](#)

A named selection of mesh faces, defined by a plane equation. Same readOn shape as Bord (name + plane), used to expose internal face sets for post-processing or coupling.

Parameters:

- **nom** (*type*: string) Name of the face group.
 - **defbord** (*type*: [defbord](#)) Definition of the plane selecting the faces.
-

internes**Inherits from:** [bord_base](#)

To indicate that the block has a set of internal faces (these faces will be duplicated automatically by the program and will be processed in a manner similar to edge faces).

Two boundaries with the same boundary conditions may have the same name (whether or not they belong to the same block).

The keyword Internes (Internal) must be used to execute a calculation with plates, followed by the equation of the surface area covered by the plates.

Parameters:

- **nom** (*type*: string) Name of block side.
 - **defbord** (*type*: [defbord](#)) Definition of block side.
-

raccord**Inherits from:** [bord_base](#)

The block side is in contact with the block of another domain (case of two coupled problems).

Parameters:

- **type1** (*type*: string into ['local', 'distant']) Contact type.
 - **type2** (*type*: string into ['homogene']) Contact type.
 - **nom** (*type*: string) Name of block side.
 - **defbord** (*type*: [defbord](#)) Definition of block side.
-

1.22.3 Keywords derived from convection_deriv**convection_deriv**[not_set](#)**convection_amont****Synonyms:** [amont](#)**Inherits from:** [convection_deriv](#)

Keyword for upwind scheme for VDF or VEF discretizations.

convection_amont_old**Synonyms:** [amont_old](#)**Inherits from:** [convection_deriv](#)

Only for VEF discretization, obsolete keyword, see [amont](#).

convection_btd**Synonyms:** [btd](#)**Inherits from:** [convection_deriv](#)

Only for EF discretization.

Parameters:

- **btd** (*type*: float) [not_set](#)
 - **facteur** (*type*: float) [not_set](#)
-

convection_centre**Synonyms:** [centre](#)**Inherits from:** [convection_deriv](#)

For VDF and VEF discretizations.

convection_centre4**Synonyms:** centre4**Inherits from:** [convection_deriv](#)

For VDF and VEF discretizations.

convection_centre_old**Synonyms:** centre_old**Inherits from:** [convection_deriv](#)

Only for VEF discretization.

convection_di_l2**Synonyms:** di_l2**Inherits from:** [convection_deriv](#)

Only for VEF discretization.

convection_ef**Synonyms:** ef**Inherits from:** [convection_deriv](#)

For VEF calculations, a centred convective scheme based on Finite Elements formulation can be called through the following data:

Convection { EF transportant_bar val transporte_bar val antisym val filtrer_resu val }

This scheme is 2nd order accuracy (and get better the property of kinetic energy conservation). Due to possible problems of instabilities phenomena, this scheme has to be coupled with stabilisation process (see Source_Qdm↔_lambdaup). These two last data are equivalent from a theoretical point of view in variationnal writing to : $\text{div}((u \cdot \text{grad } ub, vb) - (u \cdot \text{grad } vb, ub))$, where vb corresponds to the filtered reference test functions.

Remark:

This class requires to define a filtering operator : see solveur_bar

Parameters:

- **[mot1]** (*type*: string into ['default_bar']) equivalent to transportant_bar 0 transporte_bar 1 filtrer_resu 1 antisym 1
- **[bloc_ef]** (*type*: [bloc_ef](#)) not_set

convection_ef_stab**Synonyms:** ef_stab**Inherits from:** [convection_deriv](#)

Keyword for a VEF convective scheme.

Parameters:

- **[alpha]** (*type*: float) To weight the scheme centering with the factor double (between 0 (full centered) and 1 (mix between upwind and centered), by default 1). For scalar equation, it is advised to use alpha=1 and for the momentum equation, alpha=0.2 is advised.
- **[test]** (*type*: int) Developer option to compare old and new version of EF_stab
- **[tdivu]** (*type*: flag) To have the convective operator calculated as $\text{div}(TU) - T\text{div}U (= U\text{grad}T)$.
- **[old]** (*type*: flag) To use old version of EF_stab scheme (default no).
- **[volumes_etendus]** (*type*: flag) Option for the scheme to use the extended volumes (default, yes).
- **[volumes_non_etendus]** (*type*: flag) Option for the scheme to not use the extended volumes (default, no).
- **[amont_sous_zone]** (*type*: string) Option to degenerate EF_stab scheme into Amont (upwind) scheme in the sub zone of name sz_name. The sub zone may be located arbitrarily in the domain but the more often this option will be activated in a zone where EF_stab scheme generates instabilities as for free outlet for example.
- **[alpha_sous_zone]** (*type*: list of Sous_zone_valeur) List of groups of two words.

convection_generic**Synonyms:** generic**Inherits from:** [convection_deriv](#)

Keyword for generic calling of upwind and muscl convective scheme in VEF discretization. For muscl scheme, limiters and order for fluxes calculations have to be specified. The available limiters are : minmod - vanleer - vanalbada - chakravarthy - superbee, and the order of accuracy is 1 or 2. Note that chakravarthy is a non-symmetric limiter and superbee may engender results out of physical limits. By consequence, these two limiters are not recommended.

Examples:

convection { generic amount }

convection { generic muscl minmod 1 }

convection { generic muscl vanleer 2 }

In case of results out of physical limits with muscl scheme (due for instance to strong non-conformal velocity flow field), user can redefine in data file a lower order and a smoother limiter, as : convection { generic muscl minmod 1 }

Parameters:

- **type** (*type*: string into ['amount', 'muscl', 'centre']) type of scheme
- **[limiteur]** (*type*: string into ['minmod', 'vanleer', 'vanalbada', 'chakravarthy', 'superbee']) type of limiter
- **[ordre]** (*type*: int into [1, 2, 3]) order of accuracy
- **[alpha]** (*type*: float) alpha

convection_hll**Synonyms:** hll**Inherits from:** [convection_deriv](#)

Keyword for Coloc discretization. Activates the HLL conservative scheme.

convection_kquick**Synonyms:** kquick**Inherits from:** [convection_deriv](#)

Only for VEF discretization.

convection_muscl**Synonyms:** muscl**Inherits from:** [convection_deriv](#)

Keyword for muscl scheme in VEF discretization equivalent to generic muscl vanleer 2 for the 1.5 version or later. The previous muscl scheme can be used with the obsolete in future muscl_old keyword.

convection_muscl3**Synonyms:** muscl3**Inherits from:** [convection_deriv](#)

Keyword for a scheme using a ponderation between muscl and center schemes in VEF.

Parameters:

- **[alpha]** (*type*: float) To weight the scheme centering with the factor double (between 0 (full centered) and 1 (muscl), by default 1).

convection_muscl_new**Synonyms:** muscl_new**Inherits from:** [convection_deriv](#)

Only for VEF discretization.

convection_muscl_old**Synonyms:** muscl_old**Inherits from:** [convection_deriv](#)Only for VEF discretization.

convection_negligeable**Synonyms:** negligeable**Inherits from:** [convection_deriv](#)For VDF and VEF discretizations. Suppresses the convection operator.

convection_quick**Synonyms:** quick**Inherits from:** [convection_deriv](#)Only for VDF discretization.

convection_rusanov**Synonyms:** rusanov**Inherits from:** [convection_deriv](#)Keyword for Coloc discretization. Activates the Rusanov conservative scheme.

convection_supg**Synonyms:** supg**Inherits from:** [convection_deriv](#)

Only for EF discretization.

Parameters:

- **facteur** (*type*: float) not_set

1.22.4 Keywords derived from defbord

defbordClass to define an edge.

defbord_2**Inherits from:** [defbord](#)

1-D edge (straight line) in the 2-D space.

Parameters:

- **dir** (*type*: string into ['x', 'y']) Edge is perpendicular to this direction.
 - **eq** (*type*: string into ['=']) Equality sign.
 - **pos** (*type*: float) Position value.
 - **pos2_min** (*type*: float) Minimal value.
 - **inf1** (*type*: string into ['<=']) Less than or equal to sign.
 - **dir2** (*type*: string into ['x', 'y']) Edge is parallel to this direction.
 - **inf2** (*type*: string into ['<=']) Less than or equal to sign.
 - **pos2_max** (*type*: float) Maximal value.
-

defbord_3**Inherits from:** [defbord](#)

2-D edge (plane) in the 3-D space.

Parameters:

- **dir** (*type*: string into ['x', 'y', 'z']) Edge is perpendicular to this direction.
- **eq** (*type*: string into ['=']) Equality sign.
- **pos** (*type*: float) Position value.
- **pos2_min** (*type*: float) Minimal value.
- **inf1** (*type*: string into ['<=']) Less than or equal to sign.
- **dir2** (*type*: string into ['x', 'y']) Edge is parallel to this direction.
- **inf2** (*type*: string into ['<=']) Less than or equal to sign.
- **pos2_max** (*type*: float) Maximal value.
- **pos3_min** (*type*: float) Minimal value.
- **inf3** (*type*: string into ['<=']) Less than or equal to sign.
- **dir3** (*type*: string into ['y', 'z']) Edge is parallel to this direction.
- **inf4** (*type*: string into ['<=']) Less than or equal to sign.
- **pos3_max** (*type*: float) Maximal value.

1.22.5 Keywords derived from diffusion_deriv**diffusion_deriv**not_set

diffusion_negligeable**Synonyms:** negligeable**Inherits from:** [diffusion_deriv](#)the diffusivity will not taken in count

diffusion_option**Synonyms:** option**Inherits from:** [diffusion_deriv](#)

not_set

Parameters:

- **bloc_lecture** (*type*: [bloc_lecture](#)) not_set
-

diffusion_p1ncp1b**Synonyms:** p1ncp1b**Inherits from:** [diffusion_deriv](#)not_set

diffusion_stab**Synonyms:** stab**Inherits from:** [diffusion_deriv](#)

keyword allowing consistent and stable calculations even in case of obtuse angle meshes.

Parameters:

- **[standard]** (*type*: int) to recover the same results as calculations made by standard laminar diffusion operator. However, no stabilization technique is used and calculations may be unstable when working with obtuse angle meshes (by default 0)
- **[info]** (*type*: int) developer option to get the stabilizing ratio (by default 0)
- **[new_jacobian]** (*type*: int) when implicit time schemes are used, this option defines a new jacobian that may be more suitable to get stationary solutions (by default 0)
- **[nu]** (*type*: int) (respectively nut 1) takes the molecular viscosity (resp. eddy viscosity) into account in the velocity gradient part of the diffusion expression (by default nu=1 and nut=1)
- **[nut]** (*type*: int) not_set
- **[nu_transp]** (*type*: int) (respectively nut_transp 1) takes the molecular viscosity (resp. eddy viscosity) into account in the transposed velocity gradient part of the diffusion expression (by default nu_transp=0 and nut←_transp=1)
- **[nut_transp]** (*type*: int) not_set

diffusion_standard**Synonyms:** standard**Inherits from:** [diffusion_deriv](#)

A new keyword, intended for LES calculations, has been developed to optimise and parameterise each term of the diffusion operator. Remark:

1. This class requires to define a filtering operator : see solveur_bar
2. The former (original) version: diffusion { } -which omitted some of the term of the diffusion operator- can be recovered by using the following parameters in the new class :

diffusion { standard grad_Ubar 0 nu 1 nut 1 nu_transp 0 nut_transp 1 filtrer_resu 0}.

Parameters:

- **[mot1]** (*type*: string into ["defaut_bar"]) equivalent to grad_Ubar 1 nu 1 nut 1 nu_transp 1 nut←_transp 1 filtrer←_resu 1
- **[bloc_diffusion_standard]** (*type*: [bloc_diffusion_standard](#)) not_set

diffusion_turbulente_multiphase**Synonyms:** turbulente**Inherits from:** [diffusion_deriv](#)

Turbulent diffusion operator for multiphase problem

Parameters:

- **[type]** (*type*: [type_diffusion_turbulente_multiphase_deriv](#)) Turbulence model for multiphase problem

difusion_p1b**Synonyms:** p1b**Inherits from:** [diffusion_deriv](#)

not_set

1.22.6 Keywords derived from format_file_base

format_file_base

Format of the file

Parameters:

- **checkpoint_fname** (*type*: string) Name of file.
-

binaire

Inherits from: [format_file_base](#)

Format of the file - binary version

Parameters:

- **checkpoint_fname** (*type*: string) Name of file.
-

formatte

Inherits from: [format_file_base](#)

Format of the file - formatte version

Parameters:

- **checkpoint_fname** (*type*: string) Name of file.
-

pdi

Inherits from: [format_file_base](#)

Format of the file - pdi version

Parameters:

- **checkpoint_fname** (*type*: string) Name of file.
-

pdi_expert

Inherits from: [format_file_base](#)

Format of the file - PDI expert version

Parameters:

- **yaml_fname** (*type*: string) YAML file name
 - **checkpoint_fname** (*type*: string) Name of file.
-

single_hdf

Inherits from: [format_file_base](#)

Format of the file - single_hdf version

Parameters:

- **checkpoint_fname** (*type*: string) Name of file.
-

xyz

Inherits from: [format_file_base](#)

Format of the file - xyz version

Parameters:

- **checkpoint_fname** (*type*: string) Name of file.
-

1.22.7 Keywords derived from `lecture_bloc_moment_base`

`lecture_bloc_moment_base`

Auxiliary class to compute and print the moments.

`calcul`

Inherits from: [lecture_bloc_moment_base](#)

The centre of gravity will be calculated.

`centre_de_gravite`

Inherits from: [lecture_bloc_moment_base](#)

To specify the centre of gravity.

Parameters:

- **point** (*type*: [un_point](#)) A centre of gravity.

1.22.8 Keywords derived from `mailler_base`

`mailler_base`

Basic class to mesh.

- [Keywords derived from pave](#)

`domain`

Inherits from: [mailler_base](#)

Class to reuse a domain.

Parameters:

- **domain_name** (*type*: string) Name of domain.
-

`epsilon`

Inherits from: [mailler_base](#)

Two points will be confused if the distance between them is less than `eps`. By default, `eps` is set to `1e-12`. The keyword `Epsilon` allows an alternative value to be assigned to `eps`.

Parameters:

- **eps** (*type*: float) New value of precision.

1.22.8.1 Keywords derived from `pave`

`pave`

Inherits from: [mailler_base](#)

Class to create a pave (block) with boundaries.

Parameters:

- **name** (*type*: string) Name of the pave (block).
 - **bloc** (*type*: [bloc_pave](#)) Definition of the pave (block).
 - **list_bord** (*type*: list of [Bord_base](#)) The block sides.
-

pave_64**Inherits from:** [pave](#)

Same as Pave for big (64b) domain

Parameters:

- **name** (*type*: string) Name of the pave (block).
- **bloc** (*type*: [bloc_pave](#)) Definition of the pave (block).
- **list_bord** (*type*: list of Bord_base) The block sides.

1.22.9 Keywords derived from modele_turbulence_hyd_deriv**modele_turbulence_hyd_deriv**

Basic class for turbulence model for Navier-Stokes equations.

Parameters:

- **[turbulence_paroil]** (*type*: [turbulence_paroil_base](#)) Keyword to set the wall law.
- **[dt_impr_ustar]** (*type*: float) This keyword is used to print the values (U +, d +, u_{star}) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **[dt_impr_ustar_mean_only]** (*type*: [dt_impr_ustar_mean_only](#)) This keyword is used to print the mean values of u_{star} (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u_{star} , then you have to specify their names.
- **[nut_max]** (*type*: float) Upper limitation of turbulent viscosity (default value 1.e8).
- **[correction_visco_turb_pour_controle_pas_de_temps]** (*type*: flag) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **[correction_visco_turb_pour_controle_pas_de_temps_parametre]** (*type*: float) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

-
- [Keywords derived from mod_turb_hyd_ss_maille](#)

modele_turbulence_hyd_0_eq_base**Inherits from:** [modele_turbulence_hyd_deriv](#)

Turbulence model constructed using several fields.

Parameters:

- **[turbulence_paroil]** (*type*: [turbulence_paroil_base](#)) Keyword to set the wall law.
- **[dt_impr_ustar]** (*type*: float) This keyword is used to print the values (U +, d +, u_{star}) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **[dt_impr_ustar_mean_only]** (*type*: [dt_impr_ustar_mean_only](#)) This keyword is used to print the mean values of u_{star} (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u_{star} , then you have to specify their names.

- **[nut_max]** (*type*: float) Upper limitation of turbulent viscosity (default value 1.e8).
- **[correction_visco_turb_pour_controle_pas_de_temps]** (*type*: flag) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **[correction_visco_turb_pour_controle_pas_de_temps_parametre]** (*type*: float) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

modele_turbulence_hyd_null

Synonyms: null

Inherits from: [modele_turbulence_hyd_deriv](#)

Null turbulence model (turbulent viscosity = 0) which can be used with a turbulent problem.

Parameters:

- **[turbulence_paroil]** (*type*: [turbulence_paroil_base](#)) Keyword to set the wall law.
- **[dt_impr_ustar]** (*type*: float) This keyword is used to print the values (U +, d+, u\$star\$) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **[dt_impr_ustar_mean_only]** (*type*: [dt_impr_ustar_mean_only](#)) This keyword is used to print the mean values of u^* (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u^* , then you have to specify their names.
- **[nut_max]** (*type*: float) Upper limitation of turbulent viscosity (default value 1.e8).
- **[correction_visco_turb_pour_controle_pas_de_temps]** (*type*: flag) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **[correction_visco_turb_pour_controle_pas_de_temps_parametre]** (*type*: float) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

1.22.9.1 Keywords derived from mod_turb_hyd_ss_maille

mod_turb_hyd_ss_maille

Inherits from: [modele_turbulence_hyd_deriv](#)

Class for sub-grid turbulence model for Navier-Stokes equations.

Parameters:

- **[formulation_a_nb_points]** (*type*: [form_a_nb_points](#)) The structure fonction is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homegeneity planes. Example for channel flows, planes parallel to the walls.
- **[longueur_maille]** (*type*: string into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']) Different ways to calculate the characteristic length may be specified : `volume` : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another. `volume_sans_lissage` : For VEF only. Characteristic length is based

on the cubic root of the volume cells (without smoothing procedure). **scotti** : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes. **arete** : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.

- **[turbulence_paroil]** (*type: turbulence_paroil_base*) Keyword to set the wall law.
- **[dt_impr_ustar]** (*type: float*) This keyword is used to print the values (U +, d +, u_{star}) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **[dt_impr_ustar_mean_only]** (*type: dt_impr_ustar_mean_only*) This keyword is used to print the mean values of u_* (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u_* , then you have to specify their names.
- **[nut_max]** (*type: float*) Upper limitation of turbulent viscosity (default value 1.e8).
- **[correction_visco_turb_pour_controle_pas_de_temps]** (*type: flag*) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **[correction_visco_turb_pour_controle_pas_de_temps_parametre]** (*type: float*) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

longueur_melange

Inherits from: [mod_turb_hyd_ss_maille](#)

This model is based on mixing length modelling. For a non academic configuration, formulation used in the code can be expressed basically as :

$$\nu_t = (Kappa.y)^{2.5} \cdot dU/dy$$

Till a maximum distance (`dmax`) set by the user in the data file, y is set equal to the distance from the wall (`dist_w`) calculated previously and saved in file `Wall_length.xyz`. [see `Distance_paroil` keyword]

Then (from $y=dmax$), y decreases as an exponential function : $y = dmax \cdot \exp[-2 \cdot (dist_w - dmax)/dmax]$

Parameters:

- **[canalx]** (*type: float*) [height] : plane channel according to Ox direction (for the moment, formulation in the code relies on fixed height : $H=2$).
- **[tuyauz]** (*type: float*) [diameter] : pipe according to Oz direction (for the moment, formulation in the code relies on fixed diameter : $D=2$).
- **[dmax]** (*type: float*) Maximum distance.
- **[fichier]** (*type: string*) `not_set`
- **[fichier_ecriture_k_eps]** (*type: string*) When a resume with k-epsilon model is envisaged, this keyword allows to generate external MED-format file with evaluation of k and epsilon quantities (based on eddy turbulent viscosity and turbulent characteristic length returned by mixing length model). The frequency of the MED file print is set equal to `dt_impr_ustar`. Moreover, k-eps MED field is automatically saved at the last time step. MED file is then used for resuming a K-Epsilon calculation with the `Champ_Fonc_Med` keyword.
- **[formulation_a_nb_points]** (*type: form_a_nb_points*) The structure function is calculated on `nb` points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.

- **[longueur_maille]** (*type*: string into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']) Different ways to calculate the characteristic length may be specified : volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another. volume_sans_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure). scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes. arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **[turbulence_paroil]** (*type*: [turbulence_paroil_base](#)) Keyword to set the wall law.
- **[dt_impr_ustar]** (*type*: float) This keyword is used to print the values (U +, d +, u \$\star\$) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **[dt_impr_ustar_mean_only]** (*type*: [dt_impr_ustar_mean_only](#)) This keyword is used to print the mean values of u^* (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u^* , then you have to specify their names.
- **[nut_max]** (*type*: float) Upper limitation of turbulent viscosity (default value 1.e8).
- **[correction_visco_turb_pour_controle_pas_de_temps]** (*type*: flag) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **[correction_visco_turb_pour_controle_pas_de_temps_parametre]** (*type*: float) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

sous_maille_smago

Inherits from: [mod_turb_hyd_ss_maille](#)

Smagorinsky sub-grid turbulence model.

$Nut = Cs1 * Cs1 * |l| * \sqrt{2 * S * S}$

$K = Cs2 * Cs2 * |l| * 2 * S$

Parameters:

- **[cs]** (*type*: float) This is an optional keyword and the value is used to set the constant used in the Smagorinsky model (This is currently only valid for Smagorinsky models and it is set to 0.18 by default) .
- **[formulation_a_nb_points]** (*type*: [form_a_nb_points](#)) The structure fonction is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **[longueur_maille]** (*type*: string into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']) Different ways to calculate the characteristic length may be specified : volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another. volume_sans_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure). scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes. arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **[turbulence_paroil]** (*type*: [turbulence_paroil_base](#)) Keyword to set the wall law.

- **[dt_impr_ustar]** (*type*: float) This keyword is used to print the values (U +, d +, u_{star}) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **[dt_impr_ustar_mean_only]** (*type*: [dt_impr_ustar_mean_only](#)) This keyword is used to print the mean values of u_* (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u_* , then you have to specify their names.
- **[nut_max]** (*type*: float) Upper limitation of turbulent viscosity (default value $1.e8$).
- **[correction_visco_turb_pour_controle_pas_de_temps]** (*type*: flag) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **[correction_visco_turb_pour_controle_pas_de_temps_parametre]** (*type*: float) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

sous_maille_wale

Inherits from: [mod_turb_hyd_ss_maille](#)

This is the WALE-model. It is a new sub-grid scale model for eddy-viscosity in LES that has the following properties :

- it goes naturally to 0 at the wall (it doesn't need any information on the wall position or geometry)
- it has the proper wall scaling in $o(y^3)$ in the vicinity of the wall
- it reproduces correctly the laminar to turbulent transition.

Parameters:

- **[cw]** (*type*: float) The unique parameter (constant) of the WALE-model (by default value 0.5).
- **[formulation_a_nb_points]** (*type*: [form_a_nb_points](#)) The structure function is calculated on `nb` points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **[longueur_maille]** (*type*: string into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']) Different ways to calculate the characteristic length may be specified : `volume` : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another. `volume_sans_lissage` : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure). `scotti` : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes. `arete` : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **[turbulence_paroil]** (*type*: [turbulence_paroil_base](#)) Keyword to set the wall law.
- **[dt_impr_ustar]** (*type*: float) This keyword is used to print the values (U +, d +, u_{star}) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **[dt_impr_ustar_mean_only]** (*type*: [dt_impr_ustar_mean_only](#)) This keyword is used to print the mean values of u_* (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u_* , then you have to specify their names.

- **[nut_max]** (*type*: float) Upper limitation of turbulent viscosity (default value 1.e8).
- **[correction_visco_turb_pour_controle_pas_de_temps]** (*type*: flag) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **[correction_visco_turb_pour_controle_pas_de_temps_parametre]** (*type*: float) Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

1.22.10 Keywords derived from `op_non_conservativtifs_deriv`

`op_non_conservativtifs_deriv`

not set

`termes_non_conservatifs_hll`

Synonyms: hll

Inherits from: [op_non_conservativtifs_deriv](#)

Keyword for Coloc discretization. Activates the HLL non-conservative scheme.

`termes_non_conservatifs_negligeable`

Synonyms: negligeable

Inherits from: [op_non_conservativtifs_deriv](#)

For Coloc discretization. Suppresses the non_conservative operator.

1.22.11 Keywords derived from `parametre_equation_base`

`parametre_equation_base`

Basic class for `parametre_equation`

`parametre_diffusion_implicit`

Inherits from: [parametre_equation_base](#)

To specify additional parameters for the equation when using implicit diffusion

Parameters:

- **[crank]** (*type*: int into [0, 1]) Use (1) or not (0, default) a Crank Nicholson method for the diffusion implicitation algorithm. Setting crank to 1 increases the order of the algorithm from 1 to 2.
 - **[preconditionnement_diag]** (*type*: int into [0, 1]) The CG used to solve the implicitation of the equation diffusion operator is not preconditioned by default. If this option is set to 1, a diagonal preconditioning is used. Warning: this option is not necessarily more efficient, depending on the treated case.
 - **[niter_max_diffusion_implicit]** (*type*: int) Change the maximum number of iterations for the CG (Conjugate Gradient) algorithm when solving the diffusion implicitation of the equation.
 - **[seuil_diffusion_implicit]** (*type*: float) Change the threshold convergence value used by default for the CG resolution for the diffusion implicitation of this equation.
 - **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Conjugate Gradient) to solve the linear system.
-

parametre_implicit**Inherits from:** [parametre_equation_base](#)

Keyword to change for this equation only the parameter of the implicit scheme used to solve the problem.

Parameters:

- **[seuil_convergence_implicit]** (*type*: float) Keyword to change for this equation only the value of `seuil_↔convergence_implicit` used in the implicit scheme.
- **[seuil_convergence_solveur]** (*type*: float) Keyword to change for this equation only the value of `seuil_↔convergence_solveur` used in the implicit scheme
- **[solveur]** (*type*: [solveur_sys_base](#)) Keyword to change for this equation only the solver used in the implicit scheme
- **[resolution_explicite]** (*type*: flag) To solve explicitly the equation whereas the scheme is an implicit scheme.
- **[equation_non_resolue]** (*type*: flag) Keyword to specify that the equation is not solved.
- **[equation_frequence_resolue]** (*type*: string) Keyword to specify that the equation is solved only every *n* time steps (*n* is an integer or given by a time-dependent function *f(t)*).

1.22.12 Keywords derived from postraitement_base**postraitement_base**`not_set`

- [Keywords derived from postraitement](#)

1.22.12.1 Keywords derived from postraitement**postraitement****Synonyms:** `post_processing`**Inherits from:** [postraitement_base](#)

An object of post-processing (without name).

Parameters:

- **[t_debut_statistiques]** (*type*: float) `not_set` (for IJK)
- **[nb_pas_dt_post_stats_plans]** (*type*: float) `not_set` (for IJK)
- **[nb_pas_dt_post_stats_bulles]** (*type*: float) `not_set` (for IJK)
- **[expression_vx_ana]** (*type*: string) `not_set` (for IJK)
- **[expression_vy_ana]** (*type*: string) `not_set` (for IJK)
- **[expression_vz_ana]** (*type*: string) `not_set` (for IJK)
- **[expression_p_ana]** (*type*: string) `not_set` (for IJK)
- **[postprocessor_stt]** (*type*: flag) `not_set` (for STT)
- **[format_trustlata | format_ijklata]** (*type*: [format_trustlata](#)) `not_set` (for STT)
- **[interfaces]** (*type*: [interface_posts](#)) Keyword to read all the characteristics of the interfaces. Different kind of interfaces exist as well as different interface initialisations.
- **[fichier]** (*type*: string) Name of file.
- **[format]** (*type*: string into ['lml', 'lata', 'single_lata', 'lata_v2', 'med', 'cgns']) This optional parameter specifies the format of the output file. The basename used for the output file is the basename of the data file. For the `fmt` parameter, choices are `lml` or `lata`. A short description of each format can be found below. The default value is `lml`.

- **[dt_post]** (*type*: string) Field's write frequency (as a time period) - can also be specified after the 'field' keyword.
- **[nb_pas_dt_post]** (*type*: int) Field's write frequency (as a number of time steps) - can also be specified after the 'field' keyword.
- **[domaine]** (*type*: string) This optional parameter specifies the domain on which the data should be interpolated before it is written in the output file. The default is to write the data on the domain of the current problem (no interpolation).
- **[sous_domaine | sous_zone]** (*type*: string) This optional parameter specifies the sub_domaine on which the data should be interpolated before it is written in the output file. It is only available for sequential computation.
- **[parallele]** (*type*: string into ['simple', 'multiple', 'mpi-io']) Select simple (single file, sequential write), multiple (several files, parallel write), or mpi-io (single file, parallel write) for LATA format
- **[definition_champs]** (*type*: list of Definition_champ) List of definition champ
- **[definition_champs_fichier | definition_champs_file]** (*type*: [definition_champs_fichier](#)) Definition_champs read from file.
- **[sondes | probes]** (*type*: list of Sonde) List of probes.
- **[sondes_fichier | probes_file]** (*type*: [sondes_fichier](#)) Probe read from a file.
- **[sondes mobiles | mobile_probes]** (*type*: list of Sonde) List of probes.
- **[sondes mobiles_fichier | mobile_probes_file]** (*type*: [sondes_fichier](#)) Mobile probes read in a file
- **[deprecatedkeepduplicatedprobes]** (*type*: int) Flag to not remove duplicated probes in .son files (1: keep duplicate probes, 0: remove duplicate probes)
- **[champs | fields]** (*type*: [champs_posts](#)) Field's write mode.
- **[champs_fichier | fields_file]** (*type*: [champs_posts_fichier](#)) Fields read from file.
- **[statistiques | statistics]** (*type*: [stats_posts](#)) Statistics between two points fixed : start of integration time and end of integration time.
- **[statistiques_fichier | statistics_file]** (*type*: [stats_posts_fichier](#)) Statistics read from file.
- **[statistiques_en_serie | serial_statistics]** (*type*: [stats_serie_posts](#)) Statistics between two points not fixed : on period of integration.
- **[statistiques_en_serie_fichier | serial_statistics_file]** (*type*: [stats_serie_posts_fichier](#)) Serial_statistics read from a file
- **[suffix_for_reset]** (*type*: string) Suffix used to modify the postprocessing file name if the ICoCo resetTime() method is invoked.

corps_postraitement

Inherits from: [postraitement](#)

not_set

Parameters:

- **[t_debut_statistiques]** (*type*: float) not_set (for IJK)
- **[nb_pas_dt_post_stats_plans]** (*type*: float) not_set (for IJK)
- **[nb_pas_dt_post_stats_bulles]** (*type*: float) not_set (for IJK)
- **[expression_vx_ana]** (*type*: string) not_set (for IJK)
- **[expression_vy_ana]** (*type*: string) not_set (for IJK)
- **[expression_vz_ana]** (*type*: string) not_set (for IJK)

- **[expression_p_ana]** (*type*: string) not_set (for IJK)
- **[postprocessor_stt]** (*type*: flag) not_set (for STT)
- **[format_trustlata | format_ijklata]** (*type*: [format_trustlata](#)) not_set (for STT)
- **[interfaces]** (*type*: [interface_posts](#)) Keyword to read all the characteristics of the interfaces. Different kind of interfaces exist as well as different interface initialisations.
- **[fichier]** (*type*: string) Name of file.
- **[format]** (*type*: string into ['lml', 'lata', 'single_lata', 'lata_v2', 'med', 'cgns']) This optional parameter specifies the format of the output file. The basename used for the output file is the basename of the data file. For the fmt parameter, choices are lml or lata. A short description of each format can be found below. The default value is lml.
- **[dt_post]** (*type*: string) Field's write frequency (as a time period) - can also be specified after the 'field' keyword.
- **[nb_pas_dt_post]** (*type*: int) Field's write frequency (as a number of time steps) - can also be specified after the 'field' keyword.
- **[domaine]** (*type*: string) This optional parameter specifies the domain on which the data should be interpolated before it is written in the output file. The default is to write the data on the domain of the current problem (no interpolation).
- **[sous_domaine | sous_zone]** (*type*: string) This optional parameter specifies the sub_domaine on which the data should be interpolated before it is written in the output file. It is only available for sequential computation.
- **[parallele]** (*type*: string into ['simple', 'multiple', 'mpi-io']) Select simple (single file, sequential write), multiple (several files, parallel write), or mpi-io (single file, parallel write) for LATA format
- **[definition_champs]** (*type*: list of Definition_champ) List of definition champ
- **[definition_champs_fichier | definition_champs_file]** (*type*: [definition_champs_fichier](#)) Definition_champs read from file.
- **[sondes | probes]** (*type*: list of Sonde) List of probes.
- **[sondes_fichier | probes_file]** (*type*: [sondes_fichier](#)) Probe read from a file.
- **[sondes mobiles | mobile_probes]** (*type*: list of Sonde) List of probes.
- **[sondes mobiles_fichier | mobile_probes_file]** (*type*: [sondes_fichier](#)) Mobile probes read in a file
- **[deprecatedkeepduplicatedprobes]** (*type*: int) Flag to not remove duplicated probes in .son files (1: keep duplicate probes, 0: remove duplicate probes)
- **[champs | fields]** (*type*: [champs_posts](#)) Field's write mode.
- **[champs_fichier | fields_file]** (*type*: [champs_posts_fichier](#)) Fields read from file.
- **[statistiques | statistics]** (*type*: [stats_posts](#)) Statistics between two points fixed : start of integration time and end of integration time.
- **[statistiques_fichier | statistics_file]** (*type*: [stats_posts_fichier](#)) Statistics read from file.
- **[statistiques_en_serie | serial_statistics]** (*type*: [stats_serie_posts](#)) Statistics between two points not fixed : on period of integration.
- **[statistiques_en_serie_fichier | serial_statistics_file]** (*type*: [stats_serie_posts_fichier](#)) Serial_statistics read from a file
- **[suffix_for_reset]** (*type*: string) Suffix used to modify the postprocessing file name if the ICoCo resetTime() method is invoked.

1.22.13 Keywords derived from sonde_base

sonde_base

Basic probe. Probes refer to sensors that allow a value or several points of the domain to be monitored over time. The probes may be a set of points defined one by one (keyword Points) or a set of points evenly distributed over a straight segment (keyword Segment) or arranged according to a layout (keyword Plan) or according to a parallelepiped (keyword Volume). The fields allow all the values of a physical value on the domain to be known at several moments in time.

- [Keywords derived from points](#)

circle

Inherits from: [sonde_base](#)

Keyword to define several probes located on a circle.

Parameters:

- **nbr** (*type*: int) Number of probes between teta1 and teta2 (angles given in degrees).
 - **point_deb** (*type*: [un_point](#)) Center of the circle.
 - **[direction]** (*type*: int into [0, 1, 2]) Axis normal to the circle plane (0:x axis, 1:y axis, 2:z axis).
 - **radius** (*type*: float) Radius of the circle.
 - **theta1** (*type*: float) First angle.
 - **theta2** (*type*: float) Second angle.
-

circle_3

Inherits from: [sonde_base](#)

Keyword to define several probes located on a circle (in 3-D space).

Parameters:

- **nbr** (*type*: int) Number of probes between teta1 and teta2 (angles given in degrees).
 - **point_deb** (*type*: [un_point](#)) Center of the circle.
 - **direction** (*type*: int into [0, 1, 2]) Axis normal to the circle plane (0:x axis, 1:y axis, 2:z axis).
 - **radius** (*type*: float) Radius of the circle.
 - **theta1** (*type*: float) First angle.
 - **theta2** (*type*: float) Second angle.
-

numero_elem_sur_maitre

Inherits from: [sonde_base](#)

Keyword to define a probe at the special element. Useful for min/max sonde.

Parameters:

- **numero** (*type*: int) element number
-

plan

Inherits from: [sonde_base](#)

Keyword to set the number of probe layout points. The file format is type .lml

Parameters:

- **nbr** (*type*: int) Number of probes in the first direction.
 - **nbr2** (*type*: int) Number of probes in the second direction.
 - **point_deb** (*type*: [un_point](#)) First point defining the angle. This angle should be positive.
 - **point_fin** (*type*: [un_point](#)) Second point defining the angle. This angle should be positive.
 - **point_fin_2** (*type*: [un_point](#)) Third point defining the angle. This angle should be positive.
-

position_like

Inherits from: [sonde_base](#)

Keyword to define a probe at the same position of another probe named autre_sonde.

Parameters:

- **autre_sonde** (*type*: string) Name of the other probe.
-

radius

Inherits from: [sonde_base](#)

not_set

Parameters:

- **nbr** (*type*: int) Number of probe points of the segment, evenly distributed.
 - **point_deb** (*type*: [un_point](#)) First outer probe segment point.
 - **radius** (*type*: float) not_set
 - **teta1** (*type*: float) not_set
 - **teta2** (*type*: float) not_set
-

segment

Inherits from: [sonde_base](#)

Keyword to define the number of probe segment points. The file is arranged in columns.

Parameters:

- **nbr** (*type*: int) Number of probe points of the segment, evenly distributed.
 - **point_deb** (*type*: [un_point](#)) First outer probe segment point.
 - **point_fin** (*type*: [un_point](#)) Second outer probe segment point.
-

segmentfacesx

Inherits from: [sonde_base](#)

Segment probe where points are moved to the nearest x faces

Parameters:

- **nbr** (*type*: int) Number of probe points of the segment, evenly distributed.
 - **point_deb** (*type*: [un_point](#)) First outer probe segment point.
 - **point_fin** (*type*: [un_point](#)) Second outer probe segment point.
-

segmentfacesy**Inherits from:** [sonde_base](#)

Segment probe where points are moved to the nearest y faces

Parameters:

- **nbr** (*type*: int) Number of probe points of the segment, evenly distributed.
 - **point_deb** (*type*: [un_point](#)) First outer probe segment point.
 - **point_fin** (*type*: [un_point](#)) Second outer probe segment point.
-

segmentfacesz**Inherits from:** [sonde_base](#)

Segment probe where points are moved to the nearest z faces

Parameters:

- **nbr** (*type*: int) Number of probe points of the segment, evenly distributed.
 - **point_deb** (*type*: [un_point](#)) First outer probe segment point.
 - **point_fin** (*type*: [un_point](#)) Second outer probe segment point.
-

volume**Inherits from:** [sonde_base](#)

Keyword to define the probe volume in a parallelepiped passing through 4 points and the number of probes in each direction.

Parameters:

- **nbr** (*type*: int) Number of probes in the first direction.
- **nbr2** (*type*: int) Number of probes in the second direction.
- **nbr3** (*type*: int) Number of probes in the third direction.
- **point_deb** (*type*: [un_point](#)) Point of origin.
- **point_fin** (*type*: [un_point](#)) Point defining the first direction (from point of origin).
- **point_fin_2** (*type*: [un_point](#)) Point defining the second direction (from point of origin).
- **point_fin_3** (*type*: [un_point](#)) Point defining the third direction (from point of origin).

1.22.13.1 Keywords derived from points**points****Inherits from:** [sonde_base](#)

Keyword to define the number of probe points. The file is arranged in columns.

Parameters:

- **points** (*type*: list of [Un_point](#)) Points.
-

point**Inherits from:** [points](#)

Point as class-daughter of Points.

Parameters:

- **points** (*type*: list of [Un_point](#)) Points.
-

segmentpoints**Inherits from:** [points](#)

This keyword is used to define a probe segment from specifics points. The nom_champ field is sampled at ns specifics points.

Parameters:

- **points** (*type*: list of Un_point) Points.

1.22.14 Keywords derived from spec_pdc_r_base**spec_pdc_r_base**

Class to read the source term modelling the presence of a bundle of tubes in a flow. Cf=A Re-B.

longitudinale**Inherits from:** [spec_pdc_r_base](#)

Class to define the pressure loss in the direction of the tube bundle.

Parameters:

- **dir** (*type*: string into ['x', 'y', 'z']) Direction.
 - **dd** (*type*: float) Tube bundle hydraulic diameter value. This value is expressed in m.
 - **ch_a** (*type*: string into ['a', 'cf']) Keyword to be used to set law coefficient values for the coefficient of regular pressure losses.
 - **a** (*type*: float) Value of a law coefficient for regular pressure losses.
 - **[ch_b]** (*type*: string into ['b']) Keyword to be used to set law coefficient values for regular pressure losses.
 - **[b]** (*type*: float) Value of a law coefficient for regular pressure losses.
-

transversale**Inherits from:** [spec_pdc_r_base](#)

Class to define the pressure loss in the direction perpendicular to the tube bundle.

Parameters:

- **dir** (*type*: string into ['x', 'y', 'z']) Direction.
- **dd** (*type*: float) Value of the tube bundle step.
- **chaîne_d** (*type*: string into ['d']) Keyword to be used to set the value of the tube external diameter.
- **d** (*type*: float) Value of the tube external diameter.
- **ch_a** (*type*: string into ['a', 'cf']) Keyword to be used to set law coefficient values for the coefficient of regular pressure losses.
- **a** (*type*: float) Value of a law coefficient for regular pressure losses.
- **[ch_b]** (*type*: string into ['b']) Keyword to be used to set law coefficient values for regular pressure losses.
- **[b]** (*type*: float) Value of a law coefficient for regular pressure losses.

1.22.15 Keywords derived from stat_post_deriv**stat_post_deriv**

not_set

stat_post_correlation

Synonyms: champ_post_statistiques_correlation, correlation

Inherits from: [stat_post_deriv](#)

correlation between the two fields

Parameters:

- **first_field** (*type*: string) first field
 - **second_field** (*type*: string) second field
 - **[localisation]** (*type*: string into ['elem', 'som', 'faces', 'elem_dg']) Localisation of post-processed field value
-

stat_post_ecart_type

Synonyms: champ_post_statistiques_ecart_type, ecart_type

Inherits from: [stat_post_deriv](#)

to calculate the standard deviation (statistic rms) of the field

Parameters:

- **field** (*type*: string) name of the field on which statistical analysis will be performed. Possible keywords are Vitesse (velocity), Pression (pressure), Temperature, Concentration, ...
 - **[localisation]** (*type*: string into ['elem', 'som', 'faces', 'elem_dg']) Localisation of post-processed field value
-

stat_post_moyenne

Synonyms: champ_post_statistiques_moyenne, moyenne

Inherits from: [stat_post_deriv](#)

to calculate the average of the field over time

Parameters:

- **field** (*type*: string) name of the field on which statistical analysis will be performed. Possible keywords are Vitesse (velocity), Pression (pressure), Temperature, Concentration, ...
 - **[localisation]** (*type*: string into ['elem', 'som', 'faces', 'elem_dg']) Localisation of post-processed field value
-

stat_post_t_deb

Synonyms: t_deb

Inherits from: [stat_post_deriv](#)

Start of integration time

Parameters:

- **val** (*type*: float) not_set
-

stat_post_t_fin

Synonyms: t_fin

Inherits from: [stat_post_deriv](#)

End of integration time

Parameters:

- **val** (*type*: float) not_set

1.22.16 Keywords derived from traitement_particulier_base**traitement_particulier_base**

Basic class to post-process particular values.

canal**Inherits from:** [traitement_particulier_base](#)

Keyword for statistics on a periodic plane channel.

Parameters:

- **[dt_impr_moy_spat]** (*type*: float) Period to print the spatial average (default value is 1e6).
- **[dt_impr_moy_temp]** (*type*: float) Period to print the temporal average (default value is 1e6).
- **[debut_stat]** (*type*: float) Time to start the temporal averaging (default value is 1e6).
- **[fin_stat]** (*type*: float) Time to end the temporal averaging (default value is 1e6).
- **[pulsation_w]** (*type*: float) Pulsation for phase averaging (in case of pulsating forcing term) (no default value).
- **[nb_points_par_phase]** (*type*: int) Number of samples to represent phase average all along a period (no default value).
- **[reprise]** (*type*: string) val_moy_temp_xxxxxx.sauv : Keyword to resume a calculation with previous averaged quantities. Note that for thermal and turbulent problems, averages on temperature and turbulent viscosity are automatically calculated. To resume a calculation with phase averaging, val_moy_temp_xxxxxx.sauv_phase file is required on the directory where the job is submitted (this last file will be then automatically loaded by TRUST).

chmoy_faceperio**Inherits from:** [traitement_particulier_base](#)

non documente

Parameters:

- **bloc** (*type*: [bloc_lecture](#)) not_set

ec**Inherits from:** [traitement_particulier_base](#)

Keyword to print total kinetic energy into the referential linked to the domain (keyword Ec). In the case where the domain is moving into a Galilean referential, the keyword Ec_dans_repere_fixe will print total kinetic energy in the Galilean referential whereas Ec will print the value calculated into the moving referential linked to the domain

Parameters:

- **[ec]** (*type*: flag) not_set
- **[ec_dans_repere_fixe]** (*type*: flag) not_set
- **[periode]** (*type*: float) periode is the keyword to set the period of printing into the file datafile_Ec.son or datafile_Ec_dans_repere_fixe.son.

profils_thermo**Inherits from:** [traitement_particulier_base](#)

non documente

Parameters:

- **bloc** (*type*: [bloc_lecture](#)) not_set

temperature**Inherits from:** [traitement_particulier_base](#)

not_set

Parameters:

- **bord** (*type*: string) not_set
- **direction** (*type*: int) not_set

thi

Inherits from: [traitement_particulier_base](#)

Keyword for a THI (Homogeneous Isotropic Turbulence) calculation.

Parameters:

- **init_ec** (*type*: int) Keyword to renormalize initial velocity so that kinetic energy equals to the value given by keyword `val_Ec`.
- **[val_ec]** (*type*: float) Keyword to impose a value for kinetic energy by velocity renormalized if `init_Ec` value is 1.
- **[facon_init]** (*type*: int into [0, 1]) Keyword to specify how kinetic energy is computed (0 or 1).
- **[calc_spectre]** (*type*: int into [0, 1]) Calculate or not the spectrum of kinetic energy. Files called `Sorties_THI` are written with inside four columns : `time:t global_kinetic_energy:Ec enstrophy:D skewness:S` If `calc_spectre` is set to 1, a file `Sorties_THI2_2` is written with three columns : `time:t kinetic_energy_at_kc=32 enstrophy←_at_kc=32` If `calc_spectre` is set to 1, a file `spectre_xxxxx` is written with two columns at each time `xxxxx` : `frequency:k energy:E(k)`.
- **[periode_calc_spectre]** (*type*: float) Period for calculating spectrum of kinetic energy
- **[spectre_3d]** (*type*: int into [0, 1]) Calculate or not the 3D spectrum
- **[spectre_1d]** (*type*: int into [0, 1]) Calculate or not the 1D spectrum
- **[conservation_ec]** (*type*: flag) If set to 1, velocity field will be changed as to have a constant kinetic energy (default 0)
- **[longueur_boite]** (*type*: float) Length of the calculation domain

1.22.17 Keywords derived from `type_diffusion_turbulente_multiphase_deriv`

type_diffusion_turbulente_multiphase_deriv

not_set

type_diffusion_turbulente_multiphase_aire_interfaciale

Synonyms: `aire_interfaciale`, `interfacial_area`

Inherits from: [type_diffusion_turbulente_multiphase_deriv](#)

not_set

Parameters:

- **[cstdiff]** (*type*: float) Kataoka diffusion model constant. By default it is set to 0.236.
- **[ng2]** (*type*: float) not_set

type_diffusion_turbulente_multiphase_l_melange

Synonyms: `l_melange`

Inherits from: [type_diffusion_turbulente_multiphase_deriv](#)

not_set

Parameters:

- **l_melange** (*type*: float) not_set

type_diffusion_turbulente_multiphase_prandtl

Synonyms: `prandtl`

Inherits from: [type_diffusion_turbulente_multiphase_deriv](#)

Scalar Prandtl model.

Parameters:

- **[pr_t | prandtl_turbulent]** (*type*: float) Prandtl's model constant. By default it is set to 0.9.

type_diffusion_turbulente_multiphase_sgdh**Synonyms:** sgdh**Inherits from:** [type_diffusion_turbulente_multiphase_deriv](#)

not_set

Parameters:

- **[pr_t | prandtl_turbulent]** (*type*: float) not_set
- **[sigma | sigma_turbulent]** (*type*: float) not_set
- **[no_alpha]** (*type*: flag) not_set
- **[gas_turb]** (*type*: flag) not_set

type_diffusion_turbulente_multiphase_smago**Synonyms:** smago**Inherits from:** [type_diffusion_turbulente_multiphase_deriv](#)

LES Smagorinsky type.

Parameters:

- **[cs]** (*type*: float) Smagorinsky's model constant. By default it is set to 0.18.

type_diffusion_turbulente_multiphase_wale**Synonyms:** wale**Inherits from:** [type_diffusion_turbulente_multiphase_deriv](#)

LES WALE type.

Parameters:

- **[cw]** (*type*: float) WALE's model constant. By default it is set to 0.5.

1.22.18 Keywords derived from type_perte_charge_deriv**type_perte_charge_deriv**

not_set

type_perte_charge_dp**Synonyms:** dp**Inherits from:** [type_perte_charge_deriv](#)

DP field should have 3 components defining dp, dDP/dQ, Q0

Parameters:

- **dp_field** (*type*: [field_base](#)) the parameters of the previous formula ($DP = dp + dDP/dQ * (Q - Q0)$): uniform↔_field 3 dp dDP/dQ Q0 where Q0 is a mass flow rate (kg/s).

type_perte_charge_dp_regul**Synonyms:** dp_regul**Inherits from:** [type_perte_charge_deriv](#)

Keyword used to regulate the DP value in order to match a target flow rate. Syntax : dp_regul { DP0 d deb d eps e }

Parameters:

- **dp0** (*type*: string) Reference pressure drop value (possibly time-dependent)
- **deb** (*type*: string) target flow rate in kg/s
- **alpha** (*type*: string) alpha string

1.23 Keywords derived from `partitionneur_deriv`

`partitionneur_deriv`

`not_set`

Parameters:

- **[nb_parts]** (*type*: int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

`partitionneur_fichier_decoupage`

Synonyms: `fichier_decoupage`

Inherits from: [partitionneur_deriv](#)

This algorithm reads an array of integer values on the disc, one value for each mesh element. Each value is interpreted as the target part number $n \geq 0$ for this element. The number of parts created is the highest value in the array plus one. Empty parts can be created if some values are not present in the array.

The file format is ASCII, and contains space, tab or carriage-return separated integer values. The first value is the number `nb_elem` of elements in the domain, followed by `nb_elem` integer values (positive or zero).

This algorithm has been designed to work together with the '`ecrire_decoupage`' option. You can generate a partition with any other algorithm, write it to disc, modify it, and read it again to generate the `.Zone` files.

Contrary to other partitioning algorithms, no correction is applied by default to the partition (eg. element 0 on processor 0 and corrections for periodic boundaries). If '`corriger_partition`' is specified, these corrections are applied.

Parameters:

- **fichier** (*type*: string) File name
- **[corriger_partition]** (*type*: flag) `not_set`
- **[nb_parts]** (*type*: int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

`partitionneur_fichier_med`

Synonyms: `fichier_med`

Inherits from: [partitionneur_deriv](#)

Partitioning a domain using a MED file containing an integer field providing for each element the processor number on which the element should be located.

Parameters:

- **file** (*type*: string) file name of the MED file to load
- **[field]** (*type*: string) field name of the integer (or double) field to load
- **[nb_parts]** (*type*: int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

`partitionneur_metis`

Synonyms: `metis`

Inherits from: [partitionneur_deriv](#)

Metis is an external partitioning library. It is a general algorithm that will generate a partition of the domain.

Parameters:

- **[kmetis]** (*type*: flag) The default values are `pmetis`, default parameters are automatically chosen by Metis. '`kmetis`' is faster than `pmetis` option but the last option produces better partitioning quality. In both cases, the partitioning quality may be slightly improved by increasing the `nb_essais` option (by default `N=1`). It will compute `N` partitions and will keep the best one (smallest edge cut number). But this option is CPU expensive, taking `N=10` will multiply the CPU cost of partitioning by 10. Experiments show that only marginal improvements can be obtained with non default parameters.

- **[use_weights]** (*type*: flag) If `use_weights` is specified, weighting of the element-element links in the graph is used to force metis to keep opposite periodic elements on the same processor. This option can slightly improve the partitionning quality but it consumes more memory and takes more time. It is not mandatory since a correction algorithm is always applied afterwards to ensure a correct partitionning for periodic boundaries.
- **[nb_parts]** (*type*: int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

partitionneur_metis_64**Synonyms:** `metis_64`**Inherits from:** [partitionneur_deriv](#)

Metis is an external partitionning library. It is a general algorithm that will generate a partition of big (64b) domain.

Parameters:

- **[nb_parts]** (*type*: int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

partitionneur_parmetis**Synonyms:** `parmetis`**Inherits from:** [partitionneur_deriv](#)

ParMETIS is the MPI-parallel counterpart of METIS — same general-purpose mesh partitioning, run in parallel.

Parameters:

- **[use_weights]** (*type*: flag) Weight element-element links in the graph to keep periodic opposite elements on the same processor. Same meaning as in metis; costs extra memory and time but is otherwise non-mandatory (a correction pass runs unconditionally).
- **[nb_parts]** (*type*: int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

partitionneur_partition**Synonyms:** `decouper`, `partition`**Inherits from:** [partitionneur_deriv](#)This algorithm re-use the partition of the domain named `DOMAINE_NAME`. It is useful to partition for example a post processing domain. The partition should match with the calculation domain.**Parameters:**

- **domaine** (*type*: string) domain name
- **[nb_parts]** (*type*: int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

partitionneur_sous_dom**Synonyms:** `sous_dom`**Inherits from:** [partitionneur_deriv](#)Given a global partition of a global domain, 'sous-domaine' allows to produce a conform partition of a sub-domain generated from the bigger one using the keyword `create_domain_from_sub_domain`. The sub-domain will be partitionned in a conform fashion with the global domain.**Parameters:**

- **fichier** (*type*: string) fichier
- **[fichier_ssz]** (*type*: string) fichier sous zone
- **[name_ssz]** (*type*: string) nom sous zone (nom d'un objet `Sous_Domaine` declare dans le jdd)
- **[nb_parts]** (*type*: int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

partitionneur_sous_domaines**Synonyms:** partitionneur_sous_zones**Inherits from:** [partitionneur_deriv](#)

This algorithm will create one part for each specified subdomaine/domain. All elements contained in the first subdomaine/domain are put in the first part, all remaining elements contained in the second subdomaine/domain in the second part, etc...

If all elements of the current domain are contained in the specified subdomaines/domain, then N parts are created, otherwise, a supplemental part is created with the remaining elements.

If no subdomaine is specified, all subdomaines defined in the domain are used to split the mesh.

Parameters:

- **[sous_zones]** (*type:* list of str) N SUBZONE_NAME_1 SUBZONE_NAME_2 ...
- **[domaines]** (*type:* list of str) N DOMAIN_NAME_1 DOMAIN_NAME_2 ...
- **[nb_parts]** (*type:* int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

partitionneur_tranche**Synonyms:** tranche**Inherits from:** [partitionneur_deriv](#)

This algorithm will create a geometrical partitionning by slicing the mesh in the two or three axis directions, based on the geometric center of each mesh element. nz must be given if dimension=3. Each slice contains the same number of elements (slices don't have the same geometrical width, and for VDF meshes, slice boundaries are generally not flat except if the number of mesh elements in each direction is an exact multiple of the number of slices). First, nx slices in the X direction are created, then each slice is split in ny slices in the Y direction, and finally, each part is split in nz slices in the Z direction. The resulting number of parts is nx*ny*nz. If one particular direction has been declared periodic, the default slicing (0, 1, 2, ..., n-1) is replaced by (0, 1, 2, ... n-1, 0), each of the two '0' slices having twice less elements than the other slices.

Parameters:

- **[tranches]** (*type:* list of int) Partitioned by nx in the X direction, ny in the Y direction, nz in the Z direction. Works only for structured meshes. No warranty for unstructured meshes.
- **[nb_parts]** (*type:* int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

partitionneur_union**Synonyms:** union**Inherits from:** [partitionneur_deriv](#)

Let several local domains be generated from a bigger one using the keyword create_domain_from_sub_domain, and let their partitions be generated in the usual way. Provided the list of partition files for each small domain, the keyword 'union' will partition the global domain in a conform fashion with the smaller domains.

Parameters:

- **sous_domaines** (*type:* [bloc_lecture](#)) List of the partition files with the following syntaxe: {sous_domaine1 decoupage1 ... sous_domaineim decoupageim } where sous_domaine1 ... sous_zomeim are small domains names and decoupage1 ... decoupageim are partition files.
- **[nb_parts]** (*type:* int) The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

1.24 Keywords derived from pb_champ_evaluateur**pb_champ_evaluateur**

specifies problem name, the field name belonging to the problem and number of field components.

Parameters:

- **pb** (*type*: string) name of the problem where the source fields will be searched.
- **champ** (*type*: string) name of the field
- **ncomp** (*type*: int) number of components

1.25 Keywords derived from pb_gen_base

pb_gen_base

Basic class for problems.

- [Keywords derived from pb_base](#)

coupled_problem

Synonyms: probleme_couple

Inherits from: [pb_gen_base](#)

This instruction causes a probleme_couple type object to be created. This type of object has an associated problem list, that is, the coupling of n problems among them may be processed. Coupling between these problems is carried out explicitly via conditions at particular contact limits. Each problem may be associated either with the Associate keyword or with the Read/groupes keywords. The difference is that in the first case, the four problems exchange values then calculate their timestep, rather in the second case, the same strategy is used for all the problems listed inside one group, but the second group of problem exchange values with the first group of problems after the first group did its timestep. So, the first case may then also be written like this:

Probleme_Couple pbc

Read pbc { groupes { { pb1 , pb2 , pb3 , pb4 } } }

There is a physical environment per problem (however, the same physical environment could be common to several problems).

Each problem is resolved in a domain.

Warning : Presently, coupling requires coincident meshes. In case of non-coincident meshes, boundary condition 'paroi_contact' in VEF returns error message (see paroi_contact for correcting procedure).

Parameters:

- **[groupes]** (*type*: list of List_un_pb) pour les groupes
-

pb_med

Inherits from: [pb_gen_base](#)

Allows to read med files and post-process them.

Parameters:

- **list_info_med** (*type*: list of Info_med) not_set

1.25.1 Keywords derived from pb_base

pb_base

Inherits from: [pb_gen_base](#)

Resolution of equations on a domain. A problem is defined by creating an object and assigning the problem type that the user wishes to resolve. To enter values for the problem objects created, the Lire (Read) interpreter is used with a data block.

Parameters:

- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).

- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

-
- [Keywords derived from pb_multiphase](#)

pb_conduction

Inherits from: [pb_base](#)

Resolution of the heat equation.

Parameters:

- **[solide]** (*type*: [solide](#)) The medium associated with the problem.
- **[conduction]** (*type*: [conduction](#)) Heat equation.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.

- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_conduction_ibm

Inherits from: [pb_base](#)

Resolution of the IBM heat equation.

Parameters:

- **[solide]** (*type*: [solide](#)) The medium associated with the problem.
- **[conduction_ibm]** (*type*: [conduction_ibm](#)) IBM Heat equation.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_euler

Inherits from: [pb_base](#)

A problem that allows the resolution of N-phases Euler equations

Parameters:

- **[milieu_composite_euler]** (*type*: [bloc_lecture](#)) The composite medium associated with the problem.
- **momentum_euler | qdm_euler** (*type*: [momentum_euler](#)) Momentum conservation equation for a multi-phase Euler problem where the unknown is the velocity
- **masse_euler | density_euler** (*type*: [masse_euler](#)) Mass conservation equation for a multi-phase Euler problem where the unknown is the density

- **energy_euler | energie_euler** (*type*: [energy_euler](#)) Internal energy conservation equation for a multi-phase Euler problem where the unknown is the temperature
- **fraction_euler** (*type*: [fraction_euler](#)) Void fraction conservation equation for a multi-phase Euler problem where the unknown is the alpha
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique

Inherits from: [pb_base](#)

Resolution of the Navier-Stokes equations.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[correlations]** (*type*: [bloc_lecture](#)) List of correlations used in specific source terms (e.g. wall-friction adaptive law).
- **navier_stokes_standard** (*type*: [navier_stokes_standard](#)) Navier-Stokes equations.
- **[energie_cinetique_turbulente]** (*type*: [energie_cinetique_turbulente](#)) Turbulent kinetic Energy conservation equation for a turbulent mono-phase problem (available in TrioCFD).
- **[echelle_temporelle_turbulente]** (*type*: [echelle_temporelle_turbulente](#)) Turbulent Dissipation time scale equation for a turbulent mono-phase problem (available in TrioCFD).
- **[taux_dissipation_turbulent]** (*type*: [taux_dissipation_turbulent](#)) Turbulent Dissipation frequency equation for a turbulent mono-phase problem (available in TrioCFD).
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.

- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_cloned_concentration

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/multiple constituent transport equations.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituents.
- **[navier_stokes_standard]** (*type*: [navier_stokes_standard](#)) Navier-Stokes equations.
- **[convection_diffusion_concentration]** (*type*: [convection_diffusion_concentration](#)) Constituent transport vectorial equation (concentration diffusion convection).
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.

- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_cloned_concentration_turbulent

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/multiple constituent transport equations, with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type: [fluide_incompressible](#)*) The fluid medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituents.
- **[navier_stokes_turbulent]** (*type: [navier_stokes_turbulent](#)*) Navier-Stokes equations as well as the associated turbulence model equations.
- **[convection_diffusion_concentration_turbulent]** (*type: [convection_diffusion_concentration_turbulent](#)*) Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
- **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_concentration**Inherits from:** [pb_base](#)

Resolution of Navier-Stokes/multiple constituent transport equations.

Parameters:

- **fluide_incompressible** (*type:* [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type:* [constituant](#)) Constituents.
- **[navier_stokes_standard]** (*type:* [navier_stokes_standard](#)) Navier-Stokes equations.
- **[convection_diffusion_concentration]** (*type:* [convection_diffusion_concentration](#)) Constituent transport vectorial equation (concentration diffusion convection).
- **[milieu]** (*type:* [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type:* [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type:* list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type:* list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type:* list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type:* [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type:* [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type:* [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type:* [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type:* [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_concentration_scalaires_passifs**Inherits from:** [pb_base](#)

Resolution of Navier-Stokes/multiple constituent transport equations with the additional passive scalar equations.

Parameters:

- **fluide_incompressible** (*type:* [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type:* [constituant](#)) Constituents.
- **[navier_stokes_standard]** (*type:* [navier_stokes_standard](#)) Navier-Stokes equations.
- **[convection_diffusion_concentration]** (*type:* [convection_diffusion_concentration](#)) Constituent transport equations (concentration diffusion convection).
- **equations_scalaires_passifs** (*type:* list of Eqn_base) List of equations.

- **[milieu]** (*type: milieu_base*) The medium associated with the problem.
- **[postraitement | post_processing]** (*type: corps_postraitement*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: format_file_base*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: format_file_base*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: format_file_base*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: format_file_base*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: transparent_medium_radiation*) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_concentration_turbulent

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/multiple constituent transport equations, with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type: fluide_incompressible*) The fluid medium associated with the problem.
- **[constituant]** (*type: constituant*) Constituents.
- **[navier_stokes_turbulent]** (*type: navier_stokes_turbulent*) Navier-Stokes equations as well as the associated turbulence model equations.
- **[convection_diffusion_concentration_turbulent]** (*type: convection_diffusion_concentration_turbulent*) Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **[milieu]** (*type: milieu_base*) The medium associated with the problem.
- **[postraitement | post_processing]** (*type: corps_postraitement*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)

- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_concentration_turbulent_scalaires_passifs

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/multiple constituent transport equations, with turbulence modelling and with the additional passive scalar equations.

Parameters:

- **fluide_incompressible** (*type: [fluide_incompressible](#)*) The fluid medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituents.
- **[navier_stokes_turbulent]** (*type: [navier_stokes_turbulent](#)*) Navier-Stokes equations as well as the associated turbulence model equations.
- **[convection_diffusion_concentration_turbulent]** (*type: [convection_diffusion_concentration_turbulent](#)*) Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **equations_scalaires_passifs** (*type: list of Eqn_base*) List of equations.
- **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
- **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.

- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_ibm

Inherits from: [pb_base](#)

Resolution of the IBM Navier-Stokes equations.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **navier_stokes_ibm** (*type*: [navier_stokes_ibm](#)) IBM Navier-Stokes equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_ibm_turbulent

Inherits from: [pb_base](#)

Resolution of Navier-Stokes equations with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **navier_stokes_ibm_turbulent** (*type*: [navier_stokes_ibm_turbulent](#)) IBM Navier-Stokes equations as well as the associated turbulence model equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.

- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_list_concentration

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/multiple constituent transport equations.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituents.
- **[navier_stokes_standard]** (*type*: [navier_stokes_standard](#)) Navier-Stokes equations.
- **list_equations** (*type*: list of Eqn_base) List of equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.

- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class [format_file](#)). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see [schema_temps_base](#)) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_list_concentration_turbulent

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/multiple constituent transport equations, with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type: [fluide_incompressible](#)*) The fluid medium associated with the problem.
 - **[constituant]** (*type: [constituant](#)*) Constituents.
 - **[navier_stokes_turbulent]** (*type: [navier_stokes_turbulent](#)*) Navier-Stokes equations as well as the associated turbulence model equations.
 - **list_equations** (*type: list of [Eqn_base](#)*) List of equations.
 - **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
 - **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
 - **[postraitements | post_processings]** (*type: list of [Un_postraitement](#)*) Keyword to use several results files. List of objects of post-processing (with name).
 - **[liste_de_postraitements]** (*type: list of [Nom_postraitement](#)*) Keyword to use several results files. List of objects of post-processing (with name)
 - **[liste_postraitements]** (*type: list of [Un_postraitement_spec](#)*) Keyword to use several results files. List of objects of post-processing (with name)
 - **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
 - **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
 - **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class [format_file](#)). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see [schema_temps_base](#)) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
 - **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
 - **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem
-

pb_hydraulique_melange_binaire_qc**Inherits from:** [pb_base](#)

Resolution of a binary mixture problem for a quasi-compressible fluid with an iso-thermal condition.

Keywords for the unknowns other than pressure, velocity, fraction_massique are :

masse_volumique : density

pression : reduced pressure

pression_tot : total pressure.

Parameters:

- **fluide_quasi_compressible** (*type:* [fluide_quasi_compressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type:* [constituant](#)) The various constituents associated to the problem.
- **navier_stokes_qc** (*type:* [navier_stokes_qc](#)) Navier-Stokes equation for a quasi-compressible fluid.
- **convection_diffusion_espece_binaire_qc** (*type:* [convection_diffusion_espece_binaire_qc](#)) Species conservation equation for a binary quasi-compressible fluid.
- **[milieu]** (*type:* [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type:* [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type:* list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type:* list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type:* list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type:* [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type:* [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type:* [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type:* [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type:* [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_melange_binaire_turbulent_qc**Inherits from:** [pb_base](#)

Resolution of a turbulent binary mixture problem for a quasi-compressible fluid with an iso-thermal condition.

Parameters:

- **fluide_quasi_compressible** (*type:* [fluide_quasi_compressible](#)) The fluid medium associated with the problem.
- **navier_stokes_turbulent_qc** (*type:* [navier_stokes_turbulent_qc](#)) Navier-Stokes equation for a quasi-compressible fluid as well as the associated turbulence model equations.

- **convection_diffusion_espece_binaire_turbulent_qc** (*type*: [convection_diffusion_espece_binaire_turbulent_qc](#)) Species conservation equation for a quasi-compressible fluid as well as the associated turbulence model equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of [Un_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of [Nom_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of [Un_postraitement_spec](#)) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class [format_file](#)). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see [schema_temps_base](#)) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_melange_binaire_wc

Inherits from: [pb_base](#)

Resolution of a binary mixture problem for a weakly-compressible fluid with an iso-thermal condition.

Keywords for the unknowns other than pressure, velocity, fraction_massique are :

masse_volumique : density

pression : reduced pressure

pression_tot : total pressure

pression_hydro : hydro-static pressure

pression_eos : pressure used in state equation.

Parameters:

- **fluide_weakly_compressible** (*type*: [fluide_weakly_compressible](#)) The fluid medium associated with the problem.
- **navier_stokes_wc** (*type*: [navier_stokes_wc](#)) Navier-Stokes equation for a weakly-compressible fluid.
- **convection_diffusion_espece_binaire_wc** (*type*: [convection_diffusion_espece_binaire_wc](#)) Species conservation equation for a binary weakly-compressible fluid.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).

- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_hydraulique_turbulent

Inherits from: [pb_base](#)

Resolution of Navier-Stokes equations with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **navier_stokes_turbulent** (*type*: [navier_stokes_turbulent](#)) Navier-Stokes equations as well as the associated turbulence model equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.

- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#))
Read a transparent medium radiation model associated to a fluid problem

pb_post

Inherits from: [pb_base](#)

not_set

Parameters:

- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#))
Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique

Inherits from: [pb_base](#)

Resolution of thermohydraulic problem.

Parameters:

- **[fluide_incompressible]** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem (only one possibility).

- **[fluide_ostwald]** (*type: [fluide_ostwald](#)*) The fluid medium associated with the problem (only one possibility).
- **[fluide_sodium_liquide]** (*type: [fluide_sodium_liquide](#)*) The fluid medium associated with the problem (only one possibility).
- **[fluide_sodium_gaz]** (*type: [fluide_sodium_gaz](#)*) The fluid medium associated with the problem (only one possibility).
- **[correlations]** (*type: [bloc_lecture](#)*) List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **[navier_stokes_standard]** (*type: [navier_stokes_standard](#)*) Navier-Stokes equations.
- **[convection_diffusion_temperature]** (*type: [convection_diffusion_temperature](#)*) Energy equation (temperature diffusion convection).
- **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituent.
- **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_cloned_concentration

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/energy/multiple constituent transport equations.

Parameters:

- **fluide_incompressible** (*type: [fluide_incompressible](#)*) The fluid medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituents.
- **[navier_stokes_standard]** (*type: [navier_stokes_standard](#)*) Navier-Stokes equations.
- **[convection_diffusion_concentration]** (*type: [convection_diffusion_concentration](#)*) Constituent transport equations (concentration diffusion convection).

- **[convection_diffusion_temperature]** (*type*: [convection_diffusion_temperature](#)) Energy equation (temperature diffusion convection).
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of [Un_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of [Nom_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of [Un_postraitement_spec](#)) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class [format_file](#)). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see [schema_temps_base](#)) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_cloned_concentration_turbulent

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/energy/multiple constituent transport equations, with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituents.
- **[navier_stokes_turbulent]** (*type*: [navier_stokes_turbulent](#)) Navier-Stokes equations as well as the associated turbulence model equations.
- **[convection_diffusion_concentration_turbulent]** (*type*: [convection_diffusion_concentration_turbulent](#)) Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **[convection_diffusion_temperature_turbulent]** (*type*: [convection_diffusion_temperature_turbulent](#)) Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of [Un_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of [Nom_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name)

- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_concentration

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/energy/multiple constituent transport equations.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituents.
- **[navier_stokes_standard]** (*type*: [navier_stokes_standard](#)) Navier-Stokes equations.
- **[convection_diffusion_concentration]** (*type*: [convection_diffusion_concentration](#)) Constituent transport equations (concentration diffusion convection).
- **[convection_diffusion_temperature]** (*type*: [convection_diffusion_temperature](#)) Energy equation (temperature diffusion convection).
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.

- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*)
Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_concentration_scalaires_passifs

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/energy/multiple constituent transport equations, with the additional passive scalar equations.

Parameters:

- **fluide_incompressible** (*type: [fluide_incompressible](#)*) The fluid medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituents.
- **[navier_stokes_standard]** (*type: [navier_stokes_standard](#)*) Navier-Stokes equations.
- **[convection_diffusion_concentration]** (*type: [convection_diffusion_concentration](#)*) Constituent transport equations (concentration diffusion convection).
- **[convection_diffusion_temperature]** (*type: [convection_diffusion_temperature](#)*) Energy equations (temperature diffusion convection).
- **equations_scalaires_passifs** (*type: list of Eqn_base*) List of equations.
- **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
- **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*)
Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_concentration_turbulent**Inherits from:** [pb_base](#)

Resolution of Navier-Stokes/energy/multiple constituent transport equations, with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituents.
- **[navier_stokes_turbulent]** (*type*: [navier_stokes_turbulent](#)) Navier-Stokes equations as well as the associated turbulence model equations.
- **[convection_diffusion_concentration_turbulent]** (*type*: [convection_diffusion_concentration_turbulent](#)) Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **[convection_diffusion_temperature_turbulent]** (*type*: [convection_diffusion_temperature_turbulent](#)) Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_concentration_turbulent_scalaires_passifs**Inherits from:** [pb_base](#)

Resolution of Navier-Stokes/energy/multiple constituent transport equations, with turbulence modelling and with the additional passive scalar equations.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituents.

- **[navier_stokes_turbulent]** (*type*: [navier_stokes_turbulent](#)) Navier-Stokes equations as well as the associated turbulence model equations.
- **[convection_diffusion_concentration_turbulent]** (*type*: [convection_diffusion_concentration_turbulent](#)) Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **[convection_diffusion_temperature_turbulent]** (*type*: [convection_diffusion_temperature_turbulent](#)) Energy equations (temperature diffusion convection) as well as the associated turbulence model equations.
- **equations_scalaires_passifs** (*type*: list of Eqn_base) List of equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_especes_qc

Inherits from: [pb_base](#)

Resolution of thermo-hydraulic problem for a multi-species quasi-compressible fluid.

Parameters:

- **fluide_quasi_compressible** (*type*: [fluide_quasi_compressible](#)) The fluid medium associated with the problem.
- **navier_stokes_qc** (*type*: [navier_stokes_qc](#)) Navier-Stokes equation for a quasi-compressible fluid.
- **convection_diffusion_chaleur_qc** (*type*: [convection_diffusion_chaleur_qc](#)) Temperature equation for a quasi-compressible fluid.
- **equations_scalaires_passifs** (*type*: list of Eqn_base) List of equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.

- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of [Un_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of [Nom_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of [Un_postraitement_spec](#)) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class [format_file](#)). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see [schema_temps_base](#)) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_especes_turbulent_qc

Inherits from: [pb_base](#)

Resolution of turbulent thermohydraulic problem under low Mach number with passive scalar equations.

Parameters:

- **fluide_quasi_compressible** (*type*: [fluide_quasi_compressible](#)) The fluid medium associated with the problem.
- **navier_stokes_turbulent_qc** (*type*: [navier_stokes_turbulent_qc](#)) Navier-Stokes equations under low Mach number as well as the associated turbulence model equations.
- **convection_diffusion_chaleur_turbulent_qc** (*type*: [convection_diffusion_chaleur_turbulent_qc](#)) Energy equation under low Mach number as well as the associated turbulence model equations.
- **equations_scalaires_passifs** (*type*: list of [Eqn_base](#)) List of equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of [Un_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of [Nom_postraitement](#)) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of [Un_postraitement_spec](#)) Keyword to use several results files. List of objects of post-processing (with name)

- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_especes_wc

Inherits from: [pb_base](#)

Resolution of thermo-hydraulic problem for a multi-species weakly-compressible fluid.

Parameters:

- **fluide_weakly_compressible** (*type*: [fluide_weakly_compressible](#)) The fluid medium associated with the problem.
- **navier_stokes_wc** (*type*: [navier_stokes_wc](#)) Navier-Stokes equation for a weakly-compressible fluid.
- **convection_diffusion_chaleur_wc** (*type*: [convection_diffusion_chaleur_wc](#)) Temperature equation for a weakly-compressible fluid.
- **equations_scalaires_passifs** (*type*: list of Eqn_base) List of equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.

- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_ibm

Inherits from: [pb_base](#)

Resolution of IBM thermohydraulic problem.

Parameters:

- **[fluide_incompressible]** (*type: [fluide_incompressible](#)*) The fluid medium associated with the problem (only one possibility).
- **[fluide_ostwald]** (*type: [fluide_ostwald](#)*) The fluid medium associated with the problem (only one possibility).
- **[navier_stokes_ibm]** (*type: [navier_stokes_ibm](#)*) IBM Navier-Stokes equations.
- **[convection_diffusion_temperature_ibm]** (*type: [convection_diffusion_temperature_ibm](#)*) IBM Energy equation (temperature diffusion convection).
- **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituent.
- **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class [format_file](#)). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see [schema_temps_base](#)) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_ibm_turbulent

Inherits from: [pb_base](#)

Resolution of thermohydraulic problem, with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **navier_stokes_ibm_turbulent** (*type*: [navier_stokes_ibm_turbulent](#)) IBM Navier-Stokes equations as well as the associated turbulence model equations.
- **convection_diffusion_temperature_ibm_turbulent** (*type*: [convection_diffusion_temperature_ibm_turbulent](#)) Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_list_concentration

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/energy/multiple constituent transport equations.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituents.
- **[navier_stokes_standard]** (*type*: [navier_stokes_standard](#)) Navier-Stokes equations.
- **[convection_diffusion_temperature]** (*type*: [convection_diffusion_temperature](#)) Energy equation (temperature diffusion convection).
- **list_equations** (*type*: list of Eqn_base) List of equations.

- **[milieu]** (*type: milieu_base*) The medium associated with the problem.
- **[postraitement | post_processing]** (*type: corps_postraitement*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: format_file_base*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: format_file_base*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: format_file_base*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: format_file_base*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: transparent_medium_radiation*) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_list_concentration_turbulent

Inherits from: [pb_base](#)

Resolution of Navier-Stokes/energy/multiple constituent transport equations, with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type: fluide_incompressible*) The fluid medium associated with the problem.
- **[constituant]** (*type: constituant*) Constituents.
- **[navier_stokes_turbulent]** (*type: navier_stokes_turbulent*) Navier-Stokes equations as well as the associated turbulence model equations.
- **[convection_diffusion_temperature_turbulent]** (*type: convection_diffusion_temperature_turbulent*) Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **list_equations** (*type: list of Eqn_base*) List of equations.
- **[milieu]** (*type: milieu_base*) The medium associated with the problem.
- **[postraitement | post_processing]** (*type: corps_postraitement*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)

- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_qc

Inherits from: [pb_base](#)

Resolution of thermo-hydraulic problem for a quasi-compressible fluid.

Keywords for the unknowns other than pressure, velocity, temperature are :

masse_volumique : density

enthalpie : enthalpy

pression : reduced pressure

pression_tot : total pressure.

Parameters:

- **fluide_quasi_compressible** (*type: [fluide_quasi_compressible](#)*) The fluid medium associated with the problem.
- **navier_stokes_qc** (*type: [navier_stokes_qc](#)*) Navier-Stokes equation for a quasi-compressible fluid.
- **convection_diffusion_chaleur_qc** (*type: [convection_diffusion_chaleur_qc](#)*) Temperature equation for a quasi-compressible fluid.
- **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituent.
- **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.

- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_scalaires_passifs

Inherits from: [pb_base](#)

Resolution of thermohydraulic problem, with the additional passive scalar equations.

Parameters:

- **fluide_incompressible** (*type: [fluide_incompressible](#)*) The fluid medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituents.
- **[navier_stokes_standard]** (*type: [navier_stokes_standard](#)*) Navier-Stokes equations.
- **[convection_diffusion_temperature]** (*type: [convection_diffusion_temperature](#)*) Energy equations (temperature diffusion convection).
- **equations_scalaires_passifs** (*type: list of Eqn_base*) List of equations.
- **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
- **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_turbulent

Inherits from: [pb_base](#)

Resolution of thermohydraulic problem, with turbulence modelling.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **navier_stokes_turbulent** (*type*: [navier_stokes_turbulent](#)) Navier-Stokes equations as well as the associated turbulence model equations.
- **convection_diffusion_temperature_turbulent** (*type*: [convection_diffusion_temperature_turbulent](#)) Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_turbulent_qc

Inherits from: [pb_base](#)

Resolution of turbulent thermohydraulic problem under low Mach number.

Warning : Available for VDF and VEF P0/P1NC discretization only.

Parameters:

- **fluide_quasi_compressible** (*type*: [fluide_quasi_compressible](#)) The fluid medium associated with the problem.
- **navier_stokes_turbulent_qc** (*type*: [navier_stokes_turbulent_qc](#)) Navier-Stokes equations under low Mach number as well as the associated turbulence model equations.
- **convection_diffusion_chaleur_turbulent_qc** (*type*: [convection_diffusion_chaleur_turbulent_qc](#)) Energy equation under low Mach number as well as the associated turbulence model equations.

- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_turbulent_scalaires_passifs

Inherits from: [pb_base](#)

Resolution of thermohydraulic problem, with turbulence modelling and with the additional passive scalar equations.

Parameters:

- **fluide_incompressible** (*type*: [fluide_incompressible](#)) The fluid medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituents.
- **[navier_stokes_turbulent]** (*type*: [navier_stokes_turbulent](#)) Navier-Stokes equations as well as the associated turbulence model equations.
- **[convection_diffusion_temperature_turbulent]** (*type*: [convection_diffusion_temperature_turbulent](#)) Energy equations (temperature diffusion convection) as well as the associated turbulence model equations.
- **equations_scalaires_passifs** (*type*: list of Eqn_base) List of equations.
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)

- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type: [format_file_base](#)*) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type: [transparent_medium_radiation](#)*) Read a transparent medium radiation model associated to a fluid problem

pb_thermohydraulique_wc

Inherits from: [pb_base](#)

Resolution of thermo-hydraulic problem for a weakly-compressible fluid.

Keywords for the unknowns other than pressure, velocity, temperature are :

masse_volumique : density

pression : reduced pressure

pression_tot : total pressure

pression_hydro : hydro-static pressure

pression_eos : pressure used in state equation.

Parameters:

- **fluide_weakly_compressible** (*type: [fluide_weakly_compressible](#)*) The fluid medium associated with the problem.
- **navier_stokes_wc** (*type: [navier_stokes_wc](#)*) Navier-Stokes equation for a weakly-compressible fluid.
- **convection_diffusion_chaleur_wc** (*type: [convection_diffusion_chaleur_wc](#)*) Temperature equation for a weakly-compressible fluid.
- **[milieu]** (*type: [milieu_base](#)*) The medium associated with the problem.
- **[constituant]** (*type: [constituant](#)*) Constituent.
- **[postraitement | post_processing]** (*type: [corps_postraitement](#)*) One post-processing (without name).
- **[postraitements | post_processings]** (*type: list of Un_postraitement*) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type: list of Nom_postraitement*) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type: list of Un_postraitement_spec*) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type: [format_file_base](#)*) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type: [format_file_base](#)*) The same keyword than Sauvegarde except, the last time step only is saved.

- **[reprise]** (type: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (type: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (type: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

problem_read_generic

Inherits from: [pb_base](#)

The probleme_read_generic differs from the rest of the TRUST code : The problem does not state the number of equations that are enclosed in the problem. As the list of equations to be solved in the generic read problem is declared in the data file and not pre-defined in the structure of the problem, each equation has to be distinctively associated with the problem with the Associate keyword.

Parameters:

- **[milieu]** (type: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (type: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (type: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (type: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (type: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (type: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (type: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (type: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (type: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \leq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (type: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (type: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

probleme_elasticite_lineaire

Inherits from: [pb_base](#)

Resolution of Probleme_Elasticite_Lineaire

Parameters:

- **[milieu_elasticite]** (type: [milieu_elasticite](#)) Milieu_Elasticite
- **[equation_navier_cauchy]** (type: [equation_navier_cauchy](#)) Equation_Navier_Cauchy
- **[milieu]** (type: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (type: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (type: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (type: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (type: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (type: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (type: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (type: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (type: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N \neq P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (type: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (type: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

1.25.1.1 Keywords derived from pb_multiphase

pb_multiphase

Inherits from: [pb_base](#)

A problem that allows the resolution of N-phases with $3 \times N$ equations

Parameters:

- **[milieu_composite]** (type: [bloc_lecture](#)) The composite medium associated with the problem.
- **[milieu_musig]** (type: [bloc_lecture](#)) The composite medium associated with the problem.
- **[correlations]** (type: [bloc_lecture](#)) List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **[models]** (type: [bloc_lecture](#)) List of models used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **qdm_multiphase** (type: [qdm_multiphase](#)) Momentum conservation equation for a multi-phase problem where the unknown is the velocity

- **masse_multiphase** (*type*: [masse_multiphase](#)) Mass consevation equation for a multi-phase problem where the unknown is the alpha (void fraction)
- **energie_multiphase** (*type*: [energie_multiphase](#)) Internal energy conservation equation for a multi-phase problem where the unknown is the temperature
- **[echelle_temporelle_turbulente]** (*type*: [echelle_temporelle_turbulente](#)) Turbulent Dissipation time scale equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[energie_cinetique_turbulente]** (*type*: [energie_cinetique_turbulente](#)) Turbulent kinetic Energy conservation equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[energie_cinetique_turbulente_wit]** (*type*: [energie_cinetique_turbulente_wit](#)) Bubble Induced Turbulent kinetic Energy equation for a turbulent multi-phase problem (available in TrioCFD)
- **[taux_dissipation_turbulent]** (*type*: [taux_dissipation_turbulent](#)) Turbulent Dissipation frequency equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[aire_interfaciale]** (*type*: [aire_interfaciale](#)) Interfacial-area transport equation for a multi-phase problem (available in TrioCFD)
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_multiphase_enthalpie

Synonyms: pb_multiphase_h

Inherits from: [pb_multiphase](#)

A problem that allows the resolution of N-phases with 3*N equations

Parameters:

- **[milieu_composite]** (*type*: [bloc_lecture](#)) The composite medium associated with the problem.

- **[correlations]** (*type*: [bloc_lecture](#)) List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **qdm_multiphase** (*type*: [qdm_multiphase](#)) Momentum conservation equation for a multi-phase problem where the unknown is the velocity
- **masse_multiphase** (*type*: [masse_multiphase](#)) Mass conservation equation for a multi-phase problem where the unknown is the alpha (void fraction)
- **energie_multiphase_h | energie_multiphase_enthalpie** (*type*: [energie_multiphase_enthalpie](#)) Internal energy conservation equation for a multi-phase problem where the unknown is the enthalpy
- **[milieu_musig]** (*type*: [bloc_lecture](#)) The composite medium associated with the problem.
- **[models]** (*type*: [bloc_lecture](#)) List of models used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **[echelle_temporelle_turbulente]** (*type*: [echelle_temporelle_turbulente](#)) Turbulent Dissipation time scale equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[energie_cinetique_turbulente]** (*type*: [energie_cinetique_turbulente](#)) Turbulent kinetic Energy conservation equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[energie_cinetique_turbulente_wit]** (*type*: [energie_cinetique_turbulente_wit](#)) Bubble Induced Turbulent kinetic Energy equation for a turbulent multi-phase problem (available in TrioCFD)
- **[taux_dissipation_turbulent]** (*type*: [taux_dissipation_turbulent](#)) Turbulent Dissipation frequency equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[aire_interfaciale]** (*type*: [aire_interfaciale](#)) Interfacial-area transport equation for a multi-phase problem (available in TrioCFD)
- **[milieu]** (*type*: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (*type*: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (*type*: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (*type*: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (*type*: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (*type*: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (*type*: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (*type*: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.
- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

pb_multiphase_hem**Synonyms:** pb_hem**Inherits from:** pb_multiphase

A problem that allows the resolution of 2-phases mechanically and thermally coupled with 3 equations

Parameters:

- **[milieu_composite]** (type: [bloc_lecture](#)) The composite medium associated with the problem.
- **[milieu_musig]** (type: [bloc_lecture](#)) The composite medium associated with the problem.
- **[correlations]** (type: [bloc_lecture](#)) List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **[models]** (type: [bloc_lecture](#)) List of models used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **qdm_multiphase** (type: [qdm_multiphase](#)) Momentum conservation equation for a multi-phase problem where the unknown is the velocity
- **masse_multiphase** (type: [masse_multiphase](#)) Mass conservation equation for a multi-phase problem where the unknown is the alpha (void fraction)
- **energie_multiphase** (type: [energie_multiphase](#)) Internal energy conservation equation for a multi-phase problem where the unknown is the temperature
- **[echelle_temporelle_turbulente]** (type: [echelle_temporelle_turbulente](#)) Turbulent Dissipation time scale equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[energie_cinetique_turbulente]** (type: [energie_cinetique_turbulente](#)) Turbulent kinetic Energy conservation equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[energie_cinetique_turbulente_wit]** (type: [energie_cinetique_turbulente_wit](#)) Bubble Induced Turbulent kinetic Energy equation for a turbulent multi-phase problem (available in TrioCFD)
- **[taux_dissipation_turbulent]** (type: [taux_dissipation_turbulent](#)) Turbulent Dissipation frequency equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **[aire_interfaciale]** (type: [aire_interfaciale](#)) Interfacial-area transport equation for a multi-phase problem (available in TrioCFD)
- **[milieu]** (type: [milieu_base](#)) The medium associated with the problem.
- **[constituant]** (type: [constituant](#)) Constituent.
- **[postraitement | post_processing]** (type: [corps_postraitement](#)) One post-processing (without name).
- **[postraitements | post_processings]** (type: list of Un_postraitement) Keyword to use several results files. List of objects of post-processing (with name).
- **[liste_de_postraitements]** (type: list of Nom_postraitement) Keyword to use several results files. List of objects of post-processing (with name)
- **[liste_postraitements]** (type: list of Un_postraitement_spec) Keyword to use several results files. List of objects of post-processing (with name)
- **[sauvegarde]** (type: [format_file_base](#)) Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **[sauvegarde_simple]** (type: [format_file_base](#)) The same keyword than Sauvegarde except, the last time step only is saved.

- **[reprise]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file (see the class format_file). If format_reprise is xyz, the name_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ($N < > P$) processors. Should the calculation be resumed, values for the tinit (see schema_temps_base) time fields are taken from the name_file file. If there is no backup corresponding to this time in the name_file, TRUST exits in error.
- **[resume_last_time]** (*type*: [format_file_base](#)) Keyword to resume a calculation based on the name_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).
- **[transparent_medium_radiation_model | modele_rayonnement_milieu_transparent]** (*type*: [transparent_medium_radiation](#)) Read a transparent medium radiation model associated to a fluid problem

1.26 Keywords derived from porosites

porosites

To define the volume porosity and surface porosity that are uniform in every direction in space on a sub-area. Porosity was only usable in VDF discretization, and now available for VEF P1NC/P0.

Observations :

- Surface porosity values must be given in every direction in space (set this value to 1 if there is no porosity),
- Prior to defining porosity, the problem must have been discretized.

Can \t be used in VEF discretization, use Porosites_champ instead.

Parameters:

- **aco** (*type*: string into ['{'] Opening curly bracket.
- **sous_zone | sous_zone1** (*type*: string) Name of the sub-area to which porosity are allocated.
- **bloc** (*type*: [bloc_lecture_poro](#)) Surface and volume porosity values.
- **[sous_zone2]** (*type*: string) Name of the 2nd sub-area to which porosity are allocated.
- **[bloc2]** (*type*: [bloc_lecture_poro](#)) Surface and volume porosity values.
- **acof** (*type*: string into ['}'] Closing curly bracket.

1.27 Keywords derived from precondition_base

precond_base

Basic class for preconditioning.

precondsolv

Inherits from: [precond_base](#)

not_set

Parameters:

- **solveur** (*type*: [solveur_sys_base](#)) Solver type.
-

ssor

Inherits from: [precond_base](#)

Symmetric successive over-relaxation algorithm.

Parameters:

- **[omega]** (*type*: float) Over-relaxation facteur (between 1 and 2, default value 1.6).
-

ssor_bloc**Inherits from:** [precond_base](#)

not_set

Parameters:

- **[precond0]** (*type:* [precond_base](#)) not_set
- **[precond1]** (*type:* [precond_base](#)) not_set
- **[preconda]** (*type:* [precond_base](#)) not_set
- **[alpha_0]** (*type:* float) not_set
- **[alpha_1]** (*type:* float) not_set
- **[alpha_a]** (*type:* float) not_set

1.28 Keywords derived from preconditionneur_petsc_deriv**preconditionneur_petsc_deriv**Preconditioners available with petsc solvers

preconditionneur_petsc_block_jacobi_icc**Synonyms:** block_jacobi_icc**Inherits from:** [preconditionneur_petsc_deriv](#)

Incomplete Cholesky factorization for symmetric matrix with the PETSc implementation.

Parameters:

- **[level]** (*type:* int) factorization level (default value, 1). In parallel, the factorization is done by block (one per processor by default).
 - **[ordering]** (*type:* string into ['natural', 'rcm']) The ordering of the local matrix is natural by default, but rcm ordering, which reduces the bandwidth of the local matrix, may interestingly improves the quality of the decomposition and reduces the number of iterations.
-

preconditionneur_petsc_block_jacobi_ilu**Synonyms:** block_jacobi_ilu**Inherits from:** [preconditionneur_petsc_deriv](#)

preconditionner

Parameters:

- **[level]** (*type:* int) not_set
-

preconditionneur_petsc_boomeramg**Synonyms:** boomeramg**Inherits from:** [preconditionneur_petsc_deriv](#)

Multigrid preconditioner (no option is available yet, look at CLI command and Petsc documentation to try other options).

preconditionneur_petsc_c_amg**Synonyms:** c-amg**Inherits from:** [preconditionneur_petsc_deriv](#)preconditionner

preconditionneur_petsc_diag**Synonyms:** diag**Inherits from:** [preconditionneur_petsc_deriv](#)Diagonal (Jacobi) preconditioner.

preconditionneur_petsc_eisentat**Synonyms:** eisentat**Inherits from:** [preconditionneur_petsc_deriv](#)

SSOR version with Eisenstat trick which reduces the number of computations and thus CPU cost...

Parameters:

- **[omega]** (*type*: float) relaxation factor
-

preconditionneur_petsc_ilu_mumps**Synonyms:** ilu_mumps**Inherits from:** [preconditionneur_petsc_deriv](#)

Incomplete LU factorization with Block Low Ranking from the MUMPS library. Mapped at runtime onto Petsc's cholesky pc with mat_mumps_icntl_35=1 (BLR enabled).

Parameters:

- **[epsilon]** (*type*: float) BLR dropping parameter (passed through as mat_mumps_cntl_7).
-

preconditionneur_petsc_jacobi**Synonyms:** jacobi**Inherits from:** [preconditionneur_petsc_deriv](#)preconditionner

preconditionneur_petsc_lu**Synonyms:** lu**Inherits from:** [preconditionneur_petsc_deriv](#)preconditionner

preconditionneur_petsc_null**Synonyms:** null**Inherits from:** [preconditionneur_petsc_deriv](#)No preconditioner used

preconditionneur_petsc_pilut**Synonyms:** pilut**Inherits from:** [preconditionneur_petsc_deriv](#)

Dual Threshold Incomplete LU factorization.

Parameters:

- **[level]** (*type*: int) factorization level
 - **[epsilon]** (*type*: float) drop tolerance
-

preconditionneur_petsc_sa_amg**Synonyms:** sa-amg**Inherits from:** [preconditionneur_petsc_deriv](#)preconditionner

preconditionneur_petsc_spai**Synonyms:** spai**Inherits from:** [preconditionneur_petsc_deriv](#)

Spai Approximate Inverse algorithm from Parasails Hypre library.

Parameters:

- **[level]** (*type*: int) first parameter
- **[epsilon]** (*type*: float) second parameter

preconditionneur_petsc_ssor**Synonyms:** ssor**Inherits from:** [preconditionneur_petsc_deriv](#)

Symmetric Successive Over Relaxation algorithm.

Parameters:

- **[omega]** (*type*: float) relaxation factor (default value, 1.5)

1.29 Keywords derived from prepro_ibm_base**prepro_ibm_base**

To perform the intersection of an IB (Lagrange mesh) in a MED-format file .med with the Euler computational mesh.

Parameters:

- **[epsilon_prepro_ibm]** (*type*: float) geometric precision ($\ll 1$)
- **[constant_c_ibm]** (*type*: float) additive coefficient to search the purely fluid point (cf. publications G Billo)
- **[directions_pt_fluid]** (*type*: list of int) corresponding to each direction to search the purely fluid point
- **[mesh_lagrange_file]** (*type*: string) Name of the .med file including the IB surfacic mesh.
- **[mesh_lagrange_name]** (*type*: string) Name of the IB surfacic mesh.
- **[write_results_prepro]** (*type*: flag) to save output from prepro IBM
- **[out_med_file_name]** (*type*: string) Name of the .med file to save output from prepro IBM
- **[prepro_ibm_verbose]** (*type*: int) to get verbose version
- **[verify_results_prepro]** (*type*: flag) to save output from prepro IBM

prepro_ibm_ponderation**Synonyms:** methode_ibm_ponderation**Inherits from:** [prepro_ibm_base](#)

To perform the intersection of an IB (Lagrange mesh) in a MED-format file .med with the Euler computational mesh.

Parameters:

- **[type_de_ponderation]** (*type*: int) choix de la methode de ponderation
- **[epsilon_prepro_ibm]** (*type*: float) geometric precision ($\ll 1$)
- **[constant_c_ibm]** (*type*: float) additive coefficient to search the purely fluid point (cf. publications G Billo)
- **[directions_pt_fluid]** (*type*: list of int) corresponding to each direction to search the purely fluid point
- **[mesh_lagrange_file]** (*type*: string) Name of the .med file including the IB surfacic mesh.
- **[mesh_lagrange_name]** (*type*: string) Name of the IB surfacic mesh.
- **[write_results_prepro]** (*type*: flag) to save output from prepro IBM

- **[out_med_file_name]** (*type*: string) Name of the .med file to save output from prepro IBM
- **[prepro_ibm_verbose]** (*type*: int) to get verbose version
- **[verify_results_prepro]** (*type*: flag) to save output from prepro IBM

prepro_ibm_uzawa

Synonyms: methode_ibm_uzawa

Inherits from: [prepro_ibm_base](#)

To perform the intersection of an IB (Lagrange mesh) in a MED-format file .med with the Euler computational mesh.

Parameters:

- **[choix_de_la_methode_uzawa]** (*type*: int) choix de la methode d'Uzawa
- **[epsilon_prepro_ibm]** (*type*: float) geometric precision ($\ll 1$)
- **[constant_c_ibm]** (*type*: float) additive coefficient to search the purely fluid point (cf. publications G Billo)
- **[directions_pt_fluid]** (*type*: list of int) corresponding to each direction to search the purely fluid point
- **[mesh_lagrange_file]** (*type*: string) Name of the .med file including the IB surfacic mesh.
- **[mesh_lagrange_name]** (*type*: string) Name of the IB surfacic mesh.
- **[write_results_prepro]** (*type*: flag) to save output from prepro IBM
- **[out_med_file_name]** (*type*: string) Name of the .med file to save output from prepro IBM
- **[prepro_ibm_verbose]** (*type*: int) to get verbose version
- **[verify_results_prepro]** (*type*: flag) to save output from prepro IBM

1.30 Keywords derived from reorder_mesh

reorder_mesh

Reordering option to be used in a discretisation : the geometrical entities (nodes, elems, faces) of a mesh can be reordered to follow a Z-curve (Hilbert or Morton) improving data locality in memory.

Parameters:

- **[algo]** (*type*: string into ['none', 'morton', 'hilbert']) Z-Curve algorithm to use for geometrical entity renumbering.
- **[dump]** (*type*: flag) if set, will dump text files giving the numbering of the various geometrical entities before and after renumbering. Mainly used for debugging.
- **[no_nodes]** (*type*: flag) Whether to avoid node reordering.
- **[no_elems]** (*type*: flag) Whether to avoid element reordering.
- **[no_faces]** (*type*: flag) Whether to avoid face reordering.

1.31 Keywords derived from schema_temps_base

schema_temps_base

Basic class for time schemes. This scheme will be associated with a problem and the equations of this problem.

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).

- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.

- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

-
- [Keywords derived from sch_cn_iteratif](#)
 - [Keywords derived from schema_implicite_base](#)

euler_scheme

Synonyms: schema_euler_explicite, scheme_euler_explicit

Inherits from: [schema_temps_base](#)

This is the Euler explicit scheme.

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dGi/dt of all the unknown transported values Gi have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.

- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergence criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: [dt_start](#)) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

leap_frog

Inherits from: [schema_temps_base](#)

This is the leap-frog scheme.

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones

- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

runge_kutta_ordre_2**Inherits from:** [schema_temps_base](#)

This is a low-storage Runge-Kutta scheme of second order that uses 2 integration points. The method is presented by Williamson (case 1) in <https://www.sciencedirect.com/science/article/pii/0021999180900339>

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set

- **[dt_start]** (*type*: [dt_start](#)) `dt_start dt_min` : the first iteration is based on `dt_min`. `dt_start dt_calc` : the time step at first iteration is calculated in agreement with CFL condition. `dt_start dt_fixe` value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on `dt_calc`.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicit]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision Impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

runge_kutta_ordre_2_classique

Inherits from: [schema_temps_base](#)

This is a classical Runge-Kutta scheme of second order that uses 2 integration points.

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt Impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the `facsec` to 0.5. Warning: Some schemes needs a `facsec` lower than 1 (0.5 is a good start), for example `Schema_Adams_Bashforth_order_3`.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.

- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt = facsec * dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt = facsec * dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

runge_kutta_ordre_3

Inherits from: [schema_temps_base](#)

This is a low-storage Runge-Kutta scheme of third order that uses 3 integration points. The method is presented by Williamson (case 7) in <https://www.sciencedirect.com/science/article/pii/0021999180900339>

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).

- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.

- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

runge_kutta_ordre_3_classique

Inherits from: [schema_temps_base](#)

This is a classical Runge-Kutta scheme of third order that uses 3 integration points.

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.

- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergence criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: [dt_start](#)) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

runge_kutta_ordre_4

Synonyms: runge_kutta_ordre_4_d3p

Inherits from: [schema_temps_base](#)

This is a low-storage Runge-Kutta scheme of fourth order that uses 3 integration points. The method is presented by Williamson (case 17) in <https://www.sciencedirect.com/science/article/pii/0021999180900339>

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).

- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

runge_kutta_ordre_4_classique

Inherits from: [schema_temps_base](#)

This is a classical Runge-Kutta scheme of fourth order that uses 4 integration points.

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set

- **[dt_start]** (*type*: [dt_start](#)) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision Impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

runge_kutta_ordre_4_classique_3_8

Inherits from: [schema_temps_base](#)

This is a classical Runge-Kutta scheme of fourth order that uses 4 integration points and the 3/8 rule.

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt Impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.

- **[residuals]** (*type: residuals*) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type: int*) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_convection$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_max$.
- **[seuil_diffusion_implicite]** (*type: float*) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type: int*) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type: int*) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type: int*) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type: int*) not_set
- **[dt_start]** (*type: dt_start*) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type: int*) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type: int*) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type: int*) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type: float*) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type: flag*) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type: flag*) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type: flag*) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type: flag*) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type: int*) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

runge_kutta_rationnel_ordre_2

Inherits from: [schema_temps_base](#)

This is the Runge-Kutta rational scheme of second order. The method is described in the note: Wambeck - Rational Runge-Kutta methods for solving systems of ordinary differential equations, at the link: <https://link.springer.com/article/10.1007/BF02252381>. Although rational methods require more computational work than linear ones, they can have some other properties, such as a stable behaviour with explicitness, which make them preferable. The CFD application of this RRK2 scheme is described in the note: https://link.springer.com/content/pdf/10.1007%2F3-540-13917-6_112.pdf.

Parameters:

- **[tinit]** (*type: float*) Value of initial calculation time (0 by default).

- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.

- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

schema_adams_bashforth_order_2

Inherits from: [schema_temps_base](#)

not_set

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dGi/dt of all the unknown transported values Gi have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).

- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_convection$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large *facsec* value. Start with a *facsec* value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_max$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: [dt_start](#)) *dt_start dt_min* : the first iteration is based on *dt_min*. *dt_start dt_calc* : the time step at first iteration is calculated in agreement with CFL condition. *dt_start dt_fixe* value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on *dt_calc*.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

schema_adams_bashforth_order_3

Inherits from: [schema_temps_base](#)

not_set

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).

- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.

- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

schema_predictor_corrector

Inherits from: [schema_temps_base](#)

This is the predictor-corrector scheme (second order). It is more accurate and economic than MacCormack scheme. It gives best results with a second ordre convective scheme like quick, centre (VDF).

Parameters:

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpu_max]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.

- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: [dt_start](#)) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

1.31.1 Keywords derived from sch_cn_iteratif

sch_cn_iteratif

Inherits from: [schema_temps_base](#)

The Crank-Nicholson method of second order accuracy. A mid-point rule formulation is used (Euler-centered scheme). The basic scheme is: $u(t+1) = u(t) + du/dt(t+1/2)*dt$. The estimation of the time derivative du/dt at the level $(t+1/2)$ is obtained either by iterative process. The time derivative du/dt at the level $(t+1/2)$ is calculated iteratively with a simple under-relaxations method. Since the method is implicit, neither the cfl nor the fourier stability criteria must be respected. The time step is calculated in a way that the iterative procedure converges with the less iterations as possible.

Remark : for stationary or RANS calculations, no limitation can be given for time step through high value of facsec↔_max parameter (for instance : facsec_max 1000). In counterpart, for LES calculations, high values of facsec_max may engender numerical instabilities.

Parameters:

- **[seuil]** (*type*: float) criteria for ending iterative process ($\text{Max}(\|u(p) - u(p-1)\|/\text{Max} \|u(p)\|) < \text{seuil}$) (0.001 by default)
- **[niter_min]** (*type*: int) minimal number of p-iterations to satisfy convergence criteria (2 by default)
- **[niter_max]** (*type*: int) number of maximum p-iterations allowed to satisfy convergence criteria (6 by default)
- **[niter_avg]** (*type*: int) threshold of p-iterations (3 by default). If the number of p-iterations is greater than niter_avg, facsec is reduced, if lesser than niter_avg, facsec is increased (but limited by the facsec_max value).
- **[facsec_max]** (*type*: float) maximum ratio allowed between dynamical time step returned by iterative process and stability time returned by CFL condition (2 by default).

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).

- **[niter_max_diffusion_implicit]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision Impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

sch_cn_ex_iteratif

Inherits from: [sch_cn_iteratif](#)

This keyword also describes a Crank-Nicholson method of second order accuracy but here, for scalars, because of instabilities encountered when $dt > dt_CFL$, the Crank Nicholson scheme is not applied to scalar quantities. Scalars are treated according to Euler- Explicite scheme at the end of the CN treatment for velocity flow fields (by doing p Euler explicite under-iterations at $dt \leq dt_CFL$). Parameters are the same (but default values may change) compare to the Sch_CN_iterative scheme plus a relaxation keyword: niter_min (2 by default), niter_max (6 by default), niter_avg (3 by default), facsec_max (20 by default), seuil (0.05 by default)

Parameters:

- **[omega]** (*type*: float) relaxation factor (0.1 by default)
- **[seuil]** (*type*: float) criteria for ending iterative process ($\text{Max}(\|u(p) - u(p-1)\| / \text{Max} \|u(p)\|) < \text{seuil}$) (0.001 by default)
- **[niter_min]** (*type*: int) minimal number of p-iterations to satisfy convergence criteria (2 by default)
- **[niter_max]** (*type*: int) number of maximum p-iterations allowed to satisfy convergence criteria (6 by default)
- **[niter_avg]** (*type*: int) threshold of p-iterations (3 by default). If the number of p-iterations is greater than niter_avg, facsec is reduced, if lesser than niter_avg, facsec is increased (but limited by the facsec_max value).
- **[facsec_max]** (*type*: float) maximum ratio allowed between dynamical time step returned by iterative process and stability time returned by CFL condition (2 by default).
- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).

- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_{start} dt_{min} : the first iteration is based on dt_{min} . dt_{start} dt_{calc} : the time step at first iteration is calculated in agreement with CFL condition. dt_{start} dt_{fixe} value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_{calc} .
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

1.31.2 Keywords derived from `schema_implicite_base`

`schema_implicite_base`

Inherits from: [schema_temps_base](#)

Basic class for implicite time scheme.

Parameters:

- **[max_iter_implicite]** (*type*: int) Maximum number of iterations allowed for the solver (by default 200).
- **solveur** (*type*: [solveur_implicite_base](#)) This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. `solveur` is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains. Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.
- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the `.out` file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the `facsec` to 0.5. Warning: Some schemes needs a `facsec` lower than 1 (0.5 is a good start), for example `Schema_Adams_Bashforth_order_3`.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large `facsec` value. Start with a `facsec` value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.

- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergence criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: [dt_start](#)) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

schema_adams_moulton_order_2

Inherits from: [schema_implicite_base](#)

not_set

Parameters:

- **[facsec_max]** (*type*: float) Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec_max value. Warning: Some implicit schemes do not permit high facsec↔_max, example Schema_Adams_Moulton_order_3 needs facsec=facsec_max=1. Advice: The calculation may start with a facsec specified by the user and increased by the algorithm up to the facsec_max limit. But the user can also choose to specify a constant facsec (facsec_max will be set to facsec value then). Faster convergence has been seen and depends on the kind of calculation: -Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value beta low), facsec between 20-30-Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value beta high), facsec between 90-100 -Thermohydraulic with natural convection, facsec around 300 -Conduction only, facsec can be set to a very high value (1e8) as if the scheme was unconditionally stable These values can also be used as rule of thumb for initial facsec with a facsec_max limit higher.
- **[max_iter_implicite]** (*type*: int) Maximum number of iterations allowed for the solver (by default 200).

- **solveur** (*type*: [solveur_implicite_base](#)) This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. `solver` is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains. Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.
- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the `.out` file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the `facsec` to 0.5. Warning: Some schemes needs a `facsec` lower than 1 (0.5 is a good start), for example `Schema_Adams_Bashforth_order_3`.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large `facsec` value. Start with a `facsec` value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas

- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: [dt_start](#)) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision Impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

schema_adams_moulton_order_3

Inherits from: [schema_implicite_base](#)

not_set

Parameters:

- **[facsec_max]** (*type*: float) Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec_max value. Warning: Some implicit schemes do not permit high facsec↔_max, example Schema_Adams_Moulton_order_3 needs facsec=facsec_max=1. Advice: The calculation may start with a facsec specified by the user and increased by the algorithm up to the facsec_max limit. But the user can also choose to specify a constant facsec (facsec_max will be set to facsec value then). Faster convergence has been seen and depends on the kind of calculation: -Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value beta low), facsec between 20-30-Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value beta high), facsec between 90-100 -Thermohydraulic with natural convection, facsec around 300 -Conduction only, facsec can be set to a very high value (1e8) as if the scheme was unconditionally stable These values can also be used as rule of thumb for initial facsec with a facsec_max limit higher.
- **[max_iter_implicite]** (*type*: int) Maximum number of iterations allowed for the solver (by default 200).
- **solveur** (*type*: [solveur_implicite_base](#)) This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. solveur is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains. Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.

- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the `.out` file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the `facsec` to 0.5. Warning: Some schemes needs a `facsec` lower than 1 (0.5 is a good start), for example `Schema_Adams_Bashforth_order_3`.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large `facsec` value. Start with a `facsec` value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) `not_set`
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) `not_set`
- **[dt_start]** (*type*: [dt_start](#)) `dt_start dt_min` : the first iteration is based on `dt_min`. `dt_start dt_calc` : the time step at first iteration is calculated in agreement with CFL condition. `dt_start dt_fixe` value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on `dt_calc`.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).

- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision Impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

schema_backward_differentiation_order_2

Inherits from: [schema_implicite_base](#)

not_set

Parameters:

- **[facsec_max]** (*type*: float) Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec_max value. Warning: Some implicit schemes do not permit high facsec_max, example Schema_Adams_Moulton_order_3 needs facsec=facsec_max=1. Advice: The calculation may start with a facsec specified by the user and increased by the algorithm up to the facsec_max limit. But the user can also choose to specify a constant facsec (facsec_max will be set to facsec value then). Faster convergence has been seen and depends on the kind of calculation: -Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value beta low), facsec between 20-30-Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value beta high), facsec between 90-100 -Thermohydraulic with natural convection, facsec around 300 -Conduction only, facsec can be set to a very high value (1e8) as if the scheme was unconditionally stable. These values can also be used as rule of thumb for initial facsec with a facsec_max limit higher.
- **[max_iter_implicite]** (*type*: int) Maximum number of iterations allowed for the solver (by default 200).
- **solueur** (*type*: [solueur_implicite_base](#)) This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. solver is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains. Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.
- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).

- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.

- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

schema_backward_differentiation_order_3

Inherits from: [schema_implicite_base](#)

not_set

Parameters:

- **[facsec_max]** (*type*: float) Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec_max value. Warning: Some implicit schemes do not permit high facsec↔_max, example Schema_Adams_Moulton_order_3 needs facsec=facsec_max=1. Advice: The calculation may start with a facsec specified by the user and increased by the algorithm up to the facsec_max limit. But the user can also choose to specify a constant facsec (facsec_max will be set to facsec value then). Faster convergence has been seen and depends on the kind of calculation: -Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value beta low), facsec between 20-30-Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value beta high), facsec between 90-100 -Thermohydraulic with natural convection, facsec around 300 -Conduction only, facsec can be set to a very high value (1e8) as if the scheme was unconditionally stable These values can also be used as rule of thumb for initial facsec with a facsec_max limit higher.
- **[max_iter_implicite]** (*type*: int) Maximum number of iterations allowed for the solver (by default 200).
- **solueur** (*type*: [solueur_implicite_base](#)) This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. solver is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains. Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.
- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every dt_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt_sauv is in terms of physical time (not cpu time).

- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb_sauv_max timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema_Adams_Bashforth_order_3.
- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: residuals) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_{convection}$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_{max}$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: dt_start) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

schema_euler_implicite**Synonyms:** scheme_euler_implicit**Inherits from:** [schema_implicite_base](#)

This is the Euler implicit scheme.

Parameters:

- **[facsec_cfl]** (*type*: flag) Flag to compute time step based on CFL: $dt = \min(facsec, facsec_max) * dt(convection)X/$
- **[facsec_max]** (*type*: float) For old syntax, see the complete parameters of facsec for details
- **[facsec_expert]** (*type*: [facsec_expert](#)) Advanced facsec specification
- **[facsec_func]** (*type*: string) Advanced facsec specification as a function
- **[resolution_monolithique]** (*type*: [bloc_lecture](#)) Activate monolithic resolution for coupled problems. Solves together the equations corresponding to the application domains in the given order. All application domains of the coupled equations must be given to determine the order of resolution. If the monolithic solving is not wanted for a specific application domain, an underscore can be added as prefix. For example, `resolution_monolithique { dom1 { dom2 dom3 } _dom4 }` will solve in a single matrix the equations having dom1 as application domain, then the equations having dom2 or dom3 as application domain in a single matrix, then the equations having dom4 as application domain in a sequential way (not in a single matrix).
- **[max_iter_implicite]** (*type*: int) Maximum number of iterations allowed for the solver (by default 200).
- **solveur** (*type*: [solveur_implicite_base](#)) This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. `solveur` is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains. Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.
- **[tinit]** (*type*: float) Value of initial calculation time (0 by default).
- **[tmax]** (*type*: float) Time during which the calculation will be stopped (1e30s by default).
- **[tcpumax]** (*type*: float) CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **[dt_min]** (*type*: float) Minimum calculation time step (1e-16s by default).
- **[dt_max]** (*type*: string) Maximum calculation time step as function of time (1e30s by default).
- **[dt_sauv]** (*type*: float) Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **[nb_sauv_max]** (*type*: int) Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones
- **[dt_impr]** (*type*: float) Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the `.out` file.
- **[facsec]** (*type*: string) Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5. Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example `Schema_Adams_Bashforth_order_3`.

- **[seuil_statio]** (*type*: float) Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives dG_i/dt of all the unknown transported values G_i have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **[residuals]** (*type*: [residuals](#)) To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **[diffusion_implicite]** (*type*: int) Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ($dt=facsec*dt_convection$). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore $dt=facsec*dt_max$.
- **[seuil_diffusion_implicite]** (*type*: float) This keyword changes the default value (1e-6) of convergence criteria for the resolution by conjugate gradient used for implicit diffusion.
- **[impr_diffusion_implicite]** (*type*: int) Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **[impr_extremums]** (*type*: int) Print unknowns extremas
- **[no_error_if_not_converged_diffusion_implicite]** (*type*: int) not_set
- **[no_conv_subiteration_diffusion_implicite]** (*type*: int) not_set
- **[dt_start]** (*type*: [dt_start](#)) dt_start dt_min : the first iteration is based on dt_min. dt_start dt_calc : the time step at first iteration is calculated in agreement with CFL condition. dt_start dt_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity). By default, the first iteration is based on dt_calc.
- **[nb_pas_dt_max]** (*type*: int) Maximum number of calculation time steps (1e9 by default).
- **[niter_max_diffusion_implicite]** (*type*: int) This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **[precision_impr]** (*type*: int) Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **[periode_sauvegarde_securite_en_heures]** (*type*: float) To change the default period (23 hours) between the save of the fields in .sauv file.
- **[no_check_disk_space]** (*type*: flag) To disable the check of the available amount of disk space during the calculation.
- **[disable_progress]** (*type*: flag) To disable the writing of the .progress file.
- **[disable_dt_ev]** (*type*: flag) To disable the writing of the .dt_ev file.
- **[adapt_dt_tmax]** (*type*: flag) Use to adapt final dt when approaching tmax.
- **[gnuplot_header]** (*type*: int) Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

1.32 Keywords derived from solveur_implicite_base

solveur_implicite_base

not_set

- [Keywords derived from simpler](#)

newmark**Inherits from:** [solveur_implicite_base](#)

Newmark implicit solver for the resolution of the linear elastodynamic equation.

Parameters:

- **seuil_convergence_implicite** (*type*: float) Keyword to set the value of the convergence criteria for the resolution of the implicit system build to solve either the Navier_Stokes equation (only for Simple and Simpler algorithms) or a scalar equation. It is advised to use the default value (1e6) to solve the implicit system only once by time step. This value must be decreased when a coupling between problems is considered.
- **[seuil_convergence_solveur]** (*type*: float) value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier_Stokes equation and the scalar equations if any. This value **MUST** be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **[seuil_generation_solveur]** (*type*: float) Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system $Ax=B$ will be solved if residual error $||Ax-B||$ is lesser than vrel).
- **[seuil_verification_solveur]** (*type*: float) Option to check if residual error $||Ax-B||$ is lesser than vrel after the implicit linear system $Ax=B$ has been solved.
- **[seuil_test_preliminaire_solveur]** (*type*: float) Option to decide if the implicit linear system $Ax=B$ should be solved by checking if the residual error $||Ax-B||$ is bigger than vrel.
- **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gmres.
- **[controle_residu]** (*type*: flag) Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.
- **[alpha]** (*type*: float) Damping coefficient for Rayleigh damping: $C = \alpha * M$
- **[beta]** (*type*: float) Newmark parameter beta (default value 0.25 for average acceleration method).
- **[gamma]** (*type*: float) Newmark parameter gamma (default value 0.5 for average acceleration method).

solveur_lineaire_std**Inherits from:** [solveur_implicite_base](#)

not_set

Parameters:

- **[solveur]** (*type*: [solveur_sys_base](#)) not_set

1.32.1 Keywords derived from simpler**simpler****Inherits from:** [solveur_implicite_base](#)

Simpler method for incompressible systems.

Parameters:

- **seuil_convergence_implicite** (*type*: float) Keyword to set the value of the convergence criteria for the resolution of the implicit system build to solve either the Navier_Stokes equation (only for Simple and Simpler algorithms) or a scalar equation. It is advised to use the default value (1e6) to solve the implicit system only once by time step. This value must be decreased when a coupling between problems is considered.
- **[seuil_convergence_solveur]** (*type*: float) value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier_Stokes equation and the scalar equations if any. This value **MUST** be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).

- **[seuil_generation_solveur]** (*type*: float) Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system $Ax=B$ will be solved if residual error $\|Ax-B\|$ is lesser than vrel).
- **[seuil_verification_solveur]** (*type*: float) Option to check if residual error $\|Ax-B\|$ is lesser than vrel after the implicit linear system $Ax=B$ has been solved.
- **[seuil_test_preliminaire_solveur]** (*type*: float) Option to decide if the implicit linear system $Ax=B$ should be solved by checking if the residual error $\|Ax-B\|$ is bigger than vrel.
- **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **[no_qdm]** (*type*: flag) Keyword to not solve qdm equation (and turbulence models of these equation).
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gmres.
- **[controle_residu]** (*type*: flag) Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

-
- [Keywords derived from piso](#)
 - [Keywords derived from sets](#)

1.32.1.1 Keywords derived from piso

piso

Inherits from: [simpler](#)

Piso (Pressure Implicit with Split Operator) - method to solve N_S.

Parameters:

- **[seuil_convergence_implicitite]** (*type*: float) Convergence criteria.
- **[nb_corrections_max]** (*type*: int) Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections then `nb_corrections_max` if the accuracy of the projection is sufficient. (By default `nb_corrections_max` is set to 21).
- **[seuil_convergence_solveur]** (*type*: float) value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **[seuil_generation_solveur]** (*type*: float) Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system $Ax=B$ will be solved if residual error $\|Ax-B\|$ is lesser than vrel).
- **[seuil_verification_solveur]** (*type*: float) Option to check if residual error $\|Ax-B\|$ is lesser than vrel after the implicit linear system $Ax=B$ has been solved.
- **[seuil_test_preliminaire_solveur]** (*type*: float) Option to decide if the implicit linear system $Ax=B$ should be solved by checking if the residual error $\|Ax-B\|$ is bigger than vrel.
- **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **[no_qdm]** (*type*: flag) Keyword to not solve qdm equation (and turbulence models of these equation).
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gmres.
- **[controle_residu]** (*type*: flag) Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

-
- [Keywords derived from simple](#)

implicite**Inherits from:** [piso](#)

similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps. But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains.

Parameters:

- **[seuil_convergence_implicite]** (*type*: float) Convergence criteria.
- **[nb_corrections_max]** (*type*: int) Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections then `nb_corrections_max` if the accuracy of the projection is sufficient. (By default `nb_corrections_max` is set to 21).
- **[seuil_convergence_solveur]** (*type*: float) value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value **MUST** be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **[seuil_generation_solveur]** (*type*: float) Option to create a GMRES solver and use `vrel` as the convergence threshold (implicit linear system $Ax=B$ will be solved if residual error $\|Ax-B\|$ is lesser than `vrel`).
- **[seuil_verification_solveur]** (*type*: float) Option to check if residual error $\|Ax-B\|$ is lesser than `vrel` after the implicit linear system $Ax=B$ has been solved.
- **[seuil_test_preliminaire_solveur]** (*type*: float) Option to decide if the implicit linear system $Ax=B$ should be solved by checking if the residual error $\|Ax-B\|$ is bigger than `vrel`.
- **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **[no_qdm]** (*type*: flag) Keyword to not solve qdm equation (and turbulence models of these equation).
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gmres.
- **[controle_residu]** (*type*: flag) Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

1.32.1.1.1 Keywords derived from simple**simple****Inherits from:** [piso](#)

SIMPLE type algorithm

Parameters:

- **[relax_pression]** (*type*: float) Value between 0 and 1 (by default 1), this keyword is used only by the SIMPLE algorithm for relaxing the increment of pressure.
- **[seuil_convergence_implicite]** (*type*: float) Convergence criteria.
- **[nb_corrections_max]** (*type*: int) Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections then `nb_corrections_max` if the accuracy of the projection is sufficient. (By default `nb_corrections_max` is set to 21).
- **[seuil_convergence_solveur]** (*type*: float) value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value **MUST** be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **[seuil_generation_solveur]** (*type*: float) Option to create a GMRES solver and use `vrel` as the convergence threshold (implicit linear system $Ax=B$ will be solved if residual error $\|Ax-B\|$ is lesser than `vrel`).
- **[seuil_verification_solveur]** (*type*: float) Option to check if residual error $\|Ax-B\|$ is lesser than `vrel` after the implicit linear system $Ax=B$ has been solved.

- **[seuil_test_preliminaire_solveur]** (*type*: float) Option to decide if the implicit linear system $Ax=B$ should be solved by checking if the residual error $\|Ax-B\|$ is bigger than *vrel*.
- **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **[no_qdm]** (*type*: flag) Keyword to not solve qdm equation (and turbulence models of these equation).
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gmres.
- **[controle_residu]** (*type*: flag) Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

solveur_u_p

Inherits from: [simple](#)

similar to simple.

Parameters:

- **[relax_pression]** (*type*: float) Value between 0 and 1 (by default 1), this keyword is used only by the SIMPLE algorithm for relaxing the increment of pressure.
- **[seuil_convergence_implicite]** (*type*: float) Convergence criteria.
- **[nb_corrections_max]** (*type*: int) Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections then *nb_corrections_max* if the accuracy of the projection is sufficient. (By default *nb_corrections_max* is set to 21).
- **[seuil_convergence_solveur]** (*type*: float) value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **[seuil_generation_solveur]** (*type*: float) Option to create a GMRES solver and use *vrel* as the convergence threshold (implicit linear system $Ax=B$ will be solved if residual error $\|Ax-B\|$ is lesser than *vrel*).
- **[seuil_verification_solveur]** (*type*: float) Option to check if residual error $\|Ax-B\|$ is lesser than *vrel* after the implicit linear system $Ax=B$ has been solved.
- **[seuil_test_preliminaire_solveur]** (*type*: float) Option to decide if the implicit linear system $Ax=B$ should be solved by checking if the residual error $\|Ax-B\|$ is bigger than *vrel*.
- **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **[no_qdm]** (*type*: flag) Keyword to not solve qdm equation (and turbulence models of these equation).
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gmres.
- **[controle_residu]** (*type*: flag) Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

1.32.1.2 Keywords derived from sets

sets

Inherits from: [simpler](#)

Stability-Enhancing Two-Step solver which is useful for a multiphase problem. Ref : J. H. MAHAFFY, A stability-enhancing two-step method for fluid flow calculations, Journal of Computational Physics, 46, 3, 329 (1982).

Parameters:

- **[criteres_convergence]** (*type*: [bloc_criteres_convergence](#)) Set the convergence thresholds for each unknown (i.e: alpha, temperature, velocity and pressure). The default values are respectively 0.01, 0.1, 0.01 and 100

- **[iter_min]** (*type*: int) Number of minimum iterations (default value 1)
- **[iter_max]** (*type*: int) Number of maximum iterations (default value 10)
- **[seuil_convergence_implicite]** (*type*: float) Convergence criteria.
- **[nb_corrections_max]** (*type*: int) Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than `nb_corrections_max` if the accuracy of the projection is sufficient. (By default `nb_corrections_max` is set to 21).
- **[facsec_diffusion_for_sets]** (*type*: float) `facsec` to impose on the diffusion time step in sets while the total time step stays smaller than the convection time step.
- **[seuil_convergence_solveur]** (*type*: float) value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **[seuil_generation_solveur]** (*type*: float) Option to create a GMRES solver and use `vrel` as the convergence threshold (implicit linear system $Ax=B$ will be solved if residual error $\|Ax-B\|$ is lesser than `vrel`).
- **[seuil_verification_solveur]** (*type*: float) Option to check if residual error $\|Ax-B\|$ is lesser than `vrel` after the implicit linear system $Ax=B$ has been solved.
- **[seuil_test_preliminaire_solveur]** (*type*: float) Option to decide if the implicit linear system $Ax=B$ should be solved by checking if the residual error $\|Ax-B\|$ is bigger than `vrel`.
- **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **[no_qdm]** (*type*: flag) Keyword to not solve qdm equation (and turbulence models of these equation).
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gmres.
- **[controle_residu]** (*type*: flag) Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the `residu` suddenly increases.

ice

Inherits from: [sets](#)

Implicit Continuous-fluid Eulerian solver which is useful for a multiphase problem. Robust pressure reduction resolution.

Parameters:

- **[pression_degeneree]** (*type*: int) Set to 1 if the pressure field is degenerate (ex. : incompressible fluid with no imposed-pressure BCs). Default: autodetected
- **[reduction_pression | pressure_reduction]** (*type*: int) Set to 1 if the user wants a resolution with a pressure reduction. Otherwise, the flag is to be set to 0 so that the complete matrix is considered. The default value of this flag is 1.
- **[criteres_convergence]** (*type*: [bloc_criteres_convergence](#)) Set the convergence thresholds for each unknown (i.e: alpha, temperature, velocity and pressure). The default values are respectively 0.01, 0.1, 0.01 and 100
- **[iter_min]** (*type*: int) Number of minimum iterations (default value 1)
- **[iter_max]** (*type*: int) Number of maximum iterations (default value 10)
- **[seuil_convergence_implicite]** (*type*: float) Convergence criteria.
- **[nb_corrections_max]** (*type*: int) Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than `nb_corrections_max` if the accuracy of the projection is sufficient. (By default `nb_corrections_max` is set to 21).

- **[facsec_diffusion_for_sets]** (*type*: float) facsec to impose on the diffusion time step in sets while the total time step stays smaller than the convection time step.
- **[seuil_convergence_solveur]** (*type*: float) value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **[seuil_generation_solveur]** (*type*: float) Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system $Ax=B$ will be solved if residual error $\|Ax-B\|$ is lesser than vrel).
- **[seuil_verification_solveur]** (*type*: float) Option to check if residual error $\|Ax-B\|$ is lesser than vrel after the implicit linear system $Ax=B$ has been solved.
- **[seuil_test_preliminaire_solveur]** (*type*: float) Option to decide if the implicit linear system $Ax=B$ should be solved by checking if the residual error $\|Ax-B\|$ is bigger than vrel.
- **[solveur]** (*type*: [solveur_sys_base](#)) Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **[no_qdm]** (*type*: flag) Keyword to not solve qdm equation (and turbulence models of these equation).
- **[nb_it_max]** (*type*: int) Keyword to set the maximum iterations number for the Gmres.
- **[controle_residu]** (*type*: flag) Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

1.33 Keywords derived from solveur_petsc_deriv

solveur_petsc_deriv

Additional information is available in the PETSC documentation: <https://petsc.org/release/manual/>

Parameters:

- **[seuil]** (*type*: float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $\|Ax-B\|$ is less than seuil.
- **[quiet]** (*type*: flag) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type*: flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[rtol]** (*type*: float) not_set
- **[atol]** (*type*: float) not_set
- **[save_matrix_mtx_format]** (*type*: flag) not_set

solveur_petsc_bicgstab

Synonyms: bicgstab

Inherits from: [solveur_petsc_deriv](#)

Stabilized Bi-Conjugate Gradient

Parameters:

- **[precond]** (*type*: [preconditionneur_petsc_deriv](#)) not_set
- **[seuil]** (*type*: float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $\|Ax-B\|$ is less than seuil.
- **[quiet]** (*type*: flag) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type*: flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).

- **[rtol]** (*type*: float) not_set
- **[atol]** (*type*: float) not_set
- **[save_matrix_mtx_format]** (*type*: flag) not_set

solveur_petsc_cholesky

Synonyms: cholesky

Inherits from: [solveur_petsc_deriv](#)

Parallelized version of Cholesky from MUMPS library. This solver accepts an option to select a different ordering than the automatic selected one by MUMPS (and printed by using the impr option). The possible choices are Metis, Scotch, PT-Scotch or Parmetis. The two last options can only be used during a parallel calculation, whereas the two first are available for sequential or parallel calculations. It seems that the CPU cost of $A=LU$ factorization but also of the backward/forward elimination steps may sometimes be reduced by selecting a different ordering (Scotch seems often the best for b/f elimination) than the default one.

Notice that this solver requires a huge amount of memory compared to iterative methods. To know how much RAM you will need by core, then use the impr option to have detailed informations during the analysis phase and before the factorisation phase (in the following output, you will learn that the largest memory is taken by the zeroth CPU with 108MB):

Rank of proc needing largest memory in IC facto : 0

Estimated corresponding MBYTES for IC facto : 108

Thanks to the following graph, you read that in order to solve for instance a flow on a mesh with 2.6e6 cells, you will need to run a parallel calculation on 32 CPUs if you have cluster nodes with only 4GB/core (6.2GB*0.42~2.6GB) :

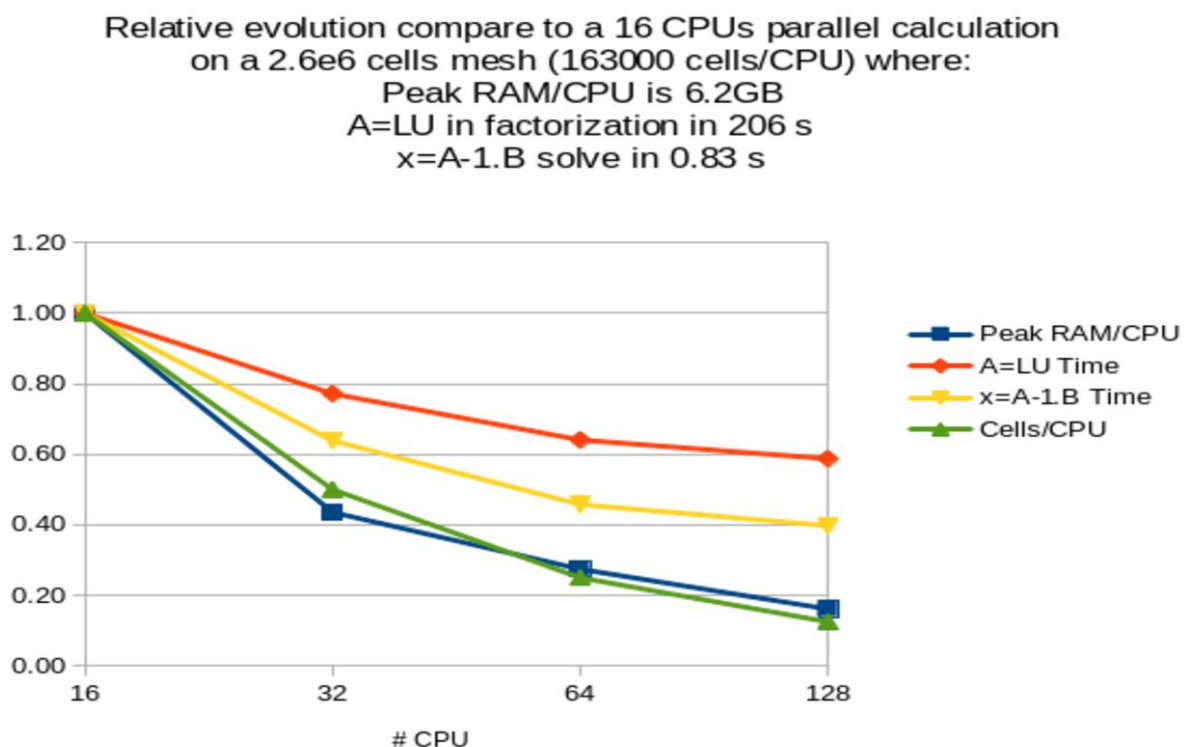


Figure 9 petscgraph

Parameters:

- **[save_matrice | save_matrix]** (*type*: flag) not_set
- **[save_matrix_petsc_format]** (*type*: flag) not_set
- **[reduce_ram]** (*type*: flag) not_set

- **[cli_quiet]** (*type*: [solveur_petsc_option_cli](#)) not_set
- **[cli]** (*type*: [solveur_petsc_option_cli](#)) not_set
- **[seuil]** (*type*: float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $\|Ax-B\|$ is less than `seuil`.
- **[quiet]** (*type*: flag) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type*: flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[rtol]** (*type*: float) not_set
- **[atol]** (*type*: float) not_set
- **[save_matrix_mtx_format]** (*type*: flag) not_set

solveur_petsc_cholesky_lapack

Synonyms: `cholesky_lapack`

Inherits from: [solveur_petsc_deriv](#)

Sequential Cholesky via LAPACK (cannot be used in parallel). Accepts `factored_matrix` to save/read/disk-cache the factorisation.

Parameters:

- **[factored_matrix]** (*type*: string into ['save', 'read', 'disk']) Cache the LU factorisation: `save` writes it after computing, `read` loads a precomputed one, `disk` reads if present and otherwise computes-then-saves.
- **[seuil]** (*type*: float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $\|Ax-B\|$ is less than `seuil`.
- **[quiet]** (*type*: flag) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type*: flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[rtol]** (*type*: float) not_set
- **[atol]** (*type*: float) not_set
- **[save_matrix_mtx_format]** (*type*: flag) not_set

solveur_petsc_cholesky_mumps_blr

Synonyms: `cholesky_mumps_blr`

Inherits from: [solveur_petsc_deriv](#)

BLR for (Block Low-Rank)

Parameters:

- **[reduce_ram]** (*type*: flag) not_set
- **[dropping_parameter]** (*type*: float) not_set
- **[cli]** (*type*: [solveur_petsc_option_cli](#)) not_set
- **[seuil]** (*type*: float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $\|Ax-B\|$ is less than `seuil`.
- **[quiet]** (*type*: flag) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type*: flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[rtol]** (*type*: float) not_set
- **[atol]** (*type*: float) not_set
- **[save_matrix_mtx_format]** (*type*: flag) not_set

solveur_petsc_cholesky_out_of_core**Synonyms:** cholesky_out_of_core**Inherits from:** [solveur_petsc_deriv](#)

Same as the previous one but with a written LU decomposition of disk (save RAM memory but add an extra CPU cost during $Ax=B$ solve).

Parameters:

- **[seuil]** (*type*: float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $||Ax-B||$ is less than **seuil**.
 - **[quiet]** (*type*: flag) is a keyword which is used to not displaying any outputs of the solver.
 - **[impr]** (*type*: flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
 - **[rtol]** (*type*: float) not_set
 - **[atol]** (*type*: float) not_set
 - **[save_matrix_mtx_format]** (*type*: flag) not_set
-

solveur_petsc_cholesky_pastix**Synonyms:** cholesky_pastix**Inherits from:** [solveur_petsc_deriv](#)

Parallelized Cholesky from PASTIX library.

Parameters:

- **[seuil]** (*type*: float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $||Ax-B||$ is less than **seuil**.
 - **[quiet]** (*type*: flag) is a keyword which is used to not displaying any outputs of the solver.
 - **[impr]** (*type*: flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
 - **[rtol]** (*type*: float) not_set
 - **[atol]** (*type*: float) not_set
 - **[save_matrix_mtx_format]** (*type*: flag) not_set
-

solveur_petsc_cholesky_superlu**Synonyms:** cholesky_superlu**Inherits from:** [solveur_petsc_deriv](#)

Parallelized Cholesky from SUPERLU_DIST library (less CPU and RAM, efficient than the previous one)

Parameters:

- **[seuil]** (*type*: float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $||Ax-B||$ is less than **seuil**.
 - **[quiet]** (*type*: flag) is a keyword which is used to not displaying any outputs of the solver.
 - **[impr]** (*type*: flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
 - **[rtol]** (*type*: float) not_set
 - **[atol]** (*type*: float) not_set
 - **[save_matrix_mtx_format]** (*type*: flag) not_set
-

solveur_petsc_cholesky_umfpack**Synonyms:** cholesky_umfpack**Inherits from:** [solveur_petsc_deriv](#)

Sequential Cholesky from UMFPACK library (seems fast).

Parameters:

- **[seuil]** (*type:* float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $\|Ax-B\|$ is less than **seuil**.
- **[quiet]** (*type:* flag) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type:* flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[rtol]** (*type:* float) not_set
- **[atol]** (*type:* float) not_set
- **[save_matrix_mtx_format]** (*type:* flag) not_set

solveur_petsc_cli**Synonyms:** cli**Inherits from:** [solveur_petsc_deriv](#)

Command Line Interface. Should be used only by advanced users, to access the whole solver/preconditioners from the PETSC API. To find all the available options, run your calculation with the -ksp_view -help options:

trust datafile [N] -ksp_view -help

-pc_type Preconditioner:(one of) none jacobi pbjacobi bjacobi sor lu shell mg eisenstat ilu icc cholesky asm ksp composite redundant nn mat fieldsplit galerkin openmp spai hypre tfs (PCSetType)

HYPRE preconditioner options:

-pc_hype_type pilut (choose one of) pilut parasails boomeramg

HYPRE ParaSails Options

-pc_hype_parasails_nlevels 1: Number of number of levels (None)

-pc_hype_parasails_thresh 0.1: Threshold (None)

-pc_hype_parasails_filter 0.1: filter (None)

-pc_hype_parasails_loadbal 0: Load balance (None)

-pc_hype_parasails_logging: FALSE Print info to screen (None)

-pc_hype_parasails_reuse: FALSE Reuse nonzero pattern in preconditioner (None)

-pc_hype_parasails_sym nonsymmetric (choose one of) nonsymmetric SPD nonsymmetric,SPD

Krylov Method (KSP) Options

-ksp_type Krylov method:(one of) cg cgne stcg gltr richardson chebychev gmres tcqmr bcgs bcgsl cgs tfqmr cr lsqr preonly qcg bicg fgmres minres symmlq lgmres lcd (KSPSetType)

-ksp_max_it 10000: Maximum number of iterations (KSPSetTolerances)

-ksp_rtol 0: Relative decrease in residual norm (KSPSetTolerances)

-ksp_atol 1e-12: Absolute value of residual norm (KSPSetTolerances)

-ksp_divtol 10000: Residual norm increase cause divergence (KSPSetTolerances)

-ksp_converged_use_initial_residual_norm: Use initial residual residual norm for computing relative convergence

-ksp_monitor_singular_value stdout: Monitor singular values (KSPMonitorSet)

-ksp_monitor_short stdout: Monitor preconditioned residual norm with fewer digits (KSPMonitorSet)

-ksp_monitor_draw: Monitor graphically preconditioned residual norm (KSPMonitorSet)

-ksp_monitor_draw_true_residual: Monitor graphically true residual norm (KSPMonitorSet)

Example to use the multigrid method as a solver, not only as a preconditioner:

Solveur_pression Petsc CLI {-ksp_type richardson -pc_type hypre -pc_hype_type boomeramg -ksp_atol 1.e-7 }

Parameters:

- **cli_bloc** (*type:* [bloc_lecture](#)) bloc

solveur_petsc_cli_quiet**Synonyms:** cli_quiet**Inherits from:** [solveur_petsc_deriv](#)

solver

Parameters:

- **cli_quiet_bloc** (*type:* [bloc_lecture](#)) bloc

solveur_petsc_gcp**Synonyms:** gcp**Inherits from:** [solveur_petsc_deriv](#)

Preconditioned Conjugate Gradient

Parameters:

- **[precond]** (*type:* [preconditionneur_petsc_deriv](#)) preconditioner
- **[precond_null]** (*type:* flag) No preconditioner used, equivalent to precondition null { }
- **[rtol]** (*type:* float) not_set
- **[reuse_preconditioner_nb_it_max]** (*type:* int) not_set
- **[cli]** (*type:* [solveur_petsc_option_cli](#)) not_set
- **[reorder_matrix]** (*type:* int) not_set
- **[read_matrix]** (*type:* flag) save_matrix|read_matrix are the keywords to save|read into a file the constant matrix A of the linear system $Ax=B$ solved (eg: matrix from the pressure linear system for an incompressible flow). It is useful when you want to minimize the MPI communications on massive parallel calculation. Indeed, in VEF discretization, the overlapping width (generally 2, specified with the `largeur_joint` option in the partition keyword partition) can be reduced to 1, once the matrix has been properly assembled and saved. The cost of the MPI communications in TRUST itself (not in PETSc) will be reduced with length messages divided by 2. So the strategy is: I) Partition your VEF mesh with a `largeur_joint` value of 2 II) Run your parallel calculation on 0 time step, to build and save the matrix with the `save_matrix` option. A file named `Matrix_NBROWS_↵rows_NCPUS_cpus.petsc` will be saved to the disk (where NBROWS is the number of rows of the matrix and NCPUS the number of CPUs used). III) Partition your VEF mesh with a `largeur_joint` value of 1 IV) Run your parallel calculation completely now and substitute the `save_matrix` option by the `read_matrix` option. Some interesting gains have been noticed when the cost of linear system solve with PETSc is small compared to all the other operations.
- **[save_matrice | save_matrix]** (*type:* flag) see read_matrix
- **[petsc_decide]** (*type:* int) not_set
- **[pcshell]** (*type:* string) not_set
- **[aij]** (*type:* flag) not_set
- **[seuil]** (*type:* float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $||Ax-B||$ is less than `seuil`.
- **[quiet]** (*type:* flag) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type:* flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[atol]** (*type:* float) not_set
- **[save_matrix_mtx_format]** (*type:* flag) not_set

solveur_petsc_gmres**Synonyms:** gmres**Inherits from:** [solveur_petsc_deriv](#)

Generalized Minimal Residual

Parameters:

- **[precond]** (*type:* [preconditionneur_petsc_deriv](#)) not_set
 - **[reuse_preconditioner_nb_it_max]** (*type:* int) not_set
 - **[save_matrix_petsc_format]** (*type:* flag) not_set
 - **[nb_it_max]** (*type:* int) In order to specify a given number of iterations instead of a condition on the residue with the keyword `seuil`. May be useful when defining a PETSc solver for the implicit time scheme where convergence is very fast: 5 or less iterations seems enough.
 - **[seuil]** (*type:* float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $\|Ax-B\|$ is less than `seuil`.
 - **[quiet]** (*type:* flag) is a keyword which is used to not displaying any outputs of the solver.
 - **[impr]** (*type:* flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
 - **[rtol]** (*type:* float) not_set
 - **[atol]** (*type:* float) not_set
 - **[save_matrix_mtx_format]** (*type:* flag) not_set
-

solveur_petsc_ibicgstab**Synonyms:** ibicgstab**Inherits from:** [solveur_petsc_deriv](#)

Improved version of previous one for massive parallel computations (only a single global reduction operation instead of the usual 3 or 4).

Parameters:

- **[precond]** (*type:* [preconditionneur_petsc_deriv](#)) not_set
 - **[seuil]** (*type:* float) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $\|Ax-B\|$ is less than `seuil`.
 - **[quiet]** (*type:* flag) is a keyword which is used to not displaying any outputs of the solver.
 - **[impr]** (*type:* flag) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
 - **[rtol]** (*type:* float) not_set
 - **[atol]** (*type:* float) not_set
 - **[save_matrix_mtx_format]** (*type:* flag) not_set
-

solveur_petsc_lu**Synonyms:** lu**Inherits from:** [solveur_petsc_deriv](#)

Several solvers through PETSc API are available.

TIPS:

A) Solver for symmetric linear systems (e.g: Pressure system from Navier-Stokes equations):

-The CHOLESKY parallel solver is from MUMPS library. It offers better performance than all others solvers if you have enough RAM for your calculation. A parallel calculation on a cluster with 4GBytes on each processor, 40000 cells/processor seems the upper limit. Seems to be very slow to initialize above 500 cpus/cores.

-When running a parallel calculation with a high number of cpus/cores (typically more than 500) where preconditioner scalability is the key for CPU performance, consider BICGSTAB with BLOCK_JACOBI_ICC(1) as preconditioner or if not converges, GCP with BLOCK_JACOBI_ICC(1) as preconditioner.

-For other situations, the first choice should be GCP/SSOR. In order to fine tune the solver choice, each one of the previous list should be considered. Indeed, the CPU speed of a solver depends of a lot of parameters. You may give a try to the OPTIMAL solver to help you to find the fastest solver on your study.

B) Solver for non symmetric linear systems (e.g.: Implicit schemes):

The BICGSTAB/DIAG solver seems to offer the best performances.

Parameters:

- **[seuil]** (*type: float*) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $||Ax-B||$ is less than **seuil**.
- **[quiet]** (*type: flag*) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type: flag*) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[rtol]** (*type: float*) not_set
- **[atol]** (*type: float*) not_set
- **[save_matrix_mtx_format]** (*type: flag*) not_set

solveur_petsc_pipecg**Synonyms:** pipecg**Inherits from:** [solveur_petsc_deriv](#)

Pipelined Conjugate Gradient (possible reduced CPU cost during massive parallel calculation due to a single non-blocking reduction per iteration, if TRUST is built with a MPI-3 implementation)... no example in TRUST

Parameters:

- **[seuil]** (*type: float*) corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard $||Ax-B||$ is less than **seuil**.
- **[quiet]** (*type: flag*) is a keyword which is used to not displaying any outputs of the solver.
- **[impr]** (*type: flag*) used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **[rtol]** (*type: float*) not_set
- **[atol]** (*type: float*) not_set
- **[save_matrix_mtx_format]** (*type: flag*) not_set

1.34 Keywords derived from source_base**source_base**

Basic class of source terms introduced in the equation.

acceleration

Inherits from: [source_base](#)

Momentum source term to take in account the forces due to rotation or translation of a non Galilean referential R\ (centre O\') into the Galilean referential R (centre O).

Parameters:

- **[vitesse]** (*type:* [field_base](#)) Keyword for the velocity of the referential R\ into the R referential ($d\vec{OO'}/dt$ term [m.s-1]). The velocity is mandatory when you want to print the total cinetic energy into the non-mobile Galilean referential R (see `Ec_dans_repere_fixe` keyword).
- **[acceleration]** (*type:* [field_base](#)) Keyword for the acceleration of the referential R\ into the R referential ($d^2\vec{OO'}/dt^2$ term [m.s-2]). `field_base` is a time dependant field (eg: `Champ_Fonc_t`).
- **[omega]** (*type:* [field_base](#)) Keyword for a rotation of the referential R\ into the R referential [rad.s-1]. `field_base` is a 3D time dependant field specified for example by a `Champ_Fonc_t` keyword. The `time_field` field should have 3 components even in 2D (In 2D: 0 0 omega).
- **[domegadt]** (*type:* [field_base](#)) Keyword to define the time derivative of the previous rotation [rad.s-2]. Should be zero if the rotation is constant. The `time_field` field should have 3 components even in 2D (In 2D: 0 0 domegadt).
- **[centre_rotation]** (*type:* [field_base](#)) Keyword to specify the centre of rotation (expressed in R\ coordinates) of R\ into R (if the domain rotates with the R\ referential, the centre of rotation is $O'=(0,0,0)$). The `time_field` should have 2 or 3 components according the dimension 2 or 3.
- **[option]** (*type:* string into ['terme_complet', 'coriolis_seul', 'entrainement_seul']) Keyword to specify the kind of calculation: `terme_complet` (default option) will calculate both the Coriolis and centrifugal forces, `coriolis_seul` will calculate the first one only, `entrainement_seul` will calculate the second one only.

boussinesq_concentration

Inherits from: [source_base](#)

Class to describe a source term that couples the movement quantity equation and constituent transport equation with the Boussinesq hypothesis.

Parameters:

- **c0** (*type:* list of float) Reference concentration field type. The only field type currently available is `Champ_Uniforme` (Uniform field).

boussinesq_temperature

Inherits from: [source_base](#)

Class to describe a source term that couples the movement quantity equation and energy equation with the Boussinesq hypothesis.

Parameters:

- **t0** (*type:* string) Reference temperature value (oC or K). It can also be a time dependant function since the 1.6.6 version.
- **[verif_boussinesq]** (*type:* int) Keyword to check (1) or not (0) the reference value in comparison with the mean value in the domain. It is set to 1 by default.

canal_perio

Inherits from: [source_base](#)

Momentum source term to maintain flow rate. The expression of the source term is:

$$S(t) = (2*(Q(0) - Q(t)) - (Q(0) - Q(t-dt)))/(coeff*dt*area)$$

Where:

`coeff`=damping coefficient

area=area of the periodic boundary

Q(t)=flow rate at time t

dt=time step

Three files will be created during calculation on a datafile named DataFile.data. The first file contains the flow rate evolution. The second file is useful for resuming a calculation with the flow rate of the previous stopped calculation, and the last one contains the pressure gradient evolution:

-DataFile_Channel_Flow_Rate_ProblemName_BoundaryName

-DataFile_Channel_Flow_Rate_repr_ProblemName_BoundaryName

-DataFile_Pressure_Gradient_ProblemName_BoundaryName

Parameters:

- **[u_etoile]** (*type*: float) not_set
- **[coeff]** (*type*: float) Damping coefficient (optional, default value is 10).
- **[h]** (*type*: float) Half height of the channel.
- **bord** (*type*: string) The name of the (periodic) boundary normal to the flow direction.
- **[debit_impose]** (*type*: float) Optional option to specify the aimed flow rate Q(0). If not used, Q(0) is computed by the code after the projection phase, where velocity initial conditions are slightly changed to verify incompressibility.

coriolis

Inherits from: [source_base](#)

Keyword for a Coriolis term in hydraulic equation. Warning: Only available in VDF.

Parameters:

- **omega** (*type*: list of float) Value of omega.

correction_antal

Inherits from: [source_base](#)

Antal correction source term for multiphase problem

correction_tomiyama

Inherits from: [source_base](#)

Tomiyama correction source term for multiphase problem

darcy

Inherits from: [source_base](#)

Class for calculation in a porous media with source term of Darcy $-\nu/K \cdot V$. This keyword must be used with a permeability model. For the moment there are two models : permeability constant or Ergun's law. Darcy source term is available for quasi compressible calculation. A new keyword is added for porosity (porosite).

Parameters:

- **bloc** (*type*: [bloc_lecture](#)) Description.

derivee_forme

Inherits from: [source_base](#)

Class to define a source term corresponding to the shape derivative in the projection equation.

dirac**Inherits from:** [source_base](#)

Class to define a source term corresponding to a volume power release in the energy equation.

Parameters:

- **position** (*type*: list of float) not_set
- **ch** (*type*: [field_base](#)) Thermal power field type. To impose a volume power on a domain sub-area, the Champ_Uniforme_Morceaux (partly_uniform_field) type must be used. Warning : The volume thermal power is expressed in W.m-3.

dispersion_bulles**Inherits from:** [source_base](#)

Base class for source terms of bubble dispersion in momentum equation.

Parameters:

- **[beta]** (*type*: float) Mutliplying factor for the output of the bubble dispersion source term.

dp_impose**Inherits from:** [source_base](#)Source term to impose a pressure difference according to the formula : $DP = dp + dDP/dQ * (Q - Q0)$ **Parameters:**

- **aco** (*type*: string into ['{'] Opening curly bracket.
- **dp_type** (*type*: [type_perte_charge_deriv](#)) mass flow rate (kg/s).
- **surface** (*type*: string into ['surface']) not_set
- **bloc_surface** (*type*: [bloc_lecture](#)) Three syntaxes are possible for the surface definition block: For VDF and VEF: { X|Y|Z = location subzone_name } Only for VEF: { Surface surface_name }. For PolyMAC_CDO { Surface surface_name Orientation champ_uniforme }.
- **acof** (*type*: string into ['] Closing curly bracket.

echange_thermique_volumique**Inherits from:** [source_base](#)

Source term that exchanges heat volumetrically between two overlapping domains using interfacial area and thermal resistances.

Parameters:

- **nom | name** (*type*: string) Tag used to match the source terms on both sides of the coupling.
- **aire_interfaciale | interfacial_area** (*type*: [field_base](#)) Interfacial area per cell used to compute the exchange.
- **[conduction_length | epaisseur_conduction]** (*type*: [field_base](#)) Conduction length (if conduction is modeled).
- **[conductivity | conductivite]** (*type*: [field_base](#)) Thermal conductivity used with the conduction length (optional).
- **[flux_parietal | heat_flux]** (*type*: [flux_parietal_base](#)) Correlation used to compute the wall heat flux on each side of the interface.

flux_interfacial**Inherits from:** [source_base](#)

Source term of mass transfer between phases connected by the saturation object defined in saturation_XXXX

forchheimer**Inherits from:** [source_base](#)

Class to add the source term of Forchheimer $-C_f/\sqrt{K} \cdot V^2$ in the Navier-Stokes equations. We must precise a permeability model : constant or Ergun's law. Moreover we can give the constant C_f : by default its value is 1. Forchheimer source term is available also for quasi compressible calculation. A new keyword is added for porosity (porosite).

Parameters:

- **bloc** (*type*: [bloc_lecture](#)) Description.

frottement_interfacial**Inherits from:** [source_base](#)

Source term which corresponds to the phases friction at the interface

Parameters:

- **[a_res]** (*type*: float) void fraction at which the gas velocity is forced to approach liquid velocity (default $\alpha_{\text{evanescence}} \cdot 100$)
- **[dv_min]** (*type*: float) minimal relative velocity used to linearize interfacial friction at low velocities
- **[exp_res]** (*type*: int) exponent that callibrates intensity of velocity convergence (default 2)

perte_charge_anisotrope**Inherits from:** [source_base](#)

Anisotropic pressure loss.

Parameters:

- **lambda_ | lambda_u | lambda** (*type*: string) Function for loss coefficient which may be Reynolds dependant (Ex: $64/Re$).
- **lambda_ortho** (*type*: string) Function for loss coefficient in transverse direction which may be Reynolds dependant (Ex: $64/Re$).
- **diam_hydr** (*type*: [champ_don_base](#)) Hydraulic diameter value.
- **direction** (*type*: [champ_don_base](#)) Field which indicates the direction of the pressure loss.
- **[sous_zone]** (*type*: string) Optional sub-area where pressure loss applies.

perte_charge_circulaire**Inherits from:** [source_base](#)

New pressure loss.

Parameters:

- **lambda_ | lambda_u | lambda** (*type*: string) Function $f(Re_{\text{tot}}, Re_{\text{long}}, t, x, y, z)$ for loss coefficient in the longitudinal direction
- **diam_hydr** (*type*: [champ_don_base](#)) Hydraulic diameter value.
- **[sous_zone]** (*type*: string) Optional sub-area where pressure loss applies.
- **lambda_ortho** (*type*: string) function: Function $f(Re_{\text{tot}}, Re_{\text{ortho}}, t, x, y, z)$ for loss coefficient in transverse direction
- **diam_hydr_ortho** (*type*: [champ_don_base](#)) Transverse hydraulic diameter value.
- **direction** (*type*: [champ_don_base](#)) Field which indicates the direction of the pressure loss.

perte_charge_directionnelle**Inherits from:** [source_base](#)

Directional pressure loss (available in VEF and PolyMAC_CDO).

Parameters:

- **lambda_** | **lambda_u** | **lambda** (*type:* string) Function for loss coefficient which may be Reynolds dependant (Ex: 64/Re).
- **diam_hydr** (*type:* [champ_don_base](#)) Hydraulic diameter value.
- **direction** (*type:* [champ_don_base](#)) Field which indicates the direction of the pressure loss.
- **[sous_zone]** (*type:* string) Optional sub-area where pressure loss applies.

perte_charge_isotrope**Inherits from:** [source_base](#)

Isotropic pressure loss (available in VEF and PolyMAC_CDO).

Parameters:

- **lambda_** | **lambda_u** | **lambda** (*type:* string) Function for loss coefficient which may be Reynolds dependant (Ex: 64/Re).
- **diam_hydr** (*type:* [champ_don_base](#)) Hydraulic diameter value.
- **[sous_zone]** (*type:* string) Optional sub-area where pressure loss applies.
- **[regul]** (*type:* [bloc_lecture](#)) Optional parameter for pressure drop coefficient regulation on target flow rate.
- **[surface]** (*type:* [bloc_lecture](#)) Optional parameter for surface used to impose the pressure drop.

perte_charge_reguliere**Inherits from:** [source_base](#)

Source term modelling the presence of a bundle of tubes in a flow.

Parameters:

- **spec** (*type:* [spec_pdcr_base](#)) Description of longitudinale or transversale type.
- **zone_name** | **name_of_zone** (*type:* string) Name of the sub-area occupied by the tube bundle. A Sous_Zone (Sub-area) type object called zone_name should have been previously created.

perte_charge_singuliere**Inherits from:** [source_base](#)

Source term that is used to model a pressure loss over a surface area (transition through a grid, sudden enlargement) defined by the faces of elements located on the intersection of a subzone named subzone_name and a X,Y, or Z plane located at X,Y or Z = location.

Parameters:

- **dir** (*type:* string into ['kx', 'ky', 'kz', 'k']) KX, KY or KZ designate directional pressure loss coefficients for respectively X, Y or Z direction. Or in the case where you chose a target flow rate with regul. Use K for isotropic pressure loss coefficient
- **[coeff]** (*type:* float) Value (float) of friction coefficient (KX, KY, KZ).
- **[regul]** (*type:* [bloc_lecture](#)) option to have adjustable K with flowrate target { K0 valeur_initiale_de_k deb debit_cible eps intervalle_variation_mutiplicatif}.
- **surface** (*type:* [bloc_lecture](#)) Three syntaxes are possible for the surface definition block: For VDF and VEF: { X|Y|Z = location subzone_name } Only for VEF: { Surface surface_name }. For PolyMAC_CDO { Surface surface_name Orientation champ_uniforme }

portance_interfaciale**Inherits from:** [source_base](#)

Base class for source term of lift force in momentum equation.

Parameters:

- **[beta]** (*type:* float) Multiplying factor for the bubble lift force source term.

puissance_thermique**Inherits from:** [source_base](#)

Class to define a source term corresponding to a volume power release in the energy equation.

Parameters:

- **ch** (*type:* [field_base](#)) Thermal power field type. To impose a volume power on a domain sub-area, the Champ_Uniforme_Morceaux ([partly_uniform_field](#)) type must be used. Warning : The volume thermal power is expressed in W.m-3 in 3D (in W.m-2 in 2D). It is a power per volume unit (in a porous media, it is a power per fluid volume unit).

radioactive_decay**Inherits from:** [source_base](#)Radioactive decay source term of the form $-\lambda_i c_i$, where $0 \leq i \leq N$, N is the number of component of the constituent, c_i and λ_i are the concentration and the decay constant of the i -th component of the constituent.**Parameters:**

- **val** (*type:* list of float) n is the number of decay constants to read (int), and $val1, val2...$ are the decay constants (double)

source_constituant**Inherits from:** [source_base](#)Keyword to specify source rates, in $[[C]/s]$, for each one of the nb constituents. $[C]$ is the concentration unit.**Parameters:**

- **ch** (*type:* [field_base](#)) Field type.

source_generique**Inherits from:** [source_base](#)

to define a source term depending on some discrete fields of the problem and (or) analytic expression. It is expressed by the way of a generic field usually used for post- processing.

Parameters:

- **champ** (*type:* [champ_generique_base](#)) the source field

source_meca_grad_pression_thermique**Inherits from:** [source_base](#)

Source term that applies the gradient of thermal pressure as an equivalent body force in the linear elasticity momentum equation.

Parameters:

- **reference_temperature_field** (*type:* [field_base](#)) Reference temperature field used to compute the thermal pressure gradient.
- **temperature_field** (*type:* [field_base](#)) Temperature field used to compute the thermal pressure gradient.

source_pdf**Inherits from:** [source_base](#)

Source term for Penalised Direct Forcing (PDF) method.

Parameters:

- **[prepro_ibm]** (*type*: [prepro_ibm_base](#)) to realise the PDF IBM preprocessing
- **[aire]** (*type*: [field_base](#)) volumic field: a boolean for the cell (0 or 1) indicating if the obstacle is in the cell
- **[get_aire_from_prepro]** (*type*: flag) get aire IBM from prepro.
- **[barycentre]** (*type*: [field_base](#)) volumic field with 3 components representing the face barycenters
- **[get_barycenter_from_prepro]** (*type*: flag) get aire IBM from prepro.
- **[rotation]** (*type*: [field_base](#)) volumic field with 9 components representing the change of basis on cells (local to global). Used for rotating cases for example.
- **[get_rotation_from_prepro]** (*type*: flag) get aire IBM from prepro.
- **[transpose_rotation]** (*type*: flag) whether to transpose the basis change matrix.
- **modele** (*type*: [bloc_pdf_model](#)) model used for the Penalized Direct Forcing
- **[interpolation]** (*type*: [interpolation_ibm_base](#)) interpolation method

source_qdm**Inherits from:** [source_base](#)

Momentum source term in the Navier-Stokes equations.

Parameters:

- **ch | champ** (*type*: [field_base](#)) Field type.

source_qdm_lambdaup**Inherits from:** [source_base](#)

This source term is a dissipative term which is intended to minimise the energy associated to non-conformal scales u' (responsible for spurious oscillations in some cases). The equation for these scales can be seen as: $\frac{du'}{dt} = -\lambda u' + \text{grad } P'$ where $-\lambda u'$ represents the dissipative term, with $\lambda = a/\Delta t$. For Crank-Nicholson temporal scheme, recommended value for a is 2.

Remark : This method requires to define a filtering operator.

Parameters:

- **lambda_ | lambda_u | lambda** (*type*: float) value of lambda
- **[lambda_min]** (*type*: float) value of lambda_min
- **[lambda_max]** (*type*: float) value of lambda_max
- **[ubar_umprim_cible]** (*type*: float) value of ubar_umprim_cible

source_th_tdivu**Inherits from:** [source_base](#)

This term source is dedicated for any scalar (called T) transport. Coupled with upwind (amont) or muscl scheme, this term gives for final expression of convection : $\text{div}(U.T) - T.\text{div}(U) = U.\text{grad}(T)$ This ensures, in incompressible flow when divergence free is badly resolved, to stay in a better way in the physical boundaries.

Warning: Only available in VEF discretization.

terme_puissance_thermique_echange_impose**Inherits from:** [source_base](#)

Source term to impose thermal power according to formula : $P = h_{imp} * (T - T_{ext})$. Where T is the Trust temperature, Text is the outside temperature with which energy is exchanged via an exchange coefficient himp

Parameters:

- **himp** (*type:* [field_base](#)) the exchange coefficient
- **text** (*type:* [field_base](#)) the outside temperature
- **[pid_controller_on_targer_power]** (*type:* [bloc_lecture](#)) PID_controller_on_targer_power bloc with parameters target_power (required), Kp, Ki and Kd (at least one of them should be provided)

travail_pression**Inherits from:** [source_base](#)

Source term which corresponds to the additional pressure work term that appears when dealing with compressible multiphase fluids

1.35 Keywords derived from sous_zone**sous_zone****Synonyms:** sous_domaine

It is an object type describing a domain sub-set.

A Sous_Zone (Sub-area) type object must be associated with a Domaine type object. The Read (Lire) interpreter is used to define the items comprising the sub-area.

Caution: The Domain type object nom_domaine must have been meshed (and triangulated or tetrahedralised in VEF) prior to carrying out the Associate (Associer) nom_sous_zone nom_domaine instruction; this instruction must always be preceded by the read instruction.

Parameters:

- **[restriction]** (*type:* string) The elements of the sub-area nom_sous_zone must be included into the other sub-area named nom_sous_zone2. This keyword should be used first in the Read keyword.
- **[rectangle]** (*type:* [bloc_origine_cotes](#)) The sub-area will include all the domain elements whose centre of gravity is within the Rectangle (in dimension 2).
- **[segment]** (*type:* [bloc_origine_cotes](#)) not_set
- **[boite | box]** (*type:* [bloc_origine_cotes](#)) The sub-area will include all the domain elements whose centre of gravity is within the Box (in dimension 3).
- **[liste]** (*type:* list of int) The sub-area will include n domain items, numbers No. 1 No. i No. n.
- **[fichier | filename]** (*type:* string) The sub-area is read into the file filename.
- **[intervalle]** (*type:* [deuxentiers](#)) The sub-area will include domain items whose number is between n1 and n2 (where $n1 \leq n2$).
- **[polynomes]** (*type:* [bloc_lecture](#)) A REPENDRE
- **[couronne]** (*type:* [bloc_couronne](#)) In 2D case, to create a couronne.
- **[tube]** (*type:* [bloc_tube](#)) In 3D case, to create a tube.
- **[fonction_sous_zone | fonction_sous_domaine]** (*type:* string) Keyword to build a sub-area with the elements included into the area defined by fonction>0.
- **[union | union_with]** (*type:* string) The elements of the sub-area nom_sous_zone3 will be added to the sub-area nom_sous_zone. This keyword should be used last in the Read keyword.

1.36 Keywords derived from transparent_medium_radiation_model

transparent_medium_radiation_model

Synonyms: modele_rayonnement_milieu_transparent

Transparent medium radiation model associated to a fluid problem

Parameters:

- **fichier_face_rayo** (*type*: string) Face radiation file (boundaries, surface and emisivity)
- **fichier_fij** (*type*: string) View factor file matrix
- **[fichier_matrice]** (*type*: string) File name to dump matrix
- **[relaxation]** (*type*: float) Relaxation parameter
- **[format_binaire]** (*type*: flag) Flag to dump matrix in a binary file. By default it is ASCII

1.37 Keywords derived from turbulence_paro_base

turbulence_paro_base

Basic class for wall laws for Navier-Stokes equations.

negligeable

Inherits from: [turbulence_paro_base](#)

Keyword to suppress the calculation of a law of the wall with a turbulence model. The wall stress is directly calculated with the derivative of the velocity, in the direction perpendicular to the wall ($\tau_{\text{tan}}/\rho = \nu \, dU/dy$).

Warning: This keyword is not available for k-epsilon models. In that case you must choose a wall law.

1.38 Keywords derived from turbulence_paro_scalaire_base

turbulence_paro_scalaire_base

Basic class for wall laws for energy equation.

negligeable_scalaire

Inherits from: [turbulence_paro_scalaire_base](#)

Keyword to suppress the calculation of a law of the wall with a turbulence model for thermohydraulic problems. The wall stress is directly calculated with the derivative of the velocity, in the direction perpendicular to the wall.

Index

Keywords derived from analyse_angle, [91](#)
Keywords derived from bloc_lecture, [156](#)
Keywords derived from bord_base, [156](#)
Keywords derived from ch_front_input, [60](#)
Keywords derived from champ_composite, [50](#)
Keywords derived from champ_don_base, [47](#)
Keywords derived from champ_fonc_fonction, [51](#)
Keywords derived from champ_fonc_med, [53](#)
Keywords derived from champ_fonc_tabule, [51](#)
Keywords derived from champ_fonc_tabule_morceaux, [51](#)
Keywords derived from champ_front_composite, [60](#)
Keywords derived from champ_front_fonc_txyz, [61](#)
Keywords derived from champ_front_tabule, [61](#)
Keywords derived from champ_generique_base, [20](#)
Keywords derived from champ_input_base, [55](#)
Keywords derived from champ_post_de_champs_post, [21](#)
Keywords derived from champ_post_operateur_base, [25](#)
Keywords derived from champ_post_statistiques_base, [25](#)
Keywords derived from champ_tabule_temps, [52](#)
Keywords derived from champ_uniforme_morceaux, [52](#)
Keywords derived from chimie, [27](#)
Keywords derived from class_generic, [27](#)
Keywords derived from comment, [31](#)
Keywords derived from condlim_base, [31](#)
Keywords derived from conduction, [124](#)
Keywords derived from cons_euler, [125](#)
Keywords derived from convection_deriv, [157](#)
Keywords derived from convection_diffusion_chaleur_qc, [127](#)
Keywords derived from convection_diffusion_concentration, [128](#)
Keywords derived from convection_diffusion_espece_binaire_qc, [129](#)
Keywords derived from convection_diffusion_temperature, [130](#)
Keywords derived from correlation_base, [43](#)
Keywords derived from defbord, [160](#)
Keywords derived from diffusion_deriv, [161](#)
Keywords derived from dirichlet, [37](#)
Keywords derived from discretisation_base, [44](#)
Keywords derived from domaine, [46](#)
Keywords derived from domaine_base, [46](#)
Keywords derived from dt_start, [27](#)
Keywords derived from eqn_base, [114](#)
Keywords derived from extract_2d_from_3d, [91](#)
Keywords derived from extruder, [92](#)
Keywords derived from field_base, [46](#)
Keywords derived from fluide_base, [107](#)
Keywords derived from fluide_incompressible, [111](#)
Keywords derived from flux_parietal_base, [43](#)
Keywords derived from format_file_base, [163](#)
Keywords derived from front_field_base, [55](#)
Keywords derived from frontiere_ouverte, [40](#)
Keywords derived from frontiere_ouverte_alpha_impose, [38](#)
Keywords derived from frontiere_ouverte_gradient_pression_impose, [41](#)
Keywords derived from frontiere_ouverte_temperature_imposee, [38](#)
Keywords derived from frontiere_ouverte_vitesse_imposee, [39](#)
Keywords derived from frontiere_ouverte_vitesse_imposee_sortie, [39](#)
Keywords derived from imprimer_flux, [92](#)
Keywords derived from interface_base, [61](#)
Keywords derived from interpolation_ibm_base, [63](#)
Keywords derived from interpolation_ibm_elem_fluid, [63](#)
Keywords derived from interpolation_ibm_mean_gradient, [66](#)
Keywords derived from interprete, [67](#)
Keywords derived from interprete_geometrique_base, [93](#)
Keywords derived from lecture_bloc_moment_base, [164](#)
Keywords derived from loi_etat_base, [101](#)
Keywords derived from loi_etat_gaz_parfait_base, [102](#)
Keywords derived from loi_etat_gaz_reel_base, [104](#)
Keywords derived from loi_etat_tppi_base, [104](#)
Keywords derived from loi_fermeture_base, [105](#)
Keywords derived from loi_horaire, [105](#)
Keywords derived from mailler, [93](#)
Keywords derived from mailler_base, [164](#)
Keywords derived from milieu_base, [106](#)
Keywords derived from mod_turb_hyd_ss_maille, [166](#)
Keywords derived from modele_turbulence_hyd_deriv, [165](#)
Keywords derived from modele_turbulence_scal_base, [112](#)
Keywords derived from mor_eqn, [114](#)
Keywords derived from moyenne_imposee_deriv, [139](#)
Keywords derived from navier_stokes_standard, [131](#)
Keywords derived from navier_stokes_turbulent, [137](#)
Keywords derived from neumann, [39](#)
Keywords derived from neumann_homogene, [41](#)
Keywords derived from neumann_paro, [41](#)
Keywords derived from nom, [141](#)
Keywords derived from objet_lecture, [141](#)
Keywords derived from op_non_conservativtifs_deriv, [170](#)
Keywords derived from parametre_equation_base, [170](#)
Keywords derived from paroi_echange_externer_impose, [42](#)
Keywords derived from paroi_echange_global_impose, [42](#)
Keywords derived from paroi_fixe, [43](#)
Keywords derived from paroi_temperature_imposee, [39](#)
Keywords derived from partitionneur_deriv, [182](#)

Keywords derived from pave, [164](#)
Keywords derived from pb_base, [185](#)
Keywords derived from pb_champ_evaluateur, [184](#)
Keywords derived from pb_gen_base, [185](#)
Keywords derived from pb_multiphase, [218](#)
Keywords derived from petsc, [29](#)
Keywords derived from piso, [262](#)
Keywords derived from points, [176](#)
Keywords derived from porosites, [222](#)
Keywords derived from postraitement, [171](#)
Keywords derived from postraitement_base, [171](#)
Keywords derived from precondition_base, [222](#)
Keywords derived from preconditionneur_petsc_deriv,
[223](#)
Keywords derived from prepro_ibm_base, [225](#)
Keywords derived from raffiner_isotrope, [93](#)
Keywords derived from read_file, [94](#)
Keywords derived from read_med, [94](#)
Keywords derived from regroupebord, [95](#)
Keywords derived from reorder_mesh, [226](#)
Keywords derived from saturation_base, [62](#)
Keywords derived from scatter, [96](#)
Keywords derived from sch_cn_iteratif, [246](#)
Keywords derived from schema_implicite_base, [250](#)
Keywords derived from schema_temps_base, [226](#)
Keywords derived from sets, [264](#)
Keywords derived from simple, [263](#)
Keywords derived from simplr, [261](#)
Keywords derived from solv_gcp, [30](#)
Keywords derived from solveur_implicite_base, [260](#)
Keywords derived from solveur_petsc_deriv, [266](#)
Keywords derived from solveur_sys_base, [28](#)
Keywords derived from sonde_base, [174](#)
Keywords derived from source_base, [273](#)
Keywords derived from sous_zone, [281](#)
Keywords derived from spec_pdcr_base, [177](#)
Keywords derived from stat_post_deriv, [177](#)
Keywords derived from symetrie, [43](#)
Keywords derived from tetraedriser, [96](#)
Keywords derived from traitement_particulier_base, [178](#)
Keywords derived from transformer, [99](#)
Keywords derived from transparent_medium_radiation_model,
[282](#)
Keywords derived from triangulate, [100](#)
Keywords derived from turbulence_paro_base, [282](#)
Keywords derived from turbulence_paro_scalaire_base,
[282](#)
Keywords derived from type_diffusion_turbulente_multiphase_deriv,
[180](#)
Keywords derived from type_perte_charge_deriv, [181](#)
Keywords derived from write_file, [101](#)
Keywords Reference Manual, [1](#)